Einführung

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 15.10.2026

Organisatorisches

- **Personen**:
  - Dr. Fabian Scheipl (VL)
  - Jana Gauß (Ü)
  - Maxim Winter (Tutorium)
- **Zeiten & Räume**:

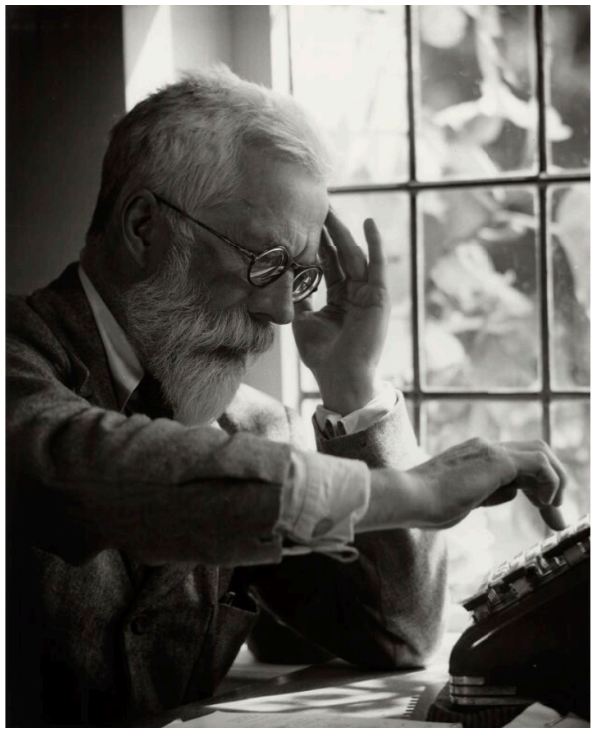
VL	Do 10:15-12:00	Hgb. – M 114
Übung 1	Di 14:15-15:45	Hgb. – M 105
Übung 2	Mi 12:15-13:45	Hgb. – B 106
Tutorium “Mathematische Grundlagen”	Di 10:00-12:00	S 003 (Schellingstr. 3)

- **Termine WS 26/27**: wöchentlich Do, **15.10.2026 – 4.2.2027** (Weihnachtspause: letzte VL 17.12.2026, weiter am 7.1.2027)

Motivation

Inhalt: Statistik früher & heute

Statistik früher



Statistik heute



Motivation

- In der Praxis ist Statistik / ML immer *computational*.
- In diesem Kurs lernen wir *mathematische Konzepte*, die bei der praktischen Lösung von Problemen helfen.
- Diese Konzepte werden benötigt, um *Computeralgorithmen* zu entwickeln, verstehen und analysieren.

  $\Longrightarrow$ fortgeschrittene (numerische) Lineare Algebra

  $\Longrightarrow$ fortgeschrittene Analysis

Warum so viel Mathe? Ein Beispiel

Aufgabe: Trainiere ein lineares Modell $y_i \approx \beta_1 x_{i1} + \cdots + \beta_p x_{ip}$ mit $p = 10.000$ Features auf $n = 100.000$ Beobachtungen.

$\Longrightarrow$ **Kleinste-Quadrate-Problem:**

$$\min_{\boldsymbol{\beta} \in \mathbb{R}^p} \left(\sum_{i=1}^{n} \left(y_i - \sum_{j=1}^{p} x_{ij} \beta_j \right)^2 \right) = \min_{\boldsymbol{\beta} \in \mathbb{R}^p} \|\boldsymbol{X}\boldsymbol{\beta} - \boldsymbol{y}\|_2^2$$

Naive Lösung (falls $\operatorname{rang}(\boldsymbol{X}) = p$)**:** $\widehat{\boldsymbol{\beta}} = (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$

Probleme in der Praxis:

1. **Kondition**: $\boldsymbol{X}^\top \boldsymbol{X}$ kann sehr schlecht “konditioniert” sein $\Longrightarrow$ numerische Instabilität, große Fehler
2. **Komplexität**: Matrixinversion kostet $O(p^3)$ Operationen $\Longrightarrow$ für $p = 10.000$: ca. 10^{12} FLOPs!
3. **Speicher**: $\boldsymbol{X}^\top \boldsymbol{X}$ ist $p \times p$ Matrix $\Longrightarrow$ für $p = 10.000$: ca. 800 MB nur für diese Matrix

Bessere Lösungen: QR-Zerlegung, SVD, iterative Verfahren $\rightarrow$ dieser Kurs

Themen

- Kondition und Komplexität
  - Was sind gute Algorithmen?
- Numerische Lineare Algebra
  - Matrix- & Tensorrechnung
  - Berechne (KQ-)Lösungen für $\boldsymbol{Ax} \approx \boldsymbol{b}$ etc.
- Fortgeschrittene Differenzialrechnung / Analysis
  - Höherdimensionale, verallgemeinerte Ableitungen
  - Wie kann man komplizierte Fkt. einfach approximieren?
- Numerische Optimierung
  - Wie finde ich Minima/Maxima von komplizierten Funktionen?
  - z.B. Training neuronaler Netze: Millionen Parameter, nicht-konvexe Verlustfunktionen $\rightarrow$ smarte Algorithmen nötig (SGD & Co.)

Fünf Beispiele

Beispiel 1: Kondition

Betrachte das lineare Gleichungssystem $\boldsymbol{Ax} = \boldsymbol{b}$ mit

$$\boldsymbol{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1.0001 \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} 2 \\ 2.0001 \end{pmatrix}$$

Exakte Lösung: $\boldsymbol{x} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$

Gestörtes Problem (z.B. durch Rundungsfehler): $\boldsymbol{Ax} = \tilde{\boldsymbol{b}}$ mit

$$\tilde{\boldsymbol{b}} = \begin{pmatrix} 2 \\ 2.0002 \end{pmatrix} \quad (\text{nur } 0.005\% \text{ Fehler!})$$

Lösung des gestörten Problems: $\tilde{\boldsymbol{x}} = \begin{pmatrix} 0 \\ 2 \end{pmatrix}$

$\Longrightarrow$ **100% Fehler** in x_1 durch 0.005% Fehler in den Daten!

$\Longrightarrow$ Das Problem ist (furchtbar) **schlecht konditioniert**.

Beispiel 2: Komplexität

Matrixmultiplikation $C = AB$ für $n \times n$ Matrizen:

Algorithmus	**Rechenops. für $n = 1000$**	**Komplexität**
Naive Methode	$\approx 10^9$ Ops	$O(n^3)$
Strassen	$\approx 6 \times 10^8$ Ops	$O(n^{2.807})$
Optimiert	$\approx 2 \times 10^8$ Ops	$O(n^{2.373})$

Für $n = 10.000$:

- Naiv: $\approx 10^{12}$ Ops
- Optimiert: $\approx 2 \times 10^{10}$ Ops

$\Longrightarrow$ **Faktor 50 schneller** durch besseren Algorithmus (...potentiell)!

Beispiel 3: Höherdimensionale Ableitungen

Beispiel AI: Gradient Descent für neuronale Netze

Verlustfunktion: $L : \mathbb{R}^{p \times K} \to \mathbb{R}$ (Matrix-Argument $\boldsymbol{W}$!)

Zum Beispiel: **MSE**

$$L(\boldsymbol{W}) = \frac{1}{2} \|\boldsymbol{X}\boldsymbol{W} - \boldsymbol{Y}\|_F^2$$

mit $\boldsymbol{X} \in \mathbb{R}^{N \times p}$, $\boldsymbol{W} \in \mathbb{R}^{p \times K}$, $\boldsymbol{Y} \in \mathbb{R}^{N \times K}$

Frage: Wie berechnen wir $\frac{\partial L}{\partial \boldsymbol{W}}$?

Naive Methode: Berechne $\frac{\partial L}{\partial W_{ij}}$ für alle i, j einzeln

$\implies$ $p \times K$ Ableitungen!

Für $p = 1000, K = 10$: **10.000 Ableitungen**!

Bessere Methode: Matrix-Kalkül

$$\frac{\partial L}{\partial \boldsymbol{W}} = \boldsymbol{X}^\top (\boldsymbol{X}\boldsymbol{W} - \boldsymbol{Y})$$

Eine Zeile statt 10.000! Und: **effizient implementierbar**.

$\implies$ Ableitungen von Matrix/Tensor-Funktionen essentiell für Deep Learning, Optimierung, etc.

Beispiel 4: Funktionsapproximation: Taylor-Reihen

Ziel: Approximiere komplizierte Funktion $f(x)$ durch ein Polynom in der Nähe von x_0

Taylor-Approximation 2. Ordnung:

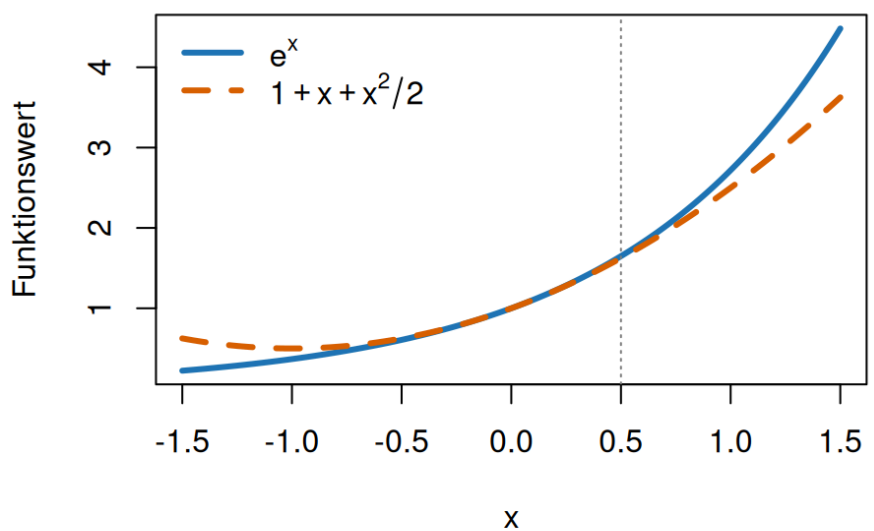
$$f(x) \approx f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)}{2}(x - x_0)^2$$

Konkretes Beispiel: $f(x) = e^x$ um $x_0 = 0$

$$f(0) = f'(0) = f''(0) = 1 \implies e^x \approx 1 + x + \frac{x^2}{2}$$

Fehler für $x = 0.5$:

- Exakt: $e^{0.5} = 1.6487\ldots$
- Approximation: $1 + 0.5 + 0.125 = 1.625$
- Relativer Fehler: $< 2\%$



Beispiel 5: Optimierung in Machine Learning

Typisches Problem: Training eines neuronalen Netzes

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^d} L(\boldsymbol{\theta}), \qquad L(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \ell(f(\boldsymbol{x}_i; \boldsymbol{\theta}), y_i)$$

Modell	**Parameter** d	**Daten** N
Logistische Regression (MNIST, 10 Klassen)	$\sim 8 \times 10^3$	6×10^4
ResNet-50	$\sim 2.5 \times 10^7$	$> 10^6$
GPT-3	$\sim 1.75 \times 10^{11}$	$\sim 3 \times 10^{11}$ Tokens

Herausforderungen:

1. **Nicht-konvex:** Viele lokale Minima / Sattelpunkte der Verlustfunktion
2. **Hochdimensional:** Naive Methoden scheitern
3. **Stochastisch:** Nur Zugriff auf Minibatches

$\Longrightarrow$ Brauchen **smarte Algorithmen**: Stochastic Gradient Descent, Momentum, etc.

Vorkenntnisse & Selbsttest

Benötigte Vorkenntnisse

- Lineare Algebra (VL Lineare Algebra I):
  - v.a. Matrixrechnung, Eigenwerte und -vektoren
- Analysis (VL Analysis I):
  - v.a. Stetigkeit & Differenzierbarkeit, Ableitungen
- R-Programmierung (VL Einführung in R)

Empfehlung:

- Intuition für LA z.B. mit *3blue1brown* o.ä. aufbauen!
- Skript **aktiv** lesen & benutzen um Grundbegriffe zu wiederholen

(Minimaler) Selbsttest: Vorkenntnisse evtl. ausreichend?

Lineare Algebra - Sie sollten wissen:

1. Was ist der *Rang* einer Matrix? Wie berechne ich ihn?
2. Was bedeutet es, wenn eine Matrix $\boldsymbol{A}$ *invertierbar* ist?
3. Was sind *Eigenwerte*/*Eigenvektoren* einer Matrix? Wie finde ich sie?
4. Was ist die geometrische Bedeutung einer Matrix-Vektor-Multiplikation $\boldsymbol{Ax}$?
5. Wann bilden Vektoren $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$ eine *Basis* des $\mathbb{R}^n$? Was ist ihr *Span*?

(Minimaler) Selbsttest: Vorkenntnisse evtl. ausreichend?

Analysis - Sie sollten wissen:

1. Was bedeutet es, wenn eine Funktion $f : \mathbb{R} \to \mathbb{R}$ *stetig* ist?
2. Was ist die *Ableitung* $f'(x_0)$ geometrisch?
3. Was ist ein lokales vs. globales Minimum?

Algorithmen und Komplexität

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 15.10.2026

Verwendete Vorkenntnisse

- **Grundlegende Analysis** (aus Analysis I)
  - Folgen und Grenzwerte
  - Grundbegriffe der Konvergenz
- **Grundlegende Lineare Algebra** (aus Lineare Algebra I)
  - Matrixoperationen (Addition, Multiplikation)
  - Verständnis des Rechenaufwands von Operationen

Numerische Probleme und Algorithmen

Numerische Probleme

- Ein numerisches Problem hat als Lösung eine oder mehrere Zahlen.
  - Wert eines Integrals
  - Lösung eines LGS
  - Basis-Koeffizienten in Funktionsapproximation
- Allgemein:

> *Gegeben Problem f mit Input $\boldsymbol{x}$, berechne Lösung $f(\boldsymbol{x})$.*

Live-Demo: Rechnen Computer richtig?

Erst raten, dann ausführen!

Was geben diese Codezeilen aus?

```r
1.0 - 1.0
```

```
[1] 0
```

```r
1.0 - 0.9 - 0.1
```

```
[1] -2.78e-17
```

```r
100 * 0.58 == 58
```

```
[1] FALSE
```

```r
x <- seq(1, 2e16, length = 10^5)
sum(x) - sum(rev(x))
```

```
[1] -262144
```

Numerische Probleme: Schwierigkeiten

- Computer sind so gut wie nie exakt.
- Sie können
  - … nur endliche und endlich viele Zahlen darstellen.
    - $\pi = 3.14159265358979323846264338327 9\ldots$ 😞
  - … nur endlich viele Berechnungen durchführen.
    - $e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$ 😞
- Deshalb ist es wichtig, *wie* wir Dinge berechnen!

Numerische Probleme: Schwierigkeiten

- Computer können nur endlich viele Zahlen darstellen.
- Das führt zu *Rundungsfehlern*, die sich *akkumulieren* können.
- Die *Reihenfolge* unserer Rechenoperationen spielt eine Rolle.
- *Fehler* einzelner Operationen sind oft vernachlässigbar, aber sie *akkumulieren*.
- **Empfehlung**: Besonders große/kleine Zahlen vermeiden.

Beispiel: Katastrophale Auslöschung

Problem: Berechne die Varianz von $\boldsymbol{x}$ mit der Formel

$$\mathrm{Var}(\boldsymbol{x}) = \frac{1}{n}\sum_{i=1}^{n} x_i^2 - \left(\frac{1}{n}\sum_{i=1}^{n} x_i\right)^2$$

Daten: $\boldsymbol{x} = (4 + 10^9,\ 7 + 10^9,\ 13 + 10^9,\ 16 + 10^9)$

Berechnung:

```
x <- c(4, 7, 13, 16) + 1e9
mean((x - mean(x))^2) # Varianz (mit 1/n)
```

```
[1] 22.5
```

```
mean(x^2) - mean(x)^2 # !! auch Varianz (mit 1/n) ?!
```

```
[1] 0
```

- **Ursache:** Subtraktion zweier fast gleich großer Zahlen ($\approx 10^{18}$).

```
mean(x^2)
```

```
[1] 1e+18
```

```
mean(x)^2
```

```
[1] 1e+18
```

Beispiel: Assoziativität verletzt

Problem: Gleitkomma-Addition ist nicht assoziativ!

Konkretes Beispiel: Seien $x = 10^{30}, y = -10^{30}, z = 1$.

$$(x + y) + z = (10^{30} + (-10^{30})) + 1 = 0 + 1 = 1$$
$$x + (y + z) = 10^{30} + ((-10^{30}) + 1) \overset{!!}{\approx} 10^{30} + (-10^{30}) = 0$$

- Die zweite Berechnung ergibt 0, weil 1 zu klein ist gegenüber 10^{30}.

$\Longrightarrow$ Reihenfolge der Summe beeinflusst Ergebnis!!

Algorithmen

Def.: Algorithmus

Ein **Algorithmus** ist ein Verfahren $\widetilde{f} = \widetilde{f}_s \circ \cdots \circ \widetilde{f}_1$, das für Inputs $\boldsymbol{x}$ eine mögliche Lösung $\widetilde{f}(\boldsymbol{x})$ des *Problems* $f(\boldsymbol{x})$ berechnet.

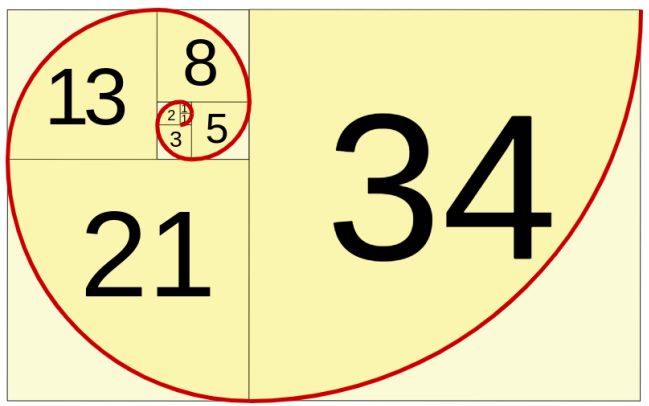
Arten von Algorithmen

- *exakt*: es gilt $f(\boldsymbol{x}) = \widetilde{f}(\boldsymbol{x})$ (bis auf Rundungsfehler)
  vs. *approximativ*: es gilt lediglich $f(\boldsymbol{x}) \approx \widetilde{f}(\boldsymbol{x})$.
- *iterativ*: gleiche Vorschrift wird mehrfach ausgeführt
  vs *direkt*: Vorschrift wird nur einmal ausgeführt.
- *probabilistisch/randomisiert*: beruht auf Zufall
  vs *deterministisch*: keine Zufallselemente.

Beispiel: Fibonacci-Zahlen

- Problem: Berechne die ersten N Fibonacci-Zahlen

$$x_1 = 0, \quad x_2 = 1, \quad x_{n+1} = x_n + x_{n-1}\ (n \geq 2).$$



- Algorithmus:

$$\widetilde{f} : \mathbb{N} \to \mathbb{N}^n$$
$$\widetilde{f}(n) = (\widetilde{f}_n \circ \cdots \circ \widetilde{f}_3 \circ \widetilde{f}_2 \circ \widetilde{f}_1)(n)$$
$$\widetilde{f}_1(n) = 0, \quad \widetilde{f}_2(x) = (x, 1),$$
$$\widetilde{f}_{i+1}(x_1, \ldots, x_i) = (x_1, \ldots, x_i, x_i + x_{i-1}) \quad (i \geq 2).$$

Fibonacci: Implementierung

- Implementierung des Algorithmus:

```
fibonacci <- function(n) {
  x <- numeric(n)  # Vektor (0, ..., 0) der Länge n
  if (n > 1) {
    x[2] = 1
  }
  if (n > 2) {
    for (i in 2:(n-1)) {
      x[i+1] <- x[i] + x[i - 1]
    }
  }
  x
}
```

- **Beispiel** ($n = 6$; schrittweise Rechnung s. Anhang):

```
fibonacci(6)
```

```
[1] 0 1 1 2 3 5
```

Quiz

Der Algorithmus ist ...

1. exakt ?
2. iterativ ?
3. probabilistisch ?

Algorithmenarten in ML / Statistik

- **Exakt/direkt:**
  - LGS-Lösung via Gauß-Elimination (später...)
  - Matrixmultiplikation
- **Approximativ:**
  - Monte-Carlo-Integration (schätzt Integrale durch Stichproben)
  - Neuronale Netze (optimieren bis zu einem Toleranzlevel)
- **Iterativ:**
  - Gradientenabstieg in Machine Learning
  - Newton-Verfahren zum Nullstellenfinden
- **Probabilistisch:**
  - Approximative Singulärwertzerlegung, Matrix Sketching
  - MCMC-Methoden in Bayesianischer Statistik

Oft kombiniert: ML-Algorithmen sind oft iterativ + approximativ + probabilistisch!

Analogie

Problem:

> “Koche Schweinebraten wie bei Oma.”

Algorithmus $\leftrightarrow$ Kochrezept

Computer $\leftrightarrow$ Koch *(der sich ans Rezept hält, aber Fehler macht)*

Gute Algorithmen

Wir wollen Algorithmen, die

- möglichst exakte Ergebnisse liefern,
- schnell sind,
- wenig Speicher brauchen.

Aufwand und Komplexität

Aufwand und Komplexität

- Der Computer führt einen Algorithmus durch elementare Operationen aus: Rechenoperationen $f_i \in \{+, -, \cdot, /\}$, aber auch Vergleiche, Zuweisungen und Speicherzugriffe — jede mit (etwa) konstantem Aufwand.
- Der *Zeitaufwand* des Algorithmus ist die Anzahl dieser Operationen.
  - höherer Aufwand $\approx$ längere Laufzeit
- Der *Speicheraufwand* ist (in etwa) die Anzahl der (zwischen-)gespeicherten Zahlen.
  - höherer Aufwand $\rightarrow$ mehr Speicherplatz

Beispiel: Matrix-Vektor-Multiplikation

Problem: Berechne $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}$ für $\boldsymbol{A} \in \mathbb{R}^{3\times 2}$, $\boldsymbol{x} \in \mathbb{R}^2$.

Konkret: $\boldsymbol{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$, $\boldsymbol{x} = \begin{pmatrix} 7 \\ 8 \end{pmatrix}$

Berechnung:

$$
\begin{aligned}
y_1 &= 1 \cdot 7 + 2 \cdot 8 = 7 + 16 = 23 \quad (2 \text{ Mult., } 1 \text{ Add.}) \\
y_2 &= 3 \cdot 7 + 4 \cdot 8 = 21 + 32 = 53 \quad (2 \text{ Mult., } 1 \text{ Add.}) \\
y_3 &= 5 \cdot 7 + 6 \cdot 8 = 35 + 48 = 83 \quad (2 \text{ Mult., } 1 \text{ Add.})
\end{aligned}
$$

- **Zeitaufwand**: $3 \times 2 = 6$ Multiplikationen + $3 \times 1 = 3$ Additionen $= 9$ Operationen
- *Allgemein:* Für $\boldsymbol{A} \in \mathbb{R}^{n\times d}$: nd Multiplikationen + $n(d-1)$ Additionen $\approx 2nd$ Operationen
- **Speicheraufwand**: nd (für $\boldsymbol{A}$) + d (für $\boldsymbol{x}$) + n (für $\boldsymbol{y}$) $= nd + n + d$ Zahlen

Quiz: Zeit- und Speicheraufwand

Sei $\boldsymbol{A} \in \mathbb{R}^{n \times d}$, $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^d$,

$$f(\boldsymbol{A}, \boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{A}(\boldsymbol{x} - \boldsymbol{y}) = \sum_{i=1}^{d} (x_i - y_i) \boldsymbol{A}_{\cdot i}.$$

```
f <- rep(0, nrow(A))
for (i in 1:ncol(A)) {
  f <- f + (x[i] - y[i]) * A[, i]
}
```

a) Wie viele elementare Rechenoperationen werden benötigt?

1. $2n + d$
2. $2dn + d$
3. dn^2
4. $2(n + d)$

b) Für wie viele Gleitkommazahlen brauchen wir Speicherplatz?

1. $3n + d$
2. $4nd$
3. $2(n + d)$
4. $n + nd + 2d$

Quiz: Lösung

a) Zeitaufwand: $2nd + d$ Operationen

Aufschlüsselung pro Iteration ($i = 1, \ldots, d$):

1. **Subtraktion:** $(x_i - y_i) \rightarrow$ 1 Operation
2. **Skalierung:** $(x_i - y_i) \cdot \boldsymbol{A}_{\cdot i} \rightarrow n$ Multiplikationen (1 Element/Zeile f. $\boldsymbol{A}_{\cdot i}$)
3. **Addition:** `f <- f + ...` $\rightarrow n$ Additionen

Pro Iteration: $1 + n + n = 2n + 1$ Operationen

Gesamt: $d \times (2n + 1) = 2nd + d$ Operationen

b) Speicheraufwand: $n + nd + 2d$ Gleitkommazahlen

- n (für `f`) + nd (für $\boldsymbol{A}$) + $2d$ (für $\boldsymbol{x}$ und $\boldsymbol{y}$)

Komplexität

- Die **exakte Anzahl** an Operationen und gespeicherten Zahlen zu kennen, ist **nicht wichtig**.
- Wir sind hauptsächlich daran interessiert **wie schnell der Aufwand** mit der Größe des Problems wächst (**“skaliert”**).
- Das nennen wir *(Laufzeit-/Speicher-)Komplexität des Algorithmus*.

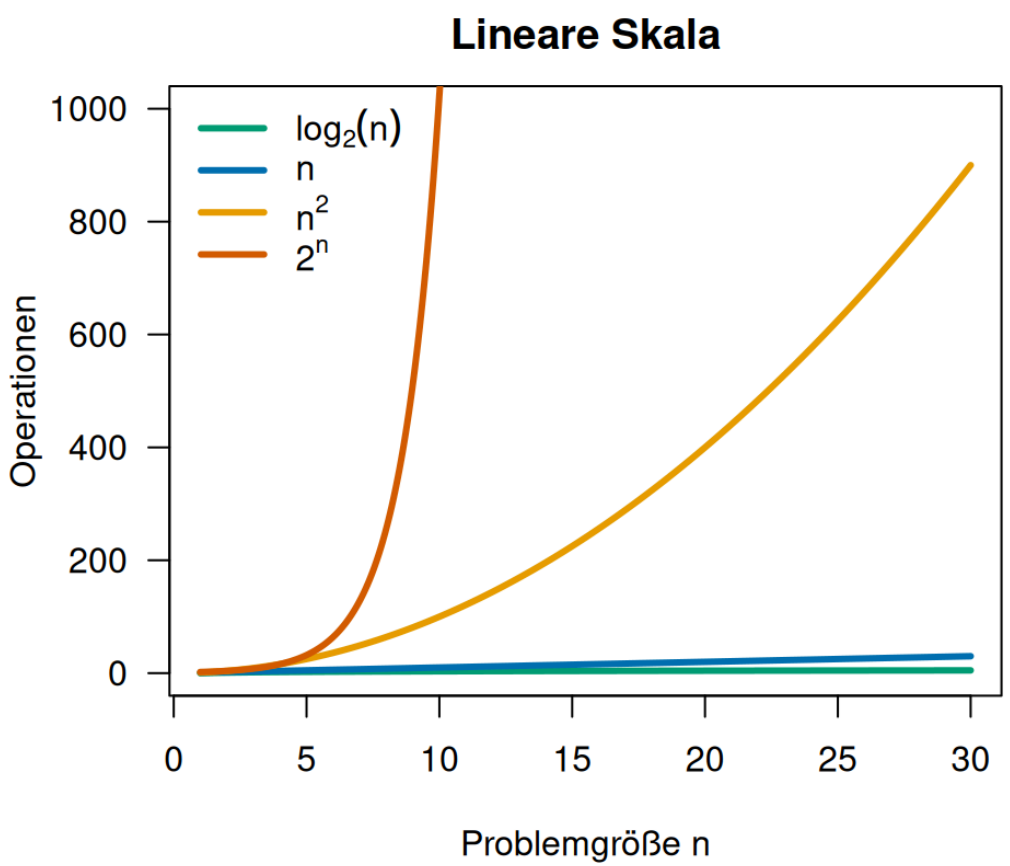
Komplexität

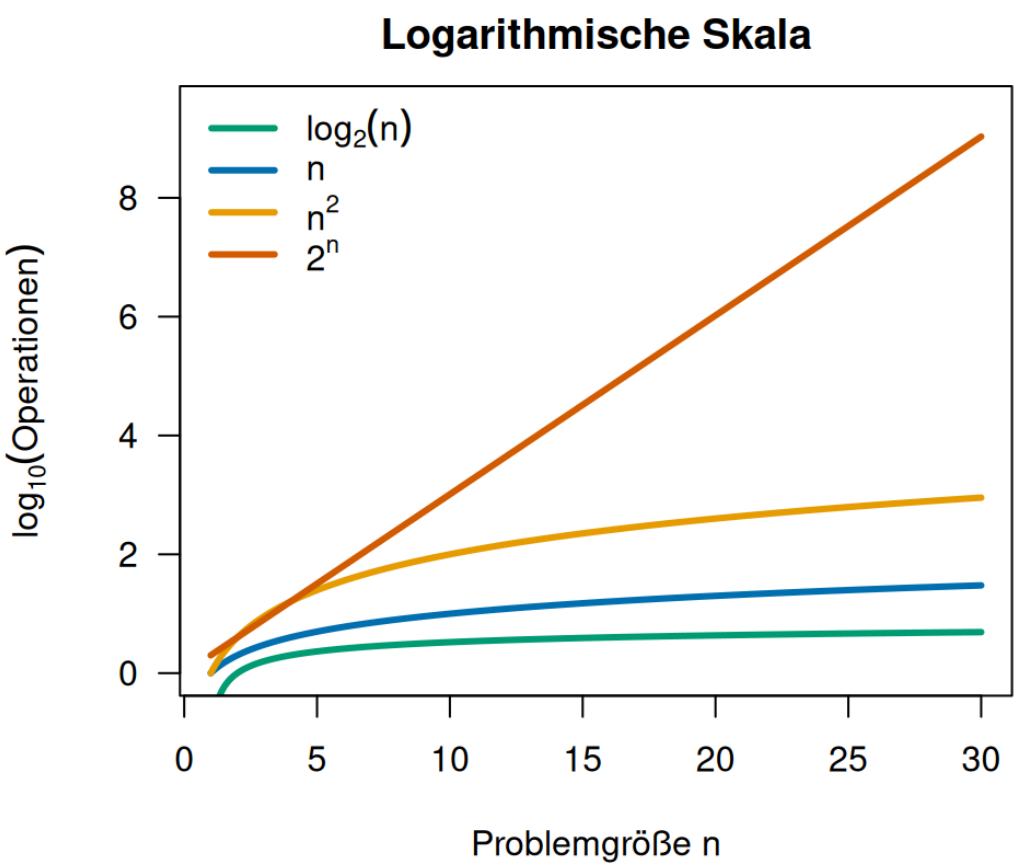
Beispiel:

- Ein Algorithmus benötigt $4n^3 + 16n^2 + 239$ Operationen.
- Für großes n ist $16n^2 + 239$ gegenüber $4n^3$ vernachlässigbar.

$\Longrightarrow$ Der Algorithmus skaliert wie $4n^3$,
die Komplexität ist *kubischer* *Ordnung*: "$O(n^3)$".

Komplexitätsklassen





Komplexitätsklassen

Wie verschiedene Komplexitäten skalieren:

Interpretation:

- $O(1)$ (konstant):
  Anzahl Operationen ist unabhängig von n
- $O(\log_b n)$ (logarithmisch):
  eine Operation mehr falls sich n ver-b-facht.
- $O(n)$ (linear):
  verdoppelt sich n, verdoppeln sich Operationen
- $O(n^2)$ (quadratisch):
  verdoppelt sich n, vervierfachen sich Operationen
- $O(2^n)$ (exponentiell):
  verdoppelt sich n, exponentieren sich Operationen

Vorsicht:

$O()$**-Notation ignoriert konstante Faktoren und Terme** – z.B.

- $1000n + 10000$ Operationen sind immer noch $O(n)$
- aber (viel) langsamer als n^2 Operationen ($O(n^2)$) für $n < 1010$!

Landausymbole

Def.: Landausymbole

Seien a_n, b_n Folgen mit $b_n \neq 0$.

- $a_n = o(b_n)$ („a_n *ist* ***klein-o*** *von* b_n"), wenn

$$\lim_{n\to\infty} \frac{a_n}{b_n} = 0.$$

- $a_n = O(b_n)$ („a_n *ist* ***groß-O*** *von* b_n"), wenn

$$\limsup_{n\to\infty} \left|\frac{a_n}{b_n}\right| < \infty.$$

Interpretation:

- $a_n = o(b_n)$: a_n **wächst langsamer als** b_n, a_n wird gegenüber b_n vernachlässigbar klein.
- $a_n = O(b_n)$: a_n **wächst höchstens so schnell wie** b_n, b_n ist gegenüber a_n nicht vernachlässigbar.
- Zusatz:

$$a_n = \Theta(b_n)$$

falls $a_n = O(b_n)$ und $b_n = O(a_n)$, d.h. a_n **wächst genauso schnell wie** b_n.

Landausymbole: Rechenbeispiel

Beispiel: Zeige, dass $3n^2 + 5n = O(n^2)$

$$\lim_{n\to\infty} \frac{3n^2 + 5n}{n^2} = \lim_{n\to\infty} \left(3 + \frac{5}{n}\right) = 3 < \infty$$
$$\Longrightarrow \; 3n^2 + 5n = O(n^2) \quad \checkmark$$

Weitere Rechenbeispiele: siehe Anhang

Landausymbole

- Meist suchen wir eine möglichst einfache Folge b_n, sodass $a_n = O(b_n)$ aber nicht $a_n = o(b_n)$.
- Schreibweise: $O(b_n)$ und $o(b_n)$ sind *Mengen* von Folgen, genau genommen also $a_n \in O(b_n)$. Das übliche Gleichheitszeichen heißt hier „ist ein" und gilt nur von links nach rechts: aus $n = O(n^2)$ und $n^2 = O(n^2)$ folgt **nicht** $n = n^2$.

Lemma: Ein paar Rechenregeln

Seien $a_n, b_n > 0$ (z.B. Anzahl von Operationen).

1. Sei $f_n = O(a_n)$ und $g_n = O(b_n)$. Dann gilt

$$f_n + g_n = O(a_n + b_n) \quad \text{und} \quad f_n \cdot g_n = O(a_n \cdot b_n).$$

2. Sei $f_n = O(a_n)$ und $g_n = o(b_n)$. Dann gilt

$$f_n \cdot g_n = o(a_n \cdot b_n)$$

3. Sei $f_n = O(a_n + b_n)$ mit $a_n = O(b_n)$. Dann gilt

$$f_n = O(b_n).$$

Rechenregeln: Anwendungsbeispiele

Beispiel Addition und Multiplikation:

Gegeben: $f_n = 3n^2 = O(n^2)$ und $g_n = 5n = O(n)$

$$\text{Addition:} \quad f_n + g_n = 3n^2 + 5n = O(n^2 + n) = O(n^2)$$
$$(\text{wegen Regel 3: } n = O(n^2))$$

$$\text{Multiplikation:} \quad f_n \cdot g_n = (3n^2) \cdot (5n) = 15n^3 = O(n^2 \cdot n) = O(n^3)$$

Beispiel Vereinfachung:

Gegeben: Algorithmus benötigt $4n^3 + 16n^2 + 239$ Operationen.

$$4n^3 + 16n^2 + 239 = O(n^3 + n^2 + 1)$$
$$= O(n^3) \quad (\text{da } n^2 = O(n^3) \text{ und } 1 = O(n^3))$$

$\Longrightarrow$ **Wir können komplexe Ausdrücke auf dominanten Term reduzieren!**

Fibonacci: Komplexitätsanalyse 1

Iterativer Algorithmus:

```r
fibonacci <- function(n) {
  x <- numeric(n)
  if (n > 1) {
    x[2] = 1
  }
  if (n > 2) {
    for (i in 2:(n-1)) {
      x[i+1] <- x[i] + x[i - 1]
    }
  }
  x
}
```

Zeitkomplexität:

- Initialisierung: `x <- numeric(n)` → $O(n)$
- Schleife: $n - 2$ Iterationen mit je konstant vielen Operationen (1 Addition + Indexrechnung) → $O(n)$
- **Gesamt:** $O(n)$ (linear)

Speicherkomplexität:

- Vektor `x` mit n Elementen → $O(n)$

Fibonacci: Komplexitätsanalyse 2

Naive Rekursion:

```
fib_recursive <- function(n) {
  if (n <= 1) {
    return(n)
  }
  return(
    fib_recursive(n-1) +
    fib_recursive(n-2)
  )
}
```

- **Zeitkomplexität:**
  jede Iteration macht 2 Aufrufe, die wiederum je 2 Aufrufe machen, usw.
  $\Longrightarrow$ insgesamt $O(2^n)$, also exponentiell!
  katastrophal langsam
- **Speicherkomplexität:**
  $O(n)$ (Rekursionstiefe)

Live: Laufzeit in Sekunden für $n = 30$ –

```
system.time(fibonacci(30))["elapsed"]
```

```
elapsed
  0.024
```

```
system.time(fib_recursive(30))["elapsed"]
```

```
elapsed
   4.27
```

$\Longrightarrow$ Iterative Lösung ist dramatisch effizienter ($\times 180$) – und liefert dabei sogar *alle* ersten n Zahlen, während die Rekursion nur F_n berechnet!

Self-Check: Landausymbole

Welche Aussagen sind wahr?

1. $2n = O(n^2)$
2. $7/n = o(1)$
3. $8n^3 + 7n^2 + n = O(n)$
4. $n^{-3} + n^{-2} = O(n^{-2})$

Zusammenfassung

- Für numerische *Probleme* f mit Input $\boldsymbol{x}$ wollen wir eine Lösung $f(\boldsymbol{x})$ berechnen.
- Gute *Algorithmen* lösen das *Problem* mit niedrigem Aufwand.
- Wir betrachten deshalb ihre *Laufzeit-/Speicher-Komplexitäten*.
- Diese analysieren wir anhand ihrer *Ordnung* mit *Landausymbolen*.

Nächste Vorlesung

- *Kondition* und *Stabilität*.

Anhang

Fibonacci: Schrittweise Berechnung

Berechne die ersten 6 Fibonacci-Zahlen ($n = 6$):

$$
\begin{aligned}
\widetilde{f}_1(6) &= 0 \\
\widetilde{f}_2(\widetilde{f}_1(6)) = \widetilde{f}_2(0) &= (0, 1) \\
\widetilde{f}_3(0, 1) &= (0, 1, 1) \\
\widetilde{f}_4(0, 1, 1) &= (0, 1, 1, 2) \\
\widetilde{f}_5(0, 1, 1, 2) &= (0, 1, 1, 2, 3) \\
\widetilde{f}_6(0, 1, 1, 2, 3) &= (0, 1, 1, 2, 3, 5)
\end{aligned}
$$

Ergebnis: $\widetilde{f}(6) = (0, 1, 1, 2, 3, 5)$

↩ zurück: „Fibonacci: Implementierung”

Landausymbole: Weitere Rechenbeispiele

Beispiel 2: Zeige, dass $5n = o(n^2)$

$$\lim_{n\to\infty} \frac{5n}{n^2} = \lim_{n\to\infty} \frac{5}{n} = 0 \implies 5n = o(n^2) \quad \checkmark$$

Beispiel 3: Ist $n^2 = O(n)$? **Nein!**

$$\lim_{n\to\infty} \frac{n^2}{n} = \lim_{n\to\infty} n = \infty \quad \text{(divergiert!)}$$
$$\implies n^2 \neq O(n)$$

↩ zurück: „Landausymbole: Rechenbeispiel"

Matrix-Spur und Matrixnormen

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- Normen als Fehlermaße
- Konditionszahlen
- **Lineare Algebra**
  - Matrixoperationen (Addition, Multiplikation, Transposition)
  - Eigenwerte und Eigenvektoren
  - Orthogonalität
  - Skalarprodukte
  - Grundlegende Vektornormen (Euklidische Norm $\|\boldsymbol{x}\|_2$, p-Normen)
- **Analysis**
  - Grenzwerte und Suprema
  - Maximum/Minimum von Funktionen

Die Spur einer Matrix

Matrix-Trace (Spur)

Def.: Spur

Die **Spur** (“trace”) einer quadratischen Matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ ist die Summe ihrer Diagonalelemente:

$$\operatorname{tr}(\boldsymbol{A}) := \sum_{i=1}^{n} a_{ii}$$

Bemerkungen:

- Nur für quadratische Matrizen definiert
- Skalare Invariante der Matrix

Beispiele:

- $\operatorname{tr}(\boldsymbol{I}_n) = n$ $\qquad\qquad$ $\operatorname{tr}\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} = 1 + 4 = 5$
- $\operatorname{tr}\begin{pmatrix} 2 & -1 & 0 \\ 0 & 3 & 5 \\ 1 & 0 & -2 \end{pmatrix} = 2 + 3 + (-2) = 3$

Grundlegende Eigenschaften der Spur

Theorem: Eigenschaften der Spur

Für Matrizen $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{n \times n}$ und $c \in \mathbb{R}$:

1. *Additiv*:
   $\operatorname{tr}(\boldsymbol{A} + \boldsymbol{B}) = \operatorname{tr}(\boldsymbol{A}) + \operatorname{tr}(\boldsymbol{B})$
2. *Homogen*:
   $\operatorname{tr}(c\boldsymbol{A}) = c \cdot \operatorname{tr}(\boldsymbol{A})$
3. *Zyklisch*:
   $\operatorname{tr}(\boldsymbol{AB}) = \operatorname{tr}(\boldsymbol{BA})$
4. *Ähnlichkeitsinvariant*:
   $\operatorname{tr}(\boldsymbol{PAP}^{-1}) = \operatorname{tr}(\boldsymbol{A})$ (für invertierbares $\boldsymbol{P} \in \mathbb{R}^{n \times n}$)
5. *Transpositionsinvariant*:
   $\operatorname{tr}(\boldsymbol{A}^\top) = \operatorname{tr}(\boldsymbol{A})$

Wichtigste Eigenschaft:

Zyklische Vertauschung:

$$\operatorname{tr}(\boldsymbol{ABC}) = \operatorname{tr}(\boldsymbol{BCA}) = \operatorname{tr}(\boldsymbol{CAB})$$

Vorsicht:

$\operatorname{tr}(\boldsymbol{AB}) = \operatorname{tr}(\boldsymbol{BA})$, aber $\operatorname{tr}(\boldsymbol{AB}) \neq \operatorname{tr}(\boldsymbol{A}) \cdot \operatorname{tr}(\boldsymbol{B})$!

Spur und Eigenwerte

Theorem: Spur als Summe der Eigenwerte

Für eine Matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ mit Eigenwerten $\lambda_1, \ldots, \lambda_n$:

$$\operatorname{tr}(\boldsymbol{A}) = \sum_{i=1}^{n} \lambda_i$$

Beweisskizze:

siehe Anhang

Kernidee:

Für diagonalisierbare $\boldsymbol{A} = \boldsymbol{P}\boldsymbol{D}\boldsymbol{P}^{-1}$ folgt die Aussage direkt aus der Ähnlichkeitsinvarianz (4).

Frobenius-Norm

Def.: Frobenius-Norm

Die **Frobenius-Norm** einer Matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ ist

$$\|\boldsymbol{A}\|_F := \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{n} a_{ij}^2}$$

Bemerkungen:

- Analog zur euklidischen Vektornorm
- Auch als **Hilbert-Schmidt-Norm** bekannt
- Misst sowas wie die “Größe” der Matrix

Spur und Frobenius-Norm

Verbindung zur Frobenius-Norm:

Für eine Matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$:

$$\|\boldsymbol{A}\|_F = \sqrt{\sum_{i=1}^{m} \sum_{j=1}^{n} a_{ij}^2} = \sqrt{\operatorname{tr}(\boldsymbol{A}^\top \boldsymbol{A})}$$

Beweis:

$$\boldsymbol{A}^\top \boldsymbol{A} = \begin{pmatrix} \sum_{i=1}^{m} a_{i1}^2 & \sum_{i=1}^{m} a_{i1} a_{i2} & \cdots \\ \sum_{i=1}^{m} a_{i2} a_{i1} & \sum_{i=1}^{m} a_{i2}^2 & \cdots \\ \vdots & \vdots & \ddots \end{pmatrix}$$

$$\implies \operatorname{tr}(\boldsymbol{A}^\top \boldsymbol{A}) = \sum_{j=1}^{n} \sum_{i=1}^{m} a_{ij}^2$$

Spur = Summe der Diagonalelemente; die Nicht-Diagonalelemente fallen weg.

Matrixnormen - Definition und Beispiele

Motivation

Warum brauchen wir `Matrixnormen`?

- Größe einer Matrix messen
- *Fehler* in Berechnungen quantifizieren
- *Konvergenz* von Algorithmen analysieren
- *Konditionierung* von Problemen verstehen

Problem mit der Frobenius-Norm:

- Ignoriert die Eigenschaften der **lineare Transformation** $\boldsymbol{x} \mapsto \boldsymbol{A}\boldsymbol{x}$, die $\boldsymbol{A}$ repräsentiert
- Behandelt Matrix wie langen Vektor:
  invariant gegen Anordnung der Einträge
- misst nur die *Summe* der quadrierten Bildlängen der Einheitsvektoren

$$\|\boldsymbol{A}\|_F^2 = \sum_j \|\boldsymbol{A}\boldsymbol{e}_j\|_2^2$$

  — nicht, wie stark $\boldsymbol{A}$ einen Vektor *maximal* verzerren kann

Frage:

Wie können wir Normen definieren, die die Transformationseigenschaften berücksichtigen?

Matrixnorm

Def.: Matrixnorm

Eine Funktion $\|\cdot\| : \mathbb{R}^{m\times n} \to [0, \infty)$ ist eine **Matrixnorm**, wenn für alle $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{m\times n}$ und $c \in \mathbb{R}$ gilt:

1. *Definitheit*:
   $\|\boldsymbol{A}\| = 0 \iff \boldsymbol{A} = \boldsymbol{0}$
2. *Absolute Homogenität*:
   $\|c\boldsymbol{A}\| = |c| \cdot \|\boldsymbol{A}\|$
3. *Dreiecksungleichung*:
   $\|\boldsymbol{A} + \boldsymbol{B}\| \leq \|\boldsymbol{A}\| + \|\boldsymbol{B}\|$

Bemerkung:

- Selbe Axiome wie für *Vektornormen*
- Eine *Vektornorm* ist eine Matrixnorm auf $\mathbb{R}^{n\times 1}$

Aus Vektornormen abgeleitete Matrixnormen

Methode: Vektorisierung

1. Schreibe Matrix als langen Vektor

$$\operatorname{vec}(\boldsymbol{A}) = \begin{pmatrix} a_{11} \\ \vdots \\ a_{m1} \\ \vdots \\ a_{1n} \\ \vdots \\ a_{mn} \end{pmatrix} \in \mathbb{R}^{mn}$$

2. Wende Vektornorm an

Beispiele:

- **Frobenius-Norm**:

$$\|\boldsymbol{A}\|_F = \|\operatorname{vec}(\boldsymbol{A})\|_2$$

- **Summennorm**:

$$\|\boldsymbol{A}\|_S = \|\operatorname{vec}(\boldsymbol{A})\|_1 = \sum_{i,j} |a_{ij}|$$

- **Maximumsnorm**:

$$\|\boldsymbol{A}\|_M = \|\operatorname{vec}(\boldsymbol{A})\|_\infty = \max_{i,j} |a_{ij}|$$

Problem mit element-weisen Normen

Verlust der Transformationsinterpretation:

Beispiel: Betrachte $\boldsymbol{A} = \boldsymbol{I}_n$ (Identitätsmatrix)

- $\|\boldsymbol{I}_n\|_F = \sqrt{n}$ (wächst mit Dimension!)
- $\|\boldsymbol{I}_n\|_S = n$
- $\|\boldsymbol{I}_n\|_M = 1$

Problem: Die Transformation $\boldsymbol{x} \mapsto \boldsymbol{I}_n\boldsymbol{x} = \boldsymbol{x}$ ändert nichts, aber Frobenius- und Summennorm wachsen mit n!

Lösung: *Operatornormen*, die die Transformation direkt messen

Beispiel: Problematische Frobenius-Norm

$$\boldsymbol{A}_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \boldsymbol{A}_2 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \boldsymbol{A}_3 = \begin{pmatrix} \sqrt{2} & 0 \\ 0 & 0 \end{pmatrix}$$

Frobenius-Norm:

$$\|\boldsymbol{A}_1\|_F = \sqrt{1^2 + 1^2} = \|\boldsymbol{A}_2\|_F = \sqrt{1^2 + 1^2} = \|\boldsymbol{A}_3\|_F = \sqrt{(\sqrt{2})^2} = \sqrt{2}$$

$\Longrightarrow$ Selbe Frobenius-Norm, **Transformationen völlig unterschiedlich:**

Mit $\boldsymbol{x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$:

- $\boldsymbol{A}_1\boldsymbol{x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$ (Identität, keine Änderung)
- $\boldsymbol{A}_2\boldsymbol{x} = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$ (Koordinatenvertauschung: Spiegelung an $x_1 = x_2$)
- $\boldsymbol{A}_3\boldsymbol{x} = \begin{pmatrix} \sqrt{2} \\ 0 \end{pmatrix}$ (Streckung der Projektion auf erste Komponente um $\sqrt{2}$)

Operatornormen

Operatornormen

Def.: Operatornorm

Sei $\|\cdot\|_V$ eine *Vektornorm* (definiert auf $\mathbb{R}^n$ und $\mathbb{R}^m$). Die **Operatornorm** von $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ ist:

$$\|\boldsymbol{A}\|_V := \max_{\boldsymbol{x} \neq \boldsymbol{0}} \frac{\|\boldsymbol{A}\boldsymbol{x}\|_V}{\|\boldsymbol{x}\|_V} = \max_{\|\boldsymbol{x}\|_V = 1} \|\boldsymbol{A}\boldsymbol{x}\|_V$$

Interpretation:

- Misst maximale **Streckung** durch $\boldsymbol{A}$ (im Sinne der Vektornorm)
  - große Norm $\rightarrow$ starke “Streckung”
  - kleine Norm $\rightarrow$ schwache “Streckung” / starke “Stauchung”
- *Streckungsfaktor* von $\boldsymbol{x}$ unter $\boldsymbol{A}$: $\frac{\|\boldsymbol{A}\boldsymbol{x}\|_V}{\|\boldsymbol{x}\|_V}$

Visualisierung: Operatornorm

$$\boldsymbol{A} = \begin{pmatrix} 2 & 1 \\ 0 & 1 \end{pmatrix}$$

```r
(A <- matrix(c(2, 1, 0, 1), nrow = 2, byrow = TRUE))
```

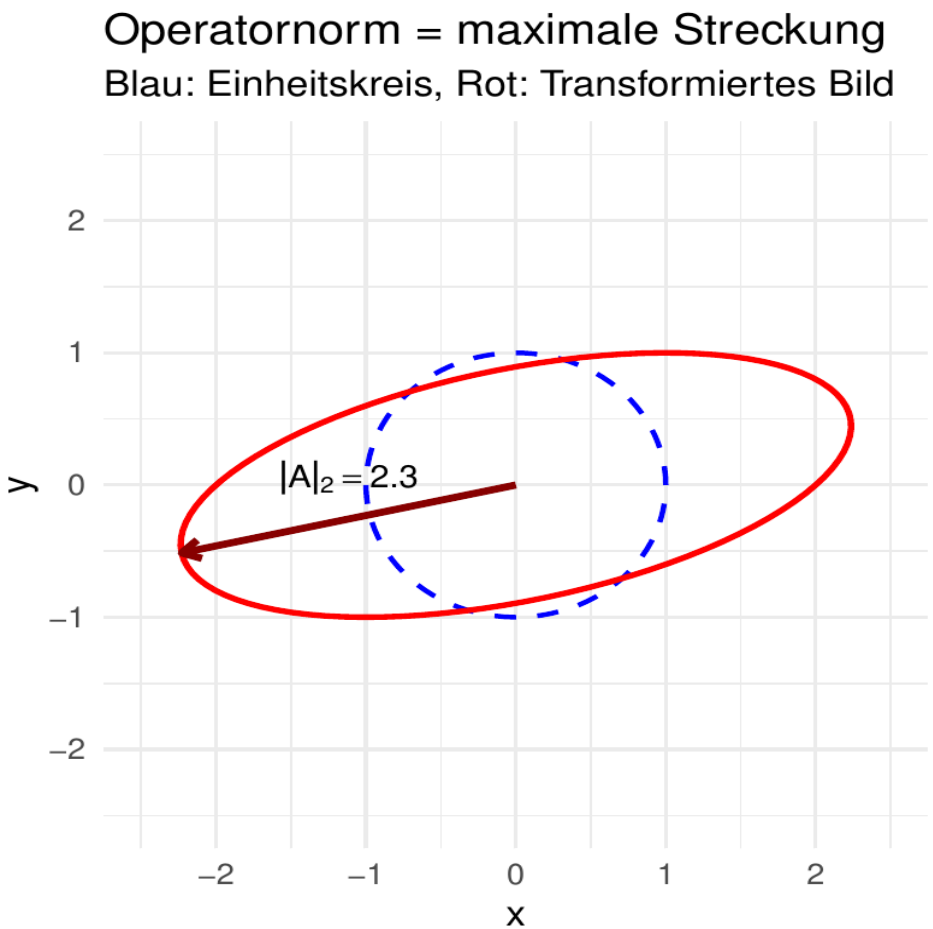
```
     [,1] [,2]
[1,]    2    1
[2,]    0    1
```

```r
(AtA <- crossprod(A))
```

```
     [,1] [,2]
[1,]    4    2
[2,]    2    2
```

```r
eigen(AtA)$values |> sqrt()
```

```
[1] 2.288 0.874
```



Wichtige Operatornormen

Basierend auf verschiedenen p-Normen

$$\|\boldsymbol{v}\|_p = \left(\sum_i |v_i|^p \right)^{1/p} \quad \text{für } 1 \leq p < \infty$$

bzw. $\|\boldsymbol{v}\|_\infty = \max_i |v_i|$:

Spektralnorm ($p = 2$):

$$\|\boldsymbol{A}\|_2 = \max_{\|\boldsymbol{x}\|_2 = 1} \|\boldsymbol{A}\boldsymbol{x}\|_2 = \sqrt{\lambda_{\max}(\boldsymbol{A}^\top \boldsymbol{A})}$$

Spaltensummennorm ($p = 1$):

$$\|\boldsymbol{A}\|_1 = \max_{\|\boldsymbol{x}\|_1 = 1} \|\boldsymbol{A}\boldsymbol{x}\|_1 = \max_{1 \leq j \leq n} \sum_{i=1}^{m} |a_{ij}|$$

Zeilensummennorm ($p = \infty$):

$$\|\boldsymbol{A}\|_\infty = \max_{\|\boldsymbol{x}\|_\infty = 1} \|\boldsymbol{A}\boldsymbol{x}\|_\infty = \max_{1 \leq i \leq m} \sum_{j=1}^{n} |a_{ij}|$$

Einschub: Orthogonalmatrizen

Def.: Orthogonalmatrix

Eine Matrix $\boldsymbol{Q} \in \mathbb{R}^{n \times n}$ heißt **Orthogonalmatrix**, wenn ihre Spalten *orthonormale Vektoren* sind, also $\boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$.

Nützliche Eigenschaften

- Geometrisch:
  $\boldsymbol{Q}$ beschreibt eine *Rotation und/oder Spiegelung* in $\mathbb{R}^n$.
- für Orthogonalmatrizen $\boldsymbol{Q}_1, \dots, \boldsymbol{Q}_n$ ist auch $\boldsymbol{Q} = \boldsymbol{Q}_1 \cdots \boldsymbol{Q}_n$ *orthogonal*.
- Normerhaltend:
  $\|\boldsymbol{Q}\boldsymbol{x}\|_2 = \sqrt{\boldsymbol{x}^\top \boldsymbol{Q}^\top \boldsymbol{Q} \boldsymbol{x}} = \sqrt{\boldsymbol{x}^\top \boldsymbol{x}} = \|\boldsymbol{x}\|_2$ für alle $\boldsymbol{x} \in \mathbb{R}^n$.
- Unitär:
  Es gilt $|\lambda_1(\boldsymbol{Q})| = \cdots = |\lambda_n(\boldsymbol{Q})| = 1$.
- $\boldsymbol{Q}^{-1} = \boldsymbol{Q}^\top$ ist wieder orthogonal, also $\|\boldsymbol{Q}\|_2 = \|\boldsymbol{Q}^{-1}\|_2 = 1$
  $\implies \kappa_2(\boldsymbol{Q}) = \|\boldsymbol{Q}\|_2 \cdot \|\boldsymbol{Q}^{-1}\|_2 = 1$
  $\implies$ Multiplikation mit $\boldsymbol{Q}$ oder $\boldsymbol{Q}^{-1}$ ist immer gut konditioniert.
  $\implies$ Multiplikation mit $\boldsymbol{Q}$ oder $\boldsymbol{Q}^{-1}$ verschlechtert nicht die Stabilität von Algorithmen.

Spektralnorm und Spektralzerlegung

Theorem

Für $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ mit Rang r und Spektralzerlegung $\boldsymbol{A}^\top \boldsymbol{A} = \boldsymbol{P} \boldsymbol{\Lambda} \boldsymbol{P}^\top$, wobei $\boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_r, 0, \ldots, 0)$ mit $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_r > 0$:

$$\|\boldsymbol{A}\|_2 = \sqrt{\lambda_1} = \sqrt{\lambda_{\max}(\boldsymbol{A}^\top \boldsymbol{A})}$$

Beweis:
siehe Anhang

Interpretation:

- Wurzel des größten Eigenwerts von $\boldsymbol{A}^\top \boldsymbol{A}$ = maximaler Streckungsfaktor
- intuitiver über *Singulärwerte* / **SVD** (später...)

Beispiele für Operatornormen

Beispiel 1:

$$\boldsymbol{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$$

Spaltensummennorm:

$$\|\boldsymbol{A}\|_1 = \max\{|2| + |0|, |1| + |3|\} = 4$$

Zeilensummennorm:

$$\|\boldsymbol{A}\|_\infty = \max\{|2| + |1|, |0| + |3|\} = 3$$

Spektralnorm: $\|\boldsymbol{A}\|_2 = \sqrt{\lambda_{\max}(\boldsymbol{A}^\top \boldsymbol{A})} \approx 3.26$

Beispiel 2: Identitätsmatrix $\boldsymbol{I}_n$

- $\|\boldsymbol{I}_n\|_1 = \|\boldsymbol{I}_n\|_2 = \|\boldsymbol{I}_n\|_\infty = 1$ ✓
- Macht Sinn: $\boldsymbol{I}_n$ verändert nichts!

Schattennormen

Schatten-p-Normen

Def.: Schatten-p-Normen

Die **Schatten-p-Norm** ($1 \leq p \leq \infty$) einer Matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, deren Kreuzprodukt $\boldsymbol{A}^\top \boldsymbol{A}$ die Eigenwerte $\boldsymbol{\lambda} = (\lambda_1, \ldots, \lambda_n)$ hat, ist:

$$\|\boldsymbol{A}\|_{S,p} := \|\boldsymbol{\lambda}^{1/2}\|_p = \left(\sum_{i=1}^{n} \lambda_i^{p/2} \right)^{1/p}$$

Wichtige Spezialfälle:

- $p = \infty$: $\|\boldsymbol{A}\|_{S,\infty} = \sqrt{\lambda_{\max}} = \|\boldsymbol{A}\|_2$ (Spektralnorm)
- $p = 2$: $\|\boldsymbol{A}\|_{S,2} = \|\boldsymbol{\lambda}^{1/2}\|_2 = \|\boldsymbol{A}\|_F$ (Frobenius-Norm!)
- $p = 1$: $\|\boldsymbol{A}\|_{S,1} = \|\boldsymbol{A}\|_* = \sum_{i=1}^{r} \sqrt{\lambda_i}$ (Nuklearnorm)

Überraschung: Frobenius-Norm ist eine Schatten-Norm!

Schatten-Normen

- Benannt nach dem Mathematiker **Robert Schatten** (1911-1977)
- Schatten-Normen sind *unitär invariant*, d.h. invariant unter orthogonalen Transformationen:
  Für orthogonale Matrizen $\boldsymbol{P}, \boldsymbol{Q}$ gilt $\|\boldsymbol{PAQ}\|_{S,p} = \|\boldsymbol{A}\|_{S,p}$
  Beweis: $(\boldsymbol{PAQ})^\top(\boldsymbol{PAQ}) = \boldsymbol{Q}^\top\boldsymbol{A}^\top\boldsymbol{AQ}$ und $\boldsymbol{A}^\top\boldsymbol{A}$ haben dieselben Eigenwerte.
- **Warum ist $\|\boldsymbol{A}\|_{S,2} = \|\boldsymbol{A}\|_F$?**
  Wie bereits gezeigt gilt $\|\boldsymbol{A}\|_F^2 = \operatorname{tr}(\boldsymbol{A}^\top\boldsymbol{A})$; mit der Spektralzerlegung $\boldsymbol{A}^\top\boldsymbol{A} = \boldsymbol{P}\boldsymbol{\Lambda}\boldsymbol{P}^\top$ folgt

$$\|\boldsymbol{A}\|_F^2 = \operatorname{tr}(\boldsymbol{P}\boldsymbol{\Lambda}\boldsymbol{P}^\top) = \operatorname{tr}(\boldsymbol{\Lambda}) = \sum_{i=1}^{r}\lambda_i = \|\boldsymbol{A}\|_{S,2}^2$$

  ($\boldsymbol{P}, \boldsymbol{P}^\top$ heben sich weg: zyklische Vertauschung und $\boldsymbol{P}^\top\boldsymbol{P} = \boldsymbol{I}$)
- **Anwendung:** Schatten-Normen sind ideal für *Approximationsprobleme*, da sie invariant unter Koordinatentransformationen sind.

Eigenschaften von Matrixnormen

Normenäquivalenz

Theorem: Alle Matrixnormen sind äquivalent

Für je zwei Matrixnormen $\|\cdot\|_a$ und $\|\cdot\|_b$ auf $\mathbb{R}^{m\times n}$ (bei festen m, n) existieren Konstanten $c, C > 0$ mit:

$$c\|\boldsymbol{A}\|_a \leq \|\boldsymbol{A}\|_b \leq C\|\boldsymbol{A}\|_a$$

Explizite Konstanten (Beispiele):

- $\|\boldsymbol{A}\|_2 \leq \|\boldsymbol{A}\|_F \leq \sqrt{\min(m,n)}\|\boldsymbol{A}\|_2$
- $\|\boldsymbol{A}\|_F \leq \|\boldsymbol{A}\|_* \leq \sqrt{\min(m,n)}\|\boldsymbol{A}\|_F$
- $\frac{1}{\sqrt{n}}\|\boldsymbol{A}\|_\infty \leq \|\boldsymbol{A}\|_2 \leq \sqrt{m}\|\boldsymbol{A}\|_\infty$

Wichtig:

- **Äquivalenz $\neq$ Gleichheit**
- Verschiedene Normen können Matrizen **unterschiedlich ordnen**
- Jede Norm hat ihre eigene **Interpretation**

Submultiplikativität

Definition: Submultiplikative Norm

Eine Matrixnorm $\|\cdot\|$ heißt **submultiplikativ**, wenn für alle $\boldsymbol{A}, \boldsymbol{B}$ mit passenden Dimensionen gilt:

$$\|\boldsymbol{AB}\| \leq \|\boldsymbol{A}\| \cdot \|\boldsymbol{B}\|$$

- Alle *Operatornormen* und *Schatten-Normen* sind submultiplikativ
- Die *Maximumsnorm* nicht (Gegenbeispiel: siehe Anhang)

Warum ist das wichtig?

- Kontrolliert **Fehlerfortpflanzung** in Matrixprodukten
- Ermöglicht **Konvergenzanalyse** iterativer Verfahren
- Verbindet Normen mit **Spektralradius**

Verträglichkeit von Normen

Def.: Verträgliche Norm

Eine Matrixnorm $\|\cdot\|$ ist **verträglich** mit einer Vektornorm $\|\cdot\|_V$, wenn:

$$\|\boldsymbol{A}\boldsymbol{x}\|_V \leq \|\boldsymbol{A}\| \cdot \|\boldsymbol{x}\|_V$$

Verträglichkeiten:

1. **Jede submultiplikative Norm** ist mit sich selbst verträglich
2. **Jede Operatornorm** ist mit ihrer induzierenden Vektornorm verträglich
3. **Frobenius-Norm** ist mit der euklidischen Norm verträglich

Anwendung:

Verträglichkeit ermöglicht **Fehlerabschätzungen**: Wenn $\|\boldsymbol{e}\|_V$ klein ist, dann ist auch $\|\boldsymbol{A}\boldsymbol{e}\|_V$ kontrolliert.

Matrixnormen und Konditionierung

Konditionszahl einer (invertierbaren) Matrix:

$$\kappa(\boldsymbol{A}) = \|\boldsymbol{A}\| \cdot \|\boldsymbol{A}^{-1}\|$$

Für Spektralnorm:

$$\kappa_2(\boldsymbol{A}) = \|\boldsymbol{A}\|_2 \cdot \|\boldsymbol{A}^{-1}\|_2 = \frac{\lambda_{\max}(\boldsymbol{A}^\top \boldsymbol{A})^{1/2}}{\lambda_{\min}(\boldsymbol{A}^\top \boldsymbol{A})^{1/2}}$$

Interpretation:

- $\kappa \approx 1$: Gut konditioniert
- $\kappa \gg 1$: Schlecht konditioniert
- $\kappa = \infty$: Singulär

Verbindung zu *Spektralzerlegung* / **SVD**: Konditionierung wird durch das Verhältnis von größtem zu kleinstem *Singulärwert* bestimmt!

Normen in der Fehleranalyse

Rückwärts-Fehleranalyse:

Problem:
Löse $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$, erhalte $\tilde{\boldsymbol{x}}$

Frage:
Für welches gestörte Problem ist $\tilde{\boldsymbol{x}}$ die exakte Lösung?

Antwort:
$(\boldsymbol{A} + \boldsymbol{\Delta A})\tilde{\boldsymbol{x}} = \boldsymbol{b}$ mit $\boldsymbol{\Delta A} = -\boldsymbol{r}\tilde{\boldsymbol{x}}^\top / \|\tilde{\boldsymbol{x}}\|^2$ für $\boldsymbol{r} = \boldsymbol{A}\tilde{\boldsymbol{x}} - \boldsymbol{b}$

Fehlerabschätzung:

$$\frac{\|\boldsymbol{\Delta A}\|_2}{\|\boldsymbol{A}\|_2} = \frac{\|\boldsymbol{r}\|_2}{\|\boldsymbol{A}\|_2 \cdot \|\tilde{\boldsymbol{x}}\|_2}$$

Interpretation:
Normen quantifizieren die **relative Störung** des Problems!

Zusammenfassung

Zusammenfassung

- *Spur*:

$$\operatorname{tr}(\boldsymbol{A}) = \sum_{i=1}^{n} a_{ii} = \sum_{i=1}^{n} \lambda_i$$

  linear, *zyklisch* ($\operatorname{tr}(\boldsymbol{AB}) = \operatorname{tr}(\boldsymbol{BA})$), ähnlichkeitsinvariant
- **Drei Familien von Matrixnormen:**
  1. *Element-weise Normen* ($\|\boldsymbol{A}\|_F$, $\|\boldsymbol{A}\|_S$, $\|\boldsymbol{A}\|_M$): einfach zu berechnen, verlieren aber die Transformationsinterpretation
  2. *Operatornormen* ($\|\boldsymbol{A}\|_1$, $\|\boldsymbol{A}\|_2$, $\|\boldsymbol{A}\|_\infty$): messen maximale Streckung, *submultiplikativ*, *verträglich*
  3. *Schatten-Normen* ($\|\boldsymbol{A}\|_{S,p}$): basieren auf Eigen-/Singulärwerten, *unitär invariant*
- **Verbindungen:**
  - *Frobenius-Norm* ist element-weise Norm **und** Schatten-Norm: $\|\boldsymbol{A}\|_F = \sqrt{\operatorname{tr}(\boldsymbol{A}^\top \boldsymbol{A})} = \sqrt{\sum_i \lambda_i} = \|\boldsymbol{A}\|_{S,2}$
  - *Spektralnorm* $\|\boldsymbol{A}\|_2 = \sqrt{\lambda_{\max}(\boldsymbol{A}^\top \boldsymbol{A})}$ verbindet Operatornormen mit Spektralzerlegung / **SVD**
  - *Konditionszahl*: $\kappa(\boldsymbol{A}) = \|\boldsymbol{A}\| \cdot \|\boldsymbol{A}^{-1}\|$

Self-Check

Quiz - Trace Eigenschaften

Welche Aussagen sind korrekt?

1. $\mathrm{tr}(\boldsymbol{A}\boldsymbol{B}) = \mathrm{tr}(\boldsymbol{A}) \cdot \mathrm{tr}(\boldsymbol{B})$
2. $\mathrm{tr}(\boldsymbol{A}\boldsymbol{B}) = \mathrm{tr}(\boldsymbol{B}\boldsymbol{A})$
3. $\mathrm{tr}(\boldsymbol{A}^\top) = -\mathrm{tr}(\boldsymbol{A})$
4. $\mathrm{tr}(\boldsymbol{I}_n) = n$
5. Wenn $\boldsymbol{A}$ invertierbar ist, dann $\mathrm{tr}(\boldsymbol{A}^{-1}) = 1/\mathrm{tr}(\boldsymbol{A})$

Quiz - Normen berechnen

Gegeben: $\boldsymbol{A} = \begin{pmatrix} 1 & 2 \\ 3 & 0 \end{pmatrix}$

Berechne:

1. $\|\boldsymbol{A}\|_1$ (Spaltensummennorm)
2. $\|\boldsymbol{A}\|_\infty$ (Zeilensummennorm)
3. $\|\boldsymbol{A}\|_F$ (Frobenius-Norm)

Anhang

Beweisskizze: Spur als Summe der Eigenwerte

Beweisskizze: (für diagonalisierbare $\boldsymbol{A}$)

Sei $\boldsymbol{A} = \boldsymbol{P}\boldsymbol{D}\boldsymbol{P}^{-1}$ eine Diagonalisierung, also $\boldsymbol{D} = \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$

$$\implies \operatorname{tr}(\boldsymbol{A}) = \operatorname{tr}\left(\boldsymbol{P}\boldsymbol{D}\boldsymbol{P}^{-1}\right) \overset{(4)}{=} \operatorname{tr}(\boldsymbol{D}) = \sum_{i=1}^{n} \lambda_i$$

($\boldsymbol{P}$, $\boldsymbol{P}^{-1}$ heben sich per Ähnlichkeitsinvarianz (4) weg.)

Beweis für allgemeine $\boldsymbol{A}$ über charakteristisches Polynom.

↩ zurück: „Spur und Eigenwerte”

Beweis: Spektralnorm und Spektralzerlegung

Sei $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ mit Rang r und Spektralzerlegung $\boldsymbol{A}^\top \boldsymbol{A} = \boldsymbol{P} \boldsymbol{\Lambda} \boldsymbol{P}^\top$, wobei $\boldsymbol{\Lambda} = \operatorname{diag}(\lambda_1, \ldots, \lambda_r, 0, \ldots, 0)$ mit $\lambda_1 \geq \cdots \geq \lambda_r > 0$.

Beweis (Skizze):

$$\|\boldsymbol{A}\boldsymbol{x}\|_2^2 = \boldsymbol{x}^\top \boldsymbol{A}^\top \boldsymbol{A} \boldsymbol{x} = \boldsymbol{x}^\top \boldsymbol{P} \boldsymbol{\Lambda} \boldsymbol{P}^\top \boldsymbol{x} = \|\boldsymbol{\Lambda}^{1/2} \boldsymbol{P}^\top \boldsymbol{x}\|_2^2$$

Weil $\boldsymbol{P}$ *orthogonal* ist, durchläuft $\boldsymbol{P}^\top \boldsymbol{x}$ die Einheitssphäre wenn $\boldsymbol{x}$ dies tut, also

$$\max_{\|\boldsymbol{x}\|_2 = 1} \|\boldsymbol{\Lambda}^{1/2} \boldsymbol{P}^\top \boldsymbol{x}\|_2^2 = \max_{\|\boldsymbol{y}\|_2 = 1} \|\boldsymbol{\Lambda}^{1/2} \boldsymbol{y}\|_2^2 = \lambda_1,$$

$$\text{da } \|\boldsymbol{\Lambda}^{1/2} \boldsymbol{y}\|_2^2 = \sum_{i=1}^{r} \lambda_i y_i^2 \leq \lambda_1 \sum_{i=1}^{r} y_i^2 \leq \lambda_1,$$

$$\text{mit Gleichheit für } \boldsymbol{y} = \boldsymbol{e}_1.$$

■

↩ zurück: „Spektralnorm und Spektralzerlegung"

Gegenbeispiel: Maximumsnorm ist nicht submultiplikativ

Beispiel: $\boldsymbol{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$

$$\boldsymbol{A}^2 = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$$

$$\|\boldsymbol{A}\|_M = 1, \quad \|\boldsymbol{A}^2\|_M = 2$$

$$\|\boldsymbol{A}^2\|_M = 2 > 1 = \|\boldsymbol{A}\|_M \cdot \|\boldsymbol{A}\|_M$$

Reparatur: Gesamtnorm

$$\|\boldsymbol{A}\|_G = \sqrt{mn}\|\boldsymbol{A}\|_M$$

Diese ist submultiplikativ!

↩ zurück: „Submultiplikativität"

Ausblick: Normen in Machine Learning

Ridge Regression (L2-Regularisierung):

$$\min_{\boldsymbol{x}} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 + \lambda \|\boldsymbol{x}\|_2^2$$

LASSO (L1-Regularisierung):

$$\min_{\boldsymbol{x}} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|_2^2 + \lambda \|\boldsymbol{x}\|_1$$

Nuclear Norm Regularisierung (Matrix Completion):

$$\min_{\boldsymbol{X}} \|\boldsymbol{P}_\Omega(\boldsymbol{X}) - \boldsymbol{P}_\Omega(\boldsymbol{M})\|_F^2 + \lambda \|\boldsymbol{X}\|_*$$

($\boldsymbol{P}_\Omega$: behält nur die beobachteten Einträge Ω, Rest $= 0$)

$\Longrightarrow$ Verschiedene Normen führen zu verschiedenen Lösungscharakteristika!

Numerische Fehleranalyse

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- Algorithmuskonzept
- Komplexitätsanalyse
- Verständnis, dass Algorithmen (meist) *approximative* Lösungen berechnen
- **Lineare Algebra I**
  - Vektor- und Matrixnormen
  - Normeigenschaften: Definitheit, Homogenität, Dreiecksungleichung
  - Matrixoperationen und Matrix-Vektor-Produkte
- **Analysis I**
  - Grenzwerte und Konvergenz
  - Stetigkeit
  - Grundlagen Taylor-Approximation

Fehleranalyse

Fehleranalysis

- Wenn ein Computer etwas speichert/rechnet, macht er Fehler.
- Der *absolute Fehler* ist dabei

$$\text{absoluter Fehler} = \text{fehlerhafter Wert} - \text{tatsächlicher Wert}.$$

- Es ist üblicher, Fehler *relativ* zu betrachten:

$$\text{relativer Fehler} = \frac{\text{fehlerhafter Wert} - \text{tatsächlicher Wert}}{\text{tatsächlicher Wert}}$$

  - Vorteil: Keine Einheiten, Fehler in %.
  - Nachteil: tatsächlicher Wert kann nicht 0 sein.
- Es gilt dann

$$\text{fehlerhafter Wert} = \text{tatsächlicher Wert} \times (1 + \text{relativer Fehler})$$

Fehlermaß

Def.: Fehlermaß

Sei V ein Vektorraum mit (Semi-)Norm $\|\cdot\|$ und $\widetilde{\boldsymbol{v}} \in V$ eine Approximation zu $\boldsymbol{v} \in V$. Dann ist

- der **absolute Fehler**

$$\boldsymbol{\Delta}_{\boldsymbol{v}} = \widetilde{\boldsymbol{v}} - \boldsymbol{v},$$

- der **relative Fehler** (nur definiert falls $\|\boldsymbol{v}\| > 0$)

$$\delta_{\boldsymbol{v}} = \frac{\|\widetilde{\boldsymbol{v}} - \boldsymbol{v}\|}{\|\boldsymbol{v}\|} = \frac{\|\boldsymbol{\Delta}_{\boldsymbol{v}}\|}{\|\boldsymbol{v}\|}.$$

- Ist $\|\boldsymbol{\Delta}_{\boldsymbol{v}}\| \leq \varepsilon$ bzw. $\delta_{\boldsymbol{v}} \leq \varepsilon$, dann nennen wir ε eine absolute bzw. relative *Fehlerschranke*.

Fehlermaß

Lemma: Fehlerschranken

Es gilt

$$\|\boldsymbol{z}\|(1-\delta_z) \leq \|\widetilde{\boldsymbol{z}}\| \leq \|\boldsymbol{z}\|(1+\delta_z).$$

Beweis:

$$\begin{aligned}
\delta_z &= \frac{\|\widetilde{\boldsymbol{z}} - \boldsymbol{z}\|}{\|\boldsymbol{z}\|} \\
\|\widetilde{\boldsymbol{z}} - \boldsymbol{z}\| &= \delta_z \|\boldsymbol{z}\| \\
|\|\widetilde{\boldsymbol{z}}\| - \|\boldsymbol{z}\|| &\leq \delta_z \|\boldsymbol{z}\| && \text{[umgekehrte Dreiecksungl.]} \\
-\delta_z \|\boldsymbol{z}\| \leq \|\widetilde{\boldsymbol{z}}\| - \|\boldsymbol{z}\| &\leq \delta_z \|\boldsymbol{z}\| && \text{[Definition von Betrag]} \\
\|\boldsymbol{z}\| - \delta_z \|\boldsymbol{z}\| \leq \|\widetilde{\boldsymbol{z}}\| &\leq \|\boldsymbol{z}\| + \delta_z \|\boldsymbol{z}\|. && \blacksquare
\end{aligned}$$

Beispiel: Fehlermaße für Vektoren

Sei $\boldsymbol{v} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$ und $\widetilde{\boldsymbol{v}} = \begin{pmatrix} 3.2 \\ 4.3 \end{pmatrix}$ mit der Euklidischen Norm $\|\cdot\|_2$.

- **Absoluter Fehler:**

$$\boldsymbol{\Delta}_v = \widetilde{\boldsymbol{v}} - \boldsymbol{v} = \begin{pmatrix} 3.2 \\ 4.3 \end{pmatrix} - \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 0.2 \\ 0.3 \end{pmatrix}$$

- **Norm des absoluten Fehlers:**

$$\|\boldsymbol{\Delta}_v\|_2 = \sqrt{0.2^2 + 0.3^2} = \sqrt{0.04 + 0.09} = \sqrt{0.13} \approx 0.361$$

- **Relativer Fehler:**

$$\delta_v = \frac{\|\boldsymbol{\Delta}_v\|_2}{\|\boldsymbol{v}\|_2} = \frac{\sqrt{0.13}}{\sqrt{3^2 + 4^2}} = \frac{\sqrt{0.13}}{5} \approx \frac{0.361}{5} \approx 0.072 = 7.2\%$$

- **Fehlerschranke:** Mit $\varepsilon = 0.1$ gilt $\delta_v \approx 0.072 < 0.1$, also ist $\varepsilon = 0.1$ eine relative Fehlerschranke.

Fehlerzerlegung

- Sei
  - f ein Problem mit idealem Input $\boldsymbol{x}$
  - $\widetilde{f}$ ein zugehöriger Algorithmus
  - und $\widetilde{\boldsymbol{x}}$ der tatsächliche Input.
- Ein Algorithmus folgt dem Schema:
  - Input $\widetilde{\boldsymbol{x}} \longrightarrow$ Algorithmus $\widetilde{f} \longrightarrow$ Ergebnis $\widetilde{f}(\widetilde{\boldsymbol{x}})$
- Ein Fehler im Ergebnis $\widetilde{f}(\widetilde{\boldsymbol{x}}) \neq f(\boldsymbol{x})$ kann entstehen durch
  - Fehler im Input $\widetilde{\boldsymbol{x}}$ (durch Messen/Speichern von $\boldsymbol{x}$)
    - unvermeidbar
  - Fehler im Algorithmus $\widetilde{f}$ (durch Rundung/Approximation)
    - können wir beeinflussen

Fehlerzerlegung

$$
\begin{aligned}
\text{Gesamtfehler} &= \widetilde{f}(\widetilde{\boldsymbol{x}}) - f(\boldsymbol{x}) \\
&= \underbrace{\widetilde{f}(\widetilde{\boldsymbol{x}}) - f(\widetilde{\boldsymbol{x}})}_{\substack{\text{Fehler im Algorithmus} \\ \text{(Maschinenarithmetik)}}} + \underbrace{f(\widetilde{\boldsymbol{x}}) - f(\boldsymbol{x})}_{\substack{\text{Folgefehler aus Input} \\ \text{(exakte Arithmetik)}}}
\end{aligned}
$$

- Erster Term misst nur den Fehler des Algorithmus (bei gegebenem/fixen Input), nicht den Inputfehler.

  Ist er "klein", nennen wir den Algorithmus $\widetilde{f}$ **stabil**.
- Zweiter Term hängt nicht vom Algorithmus $\widetilde{f}$ ab.
  Ist er "klein", nennen wir das Problem f an der Stelle $\boldsymbol{x}$ **gut konditioniert**.

Beispiel: Fehlerzerlegung

- Es gilt $e^x := \sum_{n=0}^{\infty} \frac{x^n}{n!}$
- Angenommen, wir wollen $f(\pi) = e^\pi$ berechnen und verwenden dazu die Approximationen $\widetilde{\pi} = 3$ und $\widetilde{f}(x) = \sum_{n=0}^{2} x^n/n!$

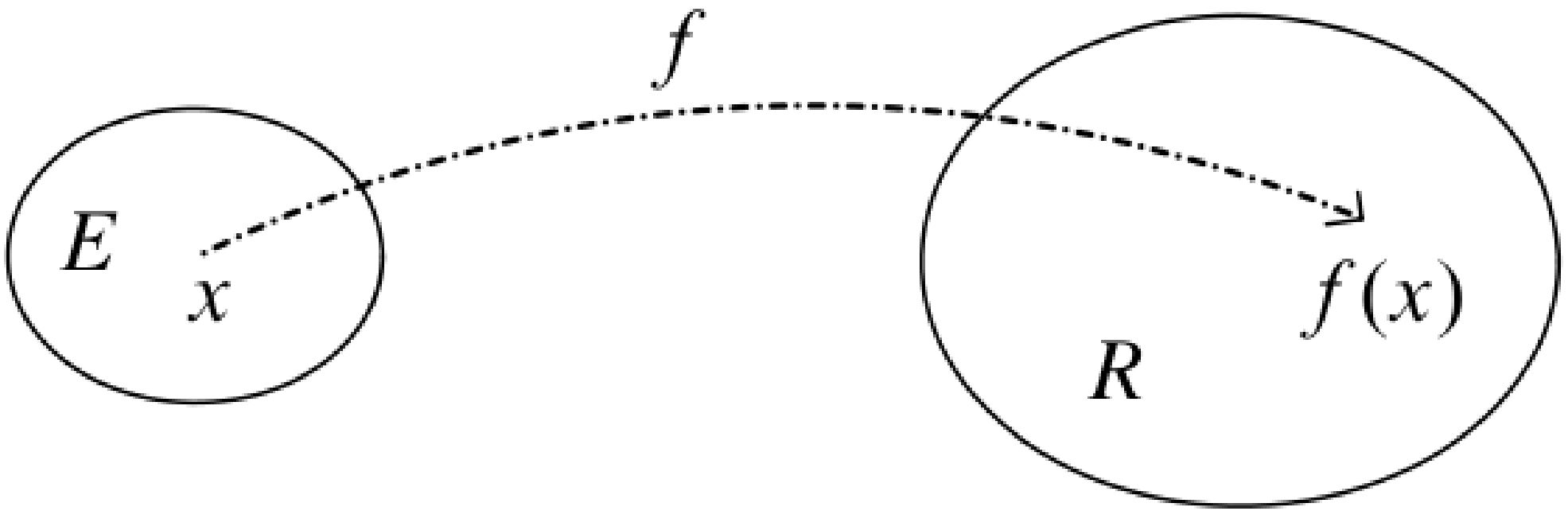
Zerlege den Gesamtfehler in die zwei erwähnten Teile.

Lösung:

$$
\begin{aligned}
\widetilde{f}(3) - f(\pi) &= \big[\widetilde{f}(3) - f(3)\big] + \big[f(3) - f(\pi)\big] \\
&= \text{Fehler im Algorithmus} + \text{Folgefehler aus Input} \\
&= \left[-\sum_{n=3}^{\infty} 3^n/n!\right] + \left[e^3 - e^\pi\right] \\
&\approx \left[\left(1 + \tfrac{3}{1} + \tfrac{9}{2}\right) - e^3\right] + [20.1 - 23.1] \\
&\approx [-11.6] + [-3]
\end{aligned}
$$

Kondition

Kondition



- Inputfehler betrachten wir als *unvermeidbar.*
- Manche Probleme sind schwer zu lösen, weil sie *zu sensitiv bzgl. des Inputs* sind:
  $\rightarrow$ Outputmenge $R = f(E)$ ist *groß* gegenüber der Menge fehlerhafter Inputs
  $E = \left\{ \widetilde{\boldsymbol{x}} \colon \frac{\|\widetilde{\boldsymbol{x}} - \boldsymbol{x}\|}{\|\boldsymbol{x}\|} \leq \delta \right\}$.
- Wir nennen das Problem dann **schlecht konditioniert**.

Kondition: Beispiel

Beispiel:

- Sei $f(x) = x^{-1}$
- Sei $x = 10^{-17}, \widetilde{x} = x + \varepsilon$ mit $\varepsilon \in (-10^{-17}, 10^{-17})$.
- $\implies f(\widetilde{x}) \in (1/2 \cdot 10^{17}, \infty)$ und

$$\left| \frac{f(\widetilde{x}) - f(x)}{f(x)} \right| \in [0, \infty).$$

Achtung:
Die Störung ist hier *relativ zu* x nicht klein (bis zu 100%).
Bei relativ *kleinen* Störungen ist $x \mapsto 1/x$ dagegen harmlos: der relative Outputfehler ist dann ungefähr so groß wie der relative Inputfehler.
Es kommt also darauf an, *welche* Art von Inputfehler unvermeidbar ist.

Kondition

- Im Folgenden betrachten wir Abbildungen $f\colon \mathbb{R}^n \to \mathbb{R}^m$.
- $\|\cdot\|$ bezeichnet dabei eine beliebige Vektornorm (bzw. die dadurch induzierte *Operatornorm* für Matrizen).

Def.: Konditionszahl

Sei f ein Problem mit Input $\boldsymbol{x}$.

- Die **absolute Konditionszahl** ist die kleinste Zahl $\kappa_{abs} = \kappa_{abs}(f, \boldsymbol{x})$ mit

$$\|f(\widetilde{\boldsymbol{x}}) - f(\boldsymbol{x})\| \leq \kappa_{abs} \|\widetilde{\boldsymbol{x}} - \boldsymbol{x}\|, \quad \text{für } \widetilde{\boldsymbol{x}} \to \boldsymbol{x}.$$

- Die **relative Konditionszahl** (für $\|\boldsymbol{x}\| > 0, \|f(\boldsymbol{x})\| > 0$) ist die kleinste Zahl $\kappa_{rel} = \kappa_{rel}(f, \boldsymbol{x})$ mit

$$\frac{\|f(\widetilde{\boldsymbol{x}}) - f(\boldsymbol{x})\|}{\|f(\boldsymbol{x})\|} \leq \kappa_{rel} \frac{\|\widetilde{\boldsymbol{x}} - \boldsymbol{x}\|}{\|\boldsymbol{x}\|}, \quad \text{für } \widetilde{\boldsymbol{x}} \to \boldsymbol{x}.$$

Interpretation

$$\|f(\widetilde{\boldsymbol{x}}) - f(\boldsymbol{x})\| \leq \kappa_{abs} \|\widetilde{\boldsymbol{x}} - \boldsymbol{x}\|, \qquad \frac{\|f(\widetilde{\boldsymbol{x}}) - f(\boldsymbol{x})\|}{\|f(\boldsymbol{x})\|} \leq \kappa_{rel} \frac{\|\widetilde{\boldsymbol{x}} - \boldsymbol{x}\|}{\|\boldsymbol{x}\|}.$$

- *gut konditioniert* ($\kappa \lesssim 1$):
  Inputfehler führt zu ähnlich großem oder kleinerem Outputfehler.
- *schlecht konditioniert* ($\kappa \gg 1$):
  Inputfehler können zu viel größeren Outputfehlern führen.
- *schlecht gestellt* ($\kappa = \infty$):
  Der Fehlerverstärkungsfaktor ist unbeschränkt — beliebig kleine Inputfehler können (relativ zu ihrer Größe) beliebig stark verstärkt werden.

Beispiel

- Sei $f(\boldsymbol{x}) = \boldsymbol{A}^{-1}\boldsymbol{x} = \boldsymbol{y}$.
- Absolute Konditionszahl:

$$\|f(\widetilde{\boldsymbol{x}}) - f(\boldsymbol{x})\| = \|\boldsymbol{A}^{-1}(\widetilde{\boldsymbol{x}} - \boldsymbol{x})\| \leq \|\boldsymbol{A}^{-1}\|\,\|\widetilde{\boldsymbol{x}} - \boldsymbol{x}\|$$
$$\Longrightarrow \quad \kappa_{abs} = \|\boldsymbol{A}^{-1}\|$$

- Relative Konditionszahl: Sei $\boldsymbol{h} = \widetilde{\boldsymbol{x}} - \boldsymbol{x}$, dann

$$\frac{\|\boldsymbol{A}^{-1}\boldsymbol{h}\| \,/\, \|\boldsymbol{A}^{-1}\boldsymbol{x}\|}{\|\boldsymbol{h}\| \,/\, \|\boldsymbol{x}\|} = \underbrace{\frac{\|\boldsymbol{A}^{-1}\boldsymbol{h}\|}{\|\boldsymbol{h}\|}}_{\leq \|\boldsymbol{A}^{-1}\|} \cdot \frac{\|\boldsymbol{x}\|}{\|\boldsymbol{A}^{-1}\boldsymbol{x}\|}$$

$$\Longrightarrow \qquad \kappa_{rel} = \|\boldsymbol{A}^{-1}\| \frac{\|\boldsymbol{x}\|}{\|\boldsymbol{A}^{-1}\boldsymbol{x}\|} = \kappa_{abs} \frac{\|\boldsymbol{A}\boldsymbol{y}\|}{\|\boldsymbol{y}\|} \leq \kappa_{abs}\|\boldsymbol{A}\| = \|\boldsymbol{A}^{-1}\|\|\boldsymbol{A}\|$$

  (Gleichheit im ersten Faktor, weil $\boldsymbol{h}$ entlang einer die Operatornorm maximierenden Richtung gewählt werden darf.)
- $\kappa(\boldsymbol{A}) = \|\boldsymbol{A}^{-1}\|\|\boldsymbol{A}\|$ wird in der linearen Algebra auch *Konditionszahl von $\boldsymbol{A}$* genannt — sie ist die von $\boldsymbol{x}$ unabhängige *obere Schranke* für κ_{rel}, nicht κ_{rel} selbst.
- Für die Operatornorm $\|\cdot\|_2$ (*Spektralnorm*) gilt

$$\kappa_2(\boldsymbol{A}) = \frac{\sqrt{\lambda_{\max}(\boldsymbol{A}^\top \boldsymbol{A})}}{\sqrt{\lambda_{\min}(\boldsymbol{A}^\top \boldsymbol{A})}}.$$

Quiz

Sei

$$\boldsymbol{A} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & 0.1 \end{pmatrix}.$$

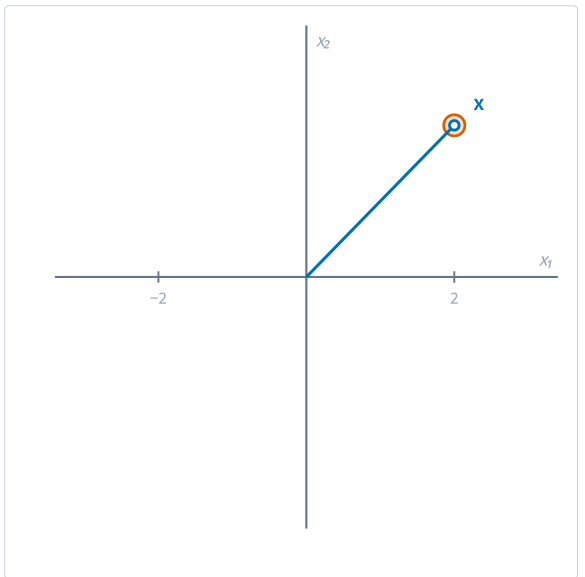
Was ist die Konditionszahl der Matrix bezüglich der 2-Norm?

1. 2
2. 20
3. 5
4. 50

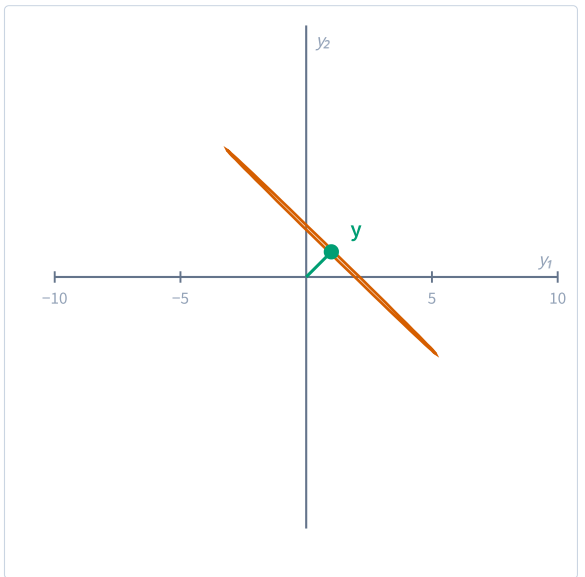
Interaktive Demo: Kondition

▸ *Ziehen wir \(\bx\) im linken Feld im Kreis herum und beobachten, wie sich die rote Ellipse rechts dabei streckt und staucht.*

Inputebene: rechte Seite \(\bx\) und die plausiblen Inputs



Lösungsebene: \(\by = \bA^{-1}\bx\) und das Bild des Kreises



Drehung (κ = 1) | mäßig gedehnt | **schlecht konditioniert** | fast singulär

x₁ 2,00

x₂ 2,05

rel. Inputfehler 5,0 %

rel. Inputfehler 5,0 %

rel. Outputfehler 410 %

`A = (1,00 1,00; 1,00 1,05)`

`y = A⁻¹x = (1,00; 1,00)`

`Schranke κ(A) = σ_max/σ_min = 82,04 · ausgeschöpft zu 100 %`

`σ_max = 2,025, σ_min = 0,0247`

`κ_rel(f, x) = 82,03`

✗ **Die ungünstige rechte Seite.** \(\kappa_{rel}(f, \bx)\) = 82,0 schöpft 100 % der Schranke \(\kappa(\bA)\) = 82,0 aus. Bemerkung 4.2.7 sagt genau das: Die Schranke ist kein Pessimismus, sie wird für ungünstige rechte Seiten angenommen. Aus 5,0 % Inputfehler werden hier 410 % Outputfehler.

Aufgabe

Sei $f(\boldsymbol{x}) = x_1 + x_2 = \binom{1}{1}^\top \boldsymbol{x} = \mathbf{1}^\top \boldsymbol{x}$.

1. Was sind die Konditionszahlen?
2. Für welche $\boldsymbol{x}$ ist das Problem schlecht konditioniert?

Lösung:

- Bezüglich der euklidischen Norm gilt (Cauchy-Schwarz)

$$|f(\widetilde{\boldsymbol{x}}) - f(\boldsymbol{x})| = |\mathbf{1}^\top(\widetilde{\boldsymbol{x}} - \boldsymbol{x})| \leq \|\mathbf{1}\|_2 \|\widetilde{\boldsymbol{x}} - \boldsymbol{x}\|_2 = \sqrt{2}\|\widetilde{\boldsymbol{x}} - \boldsymbol{x}\|_2,$$

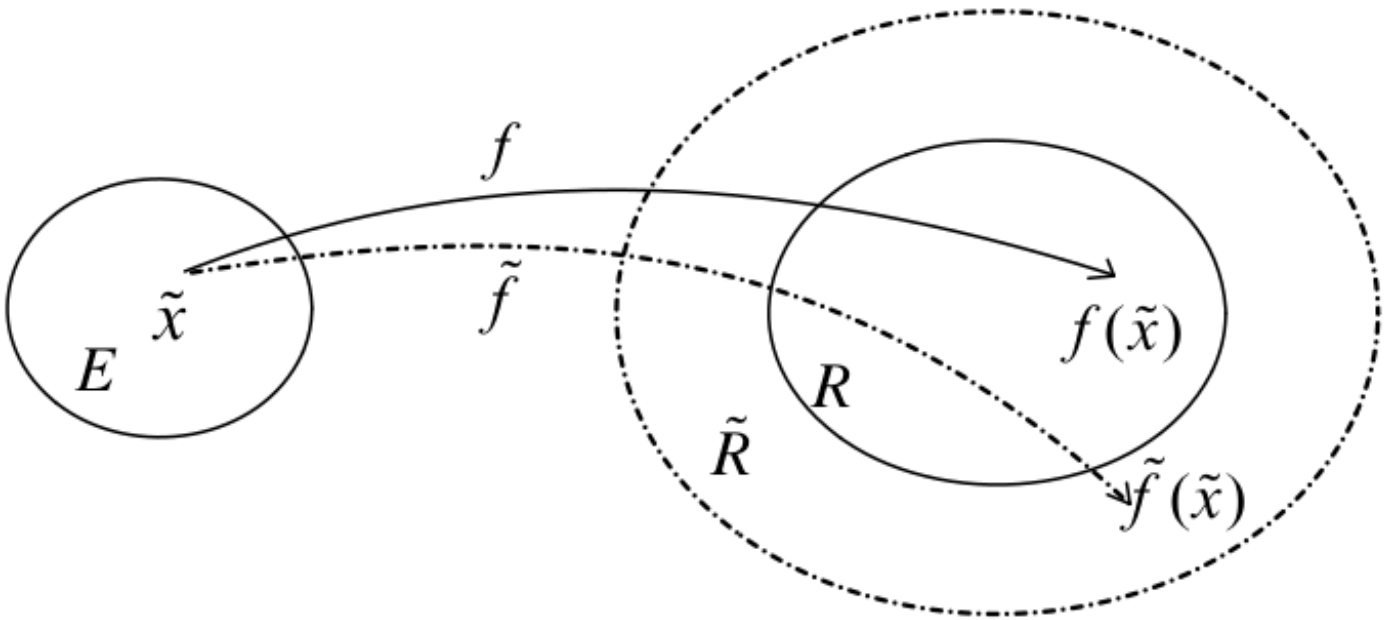
  mit Gleichheit falls $(\widetilde{\boldsymbol{x}} - \boldsymbol{x}) \parallel \mathbf{1}$, deshalb $\kappa_{abs} = \sqrt{2}$ und

$$\kappa_{rel} = \kappa_{abs} \frac{\|\boldsymbol{x}\|_2}{|x_1 + x_2|} = \sqrt{2} \frac{\|\boldsymbol{x}\|_2}{|x_1 + x_2|}.$$

- Für $0 < |x_1 + x_2| \ll \|\boldsymbol{x}\|_2$ ist $\kappa_{rel} \gg 1$ und das Problem somit schlecht konditioniert

Stabilität von Algorithmen

Stabilität



$$\begin{aligned}\text{Gesamtfehler} &= \widetilde{f}(\widetilde{\boldsymbol{x}}) - f(\boldsymbol{x}) \\ &= \underbrace{\widetilde{f}(\widetilde{\boldsymbol{x}}) - f(\widetilde{\boldsymbol{x}})}_{\text{Fehler im Algorithmus}} + \underbrace{f(\widetilde{\boldsymbol{x}}) - f(\boldsymbol{x})}_{\text{Folgefehler aus Input}}\end{aligned}$$

- Analog zur Kondition können wir auch den algorithmischen Fehler $\widetilde{f}(\widetilde{\boldsymbol{x}}) - f(\widetilde{\boldsymbol{x}})$ untersuchen.
- Ist er “klein”, nennen wir $\widetilde{f}$ **stabil**.

Beispiel: Stochastic Gradient Descent

Optimierung der Verlustfunktion $L(\theta)$ mit $\theta_{k+1} = \theta_k - \alpha\hat{\nabla}L(\theta_k)$

Stabiler Algorithmus:

- Kleine Lernrate α (z.B. 0.001)
- (Numerische) Fehler akkumulieren langsam
- $\rightarrow$ Konvergiert zuverlässig zu einem Optimum

Instabiler Algorithmus:

- Zu große Lernrate α (z.B. 1.0)
- Verstärkt Rundung-/Schätzfehler des Gradienten $\hat{\nabla}L(\theta_k)$
- $\rightarrow$ Oszilliert oder divergiert

Das ist eine *Analogie* zum Konzept der *Stabilität* — hier mischen sich numerische Fehler und die Konvergenzdynamik des Verfahrens:

- Problem f: Finde Minimum von $L(\theta)$
- Algorithmus $\widetilde{f}$: Stochastic Gradient Descent mit endlicher Präzision
- Algorithmischer Fehler: Akkumuliert Rundung-/Schätzfehler über k

Stabilität: Wichtigste Erkenntnis

Theorem: Stabilität zusammengesetzter Algorithmen

Sei f ein Problem mit Input $\boldsymbol{x}$, wobei $f(\boldsymbol{x}) = (h \circ g)(\boldsymbol{x})$ für zwei Teilprobleme g und h. Dann gilt für einen Algorithmus $\widetilde{f} = \widetilde{h} \circ \tilde{g}$:

$$\frac{\|\widetilde{f}(\boldsymbol{x}) - f(\boldsymbol{x})\|}{\|f(\boldsymbol{x})\|} \leq \frac{\|\widetilde{h}(\widetilde{\boldsymbol{y}}) - h(\widetilde{\boldsymbol{y}})\|}{\|h(\boldsymbol{y})\|} + \kappa_{rel}(h, g(\boldsymbol{x})) \frac{\|\tilde{g}(\boldsymbol{x}) - g(\boldsymbol{x})\|}{\|g(\boldsymbol{x})\|},$$

wobei $\boldsymbol{y} = g(\boldsymbol{x})$, $\widetilde{\boldsymbol{y}} = \tilde{g}(\boldsymbol{x})$.

- Ein schlecht konditioniertes Teilproblem h (an der Stelle $\boldsymbol{y} = g(\boldsymbol{x})$) verstärkt Fehler in $\tilde{g}$, wäre also ein *Indiz* für die Instabilität von $\widetilde{f}$.
- Deshalb sollten wir **schlecht konditionierte Schritte immer so früh wie möglich** im Algorithmus ausführen.

Stabilität: Wichtigste Erkenntnis

Beweis:

$$
\begin{aligned}
\frac{\|\widetilde{f}(\boldsymbol{x}) - f(\boldsymbol{x})\|}{\|f(\boldsymbol{x})\|} &= \frac{\|\widetilde{h}(\tilde{g}(\boldsymbol{x})) - h(g(\boldsymbol{x}))\|}{\|h(g(\boldsymbol{x}))\|} \\
&= \frac{\|\widetilde{h}(\tilde{g}(\boldsymbol{x})) - h(\tilde{g}(\boldsymbol{x})) + h(\tilde{g}(\boldsymbol{x})) - h(g(\boldsymbol{x}))\|}{\|h(g(\boldsymbol{x}))\|} \\
&\leq \frac{\|\widetilde{h}(\tilde{g}(\boldsymbol{x})) - h(\tilde{g}(\boldsymbol{x}))\|}{\|h(g(\boldsymbol{x}))\|} + \frac{\|h(\tilde{g}(\boldsymbol{x})) - h(g(\boldsymbol{x}))\|}{\|h(g(\boldsymbol{x}))\|} \\
&\leq \frac{\|\widetilde{h}(\widetilde{\boldsymbol{y}}) - h(\widetilde{\boldsymbol{y}})\|}{\|h(\boldsymbol{y})\|} + \kappa_{rel}(h, g(\boldsymbol{x})) \frac{\|\tilde{g}(\boldsymbol{x}) - g(\boldsymbol{x})\|}{\|g(\boldsymbol{x})\|}.
\end{aligned}
$$

Fehler von $\widetilde{h}$ selbst + durch h verstärkter Fehler von $\tilde{g}$ –

Idee: Einfügen von $0 = -h(\tilde{g}(\boldsymbol{x})) + h(\tilde{g}(\boldsymbol{x}))$, dann Dreiecksungleichung.

Beispiel: Stabilität der Varianz

Problem: Berechne Stichprobenvarianz

$$f(\boldsymbol{x}) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})^2.$$

Algorithmen:

$$\widetilde{f}_1(\boldsymbol{x}) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \overline{x})^2, \quad \widetilde{f}_2(\boldsymbol{x}) = \left(\frac{1}{n}\sum_{i=1}^{n}x_i^2\right) - \overline{x}^2$$

- Bei $\widetilde{f}_2$ berechnen wir zum Schluss die Differenz $h(a,b) = a - b$ mit

$$a = \frac{1}{n}\sum_{i=1}^{n}x_i^2 \quad \text{und} \quad b = \overline{x}^2.$$

Beispiel: Stabilität der Varianz

- Die Konditionszahl ist dabei

$$\kappa_{rel}(h, (a, b)) = \sqrt{2} \frac{\sqrt{a^2 + b^2}}{|a - b|}.$$

- Falls a, b groß sind, aber $a \approx b$ gilt, ist $\kappa_{rel}(h, (a, b))$ groß.
- In diesem Fall kann h also vorangegangene Fehler massiv verstärken!

```
1  set.seed(5)
2  x <- rnorm(10^3, mean = 0, sd = 1) + 10^10
3  mean((x - mean(x))^2)
```

```
[1] 1.02
```

```
1  mean(x^2) - mean(x)^2
```

```
[1] 16384
```

Übersicht

- *Absoluter Fehler*: $\boldsymbol{\Delta}_{\boldsymbol{v}} = \widetilde{\boldsymbol{v}} - \boldsymbol{v}$
- *Relativer Fehler*: $\delta_{\boldsymbol{v}} = \frac{\|\widetilde{\boldsymbol{v}} - \boldsymbol{v}\|}{\|\boldsymbol{v}\|}$
- *Fehlerzerlegung*:
  Gesamtfehler = *algorithmische Fehler* $(\widetilde{f}(\widetilde{\boldsymbol{x}}) - f(\widetilde{\boldsymbol{x}}))$ + *Input-Folgefehler* $(f(\widetilde{\boldsymbol{x}}) - f(\boldsymbol{x}))$
- *Kondition*: Sensitivität eines Problems bzgl. *Inputfehlern*
- *Konditionszahlen*: Absolut und relativ $(\kappa_{abs}, \kappa_{rel})$
- *Konditionszahl einer Matrix*: $\kappa(\boldsymbol{A}) = \|\boldsymbol{A}\|\|\boldsymbol{A}^{-1}\|$
- *Stabilität*:
  Ein Algorithmus ist stabil, wenn der *algorithmische Fehler* klein ist.
- *Stabilität zusammengesetzter Algorithmen*:
  Fehlerfortpflanzung über Algorithmus-Teilschritte

Wrap-up

Heute gelernt

- Die *Kondition* eines Problems beschreibt dessen **Sensitivität** bzgl. Inputfehlern.
- Die *Stabilität* eines Algorithmus beschreibt dessen **Robustheit** bzgl. Fehler.
- **Schlecht konditionierte Teilschritte möglichst früh ausführen!**

Nächste Vorlesung

- Numerische Lösung von linearen Gleichungssystemen (LGS)

Gleichungssysteme und Matrixzerlegungen

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- **Algorithmen**: Komplexität, Algorithmustypen (direkte Methoden)
- **Fehleranalyse**: Stabilität, Kondition von Problemen
- Matrixnormen, Orthogonalmatrizen
- **Lineare Algebra I**
  - Lösung linearer Gleichungssysteme (grundlegende Methoden)
  - Dreiecksmatrizen, Symmetrische Matrizen, Positiv definite Matrizen
  - Matrixinverse (wir lernen, dass wir sie *NICHT* berechnen...!)

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $\boldsymbol{U} \in \mathbb{R}^{n \times n}$ eine obere Dreiecksmatrix mit $u_{ii} \neq 0$. Mit welchem Aufwand lässt sich $\boldsymbol{U}\boldsymbol{x} = \boldsymbol{c}$ lösen?

1. $O(n)$
2. $O(n^2)$
3. $O(n^3)$
4. Gar nicht — dazu muss man erst $\boldsymbol{U}^{-1}$ berechnen.

Frage 2: Was bedeutet "$\boldsymbol{A} \in \mathbb{R}^{n \times n}$ ist *positiv definit*"?

1. Alle Einträge von $\boldsymbol{A}$ sind positiv.
2. $\det(\boldsymbol{A}) > 0$.
3. $\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} > 0$ für alle $\boldsymbol{x} \neq \boldsymbol{0}$.
4. Alle Eigenwerte von $\boldsymbol{A}$ sind verschieden.

Numerische lineare Algebra

- Wir kümmern uns nun um numerische Methoden für die lineare Algebra:
  - Matrizenrechnung
  - Lineare Gleichungssysteme
  - Kleinste-Quadrate-Probleme
  - Eigenwerte
  - ...
- Dabei achten wir insbesondere auf `Kondition` und `Komplexität`.

Matrizenrechnung

Matrizenrechnung

- Die algorithmische Berechnung folgt für gewöhnlich den direkten mathematischen Formeln

$$\boldsymbol{A} + \boldsymbol{B} = \begin{pmatrix} a_{11} + b_{11} & \cdots & a_{1m} + b_{1m} \\ \vdots & & \vdots \\ a_{n1} + b_{n1} & \cdots & a_{nm} + b_{nm} \end{pmatrix},$$

$$\boldsymbol{AB} = \begin{pmatrix} \sum_{i=1}^{m} a_{1i} b_{i1} & \cdots & \sum_{i=1}^{m} a_{1i} b_{ik} \\ \vdots & & \vdots \\ \sum_{i=1}^{m} a_{ni} b_{i1} & \cdots & \sum_{i=1}^{m} a_{ni} b_{ik} \end{pmatrix}.$$

Quiz

Was ist die algorithmische Komplexität für die Addition von $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{n \times m}$?

1. $O(nm)$
2. $O(n + m)$
3. $O(n^2 m^2)$

Quiz

Was ist die algorithmische Komplexität für die Multiplikation von $\boldsymbol{A} \in \mathbb{R}^{n \times m}$, $\boldsymbol{B} \in \mathbb{R}^{m \times k}$?

1. $O(nk)$
2. $O(nmk)$
3. $O(nm + k)$

Matrixmultiplikation

- Für die Multiplikation von zwei $(n \times n)$ Matrizen gibt es Algorithmen mit besserer Komplexität.
- Für $\boldsymbol{A}, \boldsymbol{B} \in \mathbb{R}^{n \times n}$:
  - Strassen (1969): $\approx O(n^{2.8})$
  - Alman et al. (2024): $\approx O(n^{2.372})$ (bester bekannter Exponent)



Quelle: Titelbild *Nature* **610** (6. Oktober 2022) zu AlphaTensor (Fawzi et al., 2022).

- Die Konstante, die hinter $O(\cdot)$ versteckt wird, ist aber meist so groß, dass die Algorithmen kaum verwendet werden.
- Wir rechnen der Einfachheit halber immer mit $O(nmk)$.

Lineare Gleichungssysteme

Lineare Gleichungssysteme

- Wir beschäftigen uns nun mit der Lösung eines LGS:

$$\boldsymbol{x} : \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$$

- Das ist nur sinnvoll, wenn es (genau) eine Lösung gibt. Also nehmen wir an, dass $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ invertierbar ist.
- Die Lösung kann als $\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b}$ geschrieben werden.
- Das numerische Invertieren einer Matrix ist aber
  - relativ instabil und
  - oft langsamer als direktes Lösen des LGS.

Deshalb:

Invertiere niemals eine Matrix!

Demo:

- LGS mit schlecht konditionierter Matrix mit $\kappa_2(\boldsymbol{A}) \approx 10^{14}$
- wahre Lösung $\boldsymbol{x} = (1, \ldots, 1)^\top$:

```
n <- 11 # Hilbert-Matrix:
A <- outer(1:n, 1:n,
           function(i, j) 1 / (i + j - 1))
x <- rep(1, n)
b <- A %*% x

range((solve(A) %*% b) - x) # Fehler über Inverse
range(solve(A, b) - x)      # Fehler direkte Lösung
```

```
[1] -0.195  0.288
[1] -0.000260  0.000332
```

$\Longrightarrow$ maximaler Fehler über $\boldsymbol{A}^{-1}$: ≈ 0.3;
bei direktem Lösen: $\approx 3 \cdot 10^{-4}$
— ca. **1000× genauer** (und schneller).

Lineare Gleichungssysteme

Wir erinnern uns an das Gauss'sche Eliminationsverfahren:

1. Bringe $(\boldsymbol{A} \mid \boldsymbol{b})$ in Zeilenstufenform $(\boldsymbol{U} \mid \boldsymbol{c})$ mit oberer Dreiecksmatrix $\boldsymbol{U}$.
2. Löse das System

$$\begin{array}{ccccccccc} u_{11}x_1 & + & u_{12}x_2 & + & \dots & + & u_{1n}x_n & = & c_1 \\ & & u_{22}x_2 & + & \dots & + & u_{2n}x_n & = & c_2 \\ & & & & \ddots & & \vdots & & \vdots \\ & & & & & & u_{nn}x_n & = & c_n \end{array}$$

durch Rückwärtssubstitution

$$x_n = c_n/u_{nn}, \quad x_i = \left(c_i - \sum_{j=i+1}^{n} u_{ij}x_j\right)/u_{ii} \quad (i \leq n-1).$$

Quiz

Was ist die algorithmische Komplexität der Rückwärtssubstitution?

1. $O(1)$
2. $O(n)$
3. $O(n^2)$
4. $O(n^3)$

Warum Matrixzerlegungen?

Grundidee: “Teile und herrsche” für Matrizen

Analogie: Primfaktorzerlegung von Zahlen

- $84 = 2^2 \cdot 3 \cdot 7$
- Vereinfacht Rechnung: $84 \cdot 126 = (2^2 \cdot 3 \cdot 7) \cdot (2 \cdot 3^2 \cdot 7) = 2^3 \cdot 3^3 \cdot 7^2$

Für Matrizen: $\boldsymbol{A} = \boldsymbol{BC}$ mit “einfachen” $\boldsymbol{B}, \boldsymbol{C}$

- **Dreiecksmatrizen:** Lösen in $O(n^2)$ statt $O(n^3)$
- **Orthogonale Matrizen:** Numerisch stabil, einfach zu invertieren
- **Diagonalmatrizen:** Triviale Operationen

Vorteile von Zerlegungen:

1. Effiziente Lösung mehrerer LGS mit gleicher Matrix
2. Numerische Stabilität verbessern
3. Struktur und Eigenschaften von $\boldsymbol{A}$ erkennen
4. Grundlage für viele weitere Algorithmen (Spektralzerlegung & `SVD`, …)

Die LU-Zerlegung

Intuition: LU-Zerlegung

Idee: Die Gauss-Elimination zerlegt $\boldsymbol{A}$ in:

- $\boldsymbol{L}$ (**L**ower): speichert alle Eliminationsschritte
- $\boldsymbol{U}$ (**U**pper): die resultierende obere Dreiecksmatrix

Vorteil: Sobald wir $\boldsymbol{A} = \boldsymbol{LU}$ einmal berechnet haben, können wir $\boldsymbol{Ax} = \boldsymbol{b}_i$ für *jedes* $\boldsymbol{b}_i$ effizient lösen:

Ax = b $\Longrightarrow$ **LUx = b** $\Longrightarrow$

1. Löse **Ly = b**
2. Löse **Ux = y**

Die LU-Zerlegung

- Im ersten Schritt des Eliminationsverfahrens bringen wir $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ in Zeilenstufenform.
- Dazu gehen wir von links nach rechts durch die Spalten und entfernen alle Einträge unter der Diagonalen.
- Ist das aktuelle System $(\boldsymbol{M} \mid \boldsymbol{r})$, erreichen wir das für die k-te Spalte durch die Multiplikation einer Matrix $\boldsymbol{L}_k$ von links, wobei

$$\boldsymbol{L}_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{(k+1)k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & 1 \end{pmatrix}$$

$$\text{mit } l_{ik} = m_{ik}/m_{kk}$$

Die LU-Zerlegung

- Insgesamt ist also $(\boldsymbol{U} \mid \boldsymbol{c})$ gegeben durch $\boldsymbol{U} = \boldsymbol{L}_{n-1} \cdots \boldsymbol{L}_1 \boldsymbol{A}$ und $\boldsymbol{c} = \boldsymbol{L}_{n-1} \cdots \boldsymbol{L}_1 \boldsymbol{b}$.
- Es gilt dann $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$, wobei $\boldsymbol{L} = \boldsymbol{L}_1^{-1} \cdots \boldsymbol{L}_{n-1}^{-1}$.

Die Inverse zu $\boldsymbol{L}_k$ macht deren Operation rückgängig (Vorzeichen kippen):

$$\boldsymbol{L}_k^{-1} = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & l_{(k+1)k} & 1 & \\ & & \vdots & & \ddots \\ & & l_{nk} & & 1 \end{pmatrix}$$

Jedes $\boldsymbol{L}_k^{-1}$ ist eine untere Dreiecksmatrix, also auch $\boldsymbol{L} = \boldsymbol{L}_1^{-1} \cdots \boldsymbol{L}_{n-1}^{-1}$ – genauer:

$$\boldsymbol{L} = \begin{pmatrix} 1 & & & & \\ l_{21} & \ddots & & & \\ l_{31} & l_{32} & 1 & & \\ \vdots & & \ddots & \ddots & \\ l_{n1} & \cdots & \cdots & l_{n(n-1)} & 1 \end{pmatrix}$$

$\Longrightarrow$ $\boldsymbol{L}$ sammelt einfach alle Eliminationskoeffizienten l_{ik} ein.

Beispiel: LU-Zerlegung für 3×3-Matrix

Sei $\boldsymbol{A} = \begin{pmatrix} 2 & 1 & -1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix}$.

Farben: zu eliminieren, Pivot, Eliminationskoeffizienten

Schritt 1: Eliminiere Spalte 1 mit $\boldsymbol{L}_1$:

$$l_{21} = 4/2 = 2, \quad l_{31} = -2/2 = -1$$

$$\boldsymbol{L}_1 = \begin{pmatrix} 1 & 0 & 0 \\ -2 & 1 & 0 \\ -(-1) & 0 & 1 \end{pmatrix},$$

$$\boldsymbol{L}_1\boldsymbol{A} = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 8 & 1 \end{pmatrix}$$

Schritt 2: Eliminiere Spalte 2 mit $\boldsymbol{L}_2$:

$$l_{32} = 8/(-8) = -1$$

$$\boldsymbol{L}_2 = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & -(-1) & 1 \end{pmatrix},$$

$$\boldsymbol{L}_2\boldsymbol{L}_1\boldsymbol{A} = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix} = \boldsymbol{U}$$

Ergebnis: $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$ mit

$$\boldsymbol{L} = \boldsymbol{L}_1^{-1}\boldsymbol{L}_2^{-1} = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix}, \quad \boldsymbol{U} = \begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix}$$

Die LU-Zerlegung zur Lösung von LGS

Theorem: LU-Zerlegung

Jede invertierbare Matrix $\boldsymbol{A}$ hat — ggf. erst nach Zeilenvertauschungen $\boldsymbol{A} \to \boldsymbol{PA}$ mit einer Permutationsmatrix $\boldsymbol{P}$ (→ *Pivotierung*, s.u.) — eine Darstellung

$$\boldsymbol{A} = \boldsymbol{LU} \qquad \text{bzw.} \qquad \boldsymbol{PA} = \boldsymbol{LU}$$

mit

- einer unteren Dreiecksmatrix $\boldsymbol{L}$,
- einer oberen Dreiecksmatrix $\boldsymbol{U}$.

- Eine Zerlegung ohne Pivotierung (ohne Zeilenvertauschungen, also mit $\boldsymbol{P} = \boldsymbol{I}$) erhalten wir genau dann, wenn bei der Gauß-Elimination kein Pivotelement null wird. Wird ein Pivot null, müssen wir vorher Zeilen vertauschen.
- Da $\boldsymbol{L}$ und $\boldsymbol{U}$ Dreiecksmatrizen sind, können wir das LGS $\boldsymbol{Ax} = \boldsymbol{b}$ bzw. $\boldsymbol{LUx} = \boldsymbol{b}$ jetzt leicht lösen:
  1. Löse $\boldsymbol{Ly} = \boldsymbol{b}$ (bzw. $\boldsymbol{Ly} = \boldsymbol{Pb}$) durch Vorwärtssubstitution.
  2. Löse $\boldsymbol{Ux} = \boldsymbol{y}$ durch Rückwärtssubstitution.
- **Beachte:** Die Zerlegung $\boldsymbol{A} = \boldsymbol{LU}$ ist für alle $\boldsymbol{b}$ gleich. Wir müssen sie also nur einmal ausführen.

Beispiel: Lösen mit LU-Zerlegung (Fortsetzung)

Mit $\boldsymbol{L}, \boldsymbol{U}$ von vorher und $\boldsymbol{b} = \begin{pmatrix} 5 \\ -2 \\ 9 \end{pmatrix}$.

Schritt 1: Vorwärtssubstitution für $\boldsymbol{Ly} = \boldsymbol{b}$:

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 5 \\ -2 \\ 9 \end{pmatrix} \quad \Longrightarrow \quad \begin{aligned} y_1 &= 5 \\ y_2 &= -2 - 2 \cdot 5 = -12 \\ y_3 &= 9 - (-1) \cdot 5 - (-1) \cdot (-12) = 9 + 5 - 12 = 2 \end{aligned}$$

Schritt 2: Rückwärtssubstitution für $\boldsymbol{Ux} = \boldsymbol{y}$:

$$\begin{pmatrix} 2 & 1 & -1 \\ 0 & -8 & 2 \\ 0 & 0 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ -12 \\ 2 \end{pmatrix} \Longrightarrow \begin{aligned} x_3 &= 2/3 \\ x_2 &= (-12 - 2 \cdot (2/3))/(-8) = 5/3 \\ x_1 &= (5 - 1 \cdot (5/3) - (-1) \cdot (2/3))/2 = 2 \end{aligned}$$

Pivotierte LU-Zerlegung

- Die Methode scheitert, falls im k-ten Schritt $m_{kk} = 0$.
- Das lässt sich vermeiden, indem wir Zeilen von $\boldsymbol{A}$ tauschen: $\rightarrow$ **Pivotierung**
  - löst $\boldsymbol{PA} = \boldsymbol{LU}$ mit Permutationsmatrix $\boldsymbol{P}$
- *pivoting* verbessert Stabilität auch schon für $m_{kk} \approx 0$.

Quiz

Im k-ten Schritt zum Finden von $\boldsymbol{L}$ und $\boldsymbol{U}$ multiplizieren wir

$$\boldsymbol{M} = \begin{pmatrix} * & \cdots & * & \cdots & * \\ & \ddots & \vdots & \ddots & \vdots \\ & & * & \cdots & * \\ & & \vdots & \ddots & \vdots \\ & & * & \cdots & * \end{pmatrix} \text{ mit } \boldsymbol{L}_k = \begin{pmatrix} 1 & & & & \\ & \ddots & & & \\ & & 1 & & \\ & & -l_{(k+1)k} & 1 & \\ & & \vdots & & \ddots \\ & & -l_{nk} & & 1 \end{pmatrix}.$$

Was ist die Komplexität dieses Schritts?

1. $O(n^2)$
2. $O(n^3)$
3. $O((n-k)^2)$
4. $O(k^3)$

Komplexität

- Die Berechnung der LU-Zerlegung hat also die Komplexität

$$\sum_{k=1}^{n-1} O((n-k)^2) = \sum_{k=1}^{n-1} O(n^2) = O(n^3).$$

- Das müssen wir nur einmal ausführen und können damit mehrere LGS $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}_j$, $j = 1, \ldots, J$, lösen.
- Für Vorwärts- und Rückwärtssubstitution brauchen wir je $O(n^2)$.
- Insgesamt ist die Laufzeit also $O(n^3 + Jn^2)$.

- Die Laufzeit kann verbessert werden, wenn $\boldsymbol{A}$ besondere Eigenschaften erfüllt, z.B.:
  - symmetrisch: $\boldsymbol{A} = \boldsymbol{A}^\top$,
  - positiv definit: $\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} > 0$ für alle $\boldsymbol{x} \neq \boldsymbol{0}$,
  - Bandeigenschaft: $a_{ij} = 0$ für alle $|i - j| > \beta$,
  - dünn besetzt (engl.: *sparse*): $a_{ij} = 0$ für die allermeisten (i, j).
- Für all das gibt es hervorragende Implementierungen. Man schreibt diese Algorithmen selten selbst.

Cholesky-Zerlegung

SPD-Matrizen

Ein Spezialfall ist in der Statistik besonders wichtig:

SPD-Matrizen sind

- **s**ymmetrisch und
- **p**ositiv **d**efinit.

Def.: SPD-Matrix

Eine Matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ heißt **SPD**, wenn

- $\boldsymbol{A} = \boldsymbol{A}^\top$ (*symmetrisch*) und
- $\boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{x} > 0 \; \forall \; \boldsymbol{x} \in \mathbb{R}^n \setminus \{\boldsymbol{0}\}$ (*positiv-definit*).

- Symmetrisches $\boldsymbol{A}$ ist **positiv-*semi*definit** (*PSD*) $\iff \boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{x} \geq 0 \; \forall \; \boldsymbol{x} \in \mathbb{R}^n$

SPD- und PSD-Matrizen in Statistik und ML

1. **Kovarianzmatrizen**: $\boldsymbol{\Sigma} = \mathbb{E}[(\boldsymbol{x} - \boldsymbol{\mu})(\boldsymbol{x} - \boldsymbol{\mu})^\top]$
   sind immer PSD, und SPD, wenn $\boldsymbol{x}$ nicht auf einer Untermenge von $\mathbb{R}^p$ mit Dimension $< p$ liegt.
   - Multivariate Normalverteilungen $\sim \mathcal{N}_p(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, Hauptkomponentenanalyse (PCA)
2. **Gram-Matrizen**: $\boldsymbol{K} = \boldsymbol{X}^\top \boldsymbol{X}$
   - Normalgleichungen $\boldsymbol{X}^\top \boldsymbol{X}\boldsymbol{\beta} = \boldsymbol{X}^\top \boldsymbol{y}$ für *Kleinste-Quadrate*-Probleme; Kernel-Matrizen (SVMs, Gauss-Prozesse, …)
3. **Hesse-Matrizen**: $\boldsymbol{H} = \left[\frac{\partial^2}{\partial x_i \partial x_j} f(\boldsymbol{x})\right]_{i,j}$
   - Fisher-Information; *Konvexität* & *Newton-Verfahren* in der Optimierung

Warum so wichtig?

SPD-Matrizen garantieren:

- Existenz und — bei positiver Diagonale von $\boldsymbol{L}$ — Eindeutigkeit der Cholesky-Zerlegung
- Numerische Stabilität der Cholesky-Zerlegung (auch ohne Pivotierung)
- Alle Eigenwerte sind (reell und) positiv $\implies$ voller Rang

Cholesky-Zerlegung

- Für SPD-Matrizen kann man $\boldsymbol{L}$ und $\boldsymbol{U}$ so wählen, dass $\boldsymbol{U} = \boldsymbol{L}^\top$.

Theorem: Cholesky-Zerlegung

Jede SPD-Matrix $\boldsymbol{A}$ kann in $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^\top$ zerlegt werden, wobei $\boldsymbol{L}$ eine untere Dreiecksmatrix mit positiver Diagonale ($l_{ii} > 0$) ist. Diese Zerlegung ist eindeutig.

- Der Rechen- und Speicheraufwand ist hier etwa halb so groß wie für die volle LU-Zerlegung.

(Beweis per Induktion → Anhang)

Beispiel: Cholesky-Zerlegung

Berechne Cholesky-Zerlegung von $\boldsymbol{A} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$.

Ansatz: $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^\top$ mit $\boldsymbol{L} = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix}$

Ausmultiplizieren:

$$\boldsymbol{L}\boldsymbol{L}^\top = \begin{pmatrix} l_{11} & 0 \\ l_{21} & l_{22} \end{pmatrix} \begin{pmatrix} l_{11} & l_{21} \\ 0 & l_{22} \end{pmatrix} = \begin{pmatrix} l_{11}^2 & l_{11}l_{21} \\ l_{11}l_{21} & l_{21}^2 + l_{22}^2 \end{pmatrix} = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$$

Koeffizientenvergleich:

$$\begin{aligned} l_{11}^2 = 4 \quad &\Longrightarrow \quad l_{11} = 2 \\ l_{11}l_{21} = 2 \quad &\Longrightarrow \quad l_{21} = 2/2 = 1 \\ l_{21}^2 + l_{22}^2 = 3 \quad &\Longrightarrow \quad l_{22} = \sqrt{3-1} = \sqrt{2} \end{aligned}$$

Ergebnis:

$$\boldsymbol{L} = \begin{pmatrix} 2 & 0 \\ 1 & \sqrt{2} \end{pmatrix} \Longrightarrow \qquad \boldsymbol{L}\boldsymbol{L}^\top = \begin{pmatrix} 4 & 2 \\ 2 & 3 \end{pmatrix}$$

Cholesky-Zerlegung

Beispielanwendung:

- Sei $\boldsymbol{x} \in \mathbb{R}^d$ ein Zufallsvektor mit $\mathbb{E}[\boldsymbol{x}] = \mathbf{0}$ und $\mathbb{V}\mathrm{ar}(\boldsymbol{x}) = \boldsymbol{I}_d = \mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top]$.
- Sei $\boldsymbol{\Sigma} \in \mathbb{R}^{d \times d}$ eine SPD Matrix und $\boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^\top$.
- Dann hat $\boldsymbol{y} = \boldsymbol{L}\boldsymbol{x}$ die Kovarianz

$$\mathbb{V}\mathrm{ar}(\boldsymbol{y}) = \mathbb{E}\left[\boldsymbol{y}\boldsymbol{y}^\top\right] = \mathbb{E}\left[\boldsymbol{L}\boldsymbol{x}\boldsymbol{x}^\top\boldsymbol{L}^\top\right] = \boldsymbol{L}\mathbb{E}\left[\boldsymbol{x}\boldsymbol{x}^\top\right]\boldsymbol{L}^\top = \boldsymbol{L}\boldsymbol{L}^\top = \boldsymbol{\Sigma}.$$

(das mittlere Stück fällt weg: $\mathbb{E}[\boldsymbol{x}\boldsymbol{x}^\top] = \boldsymbol{I}_d$ nach Voraussetzung)

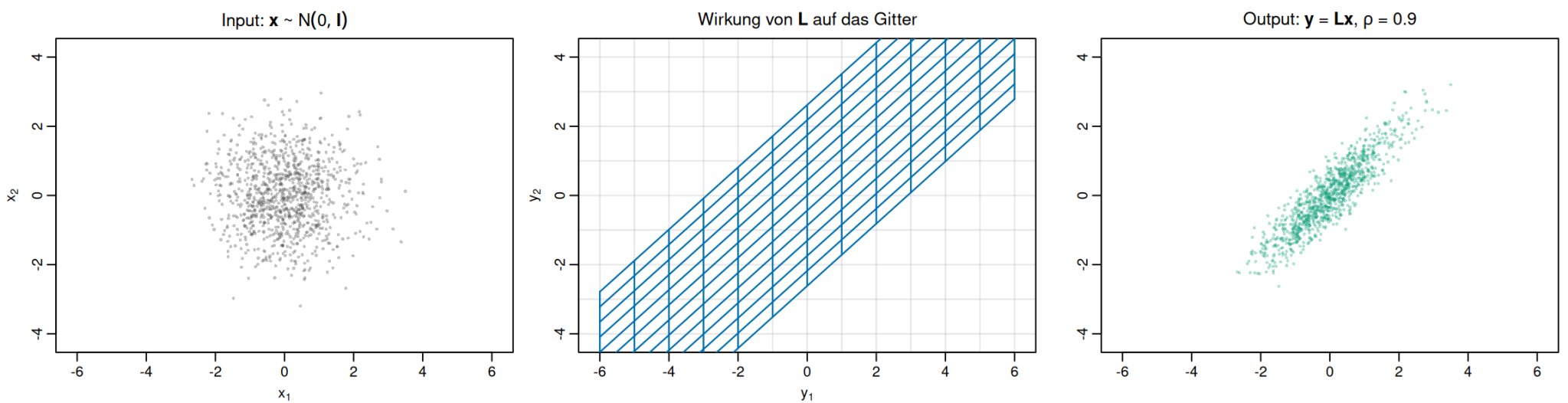
- Dies wird z.B. verwendet, um aus $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ zu simulieren:
  für $\boldsymbol{x} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}_d)$ ist $\boldsymbol{\mu} + \boldsymbol{L}\boldsymbol{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$.

Demo: Aus $\mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$ simulieren

```r
set.seed(7)
Sigma <- matrix(c(1,   0.9,
                  0.9, 1), 2, 2)
# !! chol() liefert die obere Dreiecksmatrix R mit Sigma = R'R !!
L <- t(chol(Sigma))
x <- matrix(rnorm(2 * 1000), nrow = 2)
y <- L %*% x
round(cov(t(y)), 2)
```

```
     [,1] [,2]
[1,] 1.00 0.89
[2,] 0.89 0.99
```

$$\boldsymbol{L} = \begin{pmatrix} 1 & 0 \\ 0.9 & \sqrt{1 - 0.9^2} \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0.9 & 0.436 \end{pmatrix}, \qquad \boldsymbol{\Sigma} = \boldsymbol{L}\boldsymbol{L}^\top.$$



$\Longrightarrow$ $\boldsymbol{L}$ verformt Gitter und Punktwolke auf dieselbe Weise; die empirische Kovarianz von $\boldsymbol{y}$ reproduziert $\boldsymbol{\Sigma}$.

Vergleich: LU und Cholesky

LU-Zerlegung

- **Ohne Pivotierung instabil** bei kleinen Pivotelementen
  - Division durch sehr kleine Zahlen führt zu numerischer Instabilität
- **Mit partieller Pivotierung** praktisch stabil (Higham, 2020)
- **Kondition:** Fehler $\approx \kappa(\boldsymbol{A}) \cdot \epsilon_{\text{Maschine}}$

Cholesky-Zerlegung

- **Natürlich stabil** ohne Pivotierung für *SPD-Matrizen*
- **Effizienz:** $\approx 2\times$ schneller als LU (halber Aufwand)
- **Robustheit:** In exakter Arithmetik bricht das Verfahren für **jede** nicht positiv definite Matrix ab (Wurzel aus einer nicht-positiven Zahl); bei echten SPD-Matrizen passiert das nur durch Rundungsfehler und nur bei **extrem** schlechter Kondition
  - Der Abbruch ist ein nützlicher *Diagnosehinweis*: $\boldsymbol{A}$ ist nicht SPD oder numerisch grenzwertig
- **Pivotierte Variante:** Anwendung auch für *semi*-positiv-definite Matrizen! (Details $\rightarrow$ Anhang)

Fazit

- *LU*: Universell einsetzbar, benötigt Pivotierung für Stabilität
- *Cholesky*: optimal für *SPD-Matrizen* (Statistik!), numerisch überlegen

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- **Invertiere niemals eine Matrix!**
- *LU*- und *Cholesky*-Zerlegungen

Nächste Vorlesung

- Singulärwertzerlegung (*SVD*)

Self-Check

Frage 1: Sie müssen viele LGS $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}_j$ mit derselben SPD-Matrix $\boldsymbol{A}$ lösen. Bestes Vorgehen?

1. Für jedes $\boldsymbol{b}_j$ die Gauss-Elimination komplett neu durchführen.
2. Einmal $\boldsymbol{A}^{-1}$ berechnen, dann jeweils $\boldsymbol{A}^{-1}\boldsymbol{b}_j$ multiplizieren.
3. Einmal Cholesky $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{L}^\top$, dann je zweimal Substitution.
4. Einmal pivotierte LU, da sie für SPD-Matrizen stabiler als Cholesky ist.

Frage 2: Wann braucht die LU-Zerlegung Pivotierung?

1. Nur wenn $\boldsymbol{A}$ singulär ist.
2. Wenn ein Pivotelement m_{kk} null oder nahe null ist.
3. Nie — sie verbessert nur die Laufzeit.
4. Nur für symmetrische Matrizen.

Frage 3: $\boldsymbol{A} = \boldsymbol{L}\boldsymbol{U}$ liegt bereits vor. Kosten, um *ein* LGS $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ zu lösen?

1. $O(n)$
2. $O(n^2)$
3. $O(n^3)$
4. $O(n^2 + n^3)$

Anhang

Cholesky-Zerlegung (Induktionsbeweis)

- $n = 1$: Weil $A \in \mathbb{R}$ SPD ist, gilt $A > 0$. Dann ist $A = LL^\top$ mit $L = \sqrt{A}$.
- $n > 1$:
  - Sei $\boldsymbol{A} = \begin{pmatrix} a & \boldsymbol{c}^\top \\ \boldsymbol{c} & \boldsymbol{B} \end{pmatrix}$ und die Hypothese gelte für $n-1$.
  - Setze $\boldsymbol{l} = \frac{\boldsymbol{c}}{\sqrt{a}}$. ($\boldsymbol{A}$ SPD $\implies a > 0$, da $a = \boldsymbol{e}_1^\top \boldsymbol{A} \boldsymbol{e}_1$.)
  - Außerdem ist für jedes $\boldsymbol{x} \in \mathbb{R}^{n-1} \setminus \{\boldsymbol{0}\}$:

$$\boldsymbol{x}^\top (\boldsymbol{B} - \boldsymbol{l}\boldsymbol{l}^\top)\boldsymbol{x} = \begin{pmatrix} -\frac{\boldsymbol{c}^\top \boldsymbol{x}}{a} \\ \boldsymbol{x} \end{pmatrix}^\top \boldsymbol{A} \begin{pmatrix} -\frac{\boldsymbol{c}^\top \boldsymbol{x}}{a} \\ \boldsymbol{x} \end{pmatrix} > 0$$

    also ist $\left(\boldsymbol{B} - \boldsymbol{l}\boldsymbol{l}^\top\right)$ SPD und $\left(\boldsymbol{B} - \boldsymbol{l}\boldsymbol{l}^\top\right) = \widetilde{\boldsymbol{L}}\widetilde{\boldsymbol{L}}^\top$ (Induktionshypothese).
  - Wähle nun

$$\boldsymbol{L} = \begin{pmatrix} \sqrt{a} & \boldsymbol{0}^\top \\ \boldsymbol{l} & \widetilde{\boldsymbol{L}} \end{pmatrix} \implies \boldsymbol{L}\boldsymbol{L}^\top = \begin{pmatrix} a & \boldsymbol{c}^\top \\ \boldsymbol{c} & \boldsymbol{l}\boldsymbol{l}^\top + \widetilde{\boldsymbol{L}}\widetilde{\boldsymbol{L}}^\top \end{pmatrix} = \boldsymbol{A} \quad \checkmark$$

  - *Eindeutigkeit:* $l_{11} = \sqrt{a}$ und $\boldsymbol{l} = \boldsymbol{c}/\sqrt{a}$ sind erzwungen, $\widetilde{\boldsymbol{L}}$ ist eindeutig nach Induktionsannahme.

↩ zurück: „Cholesky-Zerlegung"

Cholesky mit Pivotierung

- Notwendig falls $\boldsymbol{A}$ singulär oder numerisch rangdefizient
  - für positiv **semi**-definite Matrizen ($\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} \geq 0$ statt > 0):
- **Complete pivoting:** $\boldsymbol{P}^\top \boldsymbol{A}\boldsymbol{P} = \boldsymbol{L}\boldsymbol{L}^\top$
  - Permutationsmatrix $\boldsymbol{P}$ (zeilen- und spaltenvertauschend)
  - **Rang-aufdeckend:** Nullspalten in $\boldsymbol{L}$ zeigen Rangabfall von $\boldsymbol{A}$
- **Stabilität:** Auch für semidefinite Matrizen stabil (Higham, 1990)
- **Anwendungen** in ML/Statistik:
  - Kernel-Matrizen (Gauss-Prozesse, SVMs, …)
  - Niedrig-rangige approximative Matrixzerlegung: $O(k^2 n)$ statt $O(n^3)$
  - Gram-Matrizen $\boldsymbol{X}^\top \boldsymbol{X}$ bei $p > n$ (mehr Features als Daten)

↩ zurück: „Vergleich: LU und Cholesky"

Singulärwertzerlegung (SVD)

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- **Matrixnormen** und ihre Eigenschaften, speziell:
  - *Spektralnorm* $\|\boldsymbol{A}\|_2 = \sqrt{\lambda_{\max}(\boldsymbol{A}^\top \boldsymbol{A})}$
  - *Orthogonalmatrizen* $\boldsymbol{Q}$ mit $\boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$
- Matrixzerlegungskonzepte
- **Lineare Algebra**
  - Eigenwerte und Eigenvektoren: $\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$
  - Rang einer Matrix
  - Fundamentale Unterräume: $\mathrm{col}(\boldsymbol{A}), \ker(\boldsymbol{A}), \mathrm{col}(\boldsymbol{A}^\top)$
  - Projektionen auf Unterräume

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ *symmetrisch* ($\boldsymbol{A} = \boldsymbol{A}^\top$). Welche Aussage gilt **immer**?

1. Alle Eigenwerte von $\boldsymbol{A}$ sind ≥ 0.
2. $\boldsymbol{A}$ ist orthogonal diagonalisierbar: $\boldsymbol{A} = \boldsymbol{Q}\boldsymbol{D}\boldsymbol{Q}^\top$ mit reellen Eigenwerten.
3. $\boldsymbol{A}$ ist invertierbar.
4. $\boldsymbol{A}$ hat n verschiedene Eigenwerte.

Frage 2: Was ist der Rang von $\boldsymbol{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix}$?

1. 0
2. 1
3. 2
4. 3

Motivation

Diagonalisierung ist limitiert

- Eigenwertzerlegung funktioniert nur für **quadratische** Matrizen – und auch dort nicht immer:
  $\begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}$ hat nur den Eigenwert 0 mit eindimensionalem Eigenraum, also keine Eigenvektorbasis
- Orthogonale Diagonalisierung bisher nur für **symmetrische** Matrizen
- **Problem**:
  Die meisten Matrizen sind weder quadratisch noch symmetrisch.

Was wir wollen: Eine Zerlegung $\boldsymbol{A = PDQ^{-1}}$

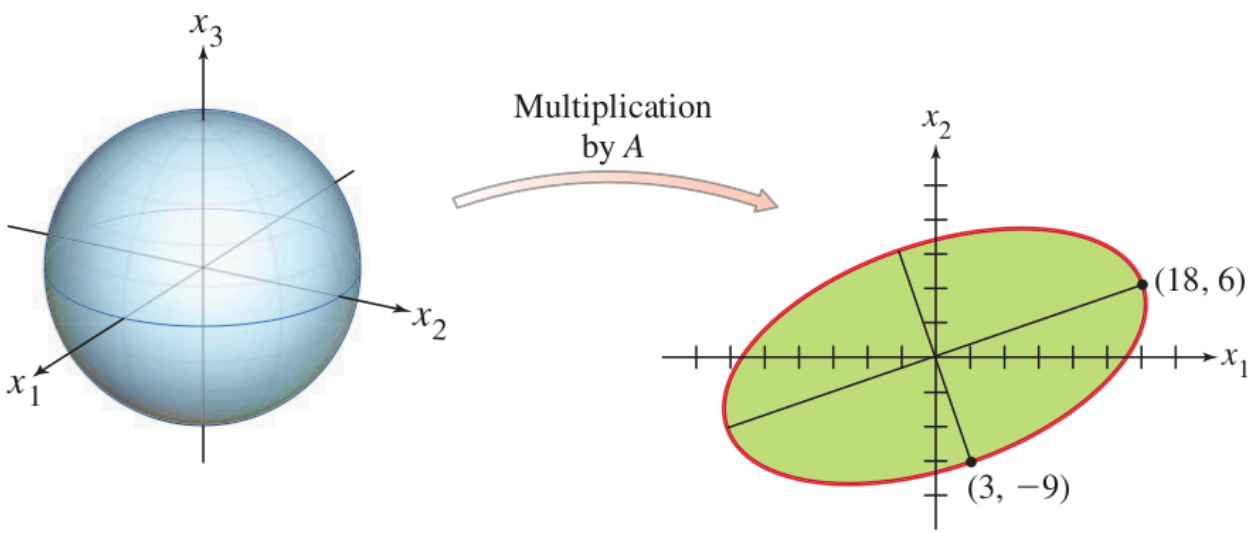
- für *beliebige* $m \times n$ Matrizen, so dass
- $\boldsymbol{D}$ "diagonal" ist und
- $\boldsymbol{P}$ und $\boldsymbol{Q}$ *Orthogonalmatrizen* sind

Geometrische Intuition

Die Einheitssphäre als Testfall:

- Betrachte alle Vektoren $\boldsymbol{x}$ mit $\|\boldsymbol{x}\| = 1$
- Nach Transformation: $\boldsymbol{A}\boldsymbol{x}$ hat verschiedene Längen
- **Frage**: Welche Richtung $\boldsymbol{A}\boldsymbol{x}$ wurde am stärksten (zweitstärksten, drittstärksten, ...) gestreckt?

Beispiel: $\boldsymbol{A} \in \mathbb{R}^{2\times 3}$



- Einheitssphäre in $\mathbb{R}^3 \to$ ausgefüllte Ellipse in $\mathbb{R}^2$; stärkste Streckung: Hauptachse, zweitstärkste: Nebenachse; Richtungen in $\ker(\boldsymbol{A})$ werden auf $\mathbf{0}$ gestaucht

▶ Interaktive Visualisierung im Skript

Die Grundidee

Maximierungsproblem:

$$\max_{\|\boldsymbol{x}\|=1} \|\boldsymbol{A}\boldsymbol{x}\|$$

Trick: Maximiere $\|\boldsymbol{A}\boldsymbol{x}\|^2$ statt $\|\boldsymbol{A}\boldsymbol{x}\|$

$$\|\boldsymbol{A}\boldsymbol{x}\|^2 = (\boldsymbol{A}\boldsymbol{x})^\top(\boldsymbol{A}\boldsymbol{x}) = \boldsymbol{x}^\top(\boldsymbol{A}^\top\boldsymbol{A})\boldsymbol{x}$$

Erkenntnis: Dies ist eine quadratische Form bezüglich $\boldsymbol{A}^\top\boldsymbol{A}$!

Im Folgenden immer: $\|\cdot\| \equiv \|\cdot\|_2$!

SVD: Singulärwerte und Singulärvektoren

Die Matrix $\boldsymbol{A}^\top \boldsymbol{A}$

Theorem: Eigenschaften von $\boldsymbol{A}^\top \boldsymbol{A}$

Für $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ gilt:

1. $\boldsymbol{A}^\top \boldsymbol{A}$ ist *symmetrisch*: $\left(\boldsymbol{A}^\top \boldsymbol{A}\right)^\top = \boldsymbol{A}^\top \boldsymbol{A}$
2. $\boldsymbol{A}^\top \boldsymbol{A}$ ist *positiv semidefinit*: $\boldsymbol{x}^\top \left(\boldsymbol{A}^\top \boldsymbol{A}\right)\boldsymbol{x} \geq 0$ für alle $\boldsymbol{x} \in \mathbb{R}^n$; insbesondere sind alle Eigenwerte $\lambda_i \geq 0$

Beweis für (2): $\boldsymbol{x}^\top \left(\boldsymbol{A}^\top \boldsymbol{A}\right)\boldsymbol{x} = \|\boldsymbol{A}\boldsymbol{x}\|_2^2 \geq 0$; für jeden normierten Eigenvektor $\boldsymbol{v}_i$ ($\|\boldsymbol{v}_i\| = 1$) gilt damit:

$$0 \leq \|\boldsymbol{A}\boldsymbol{v}_i\|_2^2 = \boldsymbol{v}_i^\top \left(\boldsymbol{A}^\top \boldsymbol{A}\right)\boldsymbol{v}_i = \boldsymbol{v}_i^\top \left(\lambda_i \boldsymbol{v}_i\right) = \lambda_i$$

Folgerung: $\boldsymbol{A}^\top \boldsymbol{A}$ ist orthogonal diagonalisierbar!

Bsp: Eigenschaften von $\boldsymbol{A}^\top \boldsymbol{A}$

Sei $\boldsymbol{A} = \begin{pmatrix} \boldsymbol{a}_1 & \boldsymbol{a}_2 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} \in \mathbb{R}^{3\times 2}$

$$\boldsymbol{A}^\top \boldsymbol{A} = \begin{pmatrix} \boldsymbol{a}_1^\top \\ \boldsymbol{a}_2^\top \end{pmatrix} \begin{pmatrix} \boldsymbol{a}_1 & \boldsymbol{a}_2 \end{pmatrix} = \begin{pmatrix} \boldsymbol{a}_1^\top \boldsymbol{a}_1 & \boldsymbol{a}_1^\top \boldsymbol{a}_2 \\ \boldsymbol{a}_2^\top \boldsymbol{a}_1 & \boldsymbol{a}_2^\top \boldsymbol{a}_2 \end{pmatrix}$$
$$= \begin{pmatrix} 1 & 2 & 1 \\ 2 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} = \begin{pmatrix} 6 & 4 \\ 4 & 5 \end{pmatrix}$$

Symmetrie: ✓

Positiv semidefinit:

$$\det\left(\boldsymbol{A}^\top \boldsymbol{A} - \lambda \boldsymbol{I}\right) = (6-\lambda)(5-\lambda) - 16 = \lambda^2 - 11\lambda + 14 = 0$$
$$\lambda_{1,2} = \frac{11 \pm \sqrt{11^2 - 4\cdot 1 \cdot 14}}{2} = \frac{11 \pm \sqrt{65}}{2}$$
$$\implies \lambda_1 \approx 9.53, \quad \lambda_2 \approx 1.47$$

$\lambda_1, \lambda_2 > 0 \implies \boldsymbol{A}^\top \boldsymbol{A}$ hier sogar positiv *definit* – mehr als das vom Theorem garantierte “semidefinit”, da $\boldsymbol{A}$ vollen Spaltenrang hat ($\operatorname{rang}(\boldsymbol{A}) = n = 2$)

Definition: Singulärwerte

Seien $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$ die Eigenwerte von $\boldsymbol{A}^\top \boldsymbol{A}$.

Def.: Singulärwerte

Die **Singulärwerte** von $\boldsymbol{A}$ sind:

$$\sigma_i = \sqrt{\lambda_i}, \quad i = 1, 2, \ldots, n$$

Interpretation:

- $\sigma_1 = \max_{\|\boldsymbol{x}\|=1} \|\boldsymbol{A}\boldsymbol{x}\|$ (maximale Streckung)
- σ_n entspricht der minimalen Streckung / maximalen Stauchung
- $\sigma_i = 0 \iff \lambda_i = 0$

Bsp: Singulärwerte

Matrix $\boldsymbol{A} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix}$: $\quad$ $\boldsymbol{A}^\top \boldsymbol{A}$ mit $\boldsymbol{\lambda} \approx (9.53, 1.47)$:

Singulärwerte:

$$\sigma_1 = \sqrt{\lambda_1} = \sqrt{9.53} \approx 3.09$$
$$\sigma_2 = \sqrt{\lambda_2} = \sqrt{1.47} \approx 1.21$$

Interpretation:

- $\sigma_1 \approx 3.09$: Maximale Streckung der Matrix $\boldsymbol{A}$ ($\boldsymbol{v}_1$ wird um Faktor 3.1 gestreckt)
- $\sigma_2 \approx 1.21$: Minimale Streckung ($\boldsymbol{v}_2$ wird um Faktor 1.2 gestreckt)
- Verhältnis $\frac{\sigma_{\max}}{\sigma_{\min}} \approx 2.55$ zeigt, wie unterschiedlich die Streckung je nach Richtung ist (*anisotrop*).

Definition: Singulärvektoren

Def.: Rechte und linke Singulärvektoren

Rechte Singulärvektoren $\boldsymbol{v}_i$:

- Normierte Eigenvektoren von $\boldsymbol{A}^\top \boldsymbol{A}$
- $\boldsymbol{A}^\top \boldsymbol{A} \boldsymbol{v}_i = \lambda_i \boldsymbol{v}_i = \sigma_i^2 \boldsymbol{v}_i$

Linke Singulärvektoren $\boldsymbol{u}_i$:

- Definiert durch: $\boldsymbol{u}_i = \frac{\boldsymbol{A}\boldsymbol{v}_i}{\sigma_i}$ (für $\sigma_i > 0$)

Wichtige Eigenschaften:

- $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$ ist orthonormal
- $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r\}$ ist orthonormal ($r = \operatorname{rang}(\boldsymbol{A})$)

Beweis der Orthonormalität: siehe Anhang

Bsp: Singulärvektoren

Matrix $\boldsymbol{A} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix}$: $\quad \boldsymbol{A}^\top \boldsymbol{A} = \begin{pmatrix} 6 & 4 \\ 4 & 5 \end{pmatrix}$ mit $\boldsymbol{\lambda} \approx (9.53, 1.47)$:

Rechte Singulärvektoren (normierte Eigenvektoren von $\boldsymbol{A}^\top \boldsymbol{A}$):

$$\begin{pmatrix} 6 & 4 \\ 4 & 5 \end{pmatrix} \boldsymbol{v}_1 = 9.53\, \boldsymbol{v}_1 \implies \boldsymbol{v}_1 \approx \begin{pmatrix} -0.75 \\ -0.662 \end{pmatrix}, \qquad \boldsymbol{v}_2 \approx \begin{pmatrix} -0.662 \\ 0.75 \end{pmatrix} \text{ (normiert, orthogonal zu } \boldsymbol{v}_1)$$

Linke Singulärvektoren ($\boldsymbol{u}_i = \boldsymbol{A}\boldsymbol{v}_i / \sigma_i$), eine Rechnung exemplarisch:

$$\boldsymbol{u}_1 = \frac{\boldsymbol{A}\boldsymbol{v}_1}{\sigma_1} \approx \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} -0.75 \\ -0.662 \end{pmatrix} / 3.09 = \begin{pmatrix} -0.672 \\ -0.7 \\ -0.243 \end{pmatrix}, \qquad \text{analog: } \boldsymbol{u}_2 \approx \begin{pmatrix} 0.691 \\ -0.474 \\ -0.546 \end{pmatrix}$$

Check: $\boldsymbol{u}_1^\top \boldsymbol{u}_2 \approx 0$ und $\|\boldsymbol{u}_1\| \approx 1$ ✓ (vollständige Rechnung: siehe Anhang)

Live-Demo: SVD in R

Stimmt unsere Handrechnung? **Vorhersage:** Was liefert `svd(A)`?

```r
A <- matrix(c(1, 2,
              2, 1,
              1, 0), nrow = 3, byrow = TRUE)
svd(A)
```

```
$d
[1] 3.09 1.21

$u
      [,1]   [,2]
[1,] -0.672  0.691
[2,] -0.700 -0.474
[3,] -0.243 -0.546

$v
      [,1]   [,2]
[1,] -0.750 -0.662
[2,] -0.662  0.750
```

- d $\approx (3.09, 1.21)$: wie von Hand berechnet ✓
- u, v: Singulärvektoren – Vorzeichen nicht eindeutig ($\pm \boldsymbol{u}_i$, $\pm \boldsymbol{v}_i$ ebenfalls gültig)!

Fundamentale Unterräume

Theorem: Charakterisierung der Unterräume

Angenommen $\boldsymbol{A}$ hat r Singulärwerte ungleich Null. Dann:

1. $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_r\}$ ist Orthonormalbasis für $\mathrm{col}(\boldsymbol{A}^\top)$ (*Zeilenraum*)
2. $\{\boldsymbol{A}\boldsymbol{v}_1, \ldots, \boldsymbol{A}\boldsymbol{v}_r\}$ ist Orthogonalbasis für $\mathrm{col}(\boldsymbol{A})$ (*Spaltenraum*) – normiert ($\|\boldsymbol{A}\boldsymbol{v}_i\| = \sigma_i$) sind das die $\{\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r\}$
3. $\{\boldsymbol{v}_{r+1}, \ldots, \boldsymbol{v}_n\}$ ist Orthonormalbasis für $\ker(\boldsymbol{A})$ (*Nullraum*)
4. $\mathrm{rang}(\boldsymbol{A}) = r$

Interpretation: SVD gibt uns explizite Orthonormalbasen für alle Fundamentalräume! (für den *linken Nullraum* $\ker(\boldsymbol{A}^\top)$: erweiterte $\boldsymbol{u}_{r+1}, \ldots, \boldsymbol{u}_m$ → SVD-Hauptsatz & Beispiel)

Rechenbeispiel: Fundamentale Unterräume (1)

Für $\boldsymbol{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix}$ mit $\boldsymbol{A}^\top \boldsymbol{A} = \begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$ und rang($\boldsymbol{A}$) = 1:

- **Singulärwerte**:
  $\lambda_1(\boldsymbol{A}^\top \boldsymbol{A}) = 4,\ \lambda_2(\boldsymbol{A}^\top \boldsymbol{A}) = 0$
  $\implies \sigma_1(\boldsymbol{A}) = 2$: Maximale Streckung durch $\boldsymbol{A}$ um Faktor 2 in Richtung $\boldsymbol{v}_1$; Faktor 0 in Richtung $\boldsymbol{v}_2$
- **Zeilenraum: Rechte Singulärvektoren**
  $\boldsymbol{v}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \boldsymbol{v}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$
  $\implies$ col($\boldsymbol{A}^\top$) = span$\{\boldsymbol{v}_1\}$ = span$\left\{ \begin{pmatrix} 1 \\ 1 \end{pmatrix} \right\}$
- **Spaltenraum: Linke Singulärvektoren** $\boldsymbol{u}_1 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix}, \quad \boldsymbol{u}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}, \quad \boldsymbol{u}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$

  (*$\boldsymbol{u}_2, \boldsymbol{u}_3$: orthonormale Ergänzung, da $\sigma_2 = 0$; → SVD-Hauptsatz)*
  $\implies$ col($\boldsymbol{A}$) = span$\{\boldsymbol{u}_1\}$ = span$\left\{ \begin{pmatrix} 1 \\ 1 \\ 0 \end{pmatrix} \right\}$

Rechenbeispiel: Fundamentale Unterräume (2)

Weiter mit $\boldsymbol{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 0 & 0 \end{pmatrix}$, $\boldsymbol{v}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix}$, $\boldsymbol{u}_2 = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}$, $\boldsymbol{u}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$:

(Rechter) Nullraum: $\ker(\boldsymbol{A}) = \operatorname{span}\{\boldsymbol{v}_2\} = \operatorname{span}\left\{ \begin{pmatrix} 1 \\ -1 \end{pmatrix} \right\}$ Verifikation: $\boldsymbol{A} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix}$ ✓

Linker Nullraum: $\ker(\boldsymbol{A}^\top) = \operatorname{span}\{\boldsymbol{u}_2, \boldsymbol{u}_3\} = \operatorname{span}\left\{ \begin{pmatrix} -1 \\ 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} \right\}$ Verifikation:

$$\boldsymbol{A}^\top \begin{pmatrix} -1 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix}$$ ✓

Der SVD-Hauptsatz

Theorem: Singulärwertzerlegung

Sei $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ mit $\operatorname{rang}(\boldsymbol{A}) = r$. Dann existiert eine Zerlegung:

$$\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$$

wobei:

- $\boldsymbol{V} \in \mathbb{R}^{n \times n}$: Orthogonalmatrix (rechte Singulärvektoren in Spalten)
- $\boldsymbol{U} \in \mathbb{R}^{m \times m}$: Orthogonalmatrix (linke Singulärvektoren erweitert zu Basis)
- $\boldsymbol{\Sigma} \in \mathbb{R}^{m \times n}$: “Diagonalmatrix” mit Singulärwerten

Beweis (Konstruktionsskizze): siehe Anhang

Die Struktur von $\boldsymbol{\Sigma}$

Form der Matrix $\boldsymbol{\Sigma}$:

$$\boldsymbol{\Sigma} = \begin{pmatrix} \operatorname{diag}(\sigma_1, \ldots, \sigma_r) & 0_{r \times (n-r)} \\ 0_{(m-r) \times r} & 0_{(m-r) \times (n-r)} \end{pmatrix}$$

Beispiele:

$\boldsymbol{A}$ *quadratisch* ($m = n = 3, r = 2$):

$$\boldsymbol{\Sigma} = \begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

$\boldsymbol{A}$ *rechteckig* ($m = 2, n = 3, r = 2$):

$$\boldsymbol{\Sigma} = \begin{pmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \end{pmatrix}$$

Geometrische Interpretation

Transformation: $\boldsymbol{x} \mapsto \boldsymbol{A}\boldsymbol{x} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top\boldsymbol{x}$

Drei Schritte:

1. $\boldsymbol{V}^\top$:
   Koordinatentransformation (Rotation/Spiegelung) zu “optimalen” Richtungen
2. $\boldsymbol{\Sigma}$:
   “Streckung” entlang dieser Achsen um Faktoren σ_i, Nullsetzen für $i > r$
3. $\boldsymbol{U}$:
   Koordinatentransformation (Rotation/Spiegelung) zurück in den Zielraum

$\Longrightarrow$ **SVD zerlegt *jede* lineare Transformation von $\mathbb{R}^n$ nach $\mathbb{R}^m$ in “Drehen-Strecken-Drehen”.**

Reduzierte SVD und Pseudoinverse

Reduzierte SVD

Problem: Viele Nulleinträge in $\boldsymbol{\Sigma} \in \mathbb{R}^{m \times n}$, sobald $\operatorname{rang}(\boldsymbol{A}) = r < m$ oder $r < n$ ($\boldsymbol{A}$ rangdefizient *oder* nicht quadratisch):

- $\boldsymbol{\Sigma}$ enthält $(m - r)$ Nullzeilen **und** $(n - r)$ Nullspalten
- Entsprechende Spalten in $\boldsymbol{U}$ bzw. $\boldsymbol{V}$ werden mit 0 multipliziert

Idee: Partitioniere $\boldsymbol{U} = (\boldsymbol{U}_r \mid \boldsymbol{U}_{m-r})$, $\boldsymbol{V} = (\boldsymbol{V}_r \mid \boldsymbol{V}_{n-r})$ mit $\boldsymbol{U}_r \in \mathbb{R}^{m \times r}$, $\boldsymbol{V}_r \in \mathbb{R}^{n \times r}$ und behalte nur die ersten r Singulärwerte und -vektoren:

Def.: Reduzierte SVD

Die **reduzierte SVD** von $\boldsymbol{A}$ mit $\operatorname{rang}(\boldsymbol{A}) = r$ ist

$$\boldsymbol{A} = \boldsymbol{U}_r \boldsymbol{\Sigma}_r \boldsymbol{V}_r^\top$$

wobei:

- $\boldsymbol{U}_r \in \mathbb{R}^{m \times r}$: Erste r linke Singulärvektoren
- $\boldsymbol{V}_r \in \mathbb{R}^{n \times r}$: Erste r rechte Singulärvektoren
- $\boldsymbol{\Sigma}_r \in \mathbb{R}^{r \times r}$: $\operatorname{diag}(\sigma_1, \ldots, \sigma_r)$

Vorteile:

Moore-Penrose Pseudoinverse

Motivation: Was wenn $\boldsymbol{A}$ nicht quadratisch/invertierbar?

Def.: Moore-Penrose Pseudoinverse

Für jede Matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ mit Rang $r \geq 1$ und *reduzierter SVD* $\boldsymbol{A} = \boldsymbol{U}_r \boldsymbol{\Sigma}_r \boldsymbol{V}_r^\top$ ist die **Moore-Penrose Pseudoinverse**:

$$\boldsymbol{A}^+ := \boldsymbol{V}_r \boldsymbol{\Sigma}_r^{-1} \boldsymbol{U}_r^\top \in \mathbb{R}^{n \times m}$$

Bemerkung:

$\boldsymbol{\Sigma}_r^{-1} = \operatorname{diag}(1/\sigma_1, \ldots, 1/\sigma_r)$; für $\boldsymbol{A} = \boldsymbol{0}$ (also $r = 0$) setzt man $\boldsymbol{A}^+ = \boldsymbol{0} \in \mathbb{R}^{n \times m}$

Eigenschaften der Pseudoinverse

Theorem: Eigenschaften von $\boldsymbol{A}^+$

1. $\boldsymbol{A}\boldsymbol{A}^+\boldsymbol{x} = \text{proj}_{\text{col}(\boldsymbol{A})}\boldsymbol{x}$ für alle $\boldsymbol{x} \in \mathbb{R}^m$
2. $\boldsymbol{A}^+\boldsymbol{A}\boldsymbol{x} = \text{proj}_{\text{col}(\boldsymbol{A}^\top)}\boldsymbol{x}$ für alle $\boldsymbol{x} \in \mathbb{R}^n$
3. $\boldsymbol{A}\boldsymbol{A}^+\boldsymbol{A} = \boldsymbol{A}$ *(Konsistenz)*
4. $\boldsymbol{A}^+\boldsymbol{A}\boldsymbol{A}^+ = \boldsymbol{A}^+$ *(Konsistenz)*

Warum Projektion?
$\boldsymbol{A}\boldsymbol{A}^+ = \boldsymbol{U}_r\boldsymbol{\Sigma}_r\boldsymbol{V}_r^\top\boldsymbol{V}_r\boldsymbol{\Sigma}_r^{-1}\boldsymbol{U}_r^\top = \boldsymbol{U}_r\boldsymbol{U}_r^\top$, und die Spalten von $\boldsymbol{U}_r$ sind eine Orthonormalbasis von $\text{col}(\boldsymbol{A})$.

Spezialfälle:

- Wenn $\text{rang}(\boldsymbol{A}) = m$: $\boldsymbol{A}\boldsymbol{A}^+ = \boldsymbol{I}_m$
- Wenn $\text{rang}(\boldsymbol{A}) = n$: $\boldsymbol{A}^+\boldsymbol{A} = \boldsymbol{I}_n$
- Wenn $\boldsymbol{A}$ quadratisch und invertierbar: $\boldsymbol{A}^+ = \boldsymbol{A}^{-1}$

Anwendungen

Spektralnorm und SVD

Bisher: $\|\boldsymbol{A}\|_2 = \sqrt{\lambda_{\max}\left(\boldsymbol{A}^\top \boldsymbol{A}\right)}$

Aus Definition SVD ist bekannt, dass $\lambda_j\left(\boldsymbol{A}^\top \boldsymbol{A}\right) = \sigma_j(\boldsymbol{A})^2$

Also:

$$\|\boldsymbol{A}\|_2 = \sigma_{\max}(\boldsymbol{A})$$

Interpretation: Größter Singulärwert = maximale Streckung

Die SVD als Summe von Rang-1 Matrizen

SVD in Summenform:

$$\boldsymbol{A} = \boldsymbol{U}_r \boldsymbol{\Sigma}_r \boldsymbol{V}_r^\top = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^\top$$

Interpretation:

- Jeder Term $\boldsymbol{u}_i \boldsymbol{v}_i^\top$ ist eine **Rang-1 Matrix** (mit $\left\| \boldsymbol{u}_i \boldsymbol{v}_i^\top \right\|_F = 1$)
- σ_i gewichtet die Bedeutung des i-ten Terms
- Große $\sigma_i \rightarrow$ wichtige Richtungen
- Kleine $\sigma_i \rightarrow$ weniger wichtige Richtungen

Idee für Approximation: Verwende nur die wichtigsten Terme!

Rang-k Approximation

Low-Rank Approximation:

$$\boldsymbol{A}_k = \sum_{i=1}^{k} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^\top \quad (k < r)$$

Theorem: Optimale Approximation

$\boldsymbol{A}_k$ ist *beste Rang-k Approximation* von $\boldsymbol{A}$ bezüglich der Frobenius- und Spektral-Normen, d.h. $\boldsymbol{A}_k$ löst beide Approximationsprobleme:

$$\boldsymbol{A}_k \in \arg \min_{\substack{\boldsymbol{B} \in \mathbb{R}^{m \times n} \\ \operatorname{rang}(\boldsymbol{B}) \leq k}} \|\boldsymbol{A} - \boldsymbol{B}\|_F \quad \text{und} \quad \boldsymbol{A}_k \in \arg \min_{\substack{\boldsymbol{B} \in \mathbb{R}^{m \times n} \\ \operatorname{rang}(\boldsymbol{B}) \leq k}} \|\boldsymbol{A} - \boldsymbol{B}\|_2$$

Approximationsfehler:

- $\|\boldsymbol{A} - \boldsymbol{A}_k\|_F = \sqrt{\sum_{i=k+1}^{r} \sigma_i^2}$
- $\|\boldsymbol{A} - \boldsymbol{A}_k\|_2 = \sigma_{k+1}$

Interpretation: Fehler bestimmt durch *weggelassene* Singulärwerte

Nicht-Eindeutigkeit in Spektralnorm: Für $\boldsymbol{A} = \operatorname{diag}(3, 2)$, $k = 1$ und $\boldsymbol{B}_b = \operatorname{diag}(b, 0)$ gilt $\|\boldsymbol{A} - \boldsymbol{B}_b\|_2 = \max\{|3 - b|, 2\}$. Damit sind alle $b \in [1, 5]$ optimal; die abgeschnittene SVD entspricht $b = 3$. In Frobenius-Norm ist nur $b = 3$ optimal, denn der Fehler ist $\sqrt{(3 - b)^2 + 2^2}$.

Wahl der optimalen Rang-k Approximation

Kriterien für die Wahl von k:

1. Relativer Fehler (Spektralnorm):

$$\frac{\|\boldsymbol{A} - \boldsymbol{A}_k\|_2}{\|\boldsymbol{A}\|_2} = \frac{\sigma_{k+1}}{\sigma_1}$$

2. Relativer Fehler (Frobenius-Norm):

$$\frac{\|\boldsymbol{A} - \boldsymbol{A}_k\|_F}{\|\boldsymbol{A}\|_F} = \frac{\sqrt{\sum_{i=k+1}^{r} \sigma_i^2}}{\sqrt{\sum_{i=1}^{r} \sigma_i^2}}$$

3. "Energieerhaltung":

$$\frac{\sum_{i=1}^{k} \sigma_i^2}{\sum_{i=1}^{r} \sigma_i^2} \geq p \quad (\text{p*100\% der "Energie"})$$

Praktische Heuristik: k am "Ellenbogen" im $\log(\sigma_i)$-Plot wählen – dort, wo die Singulärwerte abknicken.
(Typische Verläufe: siehe Anhang)

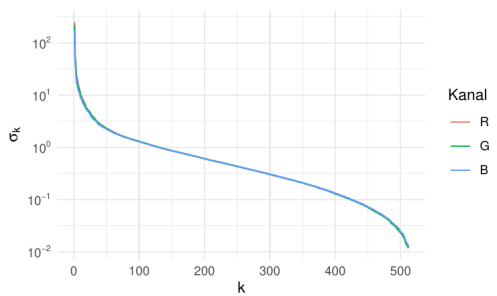
Anwendung: Datenkompression

Bildkompression – *Originalbild*: 659×512 Pixel-Matrix (pro Farbkanal)

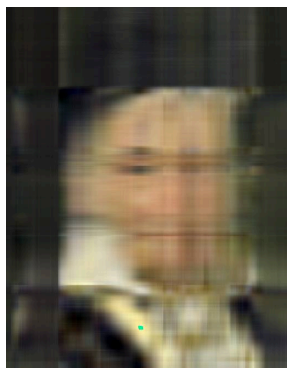
- Volle Speicherung: $659 \times 512 = 337.408$ Zahlen; **Rang-50 SVD**: $(659 \times 50) + 50 + (50 \times 512) = 58.600$ Zahlen $\implies$ *Speicherersparnis*: $\approx 83\%$ (Speicheranteil $\approx 17\%$)!
- **Vorgehen:** SVD berechnen, k anhand der Singulärwerte wählen, nur $\boldsymbol{U}_k, \boldsymbol{\Sigma}_k, \boldsymbol{V}_k$ speichern.



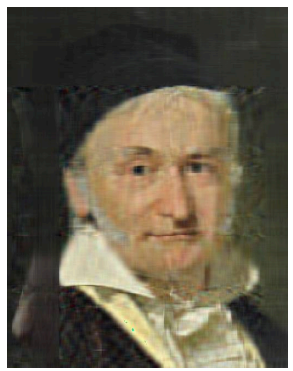
Original



Singulärwerte pro Kanal



$k = 5$



$k = 20$



$k = 50$



$k = 200$

Bildquelle: C. A. Jensen, Porträt C. F. Gauß (1840), gemeinfrei (Wikimedia Commons).

Interaktive Demo: Bildkompression

Fallback: ▶ App direkt öffnen — Screenshot im Anhang.

Anwendung: Empfehlungssysteme

Collaborative Filtering / Recommender Systems

Problem: Netflix-Szenario: m Nutzer, n Filme, aber Daten sehr *sparse*: meiste Nutzer haben meiste Filme nicht bewertet

$$\boldsymbol{R} = \begin{pmatrix} 5 & ? & 1 & ? & 4 \\ ? & 3 & ? & 4 & ? \\ 2 & 1 & ? & ? & 5 \\ ? & 5 & 4 & 3 & ? \end{pmatrix} \in \mathbb{R}^{m \times n}$$

Idee: Nutzer-Film-Präferenzen liegen in einem *niedrigdimensionalen Raum* (Genres und Nutzerklassen).

SVD-Ansatz:

1. Ergänze fehlende Einträge (z.B. durch Spaltenmittelwerte)
2. Berechne Rang-k Approximation: $\boldsymbol{R}_k = \boldsymbol{U}_k \boldsymbol{\Sigma}_k \boldsymbol{V}_k^\top$
3. Nutze $\boldsymbol{R}_k$ für Vorhersagen fehlender Ratings

Interpretation: $\boldsymbol{v}_j$ = "gelernte" Film-Genres, $\boldsymbol{u}_i$ = Nutzer-Präferenzen in diesen Genre-Dimensionen, σ_i = Wichtigkeit der Dimension *(mehr: siehe Anhang)*

Wann ist SVD besonders nützlich?

Ideale Situationen für SVD:

1. *Niedrig-Rang Strukturen*: $\text{rang}(\boldsymbol{A}) \ll \min(m, n)$
2. *Schnell abfallende Singulärwerte*: Wenige dominieren
3. *Überbestimmte Systeme*: $m > n$, möglicherweise inkonsistent
4. *Datenanalyse*: Hauptkomponentenanalyse (PCA) ohne Kreuzprodukte zu berechnen.
5. *Numerische Robustheit*: Alternative zu Normalgleichungen

Rechenaufwand:

- *SVD*: $O(\min(m^2n, mn^2))$, für $m \gg n$ etwa $4mn^2$
- *Normalgleichungen*: $O(mn^2 + n^3)$, für $m \gg n$ etwa mn^2.
- Für $m \gg n$ sind Normalgleichungen schneller, aber SVD ist robuster $\rightarrow$ trade-off zwischen Geschwindigkeit und Stabilität

SVD vs. Eigenwertzerlegung

Eigenschaft	**Eigenwertzerlegung**	**SVD**
Matrix-Typ	Quadratisch	Beliebig $m \times n$
Orthogonale Zerlegung	Nur symmetrisch ($\rightarrow$ Spektralzerlegung)	Immer
Numerische Stabilität	Kann instabil sein	Stabil
Existenz	Nur für diagonalisierbare $\boldsymbol{A}$	Immer
Geometrische Interpretation	Hauptachsen (symmetrisch)	“Optimale” Koordinaten

Zusammenfassung & Self-Check

Zusammenfassung

Kernkonzepte:

1. *SVD erweitert Eigenwertzerlegung* auf beliebige $m \times n$ Matrizen
2. *Singulärwerte vs. Eigenwerte*: $\sigma_i(\boldsymbol{A}) = \sqrt{\lambda_i\left(\boldsymbol{A}^\top \boldsymbol{A}\right)}$
3. *Hauptsatz*: $\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$ mit orthogonalen $\boldsymbol{U}$ $(m \times m)$, $\boldsymbol{V}$ $(n \times n)$, diagonalem $\boldsymbol{\Sigma}$ $(m \times n)$
4. *Reduzierte SVD*: $\boldsymbol{A} = \boldsymbol{U}_r\boldsymbol{\Sigma}_r\boldsymbol{V}_r^\top$ für Effizienz $(r = \operatorname{rang}(\boldsymbol{A}))$
5. *Pseudoinverse*: $\boldsymbol{A}^+ = \boldsymbol{V}_r\boldsymbol{\Sigma}_r^{-1}\boldsymbol{U}_r^\top$

Anwendungen: Lösung von *Kleinste-Quadrate-Problemen*, optimale *Low-Rank Approximation* (Datenkompression, -analyse)

Praktische Hinweise:

- SVD **nicht** über $\boldsymbol{A}^\top \boldsymbol{A}$ berechnen (numerisch instabil) – spezielle Algorithmen (z.B. Golub-Reinsch) nutzen: in R `svd(A)`
- *truncated SVD* für sehr große Matrizen: `irlba::irlba(A, nv = k)` (iterativ, Lanczos) bzw. `irlba::svdr(A, k)` (randomisiert)

Self-Check

Welche Aussagen sind korrekt?

1. Wenn $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, dann ist $\boldsymbol{A}^\top \boldsymbol{A} \in \mathbb{R}^{m \times m}$
2. Wenn $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, dann ist $\boldsymbol{A}^\top \boldsymbol{A}$ symmetrisch
3. Die Eigenwerte von $\boldsymbol{A}^\top \boldsymbol{A}$ sind die Singulärwerte von $\boldsymbol{A}$
4. Wenn $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, dann ist $\boldsymbol{A}$ immer diagonalisierbar
5. $\boldsymbol{A}^\top \boldsymbol{A}$ ist immer orthogonal diagonalisierbar

Anhang

Orthogonalität der Singulärvektoren

Warum sind die $\sigma_i \boldsymbol{u}_i = \boldsymbol{A}\boldsymbol{v}_i$ orthogonal?

Für $i \neq j$:

$$\begin{aligned}(\boldsymbol{A}\boldsymbol{v}_i)^\top (\boldsymbol{A}\boldsymbol{v}_j) &= \boldsymbol{v}_i^\top \left(\boldsymbol{A}^\top \boldsymbol{A}\right) \boldsymbol{v}_j \\ &= \boldsymbol{v}_i^\top (\lambda_j \boldsymbol{v}_j) \\ &= \lambda_j \left(\boldsymbol{v}_i^\top \boldsymbol{v}_j\right) \\ &= \lambda_j \cdot 0 = 0\end{aligned}$$

Für $i = j$:

$$\|\boldsymbol{A}\boldsymbol{v}_i\|^2 = \boldsymbol{v}_i^\top \left(\boldsymbol{A}^\top \boldsymbol{A}\right) \boldsymbol{v}_i = \lambda_i \|\boldsymbol{v}_i\|^2 = \lambda_i = \sigma_i^2$$

$\Longrightarrow$ Für $i, j \leq r$ (also $\sigma_i, \sigma_j > 0$) ist $\boldsymbol{u}_i^\top \boldsymbol{u}_j = \frac{(\boldsymbol{A}\boldsymbol{v}_i)^\top (\boldsymbol{A}\boldsymbol{v}_j)}{\sigma_i \sigma_j} = 0$ für $i \neq j$ bzw. $= 1$ für $i = j$: Die $\boldsymbol{u}_1, \ldots, \boldsymbol{u}_r$ sind orthonormal! *(Die übrigen $\boldsymbol{u}_{r+1}, \ldots, \boldsymbol{u}_m$ werden als orthonormale Ergänzung gewählt.)*

↩ zurück: „Definition: Singulärvektoren"

Bsp: Singulärvektoren – vollständige Rechnung (1)

Matrix $\boldsymbol{A} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix}$: $\quad \boldsymbol{A}^\top \boldsymbol{A}$ mit $\boldsymbol{\lambda} \approx (9.53, 1.47)$:

Rechte Singulärvektoren (Eigenvektoren von $\boldsymbol{A}^\top \boldsymbol{A}$):

Für $\lambda_1 \approx 9.53$:

$$\begin{pmatrix} 6 & 4 \\ 4 & 5 \end{pmatrix} \begin{pmatrix} v_1^{(1)} \\ v_2^{(1)} \end{pmatrix} = 9.53 \begin{pmatrix} v_1^{(1)} \\ v_2^{(1)} \end{pmatrix}$$

Lösung: $\boldsymbol{v}_1 \approx \begin{pmatrix} -0.75 \\ -0.662 \end{pmatrix}$

Für $\lambda_2 \approx 1.47$: $\boldsymbol{v}_2 \approx \begin{pmatrix} -0.662 \\ 0.75 \end{pmatrix}$ (normiert, orthogonal zu $\boldsymbol{v}_1$)

Bsp: Singulärvektoren – vollständige Rechnung (2)

Linke Singulärvektoren:

$$\boldsymbol{u}_1 = \frac{\boldsymbol{A}\boldsymbol{v}_1}{\sigma_1} \approx \begin{pmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} -0.75 \\ -0.662 \end{pmatrix} / 3.09 = \begin{pmatrix} -0.672 \\ -0.7 \\ -0.243 \end{pmatrix}, \qquad \text{analog: } \boldsymbol{u}_2 = \frac{\boldsymbol{A}\boldsymbol{v}_2}{\sigma_2} \approx \begin{pmatrix} 0.691 \\ -0.474 \\ -0.546 \end{pmatrix}$$

Verifikation Orthonormalität:

- $\boldsymbol{u}_1^\top \boldsymbol{u}_2 \approx (-0.672)(0.691) + (-0.7)(-0.474) + (-0.243)(-0.546) = -0.464 + 0.332 + 0.133 \approx 0$ ✓
- $\|\boldsymbol{u}_1\| \approx \sqrt{0.672^2 + 0.7^2 + 0.243^2} = \sqrt{0.452 + 0.49 + 0.059} \approx 1$ ✓

↩ zurück: „Bsp: Singulärvektoren"

Beweis des SVD-Theorems (Skizze)

Konstruktion der Zerlegung:

1. *Rechte Singulärvektoren*: Eigenvektoren von $\boldsymbol{A}^\top \boldsymbol{A}$

$$\boldsymbol{V} = (\boldsymbol{v}_1 \mid \boldsymbol{v}_2 \mid \cdots \mid \boldsymbol{v}_n)$$

2. *Linke Singulärvektoren*: $\boldsymbol{u}_i = \boldsymbol{A}\boldsymbol{v}_i / \sigma_i$ für $i \leq r$

$$\boldsymbol{U}_r = (\boldsymbol{u}_1 \mid \boldsymbol{u}_2 \mid \cdots \mid \boldsymbol{u}_r)$$

3. *Vervollständigung*: Erweitere $\boldsymbol{U}_r$ zu Orthogonalbasis des $\mathbb{R}^m$
4. *Verifikation*: $\boldsymbol{A}\boldsymbol{v}_i = \sigma_i \boldsymbol{u}_i$ für $i \leq r$, $\boldsymbol{A}\boldsymbol{v}_i = \boldsymbol{0}$ für $i > r$

↩ zurück: „Der SVD-Hauptsatz”

Typische Singulärwert-Verläufe

1. Schneller Abfall: $\sigma_i \approx c \cdot e^{-\alpha i}$

- Gut geeignet für Low-Rank Approximation
- Wenige dominante Singulärwerte

2. Linearer Abfall: $\sigma_i \approx c - \alpha i$

- Moderate Approximation möglich
- Mittlerer Kompressionsfaktor

3. Langsamer Abfall: $\sigma_i \approx c \cdot i^{-\alpha}$

- Schwierige Approximation
- Viele wichtige Komponenten

↩ zurück: „Wahl der optimalen Rang-k Approximation”

Empfehlungssysteme: Details

Interpretation der Faktoren:

- **Rechte Singulärvektoren $\boldsymbol{v}_j$**: “gelernte” Film-Genres
  - $\boldsymbol{v}_1$ könnte “Action vs. Drama” kodieren, $\boldsymbol{v}_2$ “Komödie vs. Thriller”, etc
- **Linke Singulärvektoren $\boldsymbol{u}_i$**: Nutzer-Präferenzen in diesen Genre-Dimensionen
- **Singulärwerte σ_i**: Wichtigkeit dieser Genre-Dimension

Vorteile:

- Reduziert Speicher: $k + (m + n) \cdot k \ll m \cdot n$ für $k \ll \min(m, n)$
- “Glättet” die Daten (Low-Rank = *Regularisierung*)
- Kann auch bei sehr spärlichen Daten funktionieren

Modernere Varianten:

- *Matrix Completion:* Low-Rank-Faktorisierung nur über die *beobachteten* Einträge; mit Nichtnegativitäts-Nebenbedingung: *Non-negative Matrix Factorization* (NMF)

↩ zurück: „Anwendung: Empfehlungssysteme”

Methode der Kleinsten Quadrate

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- *Kondition* & Konditionszahl $\kappa(\boldsymbol{A})$
- *Matrixnormen* (besonders $\|\cdot\|_2$) und *Orthogonalmatrizen*
- Matrixzerlegungen: *LU-* und *Cholesky-Zerlegung*
- *SVD*: $\boldsymbol{A} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^\top$, *Pseudoinverse* $\boldsymbol{A}^+$, Reduzierte SVD
- **Lineare Algebra**
  - *Normalengleichungen*: $\boldsymbol{A}^\top\boldsymbol{A}\boldsymbol{x} = \boldsymbol{A}^\top\boldsymbol{b}$
  - (Orthogonale) Projektionen auf Unterräume
- **Statistik**
  - Lineares Regressionsmodell $\boldsymbol{y} = \boldsymbol{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$
  - Methode der Kleinsten Quadrate (OLS)

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ mit $\operatorname{rang}(\boldsymbol{A}) = n$. Die Kleinste-Quadrate-Lösung $\hat{\boldsymbol{x}}$ von $\min_{\boldsymbol{x}} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|_2$ erfüllt …

1. $\boldsymbol{A}\hat{\boldsymbol{x}} = \boldsymbol{b}$
2. $\boldsymbol{A}^\top \boldsymbol{A}\hat{\boldsymbol{x}} = \boldsymbol{A}^\top \boldsymbol{b}$
3. $\boldsymbol{A}\boldsymbol{A}^\top \hat{\boldsymbol{x}} = \boldsymbol{A}\boldsymbol{b}$
4. $\hat{\boldsymbol{x}} = \boldsymbol{A}^\top \boldsymbol{b}$

Frage 2: Sei $\hat{\boldsymbol{b}} = \operatorname{proj}_{\operatorname{col}(\boldsymbol{A})} \boldsymbol{b}$ die orthogonale Projektion von $\boldsymbol{b}$ auf den Spaltenraum von $\boldsymbol{A}$. Welche Aussage gilt **immer**?

1. $\hat{\boldsymbol{b}} = \boldsymbol{b}$
2. $\boldsymbol{b} - \hat{\boldsymbol{b}} \perp \operatorname{col}(\boldsymbol{A})$
3. $\|\hat{\boldsymbol{b}}\|_2 \geq \|\boldsymbol{b}\|_2$
4. $\hat{\boldsymbol{b}} = \boldsymbol{0}$

Recap: Kleinste Quadrate

Recap: Kleinste Quadrate

Kleinste-Quadrate-Problem (KQ-Problem)

- Gegeben sei Matrix $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ und Vektor $\boldsymbol{b} \in \mathbb{R}^m$.
- Wir wollen einen Vektor $\hat{\boldsymbol{x}} \in \mathbb{R}^n$ finden, sodass $\left\| \boldsymbol{A}\hat{\boldsymbol{x}} - \boldsymbol{b} \right\|$ minimal ist.

Anmerkung:

- Hat das LGS eine Lösung $\boldsymbol{x}^*$, dann gilt $\left\| \boldsymbol{A}\boldsymbol{x}^* - \boldsymbol{b} \right\| = 0$ und $\boldsymbol{x}^*$ ist automatisch auch eine Lösung des Kleinste-Quadrate-Problems.
- Interessant ist die Problemstellung insbesondere für "Gleichungssysteme" $\boldsymbol{A}\boldsymbol{x}^* \approx \boldsymbol{b}$, die keine *exakte* Lösung besitzen.

Motivation

KQ-Methode *fundamentalst* in Statistik und Machine Learning:

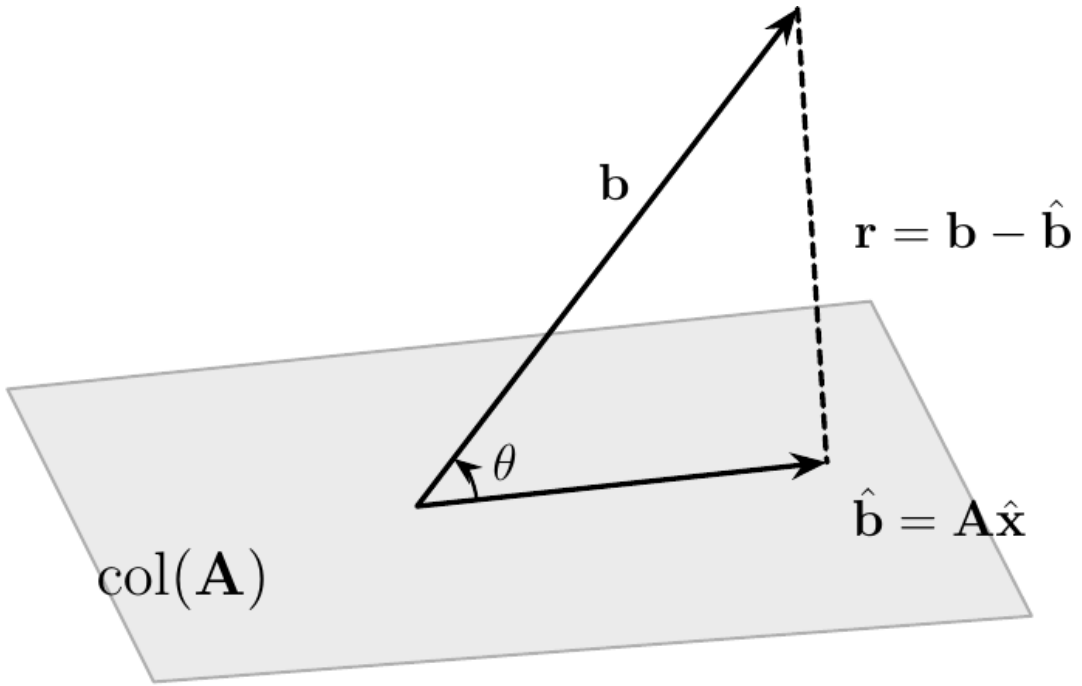
- **Beispiel: Lineare Regression**
  - **Problem:** Vorhersage von Immobilienpreisen basierend auf Wohnfläche
  - **Daten:** n Beobachtungen (x_i, y_i) mit x_i = Fläche in m^2, y_i = Preis in €
  - **Modell:** $y_i = \beta_0 + \beta_1 x_i + \epsilon_i$ mit Fehlerterm ϵ_i
  - **Matrix-Form:** $\boldsymbol{b} = \boldsymbol{A}\boldsymbol{\beta} + \boldsymbol{\epsilon}$ mit

$$\boldsymbol{A} = \begin{pmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{pmatrix}, \quad \boldsymbol{b} = \begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix}, \quad \boldsymbol{\beta} = \begin{pmatrix} \beta_0 \\ \beta_1 \end{pmatrix}$$

  - **Lösung:** $\hat{\boldsymbol{\beta}} = (\boldsymbol{A}^\top \boldsymbol{A})^{-1} \boldsymbol{A}^\top \boldsymbol{b}$ minimiert $\sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2$

- **Weitere Anwendungen:**
  - **Generalisierte Lineare Modelle, Additive Modelle:** approximative Lösungen über iterative KQ-Probleme
  - **Deep Learning:** Training neuronaler Netze (minimiert oft KQ-Fehler)
  - **Computer Vision:** Kamerakalibration, 3D-Rekonstruktion aus 2D-Bildern
  - **Signal Processing:** Rauschreduktion/De-Noising, Signalschätzung
  - **Geodäsie:** GPS-Positionsbestimmung
  - ....

Recap: Kleinste Quadrate



Theorem

Die Lösungsmenge des KQ-Problems gleicht der Lösungsmenge des LGS

$$\boldsymbol{A}\hat{\boldsymbol{x}} = \hat{\boldsymbol{b}} \quad \text{mit } \hat{\boldsymbol{b}} = \mathrm{proj}_{\mathrm{col}(\boldsymbol{A})}\boldsymbol{b}.$$

Recap: Kleinste Quadrate

Theorem

Die Lösungsmenge des KQ-Problems gleicht der Lösungsmenge der Normalengleichungen

$$\boldsymbol{A}^\top \boldsymbol{A}\hat{\boldsymbol{x}} = \boldsymbol{A}^\top \boldsymbol{b}.$$

Theorem

Falls $\operatorname{rang}(\boldsymbol{A}) = n$, ist $\boldsymbol{A}^\top \boldsymbol{A}$ invertierbar und das KQ-Problem hat die eindeutige Lösung

$$\hat{\boldsymbol{x}} = (\boldsymbol{A}^\top \boldsymbol{A})^{-1} \boldsymbol{A}^\top \boldsymbol{b} =: \boldsymbol{A}^+ \boldsymbol{b}.$$

Quiz

Welche der folgenden Aussagen sind wahr? *(mehrere Antworten möglich)*

1. $\boldsymbol{A}\boldsymbol{A}^{+} = \boldsymbol{I}_m$
2. $\boldsymbol{A}\boldsymbol{A}^{+}\boldsymbol{b} = \text{proj}_{\text{col}(\boldsymbol{A})}\boldsymbol{b}$
3. $\boldsymbol{r} = \boldsymbol{b} - \boldsymbol{A}\hat{\boldsymbol{x}} \perp \text{col}(\boldsymbol{A})$
4. $\boldsymbol{b} = \text{proj}_{V}\boldsymbol{b} + \text{proj}_{V^{\perp}}\boldsymbol{b}$ für jeden Untervektorraum V

Kondition des KQ-Problems

Kondition

Vorbemerkungen

- Wir untersuchen nun die *Kondition* von KQ-Probleme $\boldsymbol{Ax} \approx \boldsymbol{b}$ mit $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ und $\boldsymbol{b} \in \mathbb{R}^m$.
- Wir beschränken uns auf den Fall $m \geq n$ und $\operatorname{rang}(\boldsymbol{A}) = n$
  $\Longrightarrow$ eindeutige Lösung existiert.
- Im folgenden $||\cdot|| = ||\cdot||_2$ (natürliche Norm für KQ-Probleme).
- Für nicht notwendigerweise quadratische Matrizen definieren wir

$$\kappa(\boldsymbol{A}) = \begin{cases} \infty, & \operatorname{rang}(\boldsymbol{A}) < n, \\ \|\boldsymbol{A}\| \, \|\boldsymbol{A}^+\|, & \text{sonst.} \end{cases}$$

Kondition bezüglich $\boldsymbol{b}$

Theorem

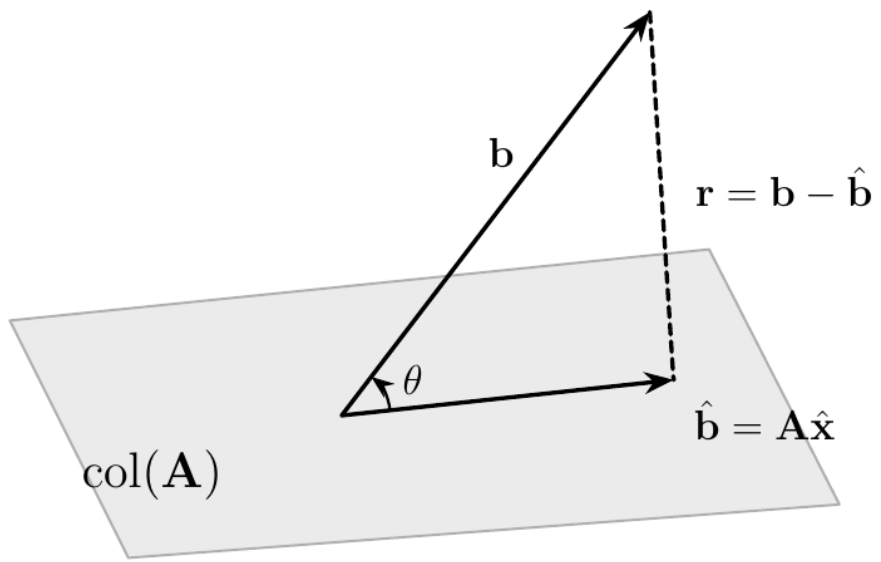
Sei $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ fix und f das Problem mit Lösung $f(\boldsymbol{b}) = \boldsymbol{A}^{+}\boldsymbol{b}$. Dann gilt

$$\frac{\|f(\tilde{\boldsymbol{b}}) - f(\boldsymbol{b})\|}{\|f(\boldsymbol{b})\|} \leq \kappa \frac{\|\tilde{\boldsymbol{b}} - \boldsymbol{b}\|}{\|\boldsymbol{b}\|} \qquad \text{mit} \qquad \kappa = \kappa_2(\boldsymbol{A}) \frac{\|\boldsymbol{b}\|}{\left\|\text{proj}_{\text{col}(\boldsymbol{A})}\boldsymbol{b}\right\|}.$$

Beweis: siehe Anhang

Kondition bezüglich b

$$\kappa = \kappa_2(A) \frac{\|b\|}{\left\|\text{proj}_{\text{col}(A)} b\right\|}$$



Interpretation

- Erinnerung:

$$\|b\|^2 = \left\|\text{proj}_{\text{col}(A)} b\right\|^2 + \left\|\text{proj}_{\text{col}(A)^\perp} b\right\|^2 \geq \left\|\text{proj}_{\text{col}(A)} b\right\|^2.$$

- **Frage:** Was sind best und worst case für κ?

Kondition bezüglich $\boldsymbol{A}$

Theorem

Sei $\boldsymbol{b} \in \mathbb{R}^m$ fix und f das Problem mit Lösung $f(\boldsymbol{A}) = \boldsymbol{A}^+\boldsymbol{b}$. Dann gilt für kleine Störungen $\tilde{\boldsymbol{A}} \to \boldsymbol{A}$ (in erster Ordnung)

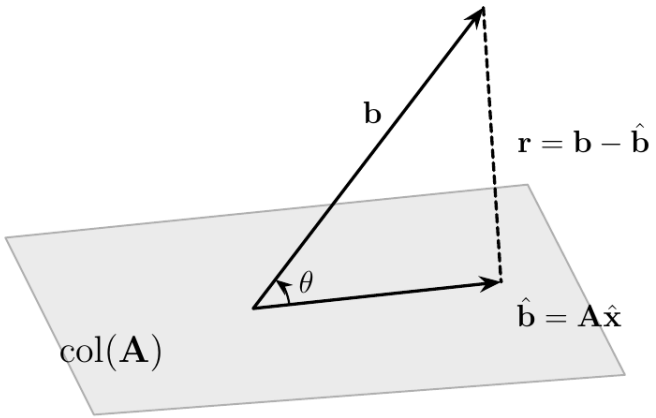
$$\frac{\|f(\tilde{\boldsymbol{A}}) - f(\boldsymbol{A})\|}{\|f(\boldsymbol{A})\|} \lesssim \kappa \frac{\|\tilde{\boldsymbol{A}} - \boldsymbol{A}\|}{\|\boldsymbol{A}\|}$$

$$\text{mit} \quad \kappa = \kappa_2(\boldsymbol{A}) + \kappa_2(\boldsymbol{A})^2 \frac{\|\boldsymbol{r}\|}{\left\|\operatorname{proj}_{\operatorname{col}(\boldsymbol{A})} \boldsymbol{b}\right\|}.$$

Beweis: Sehr aufwändige und langweilige Rechnung.

Kondition bezüglich A

$$\kappa = \kappa_2(A) + \kappa_2(A)^2 \frac{\|r\|}{\left\|\text{proj}_{\text{col}(A)} b\right\|}$$



Interpretation

- Best case: $r = 0 \Rightarrow Ax = b$ ist lösbar $\Rightarrow \kappa = \kappa_2(A)$.
- Worst case: $\text{proj}_{\text{col}(A)} b = 0$, aber $r \neq 0$
  - $\Rightarrow Ax = b$ ist nicht lösbar und $b \perp \text{col}(A)$
  - $\Rightarrow \kappa = \infty$.
- Normalerweise: $r \neq 0$ und $\text{proj}_{\text{col}(A)} b \neq 0 \Rightarrow \kappa = O(\kappa_2(A)^2)$.

Lösungsverfahren

Frage

Um die Lösung $\hat{\boldsymbol{x}} = (\boldsymbol{A}^\top \boldsymbol{A})^{-1} \boldsymbol{A}^\top \boldsymbol{b}$ zu berechnen, sei folgende Funktion gegeben:

```r
kq_solve <- function(A, b) {
  solve(t(A) %*% A) %*% t(A) %*% b
}
```

Aufgabe: Finde den Fehler in der Funktion.

Invertiere niemals eine Matrix!

`solve(M)` berechnet die volle Inverse — teurer *und* instabiler, als das LGS $\boldsymbol{M}\boldsymbol{x} = \boldsymbol{c}$ direkt zu lösen.

besser: `solve(M, c)`.

Cholesky-Zerlegung

- Die Normalengleichung $\boldsymbol{A}^\top \boldsymbol{A}\boldsymbol{x} = \boldsymbol{A}^\top \boldsymbol{b}$ gibt uns ein einfaches Lösungsverfahren:
  1. Berechne $\boldsymbol{M} = \boldsymbol{A}^\top \boldsymbol{A}$ und $\boldsymbol{c} = \boldsymbol{A}^\top \boldsymbol{b}$.
  2. Berechne Cholesky-Zerlegung $\boldsymbol{M} = \boldsymbol{L}\boldsymbol{L}^\top$.
  3. Löse $\boldsymbol{L}\boldsymbol{y} = \boldsymbol{c}$ durch Vorwärtssubstitution.
  4. Löse $\boldsymbol{L}^\top \boldsymbol{x} = \boldsymbol{y}$ durch Rückwärtssubstitution.
- Das Verfahren ist effizient, aber möglicherweise instabil:
  - Fehler in $\boldsymbol{A}^\top \boldsymbol{A}$ beeinflussen alle folgenden Schritte.
  - Man kann zeigen, dass der relative algorithmische Fehler typischerweise von der Größenordnung $\kappa_2(\boldsymbol{A})^2 \, {\epsilon_{mach}}^1$ ist.
  - Vergleiche mit Best-Case-Kondition $O(\kappa_2(\boldsymbol{A}))$.

Alternative Lösungswege

- Wir würden gerne $\boldsymbol{A}^\top \boldsymbol{A}$ vermeiden und direkt auf $\boldsymbol{A}$ operieren.
- Erinnerung: KQ-Lösung zu $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ ist

$$\widehat{\boldsymbol{x}} = \arg\min_{\boldsymbol{x}} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|_2$$

- Wir wollen das Problem mit einer Matrix $\boldsymbol{Q}$ zu einem "einfacheren" Problem $\boldsymbol{Q}\boldsymbol{A}\boldsymbol{x} \approx \boldsymbol{Q}\boldsymbol{b}$ transformieren ohne die Lösung zu ändern.

$\Longrightarrow$ $\boldsymbol{Q}$ soll *Orthogonalmatrix* sein, denn für diese gilt

$$\|\boldsymbol{Q}\boldsymbol{A}\boldsymbol{x} - \boldsymbol{Q}\boldsymbol{b}\| = \|\boldsymbol{Q}(\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b})\| = \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|.$$

Erinnerung: ***Orthogonalmatrix*** $\boldsymbol{Q}$

- $\boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$, $\boldsymbol{Q}^{-1} = \boldsymbol{Q}^\top$ ("Rotation/Spiegelung");
- Produkte von Orthogonalmatrizen sind orthogonal.
- Normerhaltend für $\|\cdot\|_2$: $\|\boldsymbol{Q}\boldsymbol{z}\|_2 = \sqrt{\boldsymbol{z}^\top \boldsymbol{Q}^\top \boldsymbol{Q}\boldsymbol{z}} = \sqrt{\boldsymbol{z}^\top \boldsymbol{z}} = \|\boldsymbol{z}\|_2$ für alle $\boldsymbol{z} \in \mathbb{R}^m$.
- Alle Singulärwerte $\sigma_i(\boldsymbol{Q}) = 1$ (da $\boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$) $\Rightarrow \kappa_2(\boldsymbol{Q}) = \frac{\sigma_{\max}}{\sigma_{\min}} = 1$
  $\Longrightarrow$ Multiplikation mit $\boldsymbol{Q}$ ist immer gut konditioniert.

QR-Zerlegung

Def.: QR-Zerlegung

Sei $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ mit $m \geq n$ und $\boldsymbol{Q} \in \mathbb{R}^{m \times m}$ eine *Orthogonalmatrix*, sodass

$$\boldsymbol{Q}^\top \boldsymbol{A} = \begin{pmatrix} \boldsymbol{R} \\ \mathbf{0}_{(m-n) \times n} \end{pmatrix},$$

wobei $\boldsymbol{R} \in \mathbb{R}^{n \times n}$ eine *obere Dreiecksmatrix* ist. Dann nennen wir

$$\boldsymbol{A} = \boldsymbol{Q} \begin{pmatrix} \boldsymbol{R} \\ \mathbf{0} \end{pmatrix}$$

eine **QR-Zerlegung** von $\boldsymbol{A}$.

Anmerkung:

- Es gibt immer eine QR-Zerlegung von $\boldsymbol{A}$.
- Die Zerlegung ist nicht eindeutig.

Beispiel: QR-Zerlegung

Gegeben:

$$\boldsymbol{A} = \begin{pmatrix} 1 & 2 \\ 0 & 0 \\ 0 & 2 \end{pmatrix}$$

Gesucht:

$\boldsymbol{Q}^\top \boldsymbol{A} = \begin{pmatrix} \boldsymbol{R} \\ \boldsymbol{0}_{1\times 2} \end{pmatrix}$ mit $\boldsymbol{Q} \in \mathbb{R}^{3\times 3}$ und $\boldsymbol{R} \in \mathbb{R}^{2\times 2}$

Methode:

Gram-Schmidt-Orthonormalisierung auf den Spalten von $\boldsymbol{A}$

Schritt 1: Erste Spalte $\boldsymbol{a}_1 = \begin{pmatrix} 1 & 0 & 0 \end{pmatrix}^\top$

- Normalisierung: $\boldsymbol{q}_1 = \frac{\boldsymbol{a}_1}{\|\boldsymbol{a}_1\|} = \begin{pmatrix} 1 & 0 & 0 \end{pmatrix}^\top \implies R_{11} = \|\boldsymbol{a}_1\| = 1$

Schritt 2: Zweite Spalte $\boldsymbol{a}_2 = \begin{pmatrix} 2 & 0 & 2 \end{pmatrix}^\top$

- Projektion auf $\boldsymbol{q}_1$: $R_{12} = \boldsymbol{q}_1{}^\top \boldsymbol{a}_2 = 2$
- Orthogonalisierung: $\tilde{\boldsymbol{q}}_2 = \boldsymbol{a}_2 - R_{12}\boldsymbol{q}_1 = \begin{pmatrix} 2 & 0 & 2 \end{pmatrix}^\top - \begin{pmatrix} 2 & 0 & 0 \end{pmatrix}^\top = \begin{pmatrix} 0 & 0 & 2 \end{pmatrix}^\top$
- Normalisierung: $\boldsymbol{q}_2 = \frac{\tilde{\boldsymbol{q}}_2}{\|\tilde{\boldsymbol{q}}_2\|} = \begin{pmatrix} 0 & 0 & 1 \end{pmatrix}^\top$, $R_{22} = \|\tilde{\boldsymbol{q}}_2\| = 2$

Schritt 3: Vervollständige zu Orthonormalbasis von $\mathbb{R}^3$: wähle $\boldsymbol{q}_3 \perp \boldsymbol{q}_1, \boldsymbol{q}_2$, z.B. $\boldsymbol{q}_3 = \begin{pmatrix} 0 & 1 & 0 \end{pmatrix}^\top$

Ergebnis: $\boldsymbol{Q} = (\boldsymbol{q}_1 \quad \boldsymbol{q}_2 \quad \boldsymbol{q}_3) = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad \boldsymbol{R} = \begin{pmatrix} 1 & 2 \\ 0 & 2 \end{pmatrix}$ *(Verifikation → Anhang)*

QR-Zerlegung: Idee

Theorem

Sei $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ mit $m \geq n$, $\boldsymbol{b} \in \mathbb{R}^m$ und $\boldsymbol{Q} \in \mathbb{R}^{m \times m}$ eine *Orthogonalmatrix*, sodass

$$\boldsymbol{Q}^\top \boldsymbol{A} = \begin{pmatrix} \boldsymbol{R} \\ \boldsymbol{0}_{(m-n)\times n} \end{pmatrix} \quad \text{und} \quad \boldsymbol{Q}^\top \boldsymbol{b} = \begin{pmatrix} \boldsymbol{c}_1 \\ \boldsymbol{c}_2 \end{pmatrix}$$

mit $\boldsymbol{R} \in \mathbb{R}^{n \times n}$ invertierbar. Dann gilt

$$\widehat{\boldsymbol{x}} = \boldsymbol{R}^{-1}\boldsymbol{c}_1 = \arg\min_{\boldsymbol{x}} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|_2.$$

Numerisch bilden wir $\boldsymbol{R}^{-1}$ **nicht** explizit, sondern lösen das Dreieckssystem $\boldsymbol{R}\widehat{\boldsymbol{x}} = \boldsymbol{c}_1$ durch Rückwärtssubstitution.

Beweis:

- Es gilt $\|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\|^2 = \left\|\boldsymbol{Q}^\top \boldsymbol{A}\boldsymbol{x} - \boldsymbol{Q}^\top \boldsymbol{b}\right\|^2 = \|\boldsymbol{R}\boldsymbol{x} - \boldsymbol{c}_1\|^2 + \|\boldsymbol{c}_2\|^2$.
- Weil der zweite Term nicht von $\boldsymbol{x}$ abhängt, gilt

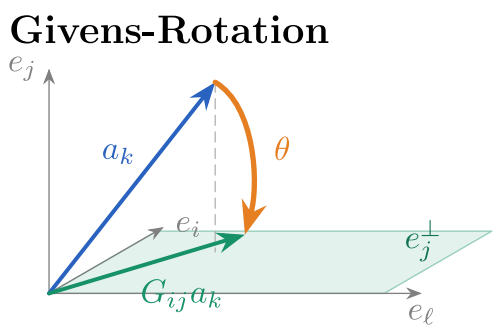
$$\arg\min_{\boldsymbol{x}} \|\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b}\| = \arg\min_{\boldsymbol{x}} \|\boldsymbol{R}\boldsymbol{x} - \boldsymbol{c}_1\|.$$

- $\widehat{\boldsymbol{x}} = \boldsymbol{R}^{-1}\boldsymbol{c}_1$ ist Lösung, weil $\left\|\boldsymbol{R}\widehat{\boldsymbol{x}} - \boldsymbol{c}_1\right\| = \left\|\boldsymbol{R}\boldsymbol{R}^{-1}\boldsymbol{c}_1 - \boldsymbol{c}_1\right\| = 0$.

Konstruktion von Q

- Wie beim Gauß-Verfahren erzeugen wir R durch Nullen unter der Diagonalen. Dazu multiplizieren wir A von links mit **orthogonalen** Transformationen; ihr Produkt ist $Q^\top$.
- In Spalte k sollen die $m-k$ Einträge unter a_{kk} null werden:

- *Givens-Rotation*:

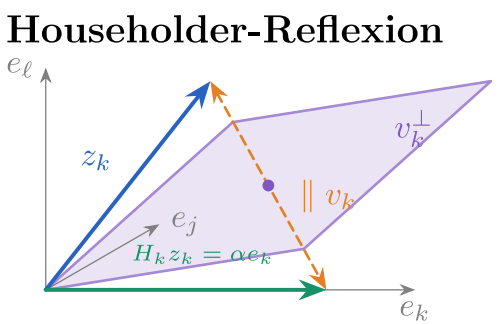


$G_{ij}(\theta)$ dreht die Projektion von a_k auf $\operatorname{span}(e_i, e_j)$ auf e_i.
So wird **ein** ausgewählter Eintrag a_{jk} null.

Bei üblicher Drehrichtung gilt

$$\theta = -\operatorname{atan2}(A_{jk}, A_{ik}).$$

- *Householder-Reflexion*:



Definiere den nach oben mit Nullen aufgefüllten Rest der k-ten Spalte von A: $z_k := \sum_{\ell=k}^{m} A_{\ell k} e_\ell$.
Spiegelebene $v_k^\perp$ halbiert den Winkel zwischen z_k und e_k.
Nach Reflexion gilt $a_{\ell k}^{\text{neu}} = 0$ **für alle** $\ell > k$, Einträge mit $\ell < k$ bleiben unverändert.

$$\text{Wähle } \alpha = \pm\|z_k\|_2, \quad z_k \neq \alpha e_k, \qquad v_k = \frac{z_k - \alpha e_k}{\|z_k - \alpha e_k\|_2},$$

$$H_k = I - 2 v_k v_k^\top, \qquad H_k z_k = \alpha e_k.$$

Pseudoinverse und Kleinste Quadrate

SVD-Lösung:

$$\boldsymbol{x} = \boldsymbol{A}^{+}\boldsymbol{b} = \boldsymbol{V}_r \boldsymbol{\Sigma}_r^{-1} \boldsymbol{U}_r^{\top} \boldsymbol{b}$$

Warum funktioniert das?

$$\boldsymbol{A}\boldsymbol{x} = \boldsymbol{A}\boldsymbol{A}^{+}\boldsymbol{b} = \mathrm{proj}_{\mathrm{col}(\boldsymbol{A})}\boldsymbol{b}$$

Vorteile gegenüber Normalengleichungen:

- Arbeitet direkt mit $\boldsymbol{A}$, nicht $\boldsymbol{A}^{\top}\boldsymbol{A} \rightarrow$ numerisch stabiler
- Funktioniert auch für rangdefiziente Matrizen ($\mathrm{rang}(\boldsymbol{A}) < n$) – liefert dann die KQ-Lösung mit minimaler Norm:

$$\boldsymbol{A}^{+}\boldsymbol{b} = \arg \min_{\substack{\boldsymbol{x} \in \mathbb{R}^n \\ \boldsymbol{A}^{\top}\boldsymbol{A}\boldsymbol{x} = \boldsymbol{A}^{\top}\boldsymbol{b}}} \|\boldsymbol{x}\|_2$$

- Explizite Konstruktion der Lösung, klare geometrische Interpretation

Demo: Ein KQ-Problem, drei Verfahren

Lineare Regression mit **fast kollinearen** Spalten — die wahre Lösung ist bekannt:

```r
set.seed(1)
m <- 100
x <- runif(m)
A <- cbind(1, x, x + 1e-6 * runif(m)) # 3. Spalte fast identisch zur 2.!
beta_wahr <- c(1, 2, 3)
b <- A %*% beta_wahr                  # exakte Daten: Loesung = beta_wahr
kappa(A, exact = TRUE)
```

```
[1] 6592188
```

- $\kappa_2(\boldsymbol{A}) \approx 7 \cdot 10^6 \implies \kappa_2(\boldsymbol{A}^\top \boldsymbol{A}) = \kappa_2(\boldsymbol{A})^2 \approx 4 \cdot 10^{13}$

Vorhersage: Doppelte Gleitkommazahlen haben ≈ 16 Dezimalstellen. Wie viele *korrekte* Stellen von $\boldsymbol{\beta}$ erwarten Sie

- für die Normalengleichungen (Fehler $\sim \kappa_2(\boldsymbol{A})^2 \, \epsilon_{mach}$)?
- für QR/SVD (Fehler $\sim \kappa_2(\boldsymbol{A}) \, \epsilon_{mach}$)?

Demo: Ergebnisse

```r
beta_ne  <- solve(t(A) %*% A, t(A) %*% b)  # Normalengleichungen (ohne Inverse!)
beta_qr  <- qr.solve(A, b)                 # QR-Zerlegung
s        <- svd(A)
beta_svd <- s$v %*% ((t(s$u) %*% b) / s$d) # SVD: A+ b
fehler   <- function(beta_hat) max(abs(beta_hat - beta_wahr))
sapply(list("Normalengl." = beta_ne, QR = beta_qr, SVD = beta_svd), fehler)
```

```
Normalengl.          QR         SVD
   8.77e-03    1.19e-11    1.55e-10
```

- **Normalengleichungen**:
  Fehler $\approx 10^{-2}$ –
  passt zu $\kappa_2(\boldsymbol{A})^2\,\epsilon_{mach} \approx 4 \cdot 10^{13} \cdot 2 \cdot 10^{-16} \approx 10^{-2}$
  $\Longrightarrow$ nur noch **2–3 korrekte Stellen**, obwohl wir die Matrixinversion vermieden haben:
  die *Quadrierung der Konditionszahl* ist das Problem, nicht (nur) `solve(M)`.
- **QR/SVD**:
  Fehler $\approx 10^{-10}$ – passt zu $\kappa_2(\boldsymbol{A})\,\epsilon_{mach} \approx 10^{-9}$
  $\Longrightarrow$ **ca. 10 korrekte Stellen** statt nur 2–3 bei den Normalengleichungen.

Vergleich der Lösungsverfahren

Verfahren	Ansatz	Komplexität ($m \gg n$)	Stabilität
Cholesky auf $\boldsymbol{A}^\top \boldsymbol{A}$	Normalengleichungen	$O(mn^2)$, kleinste Konstante ($\approx mn^2$)	Fehler $O\left(\kappa_2(\boldsymbol{A})^2\, \epsilon_{mach}\right)$ – potenziell instabil
QR-Zerlegung	löse $\boldsymbol{R}\hat{\boldsymbol{x}} = \boldsymbol{c}_1$	$O(mn^2)$ ($\approx 2mn^2$, Householder)	stabil – **Standardwahl**
SVD	$\hat{\boldsymbol{x}} = \boldsymbol{A}^+\boldsymbol{b} = \boldsymbol{V}_r \boldsymbol{\Sigma}_r^{-1} \boldsymbol{U}_r^\top \boldsymbol{b}$	$O(mn^2)$, größte Konstante	am stabilsten, auch für $\operatorname{rang}(\boldsymbol{A}) < n$

- Für $m \gg n$ haben alle drei Verfahren dieselbe Komplexitäts*ordnung* $O(mn^2)$ – die Konstanten (und die Stabilität!) unterscheiden sich aber deutlich: Normalengleichungen + Cholesky brauchen etwa halb so viele Operationen wie Householder-QR, die SVD ein Mehrfaches davon.
- SVD besonders sinnvoll, wenn $\boldsymbol{A}$ fast singulär ist ($\sigma_{\min}(\boldsymbol{A}) \approx 0 \Rightarrow \kappa(\boldsymbol{A}) \gg 1$): die $r - k$ kleinsten Singulärwerte auf 0 setzen $\rightarrow$ **Rang-k-Approximation** als Regularisierung.

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- *Kondition des KQ-Problems*:
  bzgl. $\boldsymbol{b}$ abhängig vom Winkel zwischen $\boldsymbol{b}$ und $\mathrm{col}(\boldsymbol{A})$,
  bzgl. $\boldsymbol{A}$ typischerweise $O(\kappa_2(\boldsymbol{A})^2)$.
- $\kappa_2(\boldsymbol{A}^\top \boldsymbol{A}) = \kappa_2(\boldsymbol{A})^2 \implies$ *Normalengleichungen* können instabil sein — und: **Invertiere niemals eine Matrix!**
- Drei Lösungsverfahren:
  - *Cholesky* (schnell, heikel),
  - *QR-Zerlegung* (Standardwahl),
  - *SVD* (am stabilsten, auch für rangdefiziente $\boldsymbol{A}$).

Nächste Vorlesung

- Probabilistische Methoden
- Iterative Methoden

Self-Check (1/2)

Frage 1: Welches Verfahren ist für KQ-Probleme mit $\operatorname{rang}(\boldsymbol{A}) = n$ die übliche **Standardwahl** (guter Kompromiss aus Stabilität und Geschwindigkeit)?

1. Cholesky auf $\boldsymbol{A}^\top \boldsymbol{A}$
2. QR-Zerlegung
3. SVD
4. Explizite Inversion: `solve(t(A) %*% A) %*% t(A) %*% b`

Frage 2: Für $m \gg n$ gilt für die Komplexität von Cholesky, QR und SVD:

1. Alle drei haben dieselbe Ordnung $O(mn^2 + n^3)$, aber deutlich verschiedene Konstanten.
2. Cholesky ist $O(mn)$, QR ist $O(mn^2)$, SVD ist $O(m^2 n^2)$.
3. Die SVD hat exponentielle Laufzeit.
4. QR ist immer am langsamsten.

Self-Check (2/2)

Frage 3: Die Kondition des KQ-Problems bezüglich $\boldsymbol{b}$ explodiert ($\kappa \to \infty$) genau dann, wenn …

1. $\boldsymbol{b} \in \mathrm{col}(\boldsymbol{A})$.
2. $\boldsymbol{b} \perp \mathrm{col}(\boldsymbol{A})$, d.h. $\mathrm{proj}_{\mathrm{col}(\boldsymbol{A})}\boldsymbol{b} = \boldsymbol{0}$.
3. das Residuum $\boldsymbol{r} = \boldsymbol{0}$ ist.
4. $\boldsymbol{A}$ eine Orthogonalmatrix ist.

Frage 4: Warum sind die Normalengleichungen numerisch heikel, selbst wenn man die explizite Inversion vermeidet?

1. $\boldsymbol{A}^\top \boldsymbol{A}$ ist nie invertierbar.
2. Vorwärts-/Rückwärtssubstitution ist instabil.
3. Der Übergang zu $\boldsymbol{A}^\top \boldsymbol{A}$ quadriert die Konditionszahl: $\kappa_2(\boldsymbol{A}^\top \boldsymbol{A}) = \kappa_2(\boldsymbol{A})^2$.
4. Die Cholesky-Zerlegung existiert für SPD-Matrizen nicht.

Anhang

Beweis: Kondition bezüglich $\boldsymbol{b}$

Behauptung: $\dfrac{\|f(\tilde{\boldsymbol{b}}) - f(\boldsymbol{b})\|}{\|f(\boldsymbol{b})\|} \leq \kappa_2(\boldsymbol{A}) \dfrac{\|\boldsymbol{b}\|}{\left\|\mathrm{proj}_{\mathrm{col}(\boldsymbol{A})}\boldsymbol{b}\right\|} \cdot \dfrac{\|\tilde{\boldsymbol{b}} - \boldsymbol{b}\|}{\|\boldsymbol{b}\|}$ für $f(\boldsymbol{b}) = \boldsymbol{A}^+\boldsymbol{b}$.

$$
\begin{aligned}
\frac{\left\|\boldsymbol{A}^+\tilde{\boldsymbol{b}} - \boldsymbol{A}^+\boldsymbol{b}\right\|}{\|\boldsymbol{A}^+\boldsymbol{b}\|} &= \frac{\left\|\boldsymbol{A}^+(\tilde{\boldsymbol{b}} - \boldsymbol{b})\right\|}{\|\boldsymbol{A}^+\boldsymbol{b}\|} \\
&\leq \frac{\|\boldsymbol{A}^+\| \left\|\tilde{\boldsymbol{b}} - \boldsymbol{b}\right\|}{\|\boldsymbol{A}^+\boldsymbol{b}\|} && [\text{Submultiplikativität}] \\
&= \frac{\kappa_2(\boldsymbol{A})}{\|\boldsymbol{A}\| \cdot \|\boldsymbol{A}^+\boldsymbol{b}\|} \left\|\tilde{\boldsymbol{b}} - \boldsymbol{b}\right\| && [\text{Def. von } \kappa_2(\boldsymbol{A})] \\
&\leq \frac{\kappa_2(\boldsymbol{A})}{\|\boldsymbol{A}\boldsymbol{A}^+\boldsymbol{b}\|} \left\|\tilde{\boldsymbol{b}} - \boldsymbol{b}\right\| && [\text{Submultiplikativität}]
\end{aligned}
$$

und mit $\boldsymbol{A}\boldsymbol{A}^+\boldsymbol{b} = \mathrm{proj}_{\mathrm{col}(\boldsymbol{A})}\boldsymbol{b}$ sowie Erweitern mit $\|\boldsymbol{b}\|/\|\boldsymbol{b}\|$ folgt die Behauptung. ■

↩ zurück: „Kondition bezüglich $\boldsymbol{b}$“

Beispiel: QR-Zerlegung — Verifikation

Aus dem Hauptteil: $\boldsymbol{A} = \begin{pmatrix} 1 & 2 \\ 0 & 0 \\ 0 & 2 \end{pmatrix}$, berechnet via Gram-Schmidt:

- $\boldsymbol{q}_1 = (1 \quad 0 \quad 0)^\top$; $\boldsymbol{q}_2 = (0 \quad 0 \quad 1)^\top$; $\boldsymbol{q}_3 = (0 \quad 1 \quad 0)^\top$
- $R_{11} = 1$; $R_{12} = 2$; $R_{22} = 2$

Verifikation:

$$\boldsymbol{Q} \begin{pmatrix} \boldsymbol{R} \\ \boldsymbol{0} \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 0 & 2 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 0 & 0 \\ 0 & 2 \end{pmatrix} = \boldsymbol{A} \checkmark$$

und $\boldsymbol{Q}$ ist orthogonal: $\boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}_3$ (Spalten sind orthonormale Vektoren). ✓

↩ zurück: „Beispiel: QR-Zerlegung”

Numerische Lineare Algebra: Iteration & Zufall

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- *Komplexitätsanalyse* / $O()$-Notation
- *Stabilität* und *Konditionierung*
- *Orthogonalmatrizen*
- *Spektralnorm* $\|\cdot\|_2$
- *LGS*: Problem, Direkte Algorithmen
- **Lineare Algebra:**
  - Eigenwerte und Eigenvektoren
  - Ähnlichkeitstransformationen: $\boldsymbol{B} = \boldsymbol{Q}\boldsymbol{A}\boldsymbol{Q}^{-1}$
- **Wahrscheinlichkeitstheorie**: Zufallsvektoren
- **Analysis I**:
  - Konvergenz von Folgen
  - Geometrische Reihe $\sum_k p^k$ für $|p| < 1$

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ und $\boldsymbol{v} \neq \boldsymbol{0}$ ein Eigenvektor mit $\boldsymbol{A}\boldsymbol{v} = \lambda \boldsymbol{v}$. Welche Aussage gilt dann **immer**?

1. $\boldsymbol{A}^k \boldsymbol{v} = \lambda^k \boldsymbol{v}$ für alle $k \in \mathbb{N}$
2. Es muss $\|\boldsymbol{v}\| = 1$ gelten
3. $\lambda > 0$
4. $\boldsymbol{v}$ ist der einzige Eigenvektor zu λ

Frage 2: Sei $|p| < 1$. Was ist $\sum_{k=0}^{\infty} p^k$?

1. $\frac{1}{1-p}$
2. $\frac{p}{1-p}$
3. $\frac{1}{p-1}$
4. Die Reihe divergiert.

Überblick

- Numerische lineare Algebra ist ein sehr großes Feld.
- Wir haben bisher die wichtigsten Lösungsmethoden gesehen.
- Die Kernidee dabei war *Matrixzerlegung* (*LU*, *QR*, *Cholesky*, ...).
- Heute sehen wir in kürzerer Form zwei weitere wichtige Kernideen:
  - *Iterative Methoden*
    - verkürzen *Laufzeit* auf Kosten von Genauigkeit.
  - *Probabilistische Methoden*
    - ersetzen große Probleme durch kleinere mit *höchstwahrscheinlich* sehr ähnlicher Lösung.
- An beiden wird in der numerischen linearen Algebra aktiv geforscht.

Eigenwertprobleme

Lösungsverfahren für Eigenwertprobleme

- *Direkte Methoden existieren:*
  - Charakteristisches Polynom: $\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = 0 \rightarrow$ Nullstellensuche
  - Problem:
    Polynom vom Grad n für $n \times n$ Matrix $\rightarrow$ keine direkte Lösung für $n \geq 5$
  - Numerisch instabil und teuer: $O(n^4)$ oder schlechter
- *Wann sind direkte Methoden unzureichend?*
  - Große Matrizen ($n > 1000$): $O(n^3)$ oder schlechter
  - Dünnbesetzte Matrizen: Direkte Methoden zerstören *sparsity*-Struktur
  - Oft brauchen wir nur wenige Eigenwerte (z.B. die größten $k \ll n$)
- *Warum iterativ?*
  - Schneller wenn nur wenige Eigenwerte benötigt
  - Nutzen *sparsity*-Struktur aus (nur Matrix-Vektor-Multiplikationen!)
  - Trade-off “Genauigkeit vs. Aufwand” ist steuerbar

Potenzmethode

Die **Potenzmethode** (nach Mises) liefert über Iteration den größten (separierten!) Eigenwert λ_1 und Eigenvektor $\boldsymbol{v}_1$ einer Matrix $\boldsymbol{A}$:

Idee:
Mit $\boldsymbol{x}^{(k)} = \frac{\boldsymbol{A}\boldsymbol{x}^{(k-1)}}{\left\|\boldsymbol{A}\boldsymbol{x}^{(k-1)}\right\|}$ gilt

$$\lambda^{(k)} = \left\|\boldsymbol{A}\boldsymbol{x}^{(k)}\right\| \xrightarrow{k\to\infty} |\lambda_1| \text{ und } \boldsymbol{x}^{(k)} \xrightarrow{k\to\infty} \pm\boldsymbol{v}_1$$

(Die Norm liefert nur den *Betrag* $|\lambda_1|$ – das Vorzeichen von λ_1 liefert sie nicht; für negatives λ_1 oszilliert $\boldsymbol{x}^{(k)}$ zwischen $\pm\boldsymbol{v}_1$.)

Intuition:

- Für diagonalisierbare $\boldsymbol{A}$ ist jeder Startvektor Linearkombination der Eigenvektoren $\boldsymbol{v}_i$ von $\boldsymbol{A}$: $\boldsymbol{x}^{(0)} = \sum_i^n c_i\boldsymbol{v}_i$
- Jede Iteration verstärkt die Komponente von $\boldsymbol{x}^{(k)}$ in Richtung $\boldsymbol{v}_1$ mehr als die anderen Komponenten

Algebraisch:

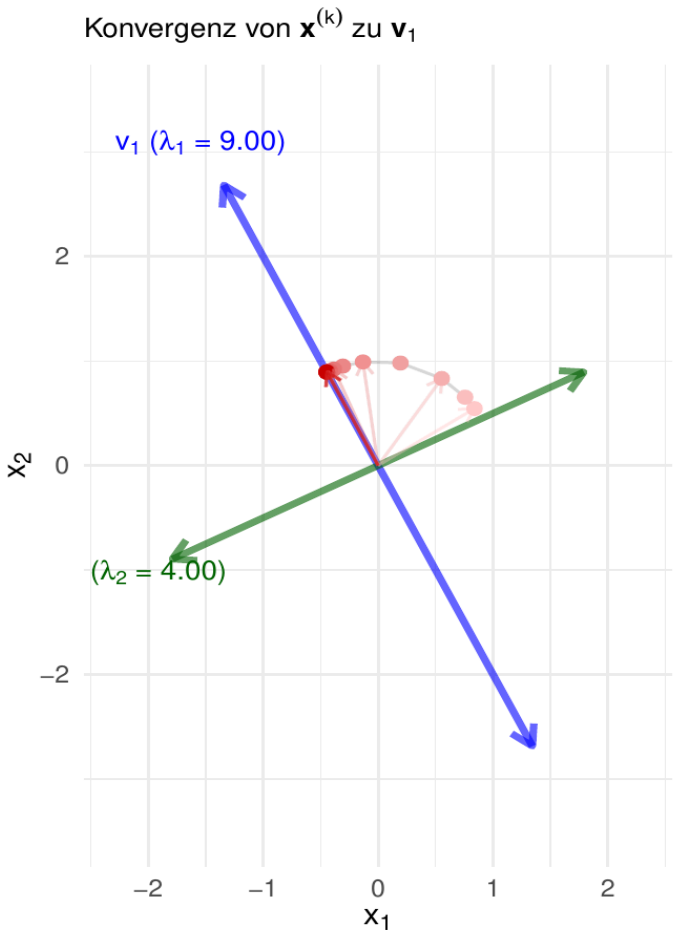
Wegen $\boldsymbol{A}\boldsymbol{v} = \lambda\boldsymbol{v} \implies \boldsymbol{A}^k\boldsymbol{v} = \lambda^k\boldsymbol{v}$ gilt auch:

- $\boldsymbol{A}^k \boldsymbol{x}^{(0)} = \lambda_1^k \left(\boldsymbol{c}_1 \boldsymbol{v}_1 + \sum_{i=2}^{n} \boldsymbol{c}_i \left(\frac{\lambda_i}{\lambda_1} \right)^k \boldsymbol{v}_i \right) \approx \lambda_1^k \boldsymbol{c}_1 \boldsymbol{v}_1$ für $k \to \infty$ weil $\left(\frac{\lambda_i}{\lambda_1} \right)^k \xrightarrow{k \to \infty} 0$ für $i \geq 2$
- $\frac{\boldsymbol{A}^k \boldsymbol{x}^{(0)}}{\| \boldsymbol{A}^k \boldsymbol{x}^{(0)} \|} \approx \frac{\lambda_1^k \boldsymbol{c}_1 \boldsymbol{v}_1}{|\lambda_1^k \boldsymbol{c}_1| \| \boldsymbol{v}_1 \|} = \pm \boldsymbol{v}_1$

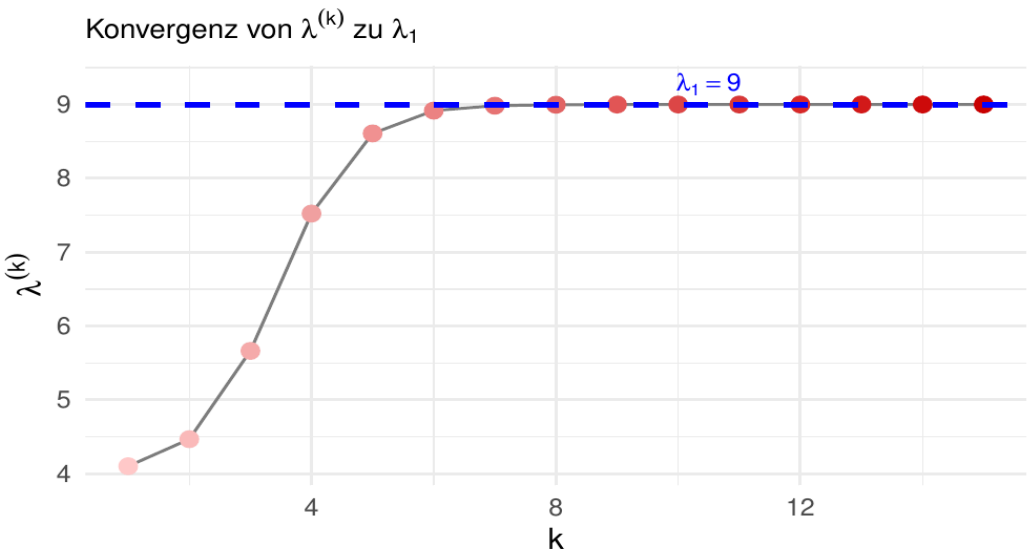
Potenzmethode: Beispiel

$$\boldsymbol{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \implies \lambda_1 = 9, \lambda_2 = 4; \boldsymbol{v}_1 = \frac{1}{\sqrt{5}} \begin{pmatrix} -1 \\ 2 \end{pmatrix}, \boldsymbol{v}_2 = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix} \text{ mit } \boldsymbol{x}^{(0)} = \begin{pmatrix} 2 \\ 1.3 \end{pmatrix}$$

:



k	Winkel zu v[1]
0	83.54°
1	75.71°
2	60.18°
3	37.79°
5	8.71°
10	0.15°
15	0.00°



Quiz

Für welche $\boldsymbol{x}^{(0)}$ versagt die Potenzmethode?

1. $\boldsymbol{x}^{(0)} = c\boldsymbol{v}_1$
2. $\boldsymbol{x}^{(0)} = c\boldsymbol{v}_2$
3. $\boldsymbol{x}^{(0)} = c(\boldsymbol{v}_1 + \boldsymbol{v}_2)$

Iterative Berechnungsverfahren

- Wir wollen (alle) Eigenwerte/-vektoren von $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ finden.
- Idee: Überführe $\boldsymbol{A}$ in spezielle Matrix $\boldsymbol{B}$, so dass das Problem leichter wird.

Theorem

Sei $\boldsymbol{B}$ eine zu $\boldsymbol{A}$ ähnliche Matrix (d.h. $\boldsymbol{B} = \boldsymbol{Q}\boldsymbol{A}\boldsymbol{Q}^{-1}$). Dann gilt:

- $\boldsymbol{A}$ und $\boldsymbol{B}$ haben die gleichen Eigenwerte (und damit auch gleiche Determinante und Spur).
- Ist $\boldsymbol{y}$ Eigenvektor von $\boldsymbol{B}$, so ist $\boldsymbol{x} = \boldsymbol{Q}^{-1}\boldsymbol{y}$ Eigenvektor von $\boldsymbol{A}$ zum gleichen Eigenwert.

Beweis:

$$\boldsymbol{B}\boldsymbol{y} = \lambda \boldsymbol{y} \implies \boldsymbol{Q}\boldsymbol{A}\boldsymbol{Q}^{-1}\boldsymbol{y} = \lambda \boldsymbol{y} \implies \boldsymbol{A}\boldsymbol{Q}^{-1}\boldsymbol{y} = \lambda \boldsymbol{Q}^{-1}\boldsymbol{y} \implies \boldsymbol{A}\boldsymbol{x} = \lambda \boldsymbol{x}.$$

Beispiel

$$\boldsymbol{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix}, \quad \boldsymbol{Q} = \tfrac{1}{\sqrt{5}} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \text{ mit } \boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$$

- $\boldsymbol{Q}$ ist **orthogonal**, also $\boldsymbol{Q}^{-1} = \boldsymbol{Q}^\top$: das Beispiel spezialisiert die Ähnlichkeitstransformation $\boldsymbol{Q}\boldsymbol{A}\boldsymbol{Q}^{-1}$ auf *orthogonale* Ähnlichkeit $\boldsymbol{Q}\boldsymbol{A}\boldsymbol{Q}^\top$.

$$\boldsymbol{B} = \boldsymbol{Q}(\boldsymbol{A}\boldsymbol{Q}^\top) = \frac{1}{5} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix}$$

(Rechnung → Anhang)

- $\boldsymbol{B}$ diagonal $\Longrightarrow$ Eigenwerte direkt ablesbar: 4 und 9
- **Der Punkt:**
  Ähnliche Matrizen haben dieselben Eigenwerte — $\boldsymbol{A}$ hat also ebenfalls die Eigenwerte 4 und 9, obwohl man sie an $\boldsymbol{A}$ nicht ablesen kann.

Verifikation:

$$\det(\boldsymbol{A} - \lambda \boldsymbol{I}) = (5-\lambda)(8-\lambda) - 4 = \lambda^2 - 13\lambda + 36 = (\lambda - 4)(\lambda - 9) = 0 \checkmark$$

Verfahren

Zur (iterativen) Berechnung einer zu $\boldsymbol{A}$ ähnlichen Matrix $\boldsymbol{B}^{(k)}$ deren Eigenwerte einfach zu bestimmen sind gibt es verschiedenste Ideen/Methoden:

- `QR-Iteration` führt zu (approximativer) Dreiecksmatrix $\boldsymbol{B}$ (*Schur-Zerlegung*).
  - ähnlich für `LU`- und `Cholesky`-Iteration
- *Lanczos-Iteration* führt zu tridiagonalem $\boldsymbol{B}$ ($b_{ij} = 0$ für $|i - j| > 1$) und kann auf die "wichtigsten" Eigenwerte eingeschränkt werden. Effizienter für große, dünn besetzte Matrizen.
- In der Praxis brauchen wir zusätzliche numerische Tricks, um diese Methoden `stabil` und akzeptabel schnell zu machen: *Shifted QR*, *Implicitly Restarted Lanczos*

QR-Iteration

Def.: QR-Iteration

Die **QR-Iteration** nutzt die *QR-Zerlegung* zur Iteration:

1. Starte mit $\boldsymbol{A}^{(0)} = \boldsymbol{A}$
2. Für $k = 1, 2, \ldots$:
   - Berechne QR-Zerlegung: $\boldsymbol{A}^{(k-1)} = \boldsymbol{Q}^{(k)} \boldsymbol{R}^{(k)}$
   - Setze $\boldsymbol{A}^{(k)} = (\boldsymbol{Q}^{(k)})^\top \boldsymbol{A}^{(k-1)} \boldsymbol{Q}^{(k)} = \boldsymbol{R}^{(k)} \boldsymbol{Q}^{(k)}$

- $\boldsymbol{A}^{(k)}$ sind ähnlich zu $\boldsymbol{A}$:

$$\boldsymbol{A}^{(k)} = \boldsymbol{R}^{(k)} \boldsymbol{Q}^{(k)} = (\boldsymbol{Q}^{(k)})^{-1} \boldsymbol{A}^{(k-1)} \boldsymbol{Q}^{(k)} = (\boldsymbol{Q}^{(1)} \cdots \boldsymbol{Q}^{(k)})^{-1} \boldsymbol{A} (\boldsymbol{Q}^{(1)} \cdots \boldsymbol{Q}^{(k)})$$

- Für $k \to \infty$ konvergiert $\boldsymbol{A}^{(k)}$ gegen eine obere Dreiecksmatrix (!) $\to$ Eigenwerte auf der Diagonalen
  (für symmetrisches $\boldsymbol{A}$ mit separierten $|\lambda_i|$; allgemein: Schur-Form)
- Berechnung der Eigenvektoren $\boldsymbol{V}$ über Produkt $\boldsymbol{Q}_k := \prod_{j=1}^{k} \boldsymbol{Q}^{(j)}$ (für symmetrische $\boldsymbol{A}$).

Beweis, dass $\boldsymbol{A}^k = \boldsymbol{Q}_k \boldsymbol{R}_k$: siehe Anhang

QR-Iteration: Interpretation

- Die QR-Iteration berechnet iterativ die *QR-Zerlegung* von $\boldsymbol{A}^k$:

$$\boldsymbol{A}^k = \boldsymbol{Q}_k \boldsymbol{R}_k \text{ mit } \boldsymbol{Q}_k := \boldsymbol{Q}^{(1)} \cdots \boldsymbol{Q}^{(k)}$$

  (Beweis $\rightarrow$ Anhang)

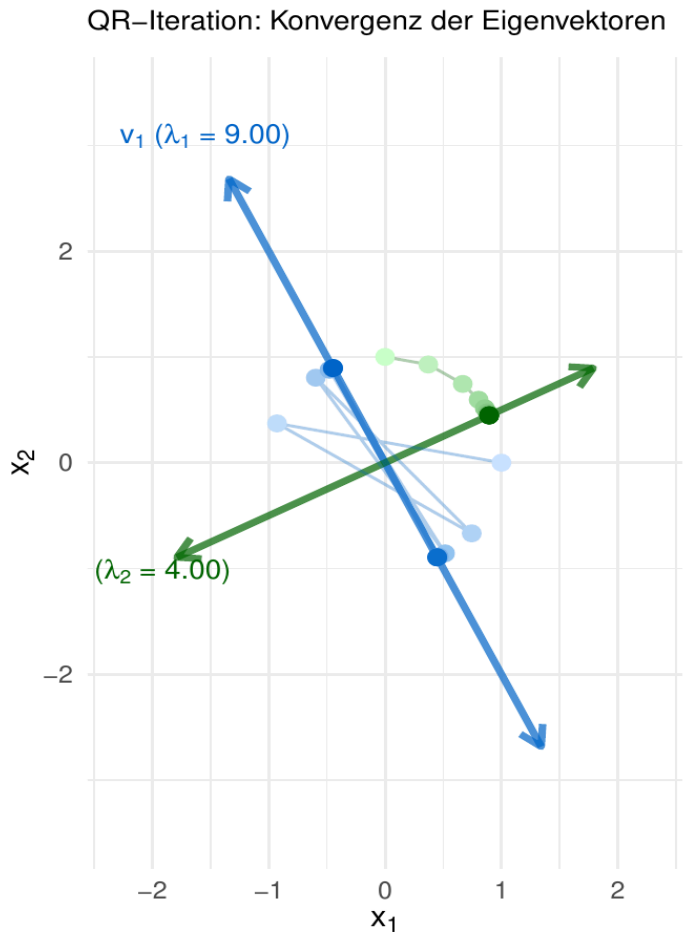
- Spalten von $\boldsymbol{Q}_k$ sind orthonormalisierte Versionen von $\boldsymbol{A}^k \boldsymbol{e}_1, \boldsymbol{A}^k \boldsymbol{e}_2, \ldots, \boldsymbol{A}^k \boldsymbol{e}_n$
  $\rightarrow$ entspricht *simultaner* *Potenzmethode* auf **allen** Einheitsvektoren!
  $\implies \boldsymbol{Q}_k \xrightarrow{k \to \infty} \boldsymbol{V}$

- $\boldsymbol{A}^{(k)} = \boldsymbol{Q}_k^\top \boldsymbol{A} \boldsymbol{Q}_k$ konvergiert wegen $\boldsymbol{Q}_k \xrightarrow{k \to \infty} \boldsymbol{V}$ gegen eine obere Dreiecksmatrix mit Eigenwerten auf der Diagonale (*Schur-Zerlegung*);
  für *symmetrische* $\boldsymbol{A} = \boldsymbol{V} \boldsymbol{\Lambda} \boldsymbol{V}^\top$ (mit *orthogonalem* $\boldsymbol{V}$) sogar gegen die Diagonalmatrix $\boldsymbol{\Lambda}$:

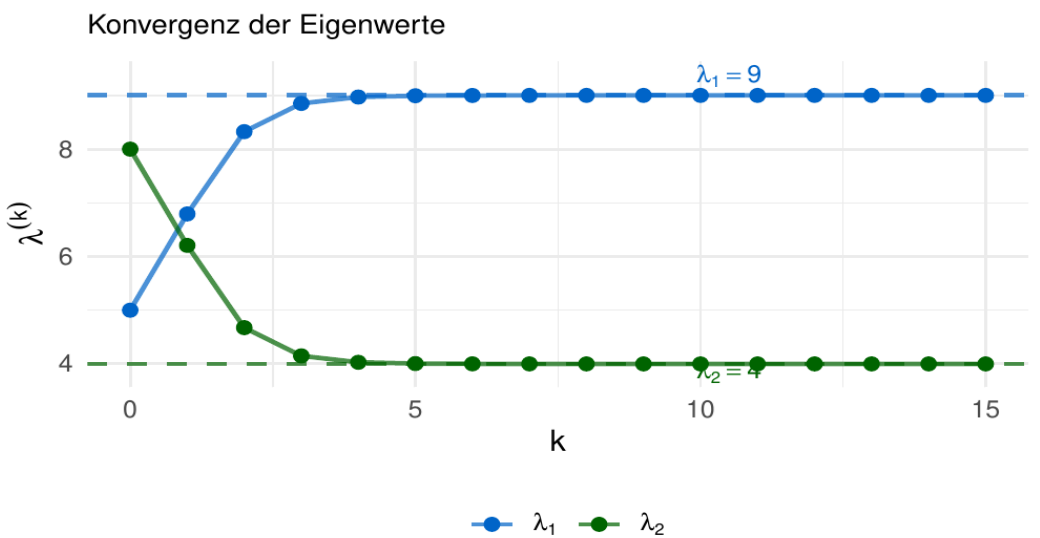
$$\boldsymbol{Q}_k^\top \boldsymbol{A} \boldsymbol{Q}_k \xrightarrow{k \to \infty} \boldsymbol{V}^\top \boldsymbol{A} \boldsymbol{V} = \boldsymbol{V}^\top \boldsymbol{V} \boldsymbol{\Lambda} \boldsymbol{V}^\top \boldsymbol{V} = \boldsymbol{\Lambda}$$

QR-Iteration: Beispiel

$$\boldsymbol{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \implies \lambda_1 = 9, \lambda_2 = 4; \boldsymbol{v}_1 = \frac{1}{\sqrt{5}} \begin{pmatrix} -1 \\ 2 \end{pmatrix}, \boldsymbol{v}_2 = \frac{1}{\sqrt{5}} \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$



k	Winkel zu v[1]	Winkel zu v[2]
0	116.57°	116.57°
1	41.63°	138.37°
2	158.44°	158.44°
3	9.96°	170.04°
5	1.99°	178.01°
10	179.97°	179.97°
15	0.00°	180.00°



Bemerkungen

`Konditionierung` von Eigenwertproblemen:

- Eigenwertprobleme sind *gut* `konditioniert`, wenn die Matrix nahezu *symmetrisch* ist.
- Eigenvektorprobleme sind *schlecht* `konditioniert`, wenn *Eigenwerte nahe beieinander* liegen.

Laufzeit:

- Jede QR-Iteration ist $O(n^3)$
- Insgesamt werden oft nur wenige Iterationen benötigt (abhängig von Eigenwertverteilung).
- Iterative `SVD`-Verfahren (z.B. *Lanczos Bi-Diagonalization*; `{irlba}`) sind noch schneller für große, dünn besetzte Matrizen

Anwendung: Google PageRank

Problem:
Rangordnung von $n \approx 10^{10}$ Webseiten im Internet bilden.

Modell:
Jede Seite i hat einen "Wichtigkeits-Score" x_i, der die Wichtigkeit der Seiten verlinkt auf i gewichtet:

$$x_i = \sum_{j:j\to i} \frac{x_j}{d_j} \quad \Longleftrightarrow \quad \boldsymbol{x} = \boldsymbol{A}\boldsymbol{x} \text{ mit } \boldsymbol{A} = \left[\frac{I(j\to i)}{d_j}\right]_{i,j} \Longrightarrow \text{Eigenwertproblem mit } \lambda = 1!$$

("$j \to i$" heißt "Seite j verlinkt auf i", d_j sei Anzahl ausgehender Links von j)

(Setzt voraus, dass jede Seite mindestens einen ausgehenden Link hat — sonst ist $d_j = 0$ und die Spalte j gar nicht definiert; solche Spalten füllt man gleichverteilt auf.)

Frage: *Warum kann Google keine $10^{10} \times 10^{10}$-Matrix invertieren — und warum funktioniert die Eigenwert-Berechnung trotzdem?*

- Direkte Methoden unmöglich: $\boldsymbol{A}$ dicht gespeichert wären 10^{20} Einträge ($\approx$ 800 Exabyte), Inversion/Zerlegung $O(n^3) = 10^{30}$ Operationen.
- Aber: $\boldsymbol{A}$ ist *extrem dünn besetzt* (jede Seite hat nur ~ 10 Links) $\Longrightarrow$ Speicherbedarf $O(n)$ statt $O(n^2)$
- **Potenzmethode**: $\boldsymbol{x}^{(k+1)} = \boldsymbol{A}\boldsymbol{x}^{(k)} / \left\|\boldsymbol{A}\boldsymbol{x}^{(k)}\right\|$ braucht nur Matrix-Vektor-Produkte: eine Iteration ist $O(n)$, Konvergenz in ~ 50 Iterationen
- **Gesamt:** $O(n)$ statt $O(n^3)$ — macht Google überhaupt erst möglich!

Anwendung: Hauptkomponentenanalyse (PCA)

Berechnung von Eigenwerten/-vektoren für Datenanalysen essentiell:

- **Problem:** Gegeben ist eine mittelwert-zentrierte Datenmatrix $\boldsymbol{X} \in \mathbb{R}^{n \times p}$ (n Beobachtungen, p Variablen)
- **Ziel:** Finde Richtungen maximaler Varianz zur Dimensionsreduktion
- **Lösung:** Berechne die Kovarianzmatrix $\boldsymbol{\Sigma} = \frac{1}{n-1} \boldsymbol{X}^\top \boldsymbol{X} \in \mathbb{R}^{p \times p}$
  - Eigenvektoren $\boldsymbol{v}_j$ von $\boldsymbol{\Sigma}$ sind die *Hauptachsen* bzw. *Loading-Richtungen* maximaler Varianz in $\mathbb{R}^p$
  - Die *Hauptkomponenten* (Scores) sind $\boldsymbol{z}_j = \boldsymbol{X}\boldsymbol{v}_j$; die Eigenwerte geben ihre Varianz an
- In der Praxis noch schneller/effizienter über *SVD* von $\boldsymbol{X}$ (z.B. `{irlba}: irlba()` (*iterativ*), `svdr()` (*iterativ* & *probabilistisch*))

Warum iterative Methoden?

- Moderne Anwendungen: $p = 10^4$ bis 10^6 (z.B. Gen-Expression, Bildanalyse, ...)
- Oft nur die k größten Eigenwerte benötigt ($k \ll p$) - vgl. Bildkompression durch *Reduzierte SVD*
- Lanczos-Iteration: Berechnet nur die k größten Eigenwerte in $O(kp^2)$ statt $O(p^3)$ für alle
  - Bei $p = 100.000$ und $k = 10$ ist Lanczos ~ 10.000 mal schneller!

Anwendung: Approximative SVD

Bildkompression durch *Reduzierte SVD*: X ist ein Graustufen-Foto als 2500×3300-Matrix *(Setup-Code, Fehlervergleich & Bildpanel → Anhang)*:

```r
r <- 50
microbenchmark::microbenchmark(
  svd_full   = svd(X),                          # exakt: alle Singulaerwerte
  svd_irlba  = irlba::irlba(X, nv = r, nu = r),  # iterativ (Lanczos): nur r
  svd_random = irlba::svdr(X, k = r),            # iterativ & randomisiert
  times = 5)
```

```
Unit: seconds
       expr   min    lq   mean median     uq    max neval cld
   svd_full 84.14 92.85 107.77 112.60 114.01 135.23     5  a
  svd_irlba  1.98  2.09   2.85   2.13   3.46   4.58     5   b
 svd_random  5.79  7.00   9.36  10.44  10.77  12.80     5   b
```

```r
X_svdr <- with(irlba::svdr(X, k = r), u %*% diag(d) %*% t(v))
norm(X - X_svdr, "F") / norm(X, "F")  # rel. Fehler der Rang-50-Approximation
```

```
[1] 0.129
```

→ iterative/randomisierte Verfahren sind hier $\sim 40\times$ bzw. $\sim 11\times$ schneller — bei praktisch identischem Ergebnis (Vergleich mit exakter Rang-50-SVD → Anhang).

Iterative Eigenwertverfahren: Vergleich

Methode	Komplexität / Iteration	Anwendung	Output
Potenz-methode	$O(\mathrm{nnz}(\boldsymbol{A}))$	Größter Eigenwert/-vektor	1 Eigenpaar
(shifted) QR	$O(n^2)$-$O(n^3)$	Alle Eigenwerte	n Eigenwerte
Lanczos	$O(m\,\mathrm{nnz}(\boldsymbol{A}) + m^2)$ für m EW	Wenige Extremal-EW, *sparse* $\boldsymbol{A}$	$m \ll n$ Eigenpaare

- Konvergenzrate *Potenzmethode*: $|\lambda_2/\lambda_1|$
- *QR-Iteration mit Shifts* oft mit quadratischer Konvergenz, am stabilsten (ohne Shifts nur linear, mit Rate $|\lambda_i/\lambda_j|$).
- Anwendung:
  - *sparse* Matrizen: Lanczos klar überlegen
  - Alle Eigenwerte, Speicher sekundär: (shifted) QR-Iteration
  - Wenige Eigenwerte: Potenzmethode (für 1), Lanczos (für $m \ll n$)

$\mathrm{nnz}(\cdot)$ = Anzahl Nicht-Null-Einträge

Iterative Lösungsverfahren

Iterative Lösungsverfahren: Intuition

- Zur Lösung von *LGS* und *KQ-Problemen* haben wir bisher direkte Methoden gesehen — *iterative Verfahren* (meist *Fixpunktiterationen*) sind oft viel schneller, wenn man etwas Genauigkeit opfert.

Wie nähern wir uns iterativ der Lösung?

- *Residuum*:
  $\boldsymbol{r}^{(k)} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(k)}$ misst, wie stark $\boldsymbol{x}^{(k)}$ die Gleichung $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$ verletzt
- Falls $\boldsymbol{A}\boldsymbol{x}^{(k)} = \boldsymbol{b}$, dann ist $\boldsymbol{r}^{(k)} = \boldsymbol{0}$ und wir haben die Lösung gefunden
- *Korrektur:*
  Solange $\boldsymbol{r}^{(k)} \neq \boldsymbol{0}$, korrigieren wir unsere Schätzung:

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \underbrace{\boldsymbol{C}\boldsymbol{r}^{(k)}}_{\text{Korrekturschritt}}$$

- $\boldsymbol{C}$ bestimmt, wie stark wir wo in Richtung des Residuums korrigieren
- **Analogie:** *Gradient Descent* - wir bewegen uns Schritt für Schritt zur Lösung

Warum nicht direkt lösen?
$\rightarrow$ Bei großen und/oder dünnbesetzten Matrizen sind iterative Methoden oft um Größenordnungen schneller als direkte Verfahren!

Iterative Lösungsverfahren für LGS

- Sei $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, $\boldsymbol{b} \in \mathbb{R}^n$ und $\boldsymbol{x}$ die Lösung des LGS $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$.
- Für einen beliebigen Startwert $\boldsymbol{x}^{(0)} \in \mathbb{R}^n$ und $\boldsymbol{C} \in \mathbb{R}^{n \times n}$, definiere die Iteration

$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(k-1)} + \boldsymbol{C}(\boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(k-1)}), \quad k = 1, 2, \ldots.$$

Theorem

Sei $\boldsymbol{x}$ die eindeutige Lösung des LGS $\boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}$.

Falls $\rho := \|\boldsymbol{I}_n - \boldsymbol{C}\boldsymbol{A}\|_2 < 1$ (*Spektralnorm*), gilt für alle $k \geq 1$:

$$\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\|_2 \leq \rho^k \|\boldsymbol{x}^{(0)} - \boldsymbol{x}\|_2.$$

- Insbesondere: $\lim_{k \to \infty} \boldsymbol{x}^{(k)} = \boldsymbol{x}$ (*Konvergenz*).

Beweis: siehe Anhang

Quiz

Sei

$$\boldsymbol{x}^{(k)} = (\boldsymbol{I} - \boldsymbol{C}\boldsymbol{A})\boldsymbol{x}^{(k-1)} + \boldsymbol{C}\boldsymbol{b}$$

mit $\boldsymbol{C} = \gamma \boldsymbol{I}_n$ für ein $\gamma > 0$. Was ist die Laufzeitkomplexität eines Iterationsschrittes?

1. $O(1)$
2. $O(n)$
3. $O(n^2)$
4. $O(n^3)$

Lösung: 3. — Mit $\boldsymbol{C} = \gamma \boldsymbol{I}$ ist $\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(k-1)} - \gamma(\boldsymbol{A}\boldsymbol{x}^{(k-1)}) + \gamma \boldsymbol{b}$: pro Schritt eine dichte Matrix-Vektor-Multiplikation $\boldsymbol{A}\boldsymbol{x}^{(k-1)}$, also $O(n^2)$; Skalierung mit γ und die zwei Vektoradditionen sind nur $O(n)$.

Iterative Lösungsverfahren für LGS: Wahl von C

- Die Frage ist nun, wie wir $\boldsymbol{C}$ so wählen, dass:
  - $\rho = \|\boldsymbol{I} - \boldsymbol{C}\boldsymbol{A}\|_2 < 1$ möglichst klein ist (optimal: $\boldsymbol{C} \approx \boldsymbol{A}^{-1}$),
  - $\boldsymbol{x}^{(k)} = (\boldsymbol{I} - \boldsymbol{C}\boldsymbol{A})\boldsymbol{x}^{(k-1)} + \boldsymbol{C}\boldsymbol{b}$ einfach zu berechnen ist.

Beliebte Wahlen für C mit Iterationsaufwand $O(n^2)$:

- *Richardson*-Iteration: $\boldsymbol{C} = \gamma \boldsymbol{I}_n$; konvergiert z.B. für SPD $\boldsymbol{A}$, falls $0 < \gamma < 2/\lambda_{\max}(\boldsymbol{A})$.
- *Jacobi*-Iteration: $\boldsymbol{C} = \operatorname{diag}(\boldsymbol{A}_{11}, \ldots, \boldsymbol{A}_{nn})^{-1}$.
- *Gauss-Seidel*-Iteration: $\boldsymbol{C}^{-1}$ gleich unterem Dreieck von $\boldsymbol{A}$.

Gesamtaufwand:

- Wegen $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\| \leq \rho^k \|\boldsymbol{x}^{(0)} - \boldsymbol{x}\|$ reichen $k = O(\log(1/\epsilon))$ Schritte für Genauigkeit ϵ (→ Self-Check).
- Damit brauchen wir $O(n^2 \log(1/\epsilon))$ Operationen für $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\| \leq \epsilon$.
- Vergleiche: $O(n^3)$ für exakte Lösungen durch *LU* oder *QR*.

Analoge iterative Verfahren gibt es auch für *KQ*-Probleme; in seltenen Fällen sind sie sogar *exakt*. Meist gilt: Trade-off zwischen *Laufzeit* und Genauigkeit — steuerbar über die Iterationszahl.

Bsp: Richardson-Iteration

Gegeben: $\boldsymbol{A} = \begin{pmatrix} 4 & 1 \\ 1 & 3 \end{pmatrix}, \boldsymbol{b} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$. Wahre Lösung: $\boldsymbol{x} = \boldsymbol{A}^{-1}\boldsymbol{b} = \frac{1}{11}\begin{pmatrix} 1 \\ 7 \end{pmatrix} \approx \begin{pmatrix} 0.091 \\ 0.636 \end{pmatrix}$.

Richardson mit $\gamma = 0.25$, Startwert $\boldsymbol{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$:

Iteration 0: $\boldsymbol{r}^{(0)} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(0)} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$, $\boldsymbol{x}^{(1)} = \boldsymbol{x}^{(0)} + 0.25\boldsymbol{r}^{(0)} = \begin{pmatrix} 0.25 \\ 0.5 \end{pmatrix}$

Iteration 1: $\boldsymbol{A}\boldsymbol{x}^{(1)} = \begin{pmatrix} 1.5 \\ 1.75 \end{pmatrix}$, $\boldsymbol{r}^{(1)} = \boldsymbol{b} - \boldsymbol{A}\boldsymbol{x}^{(1)} = \begin{pmatrix} -0.5 \\ 0.25 \end{pmatrix}$, $\boldsymbol{x}^{(2)} = \boldsymbol{x}^{(1)} + 0.25\boldsymbol{r}^{(1)} = \begin{pmatrix} 0.125 \\ 0.563 \end{pmatrix}$

Iteration 2: $\boldsymbol{A}\boldsymbol{x}^{(2)} = \begin{pmatrix} 1.063 \\ 1.813 \end{pmatrix}$, $\boldsymbol{r}^{(2)} = \begin{pmatrix} -0.063 \\ 0.188 \end{pmatrix}$, $\boldsymbol{x}^{(3)} = \begin{pmatrix} 0.109 \\ 0.609 \end{pmatrix}$

Iteration 3: $\boldsymbol{x}^{(4)} = \begin{pmatrix} 0.098 \\ 0.625 \end{pmatrix}$

Fehler: $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\|_2$: $0.643 \to 0.210 \to 0.081 \to 0.033 \to 0.013 \to \cdots$

Konvergenzrate: $\rho = \|\boldsymbol{I} - \gamma\boldsymbol{A}\|_2 \approx 0.41 \implies$ Fehler sollte ca. um Faktor 2.5 pro Iteration sinken.

Probabilistische Methoden

Zufall mit Absicht

- Oft sind Probleme so groß, dass wir gerne Genauigkeit für bessere *Komplexität* tauschen.
- Heute wird das zunehmend durch *Randomisierung* erreicht.
- **Hauptidee:**
  - *Ersetze* großes Problem durch ein
  - *zufällig* erzeugtes, *kleines* Problem,
  - das *höchstwahrscheinlich* eine *ähnliche Lösung* hat.

Idee: Matrix-Sketching

Geometrische Fakten in hoch-dimensionalem Raum $\mathbb{R}^n$ mit $n \gg 1$:

Für großes n sind zwei zufällige Vektoren $\boldsymbol{s}_1, \boldsymbol{s}_2 \in \mathbb{R}^n$ mit hoher Wahrscheinlichkeit fast orthogonal.

$\Longrightarrow$ geeignet *zufällig* gezogene Matrix $\boldsymbol{S} \in \mathbb{R}^{m \times n}$ mit $m \ll n$ erhält mit hoher Wahrscheinlichkeit

- Abstände: $\|\boldsymbol{S}\boldsymbol{x} - \boldsymbol{S}\boldsymbol{y}\|_2 \approx \|\boldsymbol{x} - \boldsymbol{y}\|_2$
- Winkel: $\angle(\boldsymbol{S}\boldsymbol{x}, \boldsymbol{S}\boldsymbol{y}) \approx \angle(\boldsymbol{x}, \boldsymbol{y})$

$\Longrightarrow$ Eine **zufällige Projektion** $\boldsymbol{S}$ in einen niedrigdimensionaleren Raum kann Geometrie oft näherungsweise erhalten.

Beispiel: Sketching in R

```r
set.seed(1312)
n <- 10000                # zwei beliebige n-dimensionale Vektoren:
x <- runif(n); y <- runif(n)
m <- 50                   # m << n !!
S <- rnorm(m * n, sd = sqrt(1 / m)) |> matrix(nrow = m, ncol = n)
Sx <- S %*% x;  Sy <- S %*% y
```

Wir ersetzen $n = 10000$ Koordinaten durch $m = 50$ zufällige Projektionen.

Vorhersage:

Um wie viel Prozent ändern sich Distanz $\|\boldsymbol{x} - \boldsymbol{y}\|_2$ und Winkel $\angle(\boldsymbol{x}, \boldsymbol{y})$ dabei wohl —
eher $\sim 50\%$, $\sim 3\%$ oder $< 0.1\%$?

```r
angle <- function(x, y) (sum(x * y) / (norm(x, "2") * norm(y, "2"))) |> acos()
c(norm(x - y, "2"), norm(Sx - Sy, "2"))  ## Distanzen  ||x - y||, ||Sx - Sy||
c(angle(x, y), angle(Sx, Sy))            ## Winkel (rad)
```

```
[1] 41.0 42.2
[1] 0.728 0.735
```

$\rightarrow$ nur $\approx 3\%$ [$\approx 1\%$] Abweichung in Distanz [Winkel] — obwohl 99.5% der Dimensionen wegfallen!

(Eine einzelne Ziehung, und eine recht glückliche: für Gauss-Skizzen streut die relative Abweichung der Distanz mit $\approx 1/\sqrt{2m} = 10\%$.)

Zufällige Einbettung eines festen Vektors

Theorem

Sei

- $\boldsymbol{S} \in \mathbb{R}^{m \times n}$ eine Matrix mit i.i.d. Zeilen $\boldsymbol{s}_i^\top$ (wobei $\boldsymbol{s}_i \in \mathbb{R}^n$),
- $\mathbb{E}[\boldsymbol{s}_i \boldsymbol{s}_i^\top] = \boldsymbol{I}_n / m$ und $\mathbb{E}[(\boldsymbol{s}_i^\top \boldsymbol{x})^4] \leq K \|\boldsymbol{x}\|^4 / m^2$ für alle $\boldsymbol{x} \in \mathbb{R}^n$.

Dann gilt für jeden *festen* Vektor $\boldsymbol{x} \in \mathbb{R}^n$ mit einer Wahrscheinlichkeit von mindestens $1 - \delta$

$$(1 - \epsilon)\|\boldsymbol{x}\|^2 \leq \|\boldsymbol{S}\boldsymbol{x}\|^2 \leq (1 + \epsilon)\|\boldsymbol{x}\|^2,$$

wobei $\delta > 0$ und $\epsilon = \sqrt{K / \delta m}$.

Beispiele:

- *Gauss:* $\boldsymbol{s}_i \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, \boldsymbol{I}_n / m)$.
- *Rademacher:* s_{ij} *i.i.d.* mit $\mathbb{P}(s_{ij} = 1/\sqrt{m}) = \mathbb{P}(s_{ij} = -1/\sqrt{m}) = 1/2$.
- *Subsampling:* $\boldsymbol{s}_i = \sqrt{n/m}\, \boldsymbol{e}_{K_i}$ skalierter Einheitsvektor mit $K_i \sim \mathrm{Unif}\{1, \ldots, n\}$.

Beweis: siehe Anhang

Anwendungen

- Zufällige Einbettungen (*Matrix-Sketching*) haben zahlreiche Anwendungen:
  - Matrixmultiplikation: $\boldsymbol{AB} \approx (\boldsymbol{AS}^\top)(\boldsymbol{SB})$
  - *KQ-Probleme*: $\min \|\boldsymbol{Ax} - \boldsymbol{b}\|_2 \approx \min \|\boldsymbol{SAx} - \boldsymbol{Sb}\|_2$
  - ...
- *Achtung:* Unser Theorem gilt je *festem* Vektor — hier hängt z.B. der Minimierer selbst von $\boldsymbol{S}$ ab!
  Dafür braucht man stärkere, *simultane* Garantien (*Subspace Embeddings*): gleiche Beweisidee, etwas größeres m.
- Wirklich schneller werden Algorithmen aber nur, wenn wir Produkte $\boldsymbol{Sx}$ sehr schnell berechnen können.
- *Beispiele:*
  - Wenn $\boldsymbol{S}$ m zufällige Koordinaten aus $\boldsymbol{x}$ zieht, braucht es dafür nur $O(m)$ Operationen.
  - Besonders leistungsfähig: *Subsampled Randomized Hadamard Transform* (SRHT) braucht $O(n \log m)$ Rechenschritte. (Vergleich der Sketching-Matrizen $\rightarrow$ Anhang)

Beispiel: Dimensionsreduktion mit Matrix-Sketching

Gegeben:
$n = 10000$ Dimensionen, $\delta = 0.05$, $\epsilon = 0.1$

Frage:
Wie klein kann m sein?

Aus dem Theorem:
Mit $\boldsymbol{s}_i \sim \mathcal{N}(\mathbf{0}, \boldsymbol{I}_n/m)$ gilt $K = 3$ und

$$\epsilon = \sqrt{\frac{K}{\delta m}} \quad \Longrightarrow \quad m = \frac{K}{\delta \epsilon^2} = \frac{3}{0.05 \cdot 0.01} = 6000$$

Ergebnis:

Mit $m = 6000$ Dimensionen können wir $n = 10000$ Dimensionen so darstellen, dass jeder gegebene Abstand mit 95% Wahrscheinlichkeit um höchstens 10% verzerrt wird.

Praktische Konsequenz:

- Speicher & Rechenzeit pro Distanzberechnung: $O(n) \to O(m)$
- Die Schranke (via Tschebyscheff-Ungleichung) ist sehr *konservativ* — in der Praxis reichen oft viel kleinere m: die Demo oben kam mit $m = 50$ auf nur $\approx 3\%$ Abweichung!
- Anwendung: Schnelle Nächste-Nachbar-Suche, Clusteranalyse in hohen Dimensionen, ...

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- Lösungsansätze für Eigenwertprobleme
- *Iterative Verfahren* nähern sich Lösung schrittweise durch *Fixpunktiteration*
- *Probabilistische Verfahren* ersetzen schweres durch zufälliges, leichteres (niedrigdimensionaleres) Problem mit approximativer Lösung
- Beide geben uns einen Trade-off zwischen *Laufzeit* und Genauigkeit

Nächste Vorlesung

- Tensoren und Tensorprodukte

Self-Check (1/2)

Frage 1: Sei $\rho \in (0, 1)$, $\epsilon > 0$, und

$$\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\| \leq \rho^k \|\boldsymbol{x}^{(0)} - \boldsymbol{x}\|.$$

Wie groß muss k mindestens sein, so dass $\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\| \leq \epsilon$?

1. $k = O(1)$
2. $k = O(\log(1/\epsilon))$
3. $k = O(1/\epsilon)$
4. $k = O(1/\epsilon^k)$

Lösung: 2. – mit $e_0 = \|\boldsymbol{x}^{(0)} - \boldsymbol{x}\|$: $\rho^k e_0 \leq \epsilon \implies \rho^k \leq \epsilon/e_0$; Logarithmieren ($0 < \rho < 1 \implies \log \rho < 0$!) gibt $k \geq \frac{\log(\epsilon) - \log(e_0)}{\log(\rho)} = \frac{\log(e_0) + \log(1/\epsilon)}{-\log(\rho)}$, also minimales $k = \left\lceil \frac{\log(e_0/\epsilon)}{-\log(\rho)} \right\rceil$ (falls $0 < \epsilon < e_0$, sonst reicht schon $k = 0$) – wächst wie $\log(1/\epsilon)$. ✓

Self-Check (2/2)

Frage 2: Worin unterscheidet sich *Subsampling* (m zufällige, skalierte Zeilen) von einer *Gauss*-Sketching-Matrix vor allem?

1. Subsampling kostet nur $O(m)$ statt $O(mn)$ (Zeit & Speicher), ist aber nur zuverlässig, wenn die Information über viele Koordinaten gestreut ist.
2. Gauss-Matrizen brauchen weniger Speicher als Subsampling.
3. Subsampling erhält alle Distanzen exakt.
4. Es gibt keinen Unterschied — beide sind optimal und gleich teuer.

Frage 3: Welcher Sketching-Typ kombiniert schnelle Anwendung ($O(n \log m)$ für $\boldsymbol{Sx}$) mit sehr guter Qualität ("State of the art")?

1. Gauss
2. Rademacher
3. Subsampling
4. Subsampled Randomized Hadamard Transform (SRHT)

Anhang

Ähnlichkeit: Beispiel — Rechnung

$$\boldsymbol{A} = \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix}, \quad \boldsymbol{Q} = \tfrac{1}{\sqrt{5}} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \text{ mit } \boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$$

Schritt 1: Berechne $\boldsymbol{AQ}^\top$:

$$\boldsymbol{AQ}^\top = \frac{1}{\sqrt{5}} \begin{pmatrix} 5 & -2 \\ -2 & 8 \end{pmatrix} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} = \frac{1}{\sqrt{5}} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix}$$

Schritt 2: Berechne $\boldsymbol{B} = \boldsymbol{Q}(\boldsymbol{AQ}^\top)$:

$$\boldsymbol{B} = \frac{1}{5} \begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} \begin{pmatrix} 8 & 9 \\ 4 & -18 \end{pmatrix} = \frac{1}{5} \begin{pmatrix} 20 & 0 \\ 0 & 45 \end{pmatrix} = \begin{pmatrix} 4 & 0 \\ 0 & 9 \end{pmatrix}$$

$\boldsymbol{B}$ diagonal $\Longrightarrow$ Eigenwerte direkt ablesbar: 4 und 9.

↩ zurück: „Beispiel”

QR-Iteration: Beweis

Definiere $\boldsymbol{Q}_k := \boldsymbol{Q}^{(1)}\boldsymbol{Q}^{(2)}\cdots\boldsymbol{Q}^{(k)}$, $\boldsymbol{R}_k = \boldsymbol{R}^{(k)}\boldsymbol{R}^{(k-1)}\cdots\boldsymbol{R}^{(1)}$

Zeige: $\boldsymbol{A}^k = \boldsymbol{Q}_k\boldsymbol{R}_k$

Beweis:

- *Induktionsanfang* ($k = 1$): $\boldsymbol{A} = \boldsymbol{Q}^{(1)}\boldsymbol{R}^{(1)} = \boldsymbol{Q}_1\boldsymbol{R}_1$ ✓
- *Induktionsschritt* ($k \to k+1$): $\boldsymbol{A}^{(i)} = \boldsymbol{R}^{(i)}\boldsymbol{Q}^{(i)}$,
  $\boldsymbol{A}^{(i)} = \boldsymbol{Q}^{(i+1)}\boldsymbol{R}^{(i+1)} \implies \boldsymbol{R}^{(i)}\boldsymbol{Q}^{(i)} = \boldsymbol{Q}^{(i+1)}\boldsymbol{R}^{(i+1)}$ $(\star)$

$$
\begin{aligned}
\boldsymbol{A}^{k+1} &= \boldsymbol{A}\boldsymbol{A}^k = (\boldsymbol{Q}^{(1)}\boldsymbol{R}^{(1)})(\boldsymbol{Q}_k\boldsymbol{R}_k) \\
&= \boldsymbol{Q}^{(1)}\big(\boldsymbol{R}^{(1)}\boldsymbol{Q}^{(1)}\big)\boldsymbol{Q}^{(2)}\cdots\boldsymbol{Q}^{(k)}\boldsymbol{R}^{(k)}\cdots\boldsymbol{R}^{(1)} \\
&\qquad [\text{nutze } k\text{-mal } (\star) : \boldsymbol{R}^{(1)}\boldsymbol{Q}^{(1)} = \boldsymbol{Q}^{(2)}\boldsymbol{R}^{(2)}, \ldots] \\
&= \boldsymbol{Q}^{(1)}\boldsymbol{Q}^{(2)}\cdots\boldsymbol{Q}^{(k+1)}\boldsymbol{R}^{(k+1)}\cdots\boldsymbol{R}^{(1)} \\
&= \boldsymbol{Q}_{k+1}\boldsymbol{R}_{k+1} \quad ✓
\end{aligned}
$$

↩ zurück: „QR-Iteration" · ↩ zurück: „QR-Iteration: Interpretation"

Iterative LGS-Verfahren: Konvergenz — Beweis

Definiere $\boldsymbol{B = I - CA}$, sodass $\|\boldsymbol{B}\|_2 = \rho$ und

$$\boldsymbol{x}^{(k)} = \boldsymbol{B}\boldsymbol{x}^{(k-1)} + \boldsymbol{C}\boldsymbol{b}, \quad k = 1, 2, \dots$$

Zunächst bemerken wir, dass $\boldsymbol{x}$ ein Fixpunkt dieser Iteration ist:

$$\boldsymbol{B}\boldsymbol{x} + \boldsymbol{C}\boldsymbol{b} = (\boldsymbol{I} - \boldsymbol{C}\boldsymbol{A})\boldsymbol{x} + \boldsymbol{C}\boldsymbol{b} = \boldsymbol{x} - \boldsymbol{C}\underbrace{(\boldsymbol{A}\boldsymbol{x} - \boldsymbol{b})}_{=0} = \boldsymbol{x}$$

also

$$\begin{aligned}
\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\|_2 &= \|\boldsymbol{B}\boldsymbol{x}^{(k-1)} + \boldsymbol{C}\boldsymbol{b} - \boldsymbol{x}\|_2 \\
&= \|\boldsymbol{B}\boldsymbol{x}^{(k-1)} + \boldsymbol{C}\boldsymbol{b} - (\boldsymbol{B}\boldsymbol{x} + \boldsymbol{C}\boldsymbol{b})\|_2 \\
&= \|\boldsymbol{B}(\boldsymbol{x}^{(k-1)} - \boldsymbol{x})\|_2 \\
&\leq \rho\|\boldsymbol{x}^{(k-1)} - \boldsymbol{x}\|_2.
\end{aligned}$$

Iterieren wir die Ungleichung, erhalten wir

$$\|\boldsymbol{x}^{(k)} - \boldsymbol{x}\|_2 \leq \rho\|\boldsymbol{x}^{(k-1)} - \boldsymbol{x}\|_2 \leq \rho^2\|\boldsymbol{x}^{(k-2)} - \boldsymbol{x}\|_2 \leq \cdots \leq \rho^k\|\boldsymbol{x}^{(0)} - \boldsymbol{x}\|_2.$$ ■

↩ zurück: „Iterative Lösungsverfahren für LGS”

Zufälliges Einbetten von Unterräumen: Beweis (1/2)

1. Für $\boldsymbol{x} = \mathbf{0}$ ist die Aussage trivial. Sei also $\boldsymbol{x} \neq \mathbf{0}$ und o.B.d.A. $\|\boldsymbol{x}\| = 1$ (andernfalls teile die Ungleichung durch $\|\boldsymbol{x}\|^2$). Zu zeigen also

$$\mathbb{P}\left(1 - \sqrt{K/\delta m} \leq \|\boldsymbol{S}\boldsymbol{x}\|^2 \leq 1 + \sqrt{K/\delta m}\right) = \mathbb{P}\left(\left|\|\boldsymbol{S}\boldsymbol{x}\|^2 - 1\right| \leq \sqrt{K/\delta m}\right) \geq 1 - \delta$$

2. Mit $\|\boldsymbol{S}\boldsymbol{x}\|^2 = \sum_{i=1}^{m} \left(\boldsymbol{s}_i^\top \boldsymbol{x}\right)^2$ gilt

$$\mathbb{E}\left[\|\boldsymbol{S}\boldsymbol{x}\|^2\right] = \sum_{i=1}^{m} \mathbb{E}\left[(\boldsymbol{s}_i^\top \boldsymbol{x})^2\right] = m\boldsymbol{x}^\top \mathbb{E}\left[\boldsymbol{s}_1 \boldsymbol{s}_1^\top\right] \boldsymbol{x} = \|\boldsymbol{x}\|^2.$$

3. Weil die Summanden unabhängig sind, gilt außerdem

$$\mathbb{V}\mathrm{ar}\left[\|\boldsymbol{S}\boldsymbol{x}\|^2\right] = \sum_{i=1}^{m} \mathbb{V}\mathrm{ar}\left[(\boldsymbol{s}_i^\top \boldsymbol{x})^2\right] \leq m\mathbb{E}\left[(\boldsymbol{s}_1^\top \boldsymbol{x})^4\right] \leq \frac{K}{m}$$

   nach der Momentannahme (mit $\|\boldsymbol{x}\| = 1$).

Zufälliges Einbetten von Unterräumen: Beweis (2/2)

(Fortsetzung)

4. Nun gibt die Tschebyscheff-Ungleichung (= Markov, angewandt auf die quadrierte Abweichung)

$$\mathbb{P}\left(\left|\|\boldsymbol{S}\boldsymbol{x}\|^2 - \mathbb{E}[\|\boldsymbol{S}\boldsymbol{x}\|^2]\right| > \epsilon\right) \leq \frac{\mathbb{V}\mathrm{ar}[\|\boldsymbol{S}\boldsymbol{x}\|^2]}{\epsilon^2} \leq \frac{m\mathbb{E}[(\boldsymbol{s}_i^\top \boldsymbol{x})^4]}{\epsilon^2} \leq \frac{K}{\epsilon^2 m}.$$

   Die Wahl $\epsilon = \sqrt{K/\delta m}$ liefert die Behauptung. ■

↩ zurück: „Zufälliges Einbetten von Unterräumen"

Approximative SVD: Setup & Fehlervergleich

Setup-Code des Benchmarks (im Foliensatz ausgeführt):

```
set.seed(1312)
library(irlba)
suppressMessages(library(magick))
img <- paste0(
  "https://upload.wikimedia.org/wikipedia/commons/7/7e/",
  "Bolzano_City_Image_-_Photo_by_Giovanni_Ussi_-_In_Black_and_White_19.jpg")
X <- (image_read(img) |>
        image_data(channels = "gray") |>
        as.numeric())[,,1]
```

Relative Frobenius-Fehler der niedrigrangigen Approximationen (der exakten Rang-50-SVD zu $\boldsymbol{X}$, und der iterativen Verfahren zur exakten Rang-50-SVD):

```
X_r <- with(svd(X, nu = r, nv = r), u %*% diag(d[1:r]) %*% t(v))
X_irlba <- with(irlba::irlba(X, nu = r, nv = r), u %*% diag(d) %*% t(v))
norm(X - X_r, "F") / norm(X, "F")
```

```
[1] 0.129
```

```
c(norm(X_r - X_irlba, "F"), norm(X_r - X_svdr, "F")) / norm(X_r, "F")
```

```
[1] 2.27e-06 1.68e-04
```

$\longrightarrow$ `irlba`/`svdr` weichen von der exakten Rang-50-SVD um Größenordnungen weniger ab, als diese vom Originalbild.

Approximative SVD: Bildvergleich

Original



SVD (r = 50)



IRLBA (r = 50)



SVDR (r = 50)



↩ zurück: „Anwendung: Approximative SVD”

Vergleich Sketching-Matrizen

Typ	Speicher	Zeit für Sx	Qualität
Gauss	$O(mn)$	$O(mn)$	Optimal
Rademacher ($\pm 1/\sqrt{m}$)	$O(mn)$	$O(mn)$	Optimal
Subsampling	$O(m)$	$O(m)$	Gut (bei über viele Koordinaten *gestreuter* Information)
SRHT (Hadamard)	$O(n)$	$O(n \log m)$	Sehr gut

Trade-offs:

- **Gauss:** Theoretisch optimal, aber teuer zu speichern und anzuwenden
- **Rademacher:** Genauso gut wie Gauss, einfacher zu generieren (nur ± 1)
- **Subsampling:** Wählt m zufällige Zeilen - extrem schnell, aber nur gut, wenn die Information über viele Koordinaten gestreut ist (schlecht, wenn sie in wenigen Koordinaten konzentriert ist)
- **SRHT:** kombiniert hohe Geschwindigkeit und gute Approximationsqualität; Speicher $O(n)$ wegen der n Vorzeichen der vorgeschalteten Diagonalmatrix (sofern man sie nicht aus einem Seed neu erzeugt)
  - *Subsampled Randomized Hadamard Transform*
  - S wird nicht explizit gespeichert, sondern durch schnelle(!) Transformationen angewendet

↩ zurück: „Anwendungen”

Tensoren und Tensorprodukte

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- **Lineare Algebra**
  - Vektorräume und Unterräume
  - Basis und Dimension
  - Lineare Abbildungen und Multilinearität
  - Äußeres Produkt: $\boldsymbol{v} \otimes \boldsymbol{w} = \boldsymbol{v}\boldsymbol{w}^T$
  - $\operatorname{span}\left(\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}\right) = \{\boldsymbol{w} : \boldsymbol{w} = \sum_i c_i \boldsymbol{v}_i; c_i \in \mathbb{R}\}$
- **Analysis**
  - Multivariate Funktionen $f(\boldsymbol{x}) = f(x_1, x_2, \ldots, x_n)$
- **Gleichungssysteme** (Kapitel 5): LGS $\boldsymbol{M}\boldsymbol{z} = \boldsymbol{c}$ lösen (LU, Cholesky)

Multilineare Abbildungen

Multilineare Abbildungen

- Bisher wurde ein Vektor $\boldsymbol{v} \in V$ linear auf einen Vektor $\boldsymbol{w} \in W$ abgebildet.
- Jetzt werden Paare oder Tupel von Vektoren $(\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n) \in V_1 \times \cdots \times V_n$ “linear” auf ein $\boldsymbol{w} \in W$ abgebildet,
  - $\rightarrow$ Wir sprechen von einer *multilinearen Abbildung*.
- Linearität bezieht sich hier auf die *einzelnen* Argumente der Abbildung.

Multilineare Abbildungen

Def.: Multilineare Abbildung

Seien $V_1, \dots, V_n, W$ Vektorräume. Eine **multilineare Abbildung** ist eine Abbildung $f\colon V_1 \times \cdots \times V_n \to W$, sodass:

$$\begin{aligned} &f(\boldsymbol{v}_1, \dots, c\boldsymbol{v}_i + c'\boldsymbol{v}_i', \dots, \boldsymbol{v}_n) \\ &= cf(\boldsymbol{v}_1, \dots, \boldsymbol{v}_i, \dots, \boldsymbol{v}_n) + c' f(\boldsymbol{v}_1, \dots, \boldsymbol{v}_i', \dots, \boldsymbol{v}_n) \end{aligned}$$

für alle $i = 1, \dots, n$, $c, c' \in \mathbb{R}$, $\boldsymbol{v}_i, \boldsymbol{v}_i' \in V_i$ und $\boldsymbol{v}_\ell \in V_\ell$ ($\ell \neq i$).

Anmerkungen:

- Im Spezialfall $n = 2$ nennen wir die Abbildung *bilinear*.
- **Achtung:** *Multilineare Abbildungen* sind in der Regel nicht linear!

Intuition: Linear vs. Multilinear

Fläche eines Rechtecks als Funktion seiner Seitenlängen:

$$f\colon \mathbb{R} \times \mathbb{R} \to \mathbb{R}, \quad (x, y) \mapsto xy$$

Warum ist f nicht linear?

- Für $c = 2$: $f(2x, 2y) = 2x \cdot 2y = 4xy = 4f(x, y) \neq 2f(x, y)$ (für $xy \neq 0$)
- Linear wäre: $f(cx, cy) = c \cdot f(x, y)$

Aber f ist bilinear:

$$\begin{aligned} y \text{ konstant: } f(x_1 + x_2, y) &= (x_1 + x_2) \cdot y = x_1 y + x_2 y = f(x_1, y) + f(x_2, y) \\ f(cx, y) &= (cx) \cdot y = c(xy) = cf(x, y) \\ x \text{ konstant: } f(x, y_1 + y_2) &= x \cdot (y_1 + y_2) = xy_1 + xy_2 = f(x, y_1) + f(x, y_2) \\ f(x, cy) &= x \cdot (cy) = c(xy) = cf(x, y) \end{aligned}$$

$\Longrightarrow$ Die Funktion ist **in jedem Argument einzeln linear**!

Multilineare Abbildungen

Beispiel:

- Multiplikation von zwei reellen Zahlen, $f\colon \mathbb{R} \times \mathbb{R} \to \mathbb{R}$, $(x_1, x_2) \mapsto x_1 x_2$, ist bilinear (s.a. Bsp. zur Fläche)

Beispiel:

- Jedes Skalarprodukt $f\colon V \times V \to \mathbb{R}$, $(\boldsymbol{v}_1, \boldsymbol{v}_2) \mapsto \langle \boldsymbol{v}_1, \boldsymbol{v}_2 \rangle$ ist bilinear.
- Das folgt aus Distributivgesetz, Homogenität und Symmetrie:
  $\langle c\boldsymbol{v} + c'\boldsymbol{v}', \boldsymbol{w} \rangle = \langle c\boldsymbol{v}, \boldsymbol{w} \rangle + \langle c'\boldsymbol{v}', \boldsymbol{w} \rangle = c\langle \boldsymbol{v}, \boldsymbol{w} \rangle + c'\langle \boldsymbol{v}', \boldsymbol{w} \rangle$

Beispiel:

- Die Matrizenmultiplikation $\mathbb{R}^{m \times k} \times \mathbb{R}^{k \times n} \to \mathbb{R}^{m \times n}$ mit $(\boldsymbol{A}, \boldsymbol{B}) \mapsto \boldsymbol{AB}$ ist bilinear.

Quiz

TRUE/FALSE?

> Für jedes multilineare f gilt $f(\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n) = \mathbf{0}$ sobald $\boldsymbol{v}_i = \mathbf{0}$ für ein i.

Darstellung von multilinearen Abbildungen

Theorem

Eine Abbildung $T\colon \mathbb{R}^{n_1} \times \cdots \times \mathbb{R}^{n_k} \to \mathbb{R}^m$ ist genau dann multilinear, wenn es reelle Koeffizienten

$$A = (a_{i_1,\ldots,i_k,j}\colon 1 \leq i_1 \leq n_1, \ldots, 1 \leq i_k \leq n_k, 1 \leq j \leq m)$$

gibt, sodass

$$T(\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(k)}) = \sum_{i_1=1}^{n_1} \cdots \sum_{i_k=1}^{n_k} \sum_{j=1}^{m} a_{i_1,\ldots,i_k,j} x_{i_1}^{(1)} \cdots x_{i_k}^{(k)} \boldsymbol{e}_j,$$

wobei $\boldsymbol{e}_j$ der j-te Einheitsvektor des $\mathbb{R}^m$ ist. Die Koeffizienten $a_{i_1,\ldots,i_k,j}$ sind eindeutig.

Tensoren

Tensoren

- Setzen wir im vorherigen Theorem $k = 1$, erhalten wir Koeffizienten $A = (a_{i,j} \colon 1 \leq i \leq n, 1 \leq j \leq m)$, sodass

$$T(\boldsymbol{x}) = \sum_{i=1}^{n} \sum_{j=1}^{m} a_{i,j} x_i \boldsymbol{e}_j = \sum_{i=1}^{n} x_i \boldsymbol{a}_i = \boldsymbol{A}\boldsymbol{x},$$

  mit $\boldsymbol{a}_i = (a_{i,1}, \ldots, a_{i,m})^\top \in \mathbb{R}^m$ und $\boldsymbol{A} = (\boldsymbol{a}_1, \ldots, \boldsymbol{a}_n) \in \mathbb{R}^{m \times n}$, d.h. $(\boldsymbol{A})_{j,i} = a_{i,j}$.
- Die Ansammlung $\boldsymbol{A}$ aus nm Koeffizienten nennen wir *Matrix*.
- Sie hat zwei "Dimensionen" weil sie durch zwei Subskripte i und j indiziert wird.
- Allgemeiner beschreibt jede solche Ansammlung von Koeffizienten mit $k + 1$ Indizes (bei $m = 1$ also mit k) eine entsprechende multilineare Funktion

Bsp: Bilineare Abbildung ($k = 2, n_1 = n_2 = 2, m = 1$)

$T\colon \mathbb{R}^2 \times \mathbb{R}^2 \to \mathbb{R}$ mit $T(\boldsymbol{x}, \boldsymbol{y}) = 2x_1y_1 + 3x_1y_2 - x_2y_1 + 4x_2y_2$

Koeffizienten $(a_{i_1,i_2} \colon 1 \leq i_1, i_2 \leq 2)$**:**

$$a_{1,1} = 2, \quad a_{1,2} = 3, \quad a_{2,1} = -1, \quad a_{2,2} = 4 \implies \boldsymbol{A} = \begin{pmatrix} 2 & 3 \\ -1 & 4 \end{pmatrix}$$

Verifikation:

$$\begin{aligned} T(\boldsymbol{x}, \boldsymbol{y}) &= \sum_{i_1=1}^{2} \sum_{i_2=1}^{2} a_{i_1,i_2} x_{i_1} y_{i_2} = a_{1,1}x_1y_1 + a_{1,2}x_1y_2 + a_{2,1}x_2y_1 + a_{2,2}x_2y_2 \\ &= 2x_1y_1 + 3x_1y_2 - x_2y_1 + 4x_2y_2 \quad \checkmark \end{aligned}$$

Numerisches Beispiel:

Für $\boldsymbol{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \boldsymbol{y} = \begin{pmatrix} 3 \\ 4 \end{pmatrix}$: $T(\boldsymbol{x}, \boldsymbol{y}) = 2 \cdot 1 \cdot 3 + 3 \cdot 1 \cdot 4 - 1 \cdot 2 \cdot 3 + 4 \cdot 2 \cdot 4 = 6 + 12 - 6 + 32 = 44$

Alternativ in Tensorschreibweise:

$$T(\boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{y} = \begin{pmatrix} 1 & 2 \end{pmatrix} \begin{pmatrix} 2 & 3 \\ -1 & 4 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = \begin{pmatrix} 0 & 11 \end{pmatrix} \begin{pmatrix} 3 \\ 4 \end{pmatrix} = 44$$

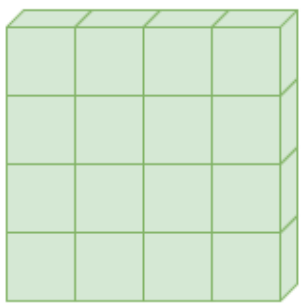
Definition: Tensor

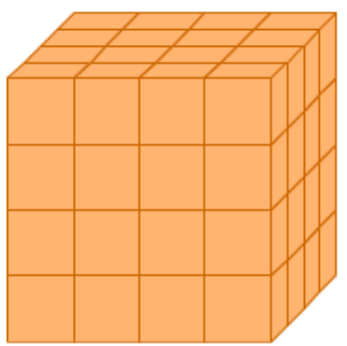
Def.: Tensor

Ein **Tensor** ist eine durch k Indizes indizierte Ansammlung von reellen Zahlen

$$A = (a_{i_1,\ldots,i_k} : 1 \leq i_1 \leq n_1, \ldots, 1 \leq i_k \leq n_k) \in \mathbb{R}^{n_1 \times \cdots \times n_k}.$$

Die Zahl k wird auch **Stufe** des Tensors genannt.





Eine (4×4)-Matrix besteht aus $4^2 = 16$ Zahlen (links); ein $(4 \times 4 \times 4)$-Tensor besteht aus $4^3 = 64$ Zahlen (rechts). (Quelle: Deisenroth/Faisal/Ong, Fig 5.13)

- Die Menge $\mathbb{R}^{n_1 \times \cdots \times n_k}$ aller $\left(n_1 \times \cdots \times n_k\right)$-Tensoren mit koeffizientenweiser Addition und Multiplikation mit Skalaren ist ein Vektorraum.

Anwendung: Tensoren in Deep Learning

- Eine Matrix stellt z.B. ein (Graustufen-)Bild dar: Dimension 1 indiziert die Pixel vertikal (Höhe), Dimension 2 horizontal (Breite).
- **RGB-Bild**: dritte Dimension indiziert die Farbkanäle $\Longrightarrow$ Stufe-3-Tensor $\boldsymbol{I} \in \mathbb{R}^{224\times224\times3}$ ($\approx 150 \cdot 10^3$ Werte pro Bild)
- **Batch/Video**: vierte Dimension indiziert mehrere Bilder — z.B. verarbeitet ein CNN 32 Bilder gleichzeitig $\Longrightarrow$ Stufe-4-Tensor $\boldsymbol{B} \in \mathbb{R}^{32\times224\times224\times3}$ ($\approx 4.8 \cdot 10^6$ Werte)
- **Feature Maps**: Convolutional Layers bilden Tensoren auf Tensoren ab: c: $\mathbb{R}^{32\times224\times224\times3} \to \mathbb{R}^{32\times56\times56\times64}$
- Rechnungen auf Tensoren können durch spezielle Hardware besonders schnell durchgeführt werden (Google TPUs, NVIDIA Tensor Cores, ...) — essentiell für KI.

(Details & Feature-Map-Visualisierung → Anhang)



$\tilde{=}$ 3 Kanal-Matrizen

$\to$



Produkt von Tensoren

Produkt von Tensoren

Def.: Äußeres Produkt

Das **äußere Produkt** von zwei Vektoren $\boldsymbol{v} \in \mathbb{R}^m$ und $\boldsymbol{w} \in \mathbb{R}^n$ ist definiert als

$$\boldsymbol{v} \otimes \boldsymbol{w} := \boldsymbol{v}\boldsymbol{w}^\top = \begin{pmatrix} v_1 w_1 & \cdots & v_1 w_n \\ \vdots & & \vdots \\ v_m w_1 & \cdots & v_m w_n \end{pmatrix} \in \mathbb{R}^{m \times n}.$$

- *Äußeres Produkt*: $\otimes \colon \mathbb{R}^m \times \mathbb{R}^n \to \mathbb{R}^{m \times n}$
- vgl Skalarprodukt (“*inneres Produkt*”): $\langle \cdot, \cdot \rangle \colon \mathbb{R}^n \times \mathbb{R}^n \to \mathbb{R}$ $\langle \boldsymbol{v}, \boldsymbol{w} \rangle = \boldsymbol{v}^\top \boldsymbol{w}$

Äußeres Produkt: Beispiel & Eigenschaften

Beispiel:

$$\boldsymbol{v} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \quad \text{und} \quad \boldsymbol{w} = \begin{pmatrix} -2 \\ 3 \\ -11 \end{pmatrix}$$

$$\boldsymbol{v} \otimes \boldsymbol{w} = \boldsymbol{v}\boldsymbol{w}^\top = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \begin{pmatrix} -2 & 3 & -11 \end{pmatrix} = \begin{pmatrix} -2 & 3 & -11 \\ -4 & 6 & -22 \end{pmatrix}$$

- Alle Spalten sind Vielfache von $\boldsymbol{v}$, alle Zeilen sind Vielfache von $\boldsymbol{w}^\top$
  - Rang ist 1, sofern $\boldsymbol{v} \neq \boldsymbol{0}$ und $\boldsymbol{w} \neq \boldsymbol{0}$
- Was für eine lineare Abbildung ist $\boldsymbol{x} \to (\boldsymbol{v} \otimes \boldsymbol{w})\boldsymbol{x}$? (im Folgenden: $\boldsymbol{v} \neq \boldsymbol{0}$, $\boldsymbol{w} \neq \boldsymbol{0}$)
  - $(\boldsymbol{v} \otimes \boldsymbol{w})\boldsymbol{x} = \boldsymbol{v}\boldsymbol{w}^\top\boldsymbol{x} = \boldsymbol{v}\langle \boldsymbol{w}, \boldsymbol{x} \rangle$: Vektor $\boldsymbol{v}$ skaliert mit Skalarprodukt von $\boldsymbol{w}$ und $\boldsymbol{x}$
  - $\mathrm{col}(\boldsymbol{v} \otimes \boldsymbol{w}) = \mathrm{span}(\boldsymbol{v})$: Bild ist eindimensional
  - $\ker(\boldsymbol{v} \otimes \boldsymbol{w}) = \{\boldsymbol{x} : \boldsymbol{w}^\top \boldsymbol{x} = 0\}$: Hyperebene orthogonal zu $\boldsymbol{w}$

Anwendungsbeispiele:

- *SVD*: $\boldsymbol{A} = \boldsymbol{U}_r \boldsymbol{\Sigma}_r \boldsymbol{V}_r^\top = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \otimes \boldsymbol{v}_i$ (Darstellung einer Matrix als gewichtete Summe von normierten Rang-1-Matrizen)
- Statistik: $\mathrm{Cov}(\boldsymbol{x}) = \mathbb{E}\left[(\boldsymbol{x} - \mathbb{E}[\boldsymbol{x}])(\boldsymbol{x} - \mathbb{E}[\boldsymbol{x}])^\top\right] = \mathbb{E}\left[\boldsymbol{x} \otimes \boldsymbol{x}\right] - \mathbb{E}\left[\boldsymbol{x}\right] \otimes \mathbb{E}[\boldsymbol{x}]$
- KI: $\mathrm{Attention}(\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}) = \mathrm{softmax}\left(\frac{1}{\sqrt{d_k}} \boldsymbol{Q}\boldsymbol{K}^\top\right)\boldsymbol{V}$

Tensorprodukt

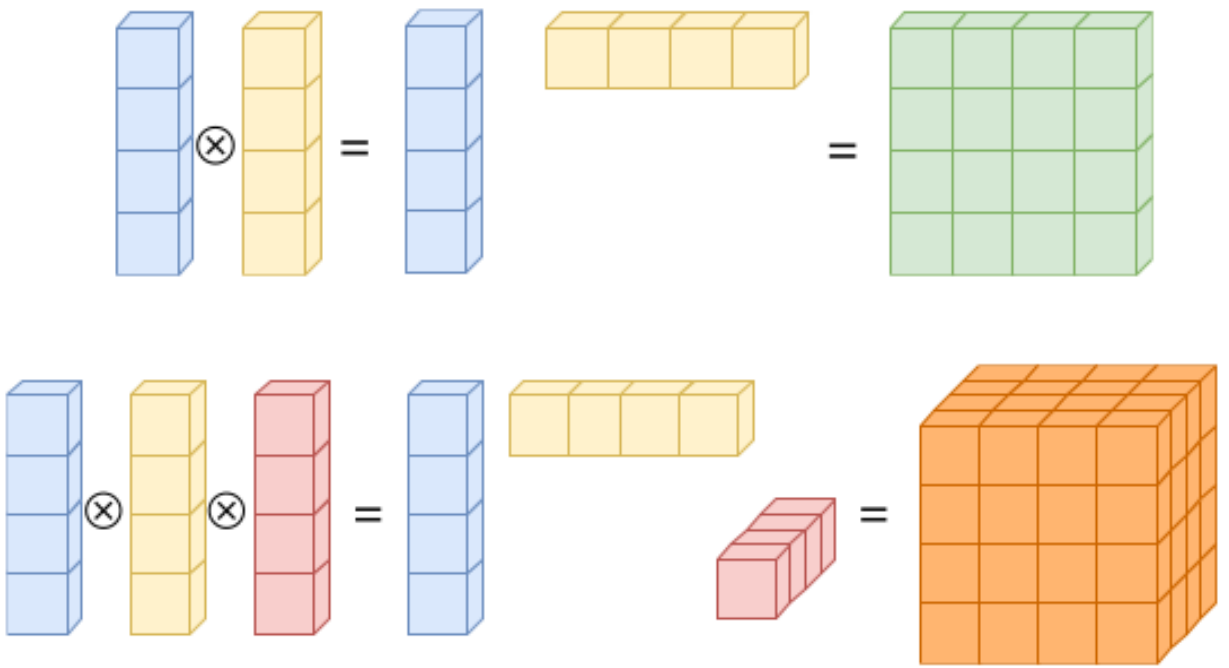
Def.: Tensorprodukt

Das **Tensorprodukt** $\otimes: \mathbb{R}^{m_1 \times \cdots \times m_p} \times \mathbb{R}^{n_1 \times \cdots \times n_q} \to \mathbb{R}^{m_1 \times \cdots \times m_p \times n_1 \times \cdots \times n_q}$ von zwei Tensoren $\boldsymbol{A} \in \mathbb{R}^{m_1 \times \cdots \times m_p}$ und $\boldsymbol{B} \in \mathbb{R}^{n_1 \times \cdots \times n_q}$ ist definiert als der Tensor $\boldsymbol{C} \in \mathbb{R}^{m_1 \times \cdots \times m_p \times n_1 \times \cdots \times n_q}$ mit

$$c_{i_1,\ldots,i_p,j_1,\ldots,j_q} = a_{i_1,\ldots,i_p} b_{j_1,\ldots,j_q}.$$

- *Äußeres Produkt* ist *Tensorprodukt* zweier Stufe-1-Tensoren ($p = q = 1$, also Vektoren)
- Das *Tensorprodukt* ist bilinear

Produkte von Tensoren



Oben: Das Produkt von zwei Stufe-1-Tensoren ergibt einen Stufe-2-Tensor. Unten: Das Produkt von drei Stufe-1-Tensoren ergibt einen Stufe-3-Tensor. Quelle: Deisenroth/Faisal/Ong, Fig 5.13

Bsp: Tensorprodukt

$$\boldsymbol{u} = \begin{pmatrix} 1 \\ 2 \end{pmatrix} \in \mathbb{R}^2, \quad \boldsymbol{v} = \begin{pmatrix} 3 \\ 5 \end{pmatrix} \in \mathbb{R}^2, \quad \boldsymbol{w} = \begin{pmatrix} 7 \\ 11 \end{pmatrix} \in \mathbb{R}^2$$

Tensorprodukt $\boldsymbol{T} = \boldsymbol{u} \otimes \boldsymbol{v} \otimes \boldsymbol{w} \in \mathbb{R}^{2\times 2\times 2}$ hat Einträge $\boldsymbol{T}_{i,j,k} = u_i v_j w_k$:

$$\begin{aligned}
\text{Slice 1 } (k=1, \text{ also } w_k = w_1 = 7)\text{:} \quad \boldsymbol{T}_{1,1,1} &= 1\cdot 3\cdot 7 = 21, & \boldsymbol{T}_{1,2,1} &= 1\cdot 5\cdot 7 = 35 \\
\boldsymbol{T}_{2,1,1} &= 2\cdot 3\cdot 7 = 42, & \boldsymbol{T}_{2,2,1} &= 2\cdot 5\cdot 7 = 70 \\
\text{Slice 2 } (k=2, \text{ also } w_k = w_2 = 11)\text{:} \quad \boldsymbol{T}_{1,1,2} &= 1\cdot 3\cdot 11 = 33, & \boldsymbol{T}_{1,2,2} &= 1\cdot 5\cdot 11 = 55 \\
\boldsymbol{T}_{2,1,2} &= 2\cdot 3\cdot 11 = 66, & \boldsymbol{T}_{2,2,2} &= 2\cdot 5\cdot 11 = 110
\end{aligned}$$

$$\implies \boldsymbol{T}_{\cdot,\cdot,1} = \begin{pmatrix} 21 & 35 \\ 42 & 70 \end{pmatrix}; \qquad \boldsymbol{T}_{\cdot,\cdot,2} = \begin{pmatrix} 33 & 55 \\ 66 & 110 \end{pmatrix}$$

Jede *slice* (Stufe-2-Tensor) dieses Stufe-3-Tensors ist eine Rang-1-Matrix:

- $\boldsymbol{T}_{\cdot,\cdot,1} = w_1 \cdot (\boldsymbol{u} \otimes \boldsymbol{v})$ und $\boldsymbol{T}_{\cdot,\cdot,2} = w_2 \cdot (\boldsymbol{u} \otimes \boldsymbol{v})$

Allgemein:
Ein Tensor der Form $\boldsymbol{u} \otimes \boldsymbol{v} \otimes \boldsymbol{w}$ hat *Rang 1* (für $\boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w} \neq \boldsymbol{0}$).

Kroneckerprodukt

Def.: Kroneckerprodukt

Das **Kroneckerprodukt** von zwei Matrizen $\boldsymbol{A} \in \mathbb{R}^{m \times n}$ und $\boldsymbol{B} \in \mathbb{R}^{p \times q}$ ist definiert als

$$\begin{aligned}
\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B} &:= \begin{pmatrix} a_{11}\boldsymbol{B} & \cdots & a_{1n}\boldsymbol{B} \\ \vdots & \vdots & \vdots \\ a_{m1}\boldsymbol{B} & \cdots & a_{mn}\boldsymbol{B} \end{pmatrix} \\
&= \begin{pmatrix} a_{11}b_{11} & \cdots & a_{11}b_{1q} & \cdots & a_{1n}b_{11} & \cdots & a_{1n}b_{1q} \\ \vdots & & \vdots & & \vdots & & \vdots \\ a_{11}b_{p1} & \cdots & a_{11}b_{pq} & \cdots & a_{1n}b_{p1} & \cdots & a_{1n}b_{pq} \\ \vdots & & \vdots & & \vdots & & \vdots \\ a_{m1}b_{11} & \cdots & a_{m1}b_{1q} & \cdots & a_{mn}b_{11} & \cdots & a_{mn}b_{1q} \\ \vdots & & \vdots & & \vdots & & \vdots \\ a_{m1}b_{p1} & \cdots & a_{m1}b_{pq} & \cdots & a_{mn}b_{p1} & \cdots & a_{mn}b_{pq} \end{pmatrix} \in \mathbb{R}^{mp \times nq}.
\end{aligned}$$

Obacht – Notation:

$\otimes$ ist oft doppelt belegt (Tensorprodukt und Kroneckerprodukt) – wir schreiben ab hier $\otimes_{\mathrm{K}}$ fürs **Kroneckerprodukt**.

Kroneckerprodukt vs. Tensorprodukt

Kroneckerprodukt ist Variante des Tensorprodukts für Matrizen – **dasselbe Zahlenmaterial, andere Anordnung**:

Für $\boldsymbol{A} \in \mathbb{R}^{m \times n}, \boldsymbol{B} \in \mathbb{R}^{p \times q}$

- `Tensorprodukt` ergibt hier **Stufe-4-Tensor**

$$\boldsymbol{C} = \boldsymbol{A} \otimes \boldsymbol{B} \in \mathbb{R}^{m \times n \times p \times q} \quad \text{mit } c_{i_1,i_2,j_1,j_2} = a_{i_1,i_2} b_{j_1,j_2}$$

- `Kroneckerprodukt` ergibt **Matrix**

$$\boldsymbol{D} = \boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B} \in \mathbb{R}^{mp \times nq} \quad \text{mit } d_{(i_1-1)p+j_1,(i_2-1)q+j_2} = a_{i_1,i_2} b_{j_1,j_2}$$

- $\boldsymbol{C}$ und $\boldsymbol{D}$ unterscheiden sich (nur) darin wie ihre Einträge angeordnet sind – $\boldsymbol{D}$ ist *"abgeflachte"* Variante von $\boldsymbol{C}$

Notation:

- In der Literatur steht meist $\otimes$ für *beides* (!); welche Bedeutung gemeint ist, verrät dort das **Format** des Ergebnisses.
- In FMM ist $\otimes_{\mathrm{K}}$ immer das `Kroneckerprodukt`
- $\otimes$ bleibt `äußerem Produkt` bzw. `Tensorprodukt` vorbehalten.

> ⓘ **Hinweis**

Bsp: Kroneckerprodukt

Beispiel:

- Für

$$\boldsymbol{A} = \begin{pmatrix} 1 & 0 \\ 2 & 5 \end{pmatrix}, \qquad \boldsymbol{B} = \begin{pmatrix} 3 & 0 & 0 \\ 0 & 2 & 0 \\ -1 & 0 & -1 \end{pmatrix}$$

- Gilt

$$\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B} = \begin{pmatrix} 1 \cdot \boldsymbol{B} & 0 \cdot \boldsymbol{B} \\ 2 \cdot \boldsymbol{B} & 5 \cdot \boldsymbol{B} \end{pmatrix} = \begin{pmatrix} 3 & 0 & 0 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 & 0 & 0 \\ -1 & 0 & -1 & 0 & 0 & 0 \\ 6 & 0 & 0 & 15 & 0 & 0 \\ 0 & 4 & 0 & 0 & 10 & 0 \\ -2 & 0 & -2 & -5 & 0 & -5 \end{pmatrix}$$

Vorsicht: $\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B} \neq \boldsymbol{B} \otimes_{\mathrm{K}} \boldsymbol{A}$! *Aber:* $(\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B})^\top = \boldsymbol{A}^\top \otimes_{\mathrm{K}} \boldsymbol{B}^\top$ – Transponieren dreht die Reihenfolge **nicht** um.

Quiz

Frage 1: Sei $\boldsymbol{A} \in \mathbb{R}^{3\times 2}$ und $\boldsymbol{B} \in \mathbb{R}^{2\times 4}$. In welchem Raum liegt das **Kroneckerprodukt** $\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B}$?

1. $\mathbb{R}^{6\times 8}$
2. $\mathbb{R}^{5\times 6}$
3. $\mathbb{R}^{3\times 2\times 2\times 4}$
4. $\mathbb{R}^{8\times 6}$

Frage 2: Sei $\boldsymbol{S} \in \mathbb{R}^{p\times p}$ eine Matrix mit Einträgen s_{ij}. Wie sieht das Kroneckerprodukt $\boldsymbol{S} \otimes_{\mathrm{K}} \boldsymbol{I}_n$ aus?

Anwendung: Blockdiagonale Matrizen

Kroneckerprodukt um Blockdiagonalmatrizen zu erzeugen:

$$\boldsymbol{I}_n \otimes_{\mathrm{K}} \underbrace{\boldsymbol{S}}_{p \times p} = \underbrace{\begin{pmatrix} \boldsymbol{S} & & & \\ & \boldsymbol{S} & & \\ & & \ddots & \\ & & & \boldsymbol{S} \end{pmatrix}}_{np \times np}$$

- Nützlich z.B. in Statistik für Kovarianzmatrizen von Vektoren mit unabhängigen Blöcken mit identischer Kovarianz
- **Achtung:**
  $\boldsymbol{I}_n \otimes_{\mathrm{K}} \boldsymbol{S}$ ist blockdiagonal, $\boldsymbol{S} \otimes_{\mathrm{K}} \boldsymbol{I}_n$ ist es i.A. *nicht* — auf die **Reihenfolge der Faktoren** kommt es an!

Anwendung: Separierbare Kovarianz

- z.B. Daten an m verschiedenen *Orten* zu n verschiedenen *Zeitpunkten* (Annahme hier: Daten geordnet Ort 1 Zeit 1, Ort 2 Zeit 1, Ort 1 Zeit 2, Ort 2 Zeit 2)
- *Modellannahme:* **Separierbare** Kovarianz mit $\mathbf{\Sigma} = \mathbf{\Sigma}_T \otimes_{\mathrm{K}} \mathbf{\Sigma}_S \in \mathbb{R}^{mn \times mn}$, wobei
  - $\mathbf{\Sigma}_T \in \mathbb{R}^{n \times n}$: Kovarianzmatrix zwischen Zeitpunkten
  - $\mathbf{\Sigma}_S \in \mathbb{R}^{m \times m}$: Kovarianzmatrix zwischen Orten
  - **Vorteile:** Parametersparsam: $mn(mn+1)/2$ vs. $m(m+1)/2 + n(n+1)/2$; einfachere Interpretation

Beispiel $m = 2$ Orte, $n = 2$ Zeitpunkte, $\mathbf{\Sigma}_T = \begin{pmatrix} 1.0 & 0.8 \\ 0.8 & 1.0 \end{pmatrix}$, $\mathbf{\Sigma}_S = \begin{pmatrix} 2.0 & 0.5 \\ 0.5 & 2.0 \end{pmatrix}$:

$$\mathbf{\Sigma} = \mathbf{\Sigma}_T \otimes_{\mathrm{K}} \mathbf{\Sigma}_S = \begin{pmatrix} 1.0 \cdot \mathbf{\Sigma}_S & 0.8 \cdot \mathbf{\Sigma}_S \\ 0.8 \cdot \mathbf{\Sigma}_S & 1.0 \cdot \mathbf{\Sigma}_S \end{pmatrix} = \begin{pmatrix} 2.0 & 0.5 & 1.6 & 0.4 \\ 0.5 & 2.0 & 0.4 & 1.6 \\ 1.6 & 0.4 & 2.0 & 0.5 \\ 0.4 & 1.6 & 0.5 & 2.0 \end{pmatrix} \in \mathbb{R}^{4 \times 4}$$

Interpretation:
Eintrag $(1,1)$: Varianz in Ort 1 zu Zeit 1 = 2.0; Eintrag $(1,3)$: Kovarianz zwischen Ort 1, Zeit 1 und Ort 1, Zeit 2 = $1.6 = 0.8 \cdot 2.0$

Anwendung: vec-Trick — Matrixgleichungen als LGS

Für $\boldsymbol{X} \in \mathbb{R}^{m \times n}$ stapelt $\operatorname{vec}(\boldsymbol{X}) = (x_{11}, \ldots, x_{m1}, x_{12}, \ldots, x_{mn})^\top \in \mathbb{R}^{mn}$ die Spalten von $\boldsymbol{X}$ zu einem Vektor.

vec-Trick

Für $\boldsymbol{A} \in \mathbb{R}^{k \times m}$, $\boldsymbol{X} \in \mathbb{R}^{m \times n}$, $\boldsymbol{B} \in \mathbb{R}^{n \times l}$ gilt

$$\operatorname{vec}(\boldsymbol{AXB}) = \left(\boldsymbol{B}^\top \otimes_{\mathrm{K}} \boldsymbol{A}\right) \operatorname{vec}(\boldsymbol{X}).$$

- Macht lineare **Matrix**gleichungen zu gewöhnlichen **LGS** $\boldsymbol{Mz} = \boldsymbol{c} \to$ lösbar mit den Methoden aus Kapitel *Gleichungssysteme* (LU, Cholesky, …)!
- Wichtige Spezialfälle ($\boldsymbol{I}_n$: Einheitsmatrix):

$$\operatorname{vec}(\boldsymbol{AX}) = (\boldsymbol{I}_n \otimes_{\mathrm{K}} \boldsymbol{A}) \operatorname{vec}(\boldsymbol{X}), \qquad \operatorname{vec}(\boldsymbol{XB}) = \left(\boldsymbol{B}^\top \otimes_{\mathrm{K}} \boldsymbol{I}_m\right) \operatorname{vec}(\boldsymbol{X})$$

- **Reihenfolge merken:**
  $\boldsymbol{B}^\top \otimes_{\mathrm{K}} \boldsymbol{A}$ und $\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B}^\top$ haben *immer* dieselbe Größe — ein Vertauschen fällt nicht durch unpassende Formate auf!

Bsp: vec-Trick löst die Sylvester-Gleichung

Gesucht:
$\boldsymbol{X} \in \mathbb{R}^{2\times 2}$ mit $\boldsymbol{AX} + \boldsymbol{XB} = \boldsymbol{C}$ *(z.B. stationäre Kovarianzen von Zeitreihenmodellen: Lyapunov-Gleichung)*

vec-Trick auf beide Summanden ($m = n = 2$):

$$\operatorname{vec}(\boldsymbol{AX} + \boldsymbol{XB}) = \left(\boldsymbol{I}_2 \otimes_{\mathrm{K}} \boldsymbol{A} + \boldsymbol{B}^\top \otimes_{\mathrm{K}} \boldsymbol{I}_2\right)\operatorname{vec}(\boldsymbol{X}) = \operatorname{vec}(\boldsymbol{C})$$

$\Longrightarrow$ ein ganz normales 4×4-LGS für $\operatorname{vec}(\boldsymbol{X})$:

```r
A <- matrix(c(3, 1, 0, 2), 2, 2)
B <- matrix(c(0, 1, 1, 1), 2, 2)
C <- matrix(c(1, 2, 3, 4), 2, 2)
# Systemmatrix des LGS via Kroneckerprodukte:
M <- kronecker(diag(2), A) + kronecker(t(B), diag(2))
X <- matrix(solve(M, as.vector(C)), 2, 2)
A %*% X + X %*% B                  # = C ?
all.equal(A %*% X + X %*% B, C)
```

```
     [,1] [,2]
[1,]    1    3
[2,]    2    4
[1] TRUE
```

- Ohne vec-Trick keine LGS-Form für unsere Löser aus Kap. 5 — mit: `solve()` genügt. *(Für große Matrizen gibt es effizientere spezialisierte Sylvester-Löser.)*

Tensorprodukt von Vektorräumen

Tensorprodukt von Vektorräumen

Def.: Tensorprodukt von Vektorräumen

Seien V und W zwei Vektorräume. Dann ist das **Tensorprodukt** $V \otimes W$ definiert als der Vektorraum

$$V \otimes W = \operatorname{span}\{\boldsymbol{v} \otimes \boldsymbol{w} \colon \boldsymbol{v} \in V, \boldsymbol{w} \in W\},$$

wobei $(\boldsymbol{v}, \boldsymbol{w}) \mapsto \boldsymbol{v} \otimes \boldsymbol{w}$ eine bilineare Abbildung ist.

- *Bilinearität* bedeutet: $(c\boldsymbol{v}) \otimes \boldsymbol{w} = c(\boldsymbol{v} \otimes \boldsymbol{w}) = \boldsymbol{v} \otimes (c\boldsymbol{w})$, $(\boldsymbol{v}_1 + \boldsymbol{v}_2) \otimes \boldsymbol{w} = \boldsymbol{v}_1 \otimes \boldsymbol{w} + \boldsymbol{v}_2 \otimes \boldsymbol{w}$ und $\boldsymbol{v} \otimes (\boldsymbol{w}_1 + \boldsymbol{w}_2) = \boldsymbol{v} \otimes \boldsymbol{w}_1 + \boldsymbol{v} \otimes \boldsymbol{w}_2$

Beispiel:

- Sei $V = \mathbb{R}^m$, $W = \mathbb{R}^n$ und $\otimes$ das äußere Produkt. Dann ist $V \otimes W = \mathbb{R}^m \otimes \mathbb{R}^n = \mathbb{R}^{m \times n}$.

Beispiel:

- Seien $V_1 = \mathbb{R}^m$, $V_2 = \mathbb{R}^n$, $V_3 = \mathbb{R}^q$ und $\otimes$ das Tensorprodukt. Dann ist

  $V_1 \otimes V_2 \otimes V_3 = \mathbb{R}^m \otimes \mathbb{R}^n \otimes \mathbb{R}^q = (\mathbb{R}^m \otimes \mathbb{R}^n) \otimes \mathbb{R}^q = \mathbb{R}^{m \times n} \otimes \mathbb{R}^q = \mathbb{R}^{m \times n \times q}$.

Bsp: $V \otimes W$ vs. $\boldsymbol{v} \otimes \boldsymbol{w}$

Wichtige Unterscheidung:

- $\boldsymbol{v} \otimes \boldsymbol{w}$: Ein einzelnes Tensorprodukt (Rang-1-Matrix für $\boldsymbol{v}, \boldsymbol{w} \neq \mathbf{0}$)
- $V \otimes W$: Der **gesamte Vektorraum**, der von allen solchen Produkten erzeugt wird

Beispiel: Sei $V = W = \mathbb{R}^2$

- $\boldsymbol{A} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ **nicht** als einzelnes Tensorprodukt $\boldsymbol{v} \otimes \boldsymbol{w}$ von $\boldsymbol{v}, \boldsymbol{w} \in \mathbb{R}^2$ darstellbar (Rang 2 $\neq$ 1).
- **Aber:** $\boldsymbol{A} \in V \otimes W = \mathbb{R}^{2\times 2}$:

$$\boldsymbol{A} = \boldsymbol{e}_1 \otimes \boldsymbol{e}_1 + \boldsymbol{e}_2 \otimes \boldsymbol{e}_2 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & 0 \end{pmatrix} + \begin{pmatrix} 0 \\ 1 \end{pmatrix} \begin{pmatrix} 0 & 1 \end{pmatrix}$$
$$= \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \quad \checkmark$$

Allgemein:

Jede Matrix $\boldsymbol{A} \in \mathbb{R}^{m\times n}$ kann als Summe von höchstens $\min(m, n)$ Tensorprodukten geschrieben werden: $\boldsymbol{A} = \sum_{i=1}^{r} \sigma_i \boldsymbol{u}_i \otimes \boldsymbol{v}_i$ mit $r = \operatorname{rank}(\boldsymbol{A}) \rightarrow$ *Singulärwertzerlegung*!

Tensorproduktbasis für Vektorräume

Theorem

Sind $\mathcal{B}_V = \{\boldsymbol{v}_1, \dots, \boldsymbol{v}_m\}$ und $\mathcal{B}_W = \{\boldsymbol{w}_1, \dots, \boldsymbol{w}_n\}$ Basen von V bzw. W, dann ist

$$\mathcal{B}_{V\otimes W} = \{\boldsymbol{v} \otimes \boldsymbol{w} \colon \boldsymbol{v} \in \mathcal{B}_V, \boldsymbol{w} \in \mathcal{B}_W\}$$

eine Basis von $V \otimes W$ und wird **Tensorproduktbasis** genannt. Insbesondere gilt

$$V \otimes W = \operatorname{span}\left(\mathcal{B}_{V\otimes W}\right) = \left\{ \sum_{i=1}^{m} \sum_{j=1}^{n} c_{i,j} \boldsymbol{v}_i \otimes \boldsymbol{w}_j \colon c_{1,1}, \dots, c_{m,n} \in \mathbb{R} \right\}$$

und $\dim(V \otimes W) = \dim(V) \dim(W)$.

Bsp: Tensorproduktbasis für bivariate Polynome

Univariate Basis:
Raum der Polynome vom Grad ≤ 1 auf $[0,1]$: $\mathcal{P}_1 = \{p(t) = a_0 + a_1 t \colon a_0, a_1 \in \mathbb{R}\}$ mit Basis $\mathcal{B} = \{b_1, b_2\}$, $b_1(t) = 1, b_2(t) = t$.

Tensorproduktbasis für bivariate Funktionen $f \colon [0,1] \times [0,1] \to \mathbb{R}$:
Basisfunktionen $\phi_{jk} = b_j \otimes b_k$ mit $(b_j \otimes b_k)(x,y) = b_j(x) \cdot b_k(y)$, also

$$\begin{aligned} \phi_{11}(x,y) &= b_1(x)b_1(y) = 1 \quad ; \quad \phi_{12}(x,y) = b_1(x)b_2(y) = y \\ \phi_{21}(x,y) &= b_2(x)b_1(y) = x \quad ; \quad \phi_{22}(x,y) = b_2(x)b_2(y) = xy \end{aligned}$$

Approximation einer bivariaten Funktion:
Z.B. $f(x,y) = 2 - x + 3y + 5xy = 2\phi_{11}(x,y) + 3\phi_{12}(x,y) - 1\phi_{21}(x,y) + 5\phi_{22}(x,y)$

Dimension: $\dim(\mathcal{P}_1 \otimes \mathcal{P}_1) = 2 \cdot 2 = 4$ ✓

Allgemein: Für k-variate Tensorprodukt-Polynome vom Grad $\leq d$ *in jeder Variablen* (univariat: $\mathcal{P}_d = \left\{p(t) = \sum_{r=0}^{d} a_r t^r \colon a_r \in \mathbb{R}\right\}$): $\dim(\mathcal{P}_d^{\otimes k}) = (d+1)^k$

Anwendung in der Praxis:

- **Statistik:** Tensorprodukt-Splines für multivariate Regression (GAMs) — ausführlich im Kapitel *Funktionsapproximation*
- **Finite-Elemente-Methoden (FEM):** Approximation von PDEs mit tensorbasierten Polynomen
- **Numerik:** Sparse Grids vermeiden die exponentielle Anzahl $(d+1)^k$ durch geschickte Auswahl

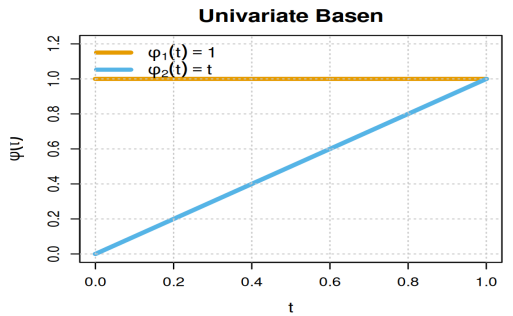
Bsp: Tensorproduktbasis für bivariate Polynome

Tensorproduktbasis

$P_1 \times P_1$

Univariate Basen:

$B_x = \{1, x\}$



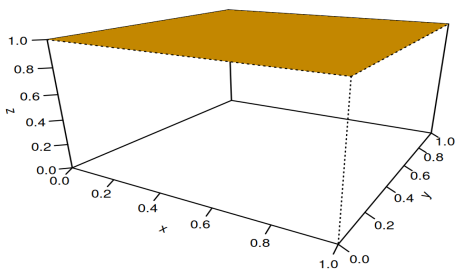
$B_y = \{1, y\}$

Tensorprodukt:

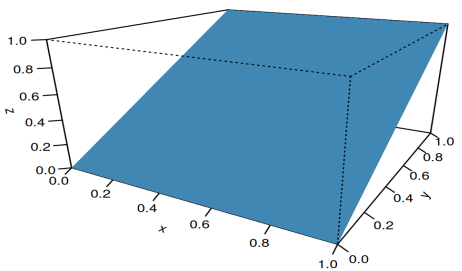
$B_x \times B_y$

erzeugt 4 bivariate

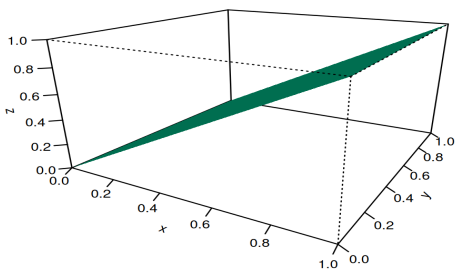
$\varphi_1 = 1 \times 1$



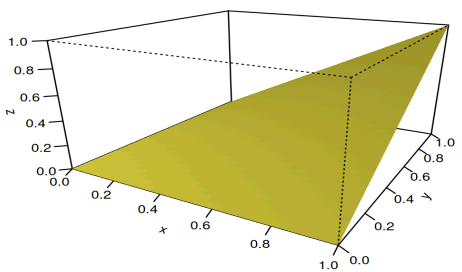
$\varphi_2 = 1 \times y$



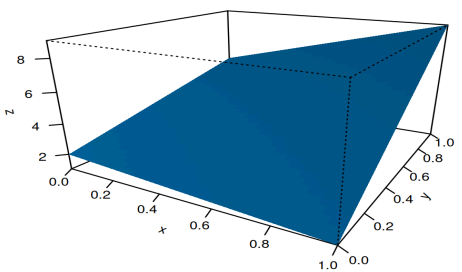
$\varphi_3 = x \times 1$



$\varphi_4 = x \times y$



Beispiel: $f(x, y) = 2 - x + 3y + 5xy$



Anwendung: Tensorprodukt-Splines — Designmatrix

Vorgriff aufs Kapitel *Funktionsapproximation*: flexible univariate Funktionen via *B-Spline*-Basis $B_1, \ldots, B_K$; bivariat via *Tensorproduktbasis* $\phi_{jk}(x, y) = B_j(x) B_k(y)$.

- Auswertung von $f(x, y) = \sum_{j,k} c_{jk} \phi_{jk}(x, y)$ auf einem **Gitter** aus x- und y-Werten:

$$\underbrace{\boldsymbol{F}}_{f(x_i, y_l)} = \boldsymbol{B}_x \boldsymbol{C} \boldsymbol{B}_y^\top$$

  mit univariaten Basismatrizen $\boldsymbol{B}_x \in \mathbb{R}^{n_x \times K}$, $(\boldsymbol{B}_x)_{ij} = B_j(x_i)$ (analog $\boldsymbol{B}_y$)

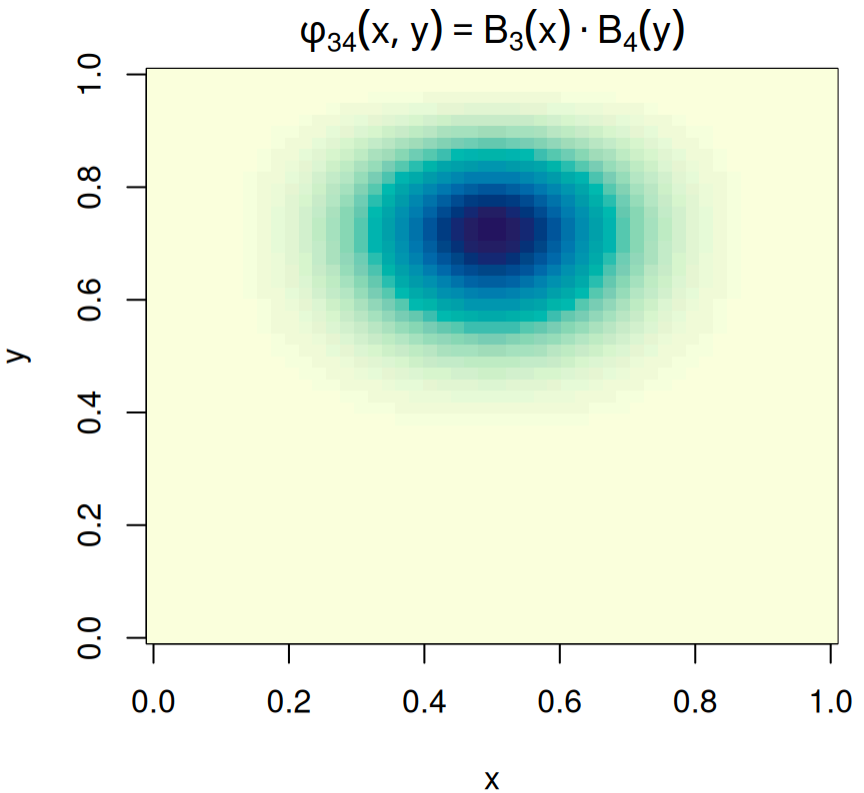
- **vec-Trick!**

$$\operatorname{vec}(\boldsymbol{F}) = (\boldsymbol{B}_y \otimes_{\mathrm{K}} \boldsymbol{B}_x) \operatorname{vec}(\boldsymbol{C})$$

  $\Longrightarrow$ Designmatrix des Gitters ist das **Kroneckerprodukt** $\boldsymbol{B} = \boldsymbol{B}_y \otimes_{\mathrm{K}} \boldsymbol{B}_x$

```r
library(splines)
x <- seq(0, 1, length.out = 50)
Bx <- bs(x, df = 6)  # B-Spline-Basis
By <- bs(x, df = 6)
B <- kronecker(By, Bx)
dim(B)  # 2500 Gitterpunkte, 6*6 Koeffizienten
```

```
[1] 2500   36
```



Eine bivariate Basisfunktion = äußeres Produkt zweier univariater B-Splines. → Kap. *Funktionsapproximation I/II*: gute univariate Basen $\otimes$-kombinieren!

Self-Check & Zusammenfassung

Kernkonzepte & Anwendungen

- *Tensoren*: höher-dimensionale Verallgemeinerung von Vektoren und Matrizen
  - Extrem wichtig für Deep Learning, Bild-/Videobearbeitung, moderne Statistik
- *Multilinearität* vs Linearität:
  - Linear: $f(a\boldsymbol{v} + b\boldsymbol{w}) = af(\boldsymbol{v}) + bf(\boldsymbol{w})$ vs. Multilinear:
    $f(\boldsymbol{v}_1, \ldots, a\boldsymbol{v}_i + b\boldsymbol{w}_i, \ldots, \boldsymbol{v}_n) = af(\boldsymbol{v}_1, \ldots, \boldsymbol{v}_i, \ldots, \boldsymbol{v}_n) + bf(\boldsymbol{v}_1, \ldots, \boldsymbol{w}_i, \ldots, \boldsymbol{v}_n)$
  - Bsp: Matrixmultiplikation bilinear aber nicht linear.
- *Äußeres Produkt*: Erzeugt (Rang-1-)Matrizen aus Vektoren ($\rightarrow$ *SVD*)
- *Tensorprodukt*: Verallgemeinerung des äußeren Produkts auf Tensoren höherer Stufe
- *Kroneckerprodukt* $\otimes_{\mathrm{K}}$: Spezielle Variante des Tensorprodukts von Matrizen ($\otimes$)
  - **vec-Trick** $\mathrm{vec}(\boldsymbol{AXB}) = (\boldsymbol{B}^\top \otimes_{\mathrm{K}} \boldsymbol{A})\mathrm{vec}(\boldsymbol{X})$: Matrixgleichungen (Sylvester/Lyapunov) $\rightarrow$ gewöhnliche LGS
  - Statistik: separierbare/Kronecker-strukturierte Kovarianzen, blockdiagonale Matrizen
- *Tensorprodukte bauen komplexe Räume auf*:
  - Konstruiere multivariate (Funktions-)Räume aus univariaten — Designmatrix auf Gittern = Kroneckerprodukt $\rightarrow$ **Funktionsapproximation**, (Höherdimensionale) *Splines*

Nächste Vorlesungen:
Fortgeschrittene Differentialrechnung

Self-Check

Frage 1: Daten an $m = 10$ Orten zu $n = 20$ Zeitpunkten. Eine unstrukturierte Kovarianzmatrix des 200-dimensionalen Datenvektors hat $\frac{200 \cdot 201}{2} = 20100$ freie Parameter. Wie viele Parameter haben die beiden Faktoren $\boldsymbol{\Sigma}_T, \boldsymbol{\Sigma}_S$ des separierbaren Modells $\boldsymbol{\Sigma} = \boldsymbol{\Sigma}_T \otimes_{\mathrm{K}} \boldsymbol{\Sigma}_S$ zusammen?

a) 265 **b)** 11550 **c)** 20100 **d)** 200

Frage 2: Sei $f\colon \mathbb{R}^2 \times \mathbb{R}^2 \to \mathbb{R}$ bilinear. Was ist $f(2\boldsymbol{v}, 3\boldsymbol{w})$?

a) $2f(\boldsymbol{v}, \boldsymbol{w})$ **b)** $5f(\boldsymbol{v}, \boldsymbol{w})$ **c)** $6f(\boldsymbol{v}, \boldsymbol{w})$ **d)** $f(\boldsymbol{v}, \boldsymbol{w})$

Frage 3: Der vec-Trick besagt $\mathrm{vec}(\boldsymbol{AXB}) = \,?$

a) $(\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B})\,\mathrm{vec}(\boldsymbol{X})$ **b)** $(\boldsymbol{B}^\top \otimes_{\mathrm{K}} \boldsymbol{A})\,\mathrm{vec}(\boldsymbol{X})$ **c)** $(\boldsymbol{B} \otimes_{\mathrm{K}} \boldsymbol{A}^\top)\,\mathrm{vec}(\boldsymbol{X})$ **d)** $(\boldsymbol{A} \otimes_{\mathrm{K}} \boldsymbol{B}^\top)\,\mathrm{vec}(\boldsymbol{X})$

Anhang

Anwendung: RGB-Bilder als Tensoren

Beispiel 1: RGB-Bild

- Farbbild der Größe 224×224 Pixel wird als Tensor $\boldsymbol{I} \in \mathbb{R}^{224 \times 224 \times 3}$ dargestellt:
  - $\boldsymbol{I}_{i,j,1}$: Rot-Intensität an Position (i, j) (Wert zwischen 0 und 255)
  - $\boldsymbol{I}_{i,j,2}$: Grün-Intensität an Position (i, j)
  - $\boldsymbol{I}_{i,j,3}$: Blau-Intensität an Position (i, j)
- Einträge: $224 \times 224 \times 3 \approx 150 \cdot 10^3$ Werte pro Bild



$\simeq$



↩ zurück: „Tensoren in Deep Learning"

Anwendung: Batch-Tensoren in CNNs

Beispiel 2: Batch von Bildern (Convolutional Neural Networks) (CNNs)

- CNN verarbeitet typischerweise mehrere Bilder gleichzeitig: $\boldsymbol{B} \in \mathbb{R}^{32\times224\times224\times3}$
  1. Dimension (Batch-Size): 32 Bilder gleichzeitig
  2. Dimension (Höhe): 224 Pixel
  3. Dimension (Breite): 224 Pixel
  4. Dimension (Kanäle): 3 Farbkanäle (RGB)
- Einträge: $32 \times 224 \times 224 \times 3 \approx 4.8 \cdot 10^6$ Werte

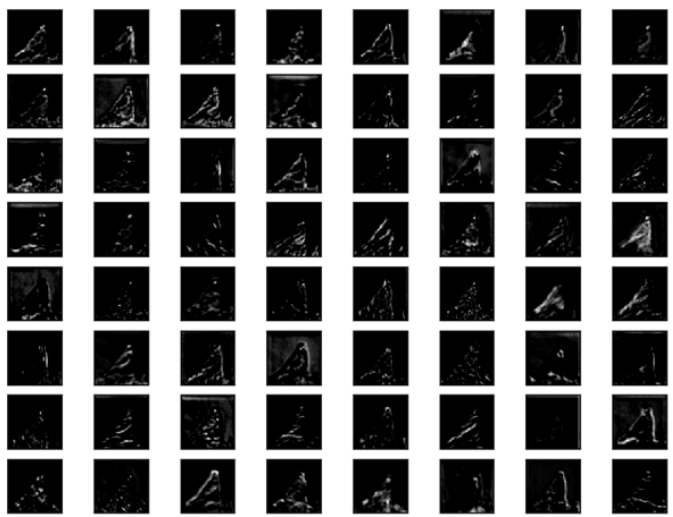
Anwendung: Feature Maps in CNNs

Beispiel 3: Feature Maps in CNNs

- Convolutional Layer c: $\mathbb{R}^{32\times224\times224\times3} \to \mathbb{R}^{32\times56\times56\times64}$
- Reduziert (üblicherweise) räumliche Auflösung $(224 \times 224 \to 56 \times 56)$
- Erhöht Anzahl der “Kanäle” (*features*) $(3 \to 64)$



$\to$



Feature Map: Block 3 des VGG16-Modells; Quelle: machinelearningmastery.com

↩ zurück: „Tensoren in Deep Learning”

Universelle Eigenschaft des Tensorprodukts

- Das *Tensorprodukt* $V \otimes W$ hat eine *universelle Eigenschaft*: **Jede** bilineare Abbildung $f\colon V \times W \to Z$ kann eindeutig als lineare Abbildung $\tilde{f}\colon V \otimes W \to Z$ faktorisiert werden:

$$f(\boldsymbol{v}, \boldsymbol{w}) = \tilde{f}(\boldsymbol{v} \otimes \boldsymbol{w}) \quad \text{für alle } \boldsymbol{v} \in V, \boldsymbol{w} \in W$$

- Diese Eigenschaft macht das Tensorprodukt zur “natürlichen” Konstruktion für die Linearisierung bilinearer Abbildungen.

Differentialrechnung I

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- Matrix-*Spur* & *Matrixnormen*
- Landau-Notation: $g(h) = o(h) \iff \lim_{h \to 0} \frac{g(h)}{h} = 0$
- **Analysis** (essentiell)
  - Ableitungen: $f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$
  - Partielle Ableitungen: $\frac{\partial f(\boldsymbol{x})}{\partial x_i}$
  - Lineare Approximation: $f(x + h) \approx f(x) + f'(x)h$
  - Grundlagen Taylorreihen
- **Lineare Algebra**
  - Matrixinversion und Determinante
  - Normierte (Vektor-)Räume
  - Beschränkte lineare Operatoren

Opener-Quiz: Was wissen Sie noch?

Frage 1: Seien $g, h\colon \mathbb{R} \to \mathbb{R}$ differenzierbar. Welche Rechenregeln stimmen?

1. $(g + h)'(x) = g'(x) + h'(x)$
2. $(g \cdot h)'(x) = g'(x) \cdot h'(x)$
3. $(g \cdot h)'(x) = g'(x)h(x) + g(x)h'(x)$
4. $(g \circ h)'(x) = g'\big(h(x)\big) \cdot h'(x)$

Frage 2: Welche der folgenden Abbildungen sind *linear*?

1. $L(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x}$ mit $\boldsymbol{A} \in \mathbb{R}^{m \times n}$, $\boldsymbol{x} \in \mathbb{R}^n$
2. $L(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}$ mit $\boldsymbol{b} \neq \boldsymbol{0}$
3. $L(x) = c \cdot x$ mit $c \in \mathbb{R}$, $x \in \mathbb{R}$
4. $L(x) = x^2$

Überblick

- Aus Analysis bekannt: Ableitungen, *Differenzierbarkeit*
- Neue wichtige Technik: **Allgemeinerer Ableitungsbegriff** (*Fréchet-Ableitung*)
  - Ableitungen mit **Matrix-/Vektor-Inputs** oder **-Outputs**
  - Ableitungen in *allgemeineren (normierten) Räumen*

Ableitung von univariaten Funktionen

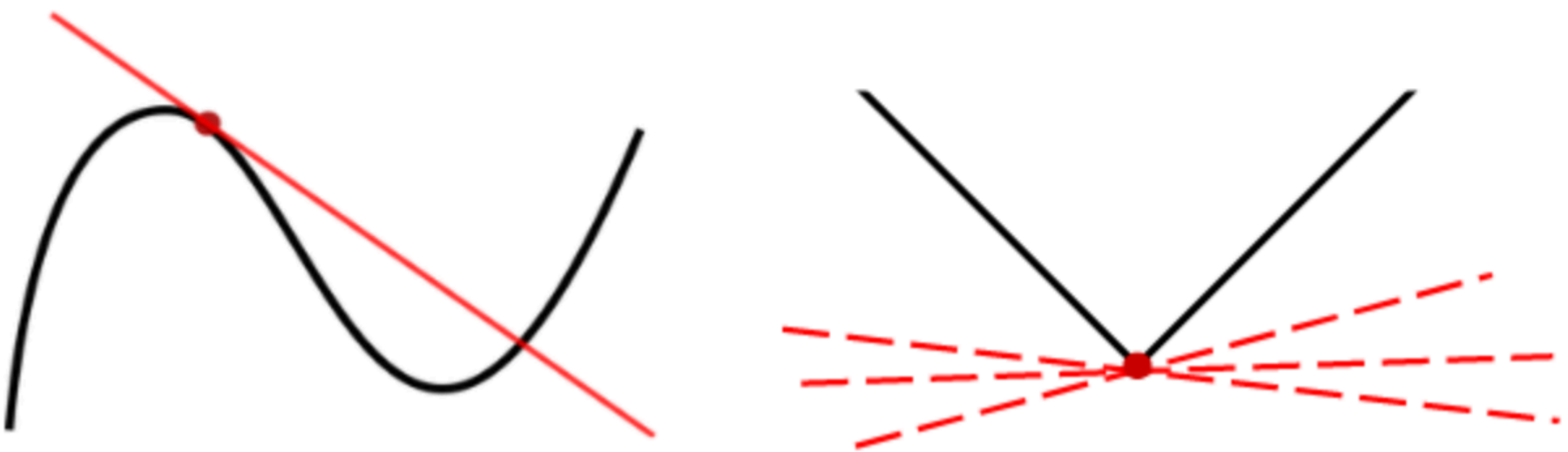
Recap: Ableitung von univariaten Funktionen

Def.: Differenzierbarkeit

Sei $S \subseteq \mathbb{R}$ offen und $f\colon S \to \mathbb{R}$ eine Funktion.
Wir nennen f an der Stelle $x \in S$ **differenzierbar** mit **Ableitung** $f'(x)$, falls

$$\lim_{h \to 0} \frac{f(x+h) - f(x)}{h} = f'(x).$$

- $f'(x)$ ist die **Steigung einer Tangente an f durch x**.
- Falls diese Steigung nicht eindeutig ist, ist f an der Stelle x nicht differenzierbar:



Quiz

Sei f differenzierbar in x mit Ableitung $f'(x)$.

Frage: Welche Aussagen sind wahr?

1. f' ist eine lineare Funktion.
2. $h \mapsto f'(x)h$ ist eine lineare Funktion (für festes x).
3. $\lim_{h \to 0} \dfrac{|f(x+h) - f(x) - f'(x)h|}{h} = 0.$
4. $f(x+h) = f(x) + f'(x)h + o(h).$

Beweis (3) $\iff$ (4): $\to$ *Anhang.*

Allgemeiner Ableitungsbegriff

Allgemeiner Ableitungsbegriff

- Ableitungen können auf verschiedene Arten auf allgemeinere, `normierte` Räume erweitert werden.
- Die Idee ist immer: ***Ableitung*** **= lineare Approximation**.
- Die Definition nach Fréchet reicht aus für
  - alle endlich-dimensionalen Räume,
  - einige unendlich-dimensionale Räume.
- Allgemeinere Definitionen: *Gateaux-*, *Hadamard-Ableitungen*.

Allgemeiner Ableitungsbegriff

Def.: (Fréchet-)differenzierbar

Seien $\mathbb{D}$ und $\mathbb{E}$ zwei normierte Räume und $f: \mathbb{D} \to \mathbb{E}$.

Wir nennen f an der Stelle $x \in \mathbb{D}$ **(Fréchet-)differenzierbar**, falls eine beschränkte lineare Abbildung $D_x f: \mathbb{D} \to \mathbb{E}$ existiert, sodass

$$f(x+h) = f(x) + D_x f(h) + o(\|h\|) \quad \text{für } h \to 0.$$

- $D_x f(h)$ *beschränkt*: Es gibt $M < \infty$, sodass $\|D_x f(h)\| \leq M \|h\| \; \forall h$.
- $h \to D_x f(h)$ ist *linear in* h, hängt aber in beliebiger Weise von x ab.
- *Beispiel:*
  - Für $f: \mathbb{R} \to \mathbb{R}$ ist $D_x f(h) = f'(x)h$.
  - $D_x f$ ist beschränkt, wenn $|f'(x)| < \infty$.
- Fehler $o(\|h\|)$: wird *relativ* zu $\|h\|$ beliebig klein, wenn $h \to 0$

Konkret: Fréchet-Ableitung von $f(x) = x^2$

Sei $f\colon \mathbb{R} \to \mathbb{R}$ mit $f(x) = x^2$. *Zu Zeigen:* f Fréchet-differenzierbar mit $D_x f(h) = 2xh$.

Prüfen der Definition:

$$\begin{aligned} f(x+h) &= (x+h)^2 = x^2 + 2xh + h^2 \\ &= f(x) + 2xh + h^2 \\ &= f(x) + D_x f(h) + h^2 \\ \implies D_x f(h) &= 2xh \ (\text{linear in } h) \end{aligned}$$

- $\lim_{h\to 0} \frac{|h^2|}{|h|} = \lim_{h\to 0} |h| = 0 \implies h^2 = o(|h|)$ ✓
- $|D_x f(h)| = |2xh| = 2|x| \cdot |h| \leq M|h| \ \forall h$ mit $M = 2|x| < \infty$
  $\implies D_x f$ beschränkt ✓

$\longrightarrow$ entspricht $f'(x) = 2x$, da $D_x f(h) = f'(x)h = 2x \cdot h$.

Wichtige Spezialfälle

Wichtige Spezialfälle

- Die bereits bekannten Ableitungen sind alle Spezialfälle der Fréchet-Ableitung.
- Besonders wichtige neue Spezialfälle: Ableitungen von Funktionen, die
  - *Skalare, Vektoren oder Matrizen*
  - als *Input oder Output* haben.
- Insbesondere betrachten wir die folgenden Fälle:

Input \ Output	**Skalar**	**Vektor**	**Matrix**
Skalar	$\frac{dy}{dx}$	$\frac{d\boldsymbol{y}}{dx}$	$\frac{d\boldsymbol{Y}}{dx}$
Vektor	$\frac{\partial y}{\partial \boldsymbol{x}}$	$\frac{\partial \boldsymbol{y}}{\partial \boldsymbol{x}}$	
Matrix	$\frac{\partial y}{\partial \boldsymbol{X}}$		

Spezialfall: Vektor zu Skalar

Spezialfall: Vektor zu Skalar

Def.: Gradient

Sei $f\colon \mathbb{R}^n \to \mathbb{R}$ eine *differenzierbare* Funktion.

Wir nennen den **Gradienten** von f in $\boldsymbol{x} \in \mathbb{R}^n$ den *Zeilenvektor*

$$\frac{\partial f(\boldsymbol{x})}{\partial \boldsymbol{x}} = \nabla f(\boldsymbol{x}) = \left(\frac{\partial f(\boldsymbol{x})}{\partial x_1}, \cdots, \frac{\partial f(\boldsymbol{x})}{\partial x_n} \right) \in \mathbb{R}^{1\times n}$$

$$\implies f(\boldsymbol{x}+\boldsymbol{h}) = f(\boldsymbol{x}) + D_{\boldsymbol{x}} f(\boldsymbol{h}) + o(\|\boldsymbol{h}\|)$$

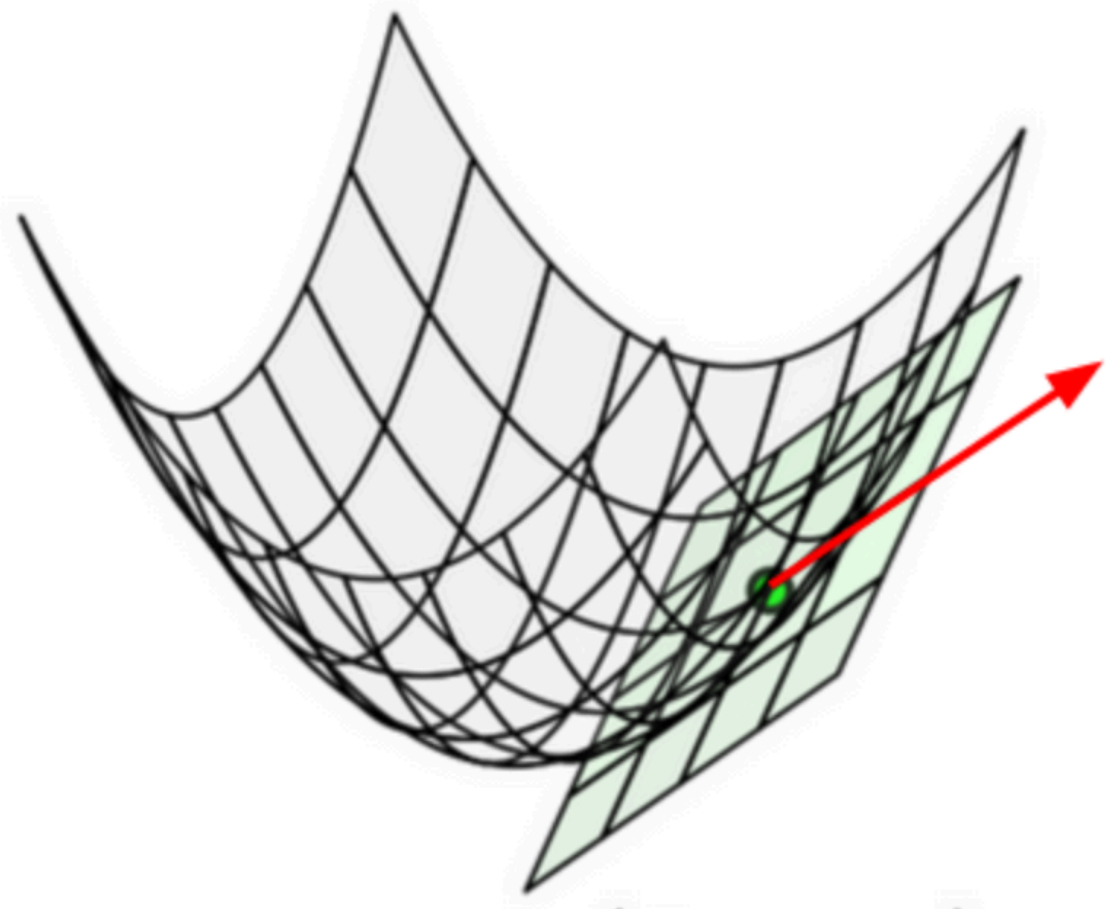
mit $D_{\boldsymbol{x}} f(\boldsymbol{h}) = \nabla f(\boldsymbol{x})\boldsymbol{h}$.

Anmerkung:

Oft wird ∇f auch als Spaltenvektor definiert.
Für uns ist er als **Zeilenvektor** $\in \mathbb{R}^{1\times n}$ konsistenter und praktischer:
- Der Gradient ist dann genau die $1 \times n$-*Jacobimatrix* von f.
- Damit wird die Kettenregel zu einer simplen Matrixmultiplikation von Jacobimatrizen, ganz ohne Transponieren.
($\to$ Kapitel *Differentialrechnung II*)

Spezialfall: Vektor zu Skalar

$f\colon \mathbb{R}^n \to \mathbb{R}$



- Graph von f in grau, der grüne Punkt ist $(\boldsymbol{x}, f(\boldsymbol{x}))$.
- Der rote Pfeil zeigt (in die Tangentialebene angehoben) die Richtung des Gradienten $\nabla f(\boldsymbol{x})^\top$. $\implies$ *Richtung, in die f an Stelle $\boldsymbol{x}$ am **stärksten** steigt*
- Die grüne Ebene ist der Graph von $\boldsymbol{u} \mapsto f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})(\boldsymbol{u} - \boldsymbol{x})$.

Beispiel: Gradient einer quadratischen Funktion

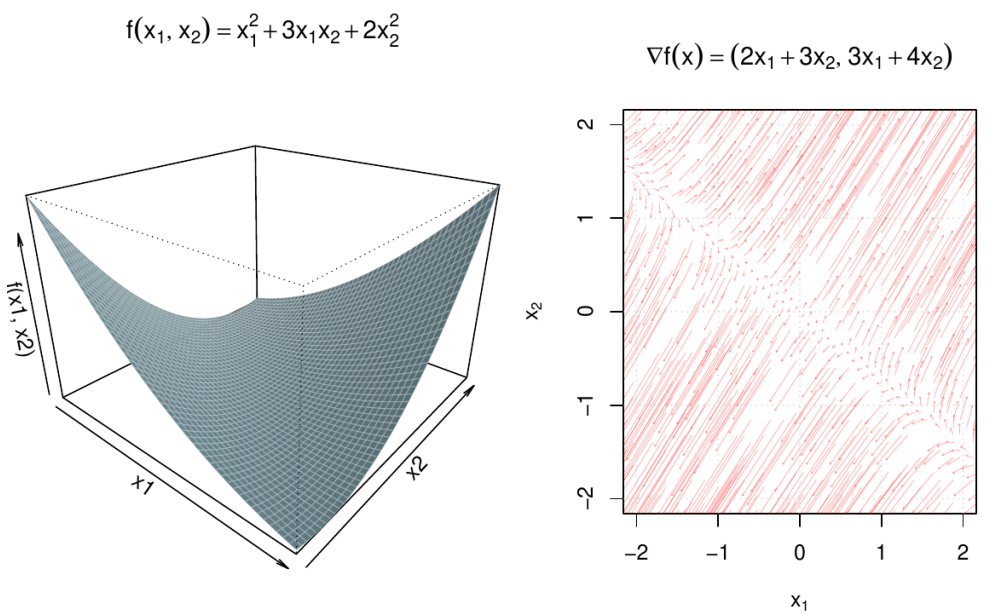
$$f\colon \mathbb{R}^2 \to \mathbb{R} \text{ mit } f(\boldsymbol{x}) = x_1^2 + 3x_1x_2 + 2x_2^2$$

Schritt 1: Berechne partielle Ableitungen

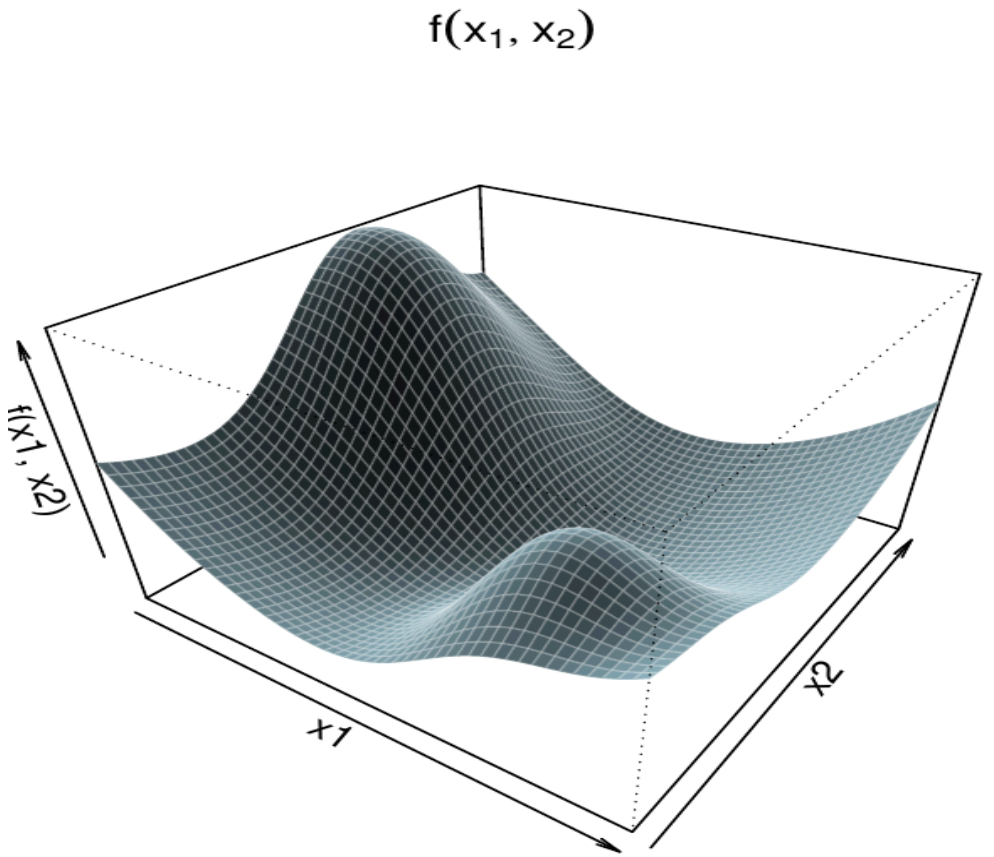
$$\frac{\partial f(\boldsymbol{x})}{\partial x_1} = 2x_1 + 3x_2; \qquad \frac{\partial f(\boldsymbol{x})}{\partial x_2} = 3x_1 + 4x_2$$

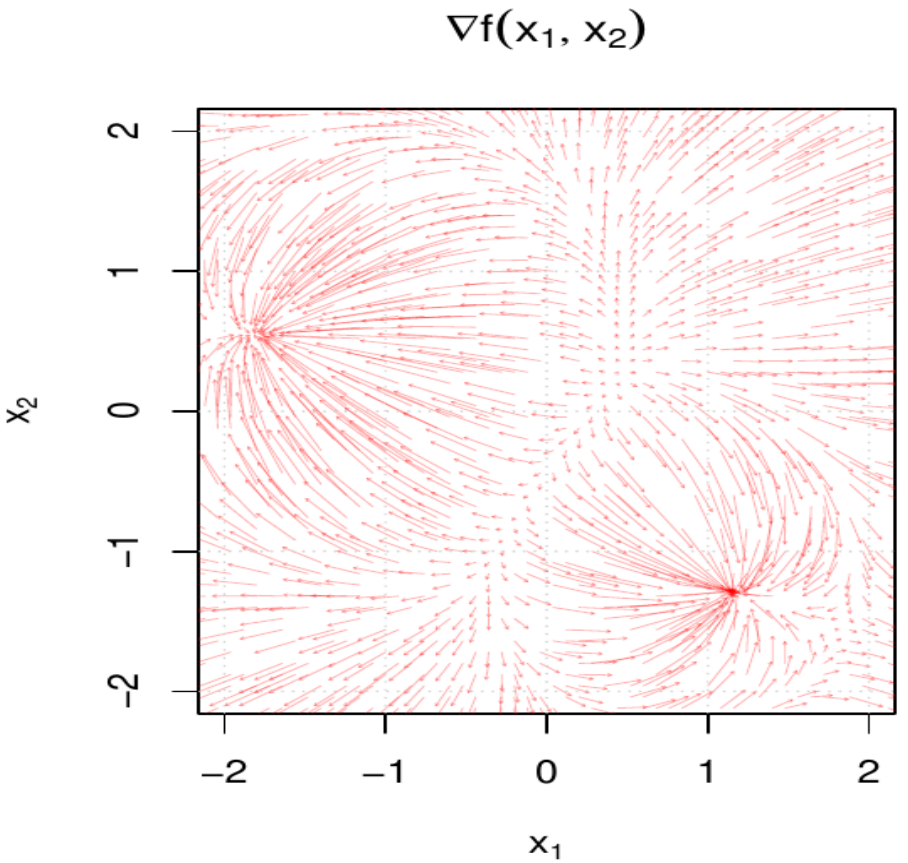
Schritt 2: Gradient als Zeilenvektor

$$\nabla f(\boldsymbol{x}) = (2x_1 + 3x_2 \quad 3x_1 + 4x_2) \qquad (\implies \nabla f\colon \mathbb{R}^2 \to \mathbb{R}^{1\times 2}!)$$



Beispiel: Gradient einer komplexeren Funktion





Übung: Vektor zu Skalar

Sei $\boldsymbol{a}, \boldsymbol{x} \in \mathbb{R}^n$ und

$$f(\boldsymbol{x}) = \boldsymbol{a}^\top \boldsymbol{x}.$$

Frage:

Was ist $\nabla f(\boldsymbol{x})$?

Antwort:

- Es gilt $f(\boldsymbol{x}) = \sum_{i=1}^{n} a_i x_i$.
- Deshalb $\frac{\partial f(\boldsymbol{x})}{\partial x_i} = a_i$.
- Also

$$\nabla f(\boldsymbol{x}) := \left(\frac{\partial f(\boldsymbol{x})}{\partial x_1}, \cdots, \frac{\partial f(\boldsymbol{x})}{\partial x_n} \right) = (a_1, \ldots, a_n) = \boldsymbol{a}^\top.$$

Übung: Gradient der quadratischen Form

Sei $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ symmetrische Matrix und $f\colon \mathbb{R}^n \to \mathbb{R}$ mit

$$f(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{A} \boldsymbol{x}.$$

1. $f(\boldsymbol{x}) = \sum_{i=1}^{n} \sum_{j=1}^{n} x_i a_{ij} x_j$.
2. Ableiten nach x_k:

$$\begin{aligned}
\frac{\partial f(\boldsymbol{x})}{\partial x_k} &= \sum_{i=1}^{n} x_i a_{ik} + \sum_{j=1}^{n} a_{kj} x_j \\
&= \sum_{i=1}^{n} a_{ki} x_i + \sum_{j=1}^{n} a_{kj} x_j \quad (\text{da } \boldsymbol{A} \text{ symmetrisch}) \\
&= 2 \sum_{i=1}^{n} a_{ki} x_i = 2(\boldsymbol{A}\boldsymbol{x})_k \\
&\implies \nabla f(\boldsymbol{x}) = 2\boldsymbol{x}^\top \boldsymbol{A}
\end{aligned}$$

Spezialfall: Ist $\boldsymbol{A} = \boldsymbol{I}$, dann $f(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{x} = \|\boldsymbol{x}\|_2^2$ und $\nabla f(\boldsymbol{x}) = 2\boldsymbol{x}^\top$.

Anwendung: Gradient Descent

Problemstellung:
Minimiere eine Verlustfunktion $L(\boldsymbol{\theta})$
(z.B. Mean Squared Error eines Modells mit Parametern $\boldsymbol{\theta}$).

Gradient Descent (**Gradientenabstieg**):

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \alpha \nabla L\left(\boldsymbol{\theta}^{(t)}\right)^\top$$

- $\alpha > 0$: Lernrate (Schrittweite)
- $-\nabla L\left(\boldsymbol{\theta}^{(t)}\right)^\top$: Negativer Gradient zeigt in Richtung des steilsten *Abstiegs*
- Gradient sagt uns, in welche Richtung wir $\boldsymbol{\theta}$ ändern müssen, um näher an ein Minimum (= *"optimales"* Modell) zu kommen
- fundamentaler Basisalgorithmus zur **Optimierung** von neuronalen Netze, komplexen Regressionsmodellen, ...
  $\rightarrow$ *Optimierung I & II*

Spezialfall: Vektor zu Vektor

Spezialfall: Vektor zu Vektor

Def.: Jacobimatrix

Sei $f\colon \mathbb{R}^n \to \mathbb{R}^m$ eine differenzierbare Funktion mit $f(\boldsymbol{x}) = \big(f_1(\boldsymbol{x}), \dots, f_m(\boldsymbol{x})\big)^\top$.

Die **Jacobimatrix** $\boldsymbol{J}_f(\boldsymbol{x}) \in \mathbb{R}^{m\times n}$ von f in $\boldsymbol{x} \in \mathbb{R}^n$ ist

$$\boldsymbol{J}_f(\boldsymbol{x}) := \frac{\partial f(\boldsymbol{x})}{\partial \boldsymbol{x}} = \begin{pmatrix} \partial f_1(\boldsymbol{x})/\partial x_1 & \cdots & \partial f_1(\boldsymbol{x})/\partial x_n \\ \vdots & \ddots & \vdots \\ \partial f_m(\boldsymbol{x})/\partial x_1 & \cdots & \partial f_m(\boldsymbol{x})/\partial x_n \end{pmatrix}.$$

$ f(+) = f() + D_{} f() + o(||)$

mit $D_{\boldsymbol{x}} f(\boldsymbol{h}) = \boldsymbol{J}_f(\boldsymbol{x})\boldsymbol{h}$.

- Die *Zeilen* der *Jacobimatrix* sind *Gradienten* der Komponenten von $f(\boldsymbol{x}) = \big(f_1(\boldsymbol{x}), \dots, f_m(\boldsymbol{x})\big)^\top$:

$$\boldsymbol{J}_f(\boldsymbol{x}) = \begin{pmatrix} \nabla f_1(\boldsymbol{x}) \\ \vdots \\ \nabla f_m(\boldsymbol{x}) \end{pmatrix}$$

Spezialfall: Vektor zu Vektor

Wichtige Identitäten:

- $f(\boldsymbol{x}) = \boldsymbol{c} \in \mathbb{R}^m \quad \Longrightarrow \boldsymbol{J}_f(\boldsymbol{x}) = \boldsymbol{0}_{m \times n}$
- $f(\boldsymbol{x}) = \boldsymbol{x}$ (d.h. $m = n$) $\quad \Longrightarrow \boldsymbol{J}_f(\boldsymbol{x}) = \boldsymbol{I}_n$
- $f(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x}$ mit $\boldsymbol{A} \in \mathbb{R}^{m \times n} \quad \Longrightarrow \boldsymbol{J}_f(\boldsymbol{x}) = \boldsymbol{A}$
- $f(\boldsymbol{x}) = \left(\boldsymbol{x}^\top \boldsymbol{A}\right)^\top = \boldsymbol{A}^\top \boldsymbol{x}$ mit $\boldsymbol{A} \in \mathbb{R}^{n \times m} \quad \Longrightarrow \boldsymbol{J}_f(\boldsymbol{x}) = \boldsymbol{A}^\top$

Übung

$$f(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x} \text{ mit } \boldsymbol{A} \in \mathbb{R}^{m \times n}:$$

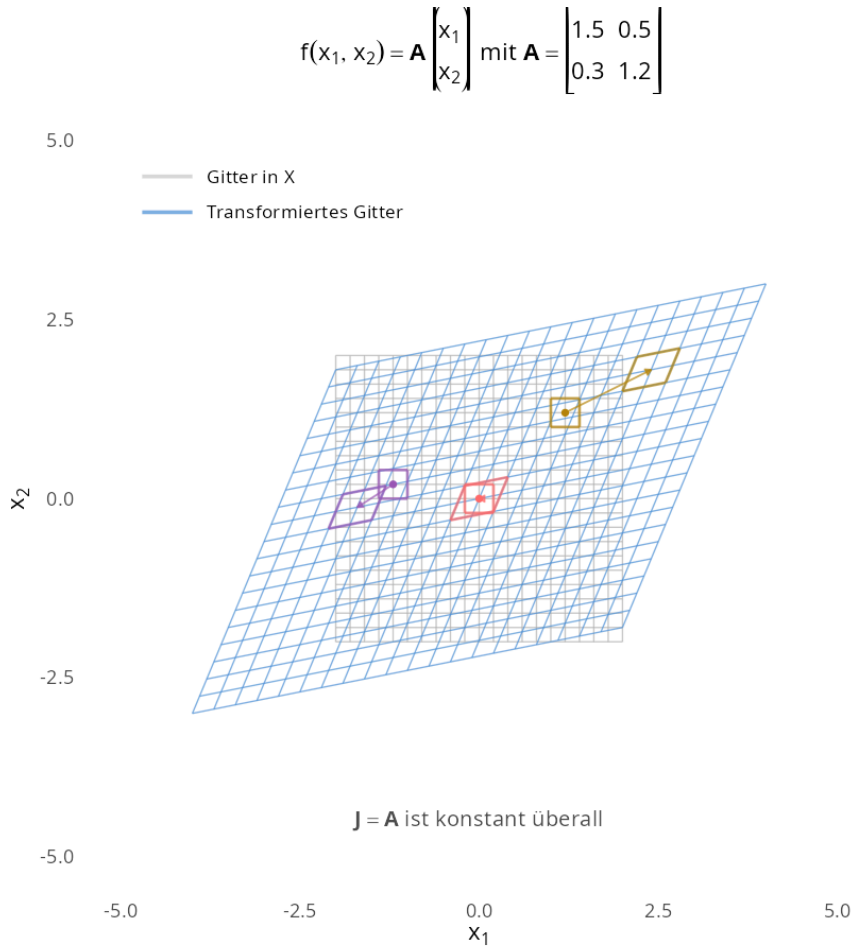
$$f_i(\boldsymbol{x}) = \sum_{j=1}^{n} a_{ij} x_j \quad \text{(komponentenweise)}$$

$$\frac{\partial f_i(\boldsymbol{x})}{\partial x_k} = \frac{\partial}{\partial x_k} \left(\sum_{j=1}^{n} a_{ij} x_j \right) = a_{ik}$$

$$\implies \boldsymbol{J}_f(\boldsymbol{x}) = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{pmatrix} = \boldsymbol{A} \quad \checkmark$$

- Lineare Funktionen haben konstante Jacobimatrizen
- Für lineare Funktionen gilt $f(\boldsymbol{x} + \boldsymbol{h}) = f(\boldsymbol{x}) + \boldsymbol{J}_f(\boldsymbol{x})\boldsymbol{h}$

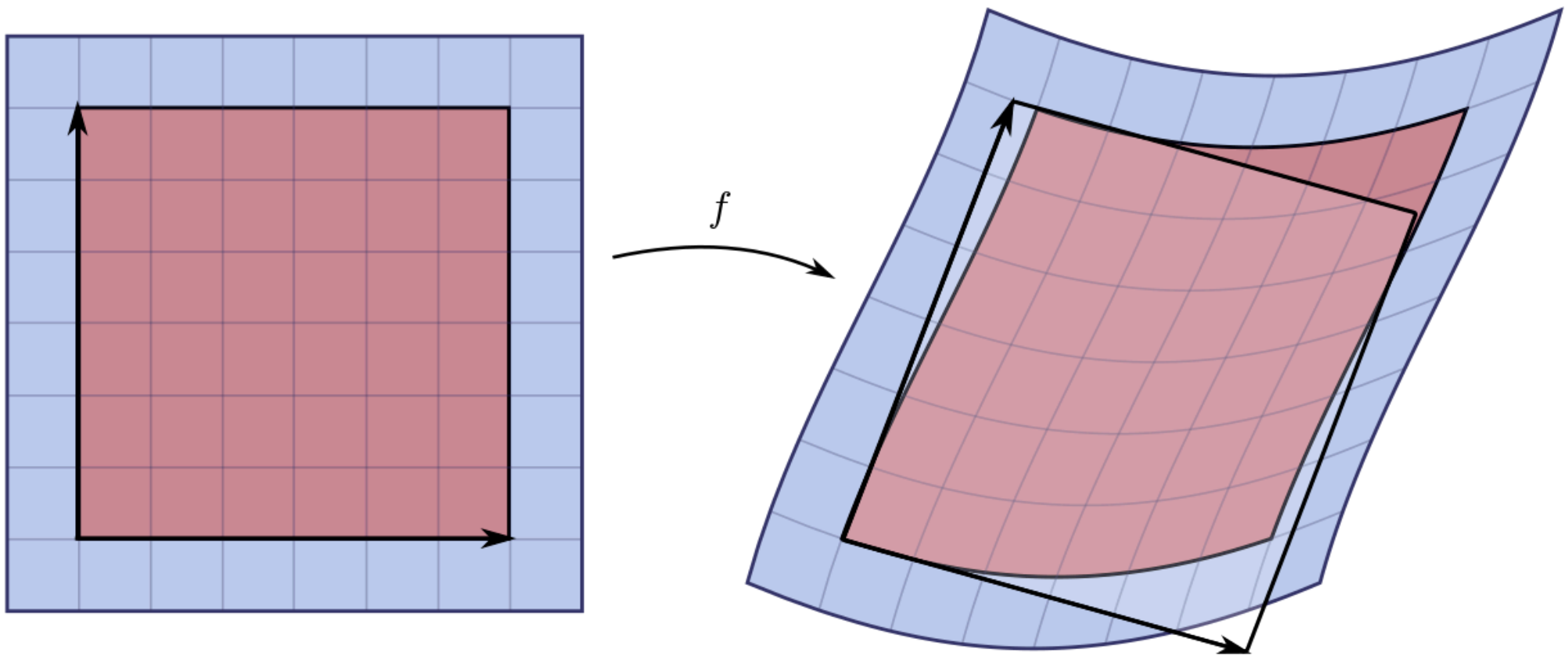
Visualisierung: Lineare Transformation $\mathbb{R}^2 \to \mathbb{R}^2$



Lineare Funktion: $f(\boldsymbol{x} + \boldsymbol{h}) = f(\boldsymbol{x}) + \boldsymbol{J}_f(\boldsymbol{x})\boldsymbol{h}$

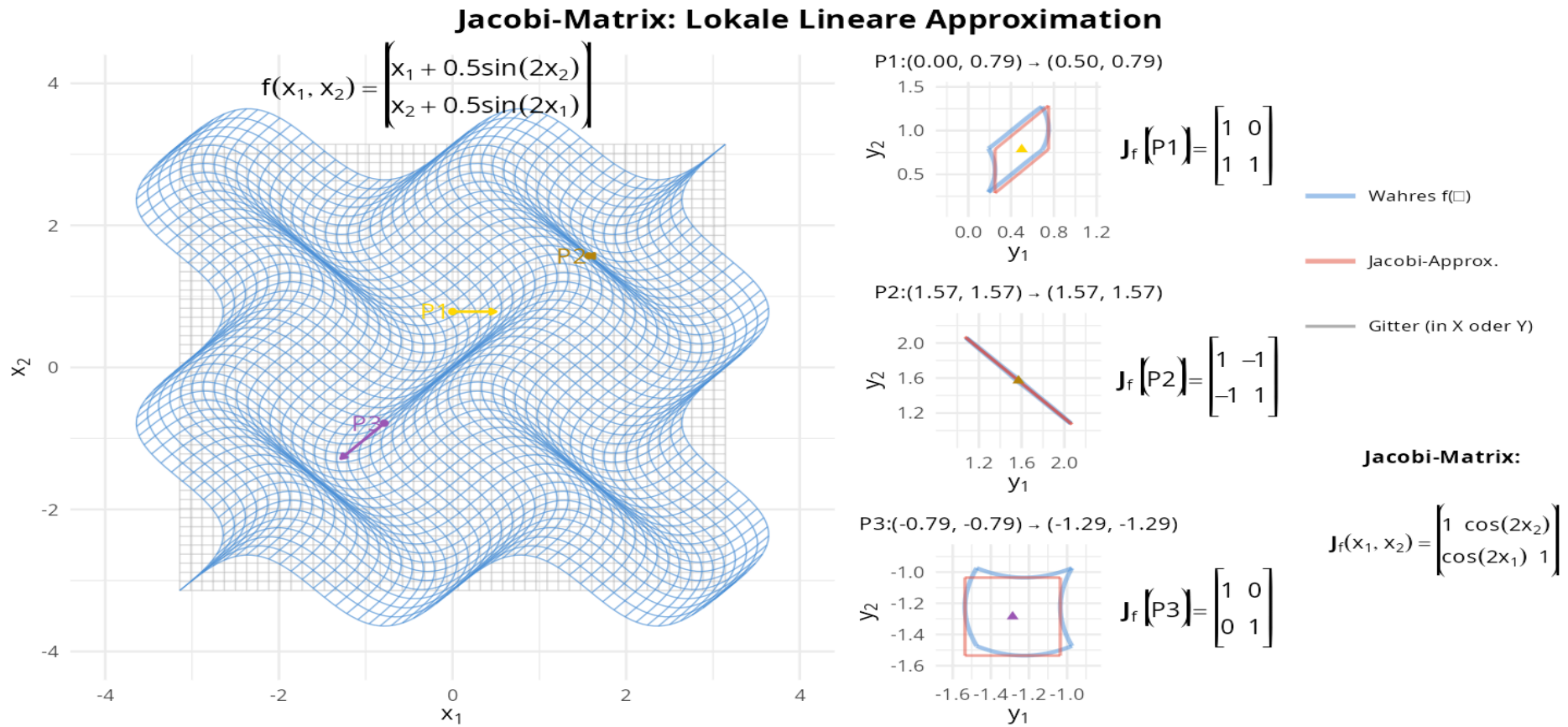
Geometrische Intuition

$f\colon \mathbb{R}^n \to \mathbb{R}^m$



- $f\colon \mathbb{R}^2 \to \mathbb{R}^2$ transformiert das rote Quadrat links zur roten Fläche rechts.
- Die gekrümmte Darstellung soll verdeutlichen, dass f den Raum lokal verbiegt und streckt, obwohl Urbild und Bild dieselbe Dimension haben.
- Das schwarze Quadrat rechts zeigt das Bild der linearen Approximation durch $\boldsymbol{J}_f$.

Geometrische Intuition — Jacobimatrix $\mathbb{R}^2 \to \mathbb{R}^2$



$$f(\boldsymbol{x} + \boldsymbol{h}) \approx f(\boldsymbol{x}) + \boldsymbol{J}_f(\boldsymbol{x})\boldsymbol{h}$$

Spezialfall: Skalar zu Matrix

Spezialfall: Skalar zu Matrix

$$\boldsymbol{F}\colon \mathbb{R} \to \mathbb{R}^{m\times n}$$

- Sei $x \in \mathbb{R}$, seien alle f_{ij} in x differenzierbar, und

$$\boldsymbol{F}(x) = \begin{pmatrix} f_{11}(x) & \cdots & f_{1n}(x) \\ \vdots & \ddots & \vdots \\ f_{m1}(x) & \cdots & f_{mn}(x) \end{pmatrix}.$$

- Definiere

$$\frac{\partial \boldsymbol{F}(x)}{\partial x} = \begin{pmatrix} \frac{\partial f_{11}(x)}{\partial x} & \cdots & \frac{\partial f_{1n}(x)}{\partial x} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_{m1}(x)}{\partial x} & \cdots & \frac{\partial f_{mn}(x)}{\partial x} \end{pmatrix} \in \mathbb{R}^{m\times n}.$$

- Dann ist

$$\boldsymbol{F}(x+h) = \boldsymbol{F}(x) + D_x\boldsymbol{F}(h) + o(|h|) \quad \text{mit} \quad D_x\boldsymbol{F}(h) = \frac{\partial \boldsymbol{F}(x)}{\partial x} h.$$

Übung: Skalar zu Matrix

Sei, für $x > 0$,

$$\boldsymbol{F}(x) = \begin{pmatrix} x^2 & 2e^x & 0 \\ 0 & x & \ln(x) \end{pmatrix}.$$

Frage:

Was ist $\frac{\partial \boldsymbol{F}(x)}{\partial x}$?

Antwort:

$$\frac{\partial \boldsymbol{F}(x)}{\partial x} = \begin{pmatrix} 2x & 2e^x & 0 \\ 0 & 1 & 1/x \end{pmatrix}.$$

Skalar zu Matrix: Wichtige Identitäten

Für *quadratisches, differenzierbares*

$$\boldsymbol{F}\colon \mathbb{R} \to \mathbb{R}^{n\times n}$$

(erste beide Formeln: $\boldsymbol{F}(x)$ zusätzlich invertierbar):

- $\dfrac{\partial \boldsymbol{F}(x)^{-1}}{\partial x} = -\boldsymbol{F}(x)^{-1}\dfrac{\partial \boldsymbol{F}(x)}{\partial x}\boldsymbol{F}(x)^{-1}.$
- $\dfrac{\partial \det\big(\boldsymbol{F}(x)\big)}{\partial x} = \det\big(\boldsymbol{F}(x)\big)\operatorname{tr}\left[\boldsymbol{F}(x)^{-1}\dfrac{\partial \boldsymbol{F}(x)}{\partial x}\right].$
- $\dfrac{\partial \operatorname{tr}\big(\boldsymbol{F}(x)\big)}{\partial x} = \operatorname{tr}\left(\dfrac{\partial \boldsymbol{F}(x)}{\partial x}\right).$

(Rechenbeispiel zur Determinanten-Formel: → Anhang — wichtig z.B. für Ableitungen von Likelihood-Funktionen!)

Beweis: Ableitung der Spur

$$\frac{\partial \operatorname{tr}\big(\boldsymbol{F}(x)\big)}{\partial x} = \operatorname{tr}\left(\frac{\partial \boldsymbol{F}(x)}{\partial x}\right)$$

Beweis:

$$\begin{aligned}
\operatorname{tr}\big(\boldsymbol{F}(x)\big) &= \sum_{i=1}^{n} f_{ii}(x) \\
\frac{\partial}{\partial x}\operatorname{tr}\big(\boldsymbol{F}(x)\big) &= \frac{\partial}{\partial x}\left(\sum_{i=1}^{n} f_{ii}(x)\right) \\
&= \sum_{i=1}^{n} \frac{\partial f_{ii}(x)}{\partial x} \\
&= \operatorname{tr}\left(\frac{\partial \boldsymbol{F}(x)}{\partial x}\right) \quad \checkmark
\end{aligned}$$

Spezialfall: Matrix zu Skalar

Spezialfall: Matrix zu Skalar

$$f\colon \mathbb{R}^{m\times n} \to \mathbb{R}$$

- Sei f in $\boldsymbol{X} \in \mathbb{R}^{m\times n}$ differenzierbar. Definiere:

$$\frac{\partial f(\boldsymbol{X})}{\partial \boldsymbol{X}} = \begin{pmatrix} \partial f(\boldsymbol{X})/\partial x_{11} & \cdots & \partial f(\boldsymbol{X})/\partial x_{1n} \\ \vdots & \ddots & \vdots \\ \partial f(\boldsymbol{X})/\partial x_{m1} & \cdots & \partial f(\boldsymbol{X})/\partial x_{mn} \end{pmatrix} \in \mathbb{R}^{m\times n}.$$

- Dann ist

$$f(\boldsymbol{X}+\boldsymbol{H}) = f(\boldsymbol{X}) + D_{\boldsymbol{X}} f(\boldsymbol{H}) + o(\|\boldsymbol{H}\|) \quad \text{mit} \quad D_{\boldsymbol{X}} f(\boldsymbol{H}) = \operatorname{tr}\left(\left(\frac{\partial f(\boldsymbol{X})}{\partial \boldsymbol{X}}\right)^{\top} \boldsymbol{H}\right).$$

Matrix zu Skalar: Beispiel

Beispiel:

- Sei $f(\boldsymbol{X}) = \boldsymbol{a}^\top \boldsymbol{X}\boldsymbol{b}$ mit $\boldsymbol{X} \in \mathbb{R}^{m\times n}$, $\boldsymbol{a} \in \mathbb{R}^m$, $\boldsymbol{b} \in \mathbb{R}^n$.
- Dann ist

$$f(\boldsymbol{X}) = \sum_{i=1}^{m}\sum_{j=1}^{n} a_i x_{ij} b_j \quad \Longrightarrow \quad \frac{\partial f(\boldsymbol{X})}{\partial x_{ij}} = a_i b_j.$$

- Daraus folgt

$$\frac{\partial \boldsymbol{a}^\top \boldsymbol{X}\boldsymbol{b}}{\partial \boldsymbol{X}} = \begin{pmatrix} a_1 b_1 & \cdots & a_1 b_n \\ \vdots & \ddots & \vdots \\ a_m b_1 & \cdots & a_m b_n \end{pmatrix} = \boldsymbol{a}\boldsymbol{b}^\top$$

$$\Longrightarrow \; D_{\boldsymbol{X}} f(\boldsymbol{H}) = \operatorname{tr}\left(\left(\boldsymbol{a}\boldsymbol{b}^\top\right)^\top \boldsymbol{H}\right) = \operatorname{tr}\left(\boldsymbol{b}\boldsymbol{a}^\top \boldsymbol{H}\right) = \boldsymbol{a}^\top \boldsymbol{H}\boldsymbol{b}$$

- also hier:

$$f(\boldsymbol{X}+\boldsymbol{H}) = \boldsymbol{a}^\top \boldsymbol{X}\boldsymbol{b} + \boldsymbol{a}^\top \boldsymbol{H}\boldsymbol{b} = f(\boldsymbol{X}) + D_{\boldsymbol{X}} f(\boldsymbol{H})$$

Wichtige Identitäten: Matrix zu Skalar

- **Quadratische Form:** $f(\boldsymbol{X}) = \operatorname{tr}\left(\boldsymbol{X}\boldsymbol{A}\boldsymbol{X}^\top\right)$ mit symmetrischem $\boldsymbol{A} \in \mathbb{R}^{n\times n}$

$$\frac{\partial f(\boldsymbol{X})}{\partial \boldsymbol{X}} = \boldsymbol{X}\left(\boldsymbol{A} + \boldsymbol{A}^\top\right) = 2\boldsymbol{X}\boldsymbol{A}$$

- **Trace-Produkt:** $f(\boldsymbol{X}) = \operatorname{tr}\left(\boldsymbol{A}^\top \boldsymbol{X}\right)$ mit $\boldsymbol{A} \in \mathbb{R}^{m\times n}$

$$\frac{\partial f(\boldsymbol{X})}{\partial \boldsymbol{X}} = \boldsymbol{A}$$

- **Frobenius-Norm:** $f(\boldsymbol{X}) = \|\boldsymbol{X}\|_F^2 = \operatorname{tr}\left(\boldsymbol{X}^\top \boldsymbol{X}\right)$

$$\frac{\partial f(\boldsymbol{X})}{\partial \boldsymbol{X}} = 2\boldsymbol{X}$$

(Anwendungsbeispiel Matrixzerlegung / Matrix Completion: → Anhang)

Bemerkungen

- Wir haben bewusst ein paar Fälle ausgelassen:
  - Matrix zu Vektor
  - Vektor zu Matrix
  - Matrix zu Matrix
- Diese Fälle werden seltener gebraucht und können (z.B.) über *Tensoren* dargestellt werden.
- Zur *Störungsanalyse* von Matrixrechnungen kann man auch $h \mapsto f(\boldsymbol{X} + h\boldsymbol{E})$ nach Skalar h ableiten (= Ableitung in "Richtung" $\boldsymbol{E}$.)
- Formeln für die meisten in Anwendungen vorkommenden Ableitungen findet man in dem *Matrix Cookbook* von Petersen & Pedersen (2012).
- Abbildungen zwischen Funktionenräumen $\longrightarrow$ Übung.

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- Fréchet-Ableitung auf normierten Räumen: **Ableitung = lineare Approximation**
- Spezialfälle für Skalare, Vektoren, Matrizen: *Gradient* (Zeilenvektor!), *Jacobimatrix*, Matrixableitungen

Nächste Vorlesung

- Eigenschaften, Rechenregeln, höhere Ableitungen

Self-Check

Frage 1: Sei $f(\boldsymbol{x}) = \boldsymbol{a}^\top \boldsymbol{x}$ mit $\boldsymbol{a}, \boldsymbol{x} \in \mathbb{R}^n$. Was ist $\nabla f(\boldsymbol{x})$?

1. $\boldsymbol{a}^\top$
2. $\boldsymbol{a}\boldsymbol{a}^\top$
3. $\boldsymbol{x}^\top$
4. $\boldsymbol{a}^\top \boldsymbol{x}$

Frage 2: Sei $f(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}$ mit *symmetrischem* $\boldsymbol{A} \in \mathbb{R}^{n \times n}$. Was ist $\nabla f(\boldsymbol{x})$?

1. $\boldsymbol{A}\boldsymbol{x}$
2. $2\boldsymbol{x}^\top \boldsymbol{A}$
3. $2\boldsymbol{A}$
4. $\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}$

Frage 3: Sei $f\colon \mathbb{R}^3 \to \mathbb{R}^2$ differenzierbar. Welche Dimension hat die Jacobimatrix $\boldsymbol{J}_f(\boldsymbol{x})$?

1. 3×2
2. 2×3
3. 3×3
4. 2×2

Anhang

Quiz: Beweis Aussage (3) $\iff$ (4)

“$\implies$”: Angenommen (3) gilt, also $\lim_{h\to 0} \frac{|f(x+h)-f(x)-f'(x)h|}{h} = 0$.

- Definiere $r(h) := f(x+h) - f(x) - f'(x)h$.
- Also: $\lim\limits_{h\to 0} \dfrac{|r(h)|}{h} = 0$, also $r(h) = o(h)$.
- Und damit: $f(x+h) = f(x) + f'(x)h + r(h) = f(x) + f'(x)h + o(h)$. ✓

“$\impliedby$”: Angenommen (4) gilt, also $f(x+h) = f(x) + f'(x)h + o(h)$.

- Dann ist $f(x+h) - f(x) - f'(x)h = o(h)$.
- Per Definition von $o(h)$: $\lim\limits_{h\to 0} \dfrac{|f(x+h) - f(x) - f'(x)h|}{h} = \lim\limits_{h\to 0} \dfrac{|o(h)|}{h} = 0$. ✓

↩ zurück: „Quiz”

Beispiel: Ableitung der Determinante

*Vertiefung zu "Skalar zu Matrix: Wichtige Identitäten" — **wichtig** für Ableitungen von Likelihood-Funktionen in der Statistik!*

$$\frac{\partial \det\big(\boldsymbol{F}(x)\big)}{\partial x} = \det\big(\boldsymbol{F}(x)\big)\operatorname{tr}\left[\boldsymbol{F}(x)^{-1}\frac{\partial \boldsymbol{F}(x)}{\partial x}\right]$$

Beispiel: $\boldsymbol{F}(x) = \begin{pmatrix} x & 0 \\ 0 & 2x \end{pmatrix}$.

Direkte Berechnung:

$$\det(\boldsymbol{F}(x)) = x \cdot 2x = 2x^2 \quad \Longrightarrow \quad \frac{\partial}{\partial x}\det(\boldsymbol{F}(x)) = 4x$$

Überprüfung (für $x \neq 0$, sonst existiert $\boldsymbol{F}(x)^{-1}$ nicht):

$$\frac{\partial \boldsymbol{F}(x)}{\partial x} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}, \quad \boldsymbol{F}(x)^{-1} = \begin{pmatrix} 1/x & 0 \\ 0 & 1/(2x) \end{pmatrix}$$

$$\boldsymbol{F}(x)^{-1}\frac{\partial \boldsymbol{F}(x)}{\partial x} = \begin{pmatrix} 1/x & 0 \\ 0 & 1/(2x) \end{pmatrix}\begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} = \begin{pmatrix} 1/x & 0 \\ 0 & 1/x \end{pmatrix}$$

$$\operatorname{tr}\left[\boldsymbol{F}(x)^{-1}\frac{\partial \boldsymbol{F}(x)}{\partial x}\right] = 1/x + 1/x = 2/x$$

$$\det(\boldsymbol{F}(x)) \cdot \operatorname{tr}\left[\boldsymbol{F}(x)^{-1}\frac{\partial \boldsymbol{F}(x)}{\partial x}\right] = 2x^2 \cdot \frac{2}{x} = 4x \quad \checkmark$$

↩ zurück: „Skalar zu Matrix: Wichtige Identitäten"

Anwendung: Matrixzerlegung & *Matrix Completion*

Vertiefung zu “Wichtige Identitäten: Matrix zu Skalar”.

Gegeben: Unvollständig nur an Indizes $(i,j) \in \Omega$ beobachtete Matrix $\boldsymbol{Y} \in \mathbb{R}^{m \times n}$ (z.B. User-Ratings von Filmen)

Ziel: Finde Low-Rank-Approximation $\boldsymbol{Y} \approx \boldsymbol{U}\boldsymbol{V}^\top$ mit $\boldsymbol{U} \in \mathbb{R}^{m \times k}$, $\boldsymbol{V} \in \mathbb{R}^{n \times k}$.

Verlustfunktion:

$$L(\boldsymbol{U}, \boldsymbol{V}) = \frac{1}{2} \sum_{(i,j) \in \Omega} \left(y_{ij} - \left[\boldsymbol{U}\boldsymbol{V}^\top \right]_{ij} \right)^2 = \frac{1}{2} \left\| \boldsymbol{P}_\Omega \odot (\boldsymbol{Y} - \boldsymbol{U}\boldsymbol{V}^\top) \right\|_F^2; \qquad \boldsymbol{P}_\Omega = \left[\mathbb{I}_{(i,j) \in \Omega} \right]$$

($\odot$: *elementweises* (Hadamard-)Produkt)

Gradienten für Optimierung:

$$\frac{\partial L}{\partial \boldsymbol{U}} = -\left(\boldsymbol{P}_\Omega \odot \left(\boldsymbol{Y} - \boldsymbol{U}\boldsymbol{V}^\top \right) \right) \boldsymbol{V}$$
$$\frac{\partial L}{\partial \boldsymbol{V}} = -\left(\boldsymbol{P}_\Omega \odot \left(\boldsymbol{Y} - \boldsymbol{U}\boldsymbol{V}^\top \right) \right)^\top \boldsymbol{U}$$

Anwendungen: Missing Data Imputation, Recommender Systems / Collaborative Filtering, …

↩ zurück: „Wichtige Identitäten: Matrix zu Skalar”

Differentialrechnung II

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- *Fréchet-Differenzierbarkeit*: $f(\boldsymbol{x}+\boldsymbol{h}) = f(\boldsymbol{x}) + D_{\boldsymbol{x}} f(\boldsymbol{h}) + o(\|\boldsymbol{h}\|)$
- *Gradient* $\nabla f(\boldsymbol{x})$ für $f : \mathbb{R}^n \to \mathbb{R}$
- *Jacobimatrix* $\boldsymbol{J}_f(\boldsymbol{x})$ für $f : \mathbb{R}^n \to \mathbb{R}^m$
- Matrixableitungen: $\partial f / \partial \boldsymbol{X}$
- **Analysis:**
  - Produktregel: $(fg)' = f'g + fg'$ &
    Kettenregel: $(g \circ f)' = g'(f) \cdot f'$
  - Höhere Ableitungen: $f'', f''', \ldots$
  - Taylorreihen: $f(x+h) = f(x) + f'(x)h + (1/2)f''(x)h^2 + \ldots$
- **Lineare Algebra:**
  - *Multilineare* & *Bilineare* Abbildungen: $\langle \cdot, \cdot \rangle$
  - Quadratische Formen: $\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}$
  - Komposition von Funktionen

Opener-Quiz: Was wissen Sie noch?

Frage 1: Sei $f\colon \mathbb{R} \to \mathbb{R}$ differenzierbar in x mit Ableitung $f'(x)$. Welche Aussagen sind wahr?

1. f' ist eine lineare Funktion.
2. $h \mapsto f'(x)h$ ist eine lineare Funktion (für festes x).
3. $f(x+h) = f(x) + f'(x)h + o(h)$ für $h \to 0$.
4. "$o(h)$" steht für eine Funktion $r(h)$ mit $\lim_{h\to 0} \frac{r(h)}{h} = 0$.

Frage 2: Sei $f\colon \mathbb{R}^n \to \mathbb{R}^m$ differenzierbar. Welche Dimension hat die Jacobimatrix $\boldsymbol{J}_f(\boldsymbol{x})$?

1. $n \times m$
2. $m \times n$
3. $n \times n$
4. $m \times m$

Eigenschaften und Rechenregeln

Wichtige Eigenschaften: Stetigkeit

Theorem: Stetigkeit aus Differenzierbarkeit

Seien $\mathbb{D}$ und $\mathbb{E}$ normierte Räume und $f\colon \mathbb{D} \to \mathbb{E}$ in $\boldsymbol{x} \in \mathbb{D}$ differenzierbar.
Dann ist f in $\boldsymbol{x}$ stetig.

Beweis.

- Per Definition ist $D_{\boldsymbol{x}} f$ beschränkt und linear. Insbesondere gilt

$$\|D_{\boldsymbol{x}} f(\boldsymbol{h})\| \leq M \|\boldsymbol{h}\|, \quad \text{d.h.} \quad D_{\boldsymbol{x}} f(\boldsymbol{h}) = O(\|\boldsymbol{h}\|).$$

- Also ist

$$f(\boldsymbol{x}+\boldsymbol{h}) = f(\boldsymbol{x}) + D_{\boldsymbol{x}} f(\boldsymbol{h}) + o(\|\boldsymbol{h}\|) = f(\boldsymbol{x}) + O(\|\boldsymbol{h}\|) \text{ für } \|\boldsymbol{h}\| \to 0.$$

- Für $\|\boldsymbol{h}\| \to 0$ erhalten wir $f(\boldsymbol{x}+\boldsymbol{h}) \to f(\boldsymbol{x})$. ■

Notation:

$r(\boldsymbol{h}) = O(\|\boldsymbol{h}\|)$ für $\|\boldsymbol{h}\| \to 0$ heißt:
$\|r(\boldsymbol{h})\| \leq C \|\boldsymbol{h}\|$ für alle hinreichend kleinen $\boldsymbol{h}$ –
im Unterschied zu $r(\boldsymbol{h}) = o(\|\boldsymbol{h}\|)$, wo $\|r(\boldsymbol{h})\| / \|\boldsymbol{h}\| \to 0$ gilt.

Bsp: Stetig aber nicht differenzierbar

Betrachte $f\colon \mathbb{R} \to \mathbb{R}$ mit $f(x) = |x|$.

- f stetig in $x = 0$: Für $h \to 0$ gilt $f(0+h) = |h| \to 0 = f(0)$. ✓
- f *nicht* differenzierbar in $x = 0$:

$$\lim_{h \to 0^+} \frac{f(h) - f(0)}{h} = \lim_{h \to 0^+} \frac{|h|}{h} = \lim_{h \to 0^+} \frac{h}{h} = 1,$$
$$\lim_{h \to 0^-} \frac{f(h) - f(0)}{h} = \lim_{h \to 0^-} \frac{|h|}{h} = \lim_{h \to 0^-} \frac{-h}{h} = -1.$$

  Da die linksseitigen und rechtsseitigen Grenzwerte verschieden sind, existiert $f'(0)$ nicht. ×

Fazit:

- Differenzierbarkeit $\implies$ Stetigkeit
- aber Stetigkeit $\not\Longleftarrow$ Differenzierbarkeit!

Wichtige Eigenschaften: Linearität

Theorem: Linearität der Ableitungsoperation

Seien $\mathbb{D}$ und $\mathbb{E}$ normierte Räume und $f, g\colon \mathbb{D} \to \mathbb{E}$ in $\boldsymbol{x} \in \mathbb{D}$ differenzierbar.
Dann ist für alle $c_1, c_2 \in \mathbb{R}$ auch $c_1 f + c_2 g$ in $\boldsymbol{x}$ differenzierbar, mit

$$D_{\boldsymbol{x}}(c_1 f + c_2 g) = c_1 D_{\boldsymbol{x}} f + c_2 D_{\boldsymbol{x}} g.$$

Beweis: $\rightarrow$ Übung

Wichtige Eigenschaften: Linearität

Beispiele

- Für $\mathbb{D} = \mathbb{R}, \mathbb{E} = \mathbb{R}$: $(c_1 f + c_2 g)' = c_1 f' + c_2 g'$
- Für $\mathbb{D} = \mathbb{R}^n, \mathbb{E} = \mathbb{R}$:

$$\nabla \left[c_1 f(\boldsymbol{x}) + c_2 g(\boldsymbol{x}) \right] = c_1 \nabla f(\boldsymbol{x}) + c_2 \nabla g(\boldsymbol{x})$$

- Für $\mathbb{D} = \mathbb{R}^n, \mathbb{E} = \mathbb{R}^m$:

$$\boldsymbol{J}_{c_1 f + c_2 g}(\boldsymbol{x}) = c_1 \boldsymbol{J}_f(\boldsymbol{x}) + c_2 \boldsymbol{J}_g(\boldsymbol{x})$$

- Für $\mathbb{D} = \mathbb{R}, \mathbb{E} = \mathbb{R}^{m \times n}$:

$$\frac{\partial \left[c_1 \boldsymbol{F}(x) + c_2 \boldsymbol{G}(x) \right]}{\partial x} = c_1 \frac{\partial \boldsymbol{F}(x)}{\partial x} + c_2 \frac{\partial \boldsymbol{G}(x)}{\partial x}$$

- Für $\mathbb{D} = \mathbb{R}^{m \times n}, \mathbb{E} = \mathbb{R}$:

$$\frac{\partial \left[c_1 f(\boldsymbol{X}) + c_2 g(\boldsymbol{X}) \right]}{\partial \boldsymbol{X}} = c_1 \frac{\partial f(\boldsymbol{X})}{\partial \boldsymbol{X}} + c_2 \frac{\partial g(\boldsymbol{X})}{\partial \boldsymbol{X}}$$

Bsp: Linearität der Ableitung

Gegeben: $f(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{x}$ und $g(\boldsymbol{x}) = \boldsymbol{a}^\top \boldsymbol{x}$ mit $\boldsymbol{a} = (1, 2)^\top$.

Berechne $\nabla h(\boldsymbol{x})$ für $h(\boldsymbol{x}) = 3f(\boldsymbol{x}) - 2g(\boldsymbol{x})$ an der Stelle $\boldsymbol{x} = (1, -1)^\top$.

(Achtung: h ist hier ausnahmsweise ein Funktionsname, nicht der Zuwachs $\boldsymbol{h}$.)

Lösung:

1. Berechne Gradienten:
   - Für $f(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{x} = x_1^2 + x_2^2$ gilt: $\nabla f(\boldsymbol{x}) = (2x_1, 2x_2)$
   - Für $g(\boldsymbol{x}) = \boldsymbol{a}^\top \boldsymbol{x} = x_1 + 2x_2$ gilt: $\nabla g(\boldsymbol{x}) = \boldsymbol{a}^\top = (1,\ 2)$
2. Wende Linearität an:

$$\nabla h(\boldsymbol{x}) = 3\nabla f(\boldsymbol{x}) - 2\nabla g(\boldsymbol{x}) = 3(2x_1,\ 2x_2) - 2(1,\ 2) = (6x_1 - 2,\ 6x_2 - 4)$$

3. Setze $\boldsymbol{x} = (1, -1)^\top$ ein:

$$\nabla h\left(\begin{pmatrix} 1 \\ -1 \end{pmatrix}\right) = (6 \cdot 1 - 2,\ 6 \cdot (-1) - 4) = (4,\ -10)$$

Rechenregeln: Produktregel

Theorem: Produktregel

Seien $\mathbb{D}, \mathbb{E}, \mathbb{B}$ normierte Räume, $f, g\colon \mathbb{D} \to \mathbb{E}$ in $\boldsymbol{x} \in \mathbb{D}$ differenzierbar, $\langle \cdot, \cdot \rangle\colon \mathbb{E} \times \mathbb{E} \to \mathbb{B}$ eine beschränkte bilineare Abbildung und $b(\boldsymbol{x}) = \langle f(\boldsymbol{x}), g(\boldsymbol{x}) \rangle$.

Dann gilt

$$D_{\boldsymbol{x}} b(\boldsymbol{h}) = \langle D_{\boldsymbol{x}} f(\boldsymbol{h}), g(\boldsymbol{x}) \rangle + \langle f(\boldsymbol{x}), D_{\boldsymbol{x}} g(\boldsymbol{h}) \rangle.$$

Beweis (kompakt):

Differenzierbarkeit von f, g einsetzen, Bilinearität ausmultiplizieren:

$$\begin{aligned} b(\boldsymbol{x}+\boldsymbol{h}) &= \big\langle f(\boldsymbol{x}) + D_{\boldsymbol{x}} f(\boldsymbol{h}) + o(\|\boldsymbol{h}\|),\ g(\boldsymbol{x}) + D_{\boldsymbol{x}} g(\boldsymbol{h}) + o(\|\boldsymbol{h}\|) \big\rangle \\ &= b(\boldsymbol{x}) + \underbrace{\langle D_{\boldsymbol{x}} f(\boldsymbol{h}), g(\boldsymbol{x}) \rangle + \langle f(\boldsymbol{x}), D_{\boldsymbol{x}} g(\boldsymbol{h}) \rangle}_{= D_{\boldsymbol{x}} b(\boldsymbol{h}) \text{ (linear in } \boldsymbol{h})} + \underbrace{\langle D_{\boldsymbol{x}} f(\boldsymbol{h}), D_{\boldsymbol{x}} g(\boldsymbol{h}) \rangle + o(\|\boldsymbol{h}\|)}_{= O(\|\boldsymbol{h}\|^2) + o(\|\boldsymbol{h}\|) = o(\|\boldsymbol{h}\|)} \end{aligned}$$

■

Kreuzterm: $D_{\boldsymbol{x}} f(\boldsymbol{h}), D_{\boldsymbol{x}} g(\boldsymbol{h})$ sind beide $O(\|\boldsymbol{h}\|)$ (beschränkte lineare Abbildungen), die bilineare Abbildung multipliziert die Größenordnungen $\to O(\|\boldsymbol{h}\|^2)$.

Rechenregeln: Produktregel

Beispiele:

- $\mathbb{D} = \mathbb{R}, \mathbb{E} = \mathbb{R}, \mathbb{B} = \mathbb{R}$:

$$b(x) = f(x)g(x) \quad \Longrightarrow \quad b'(x) = f'(x)g(x) + g'(x)f(x).$$

- $\mathbb{D} = \mathbb{R}^n, \mathbb{E} = \mathbb{R}, \mathbb{B} = \mathbb{R}$:

$$b(\boldsymbol{x}) = f(\boldsymbol{x})g(\boldsymbol{x}) \quad \Longrightarrow \quad \nabla b(\boldsymbol{x}) = g(\boldsymbol{x})\nabla f(\boldsymbol{x}) + f(\boldsymbol{x})\nabla g(\boldsymbol{x}).$$

- $\mathbb{D} = \mathbb{R}^n, \mathbb{E} = \mathbb{R}^m, \mathbb{B} = \mathbb{R}$:

$$b(\boldsymbol{x}) = f(\boldsymbol{x})^\top g(\boldsymbol{x}) \quad \Longrightarrow \quad \nabla b(\boldsymbol{x}) = g(\boldsymbol{x})^\top \boldsymbol{J}_f(\boldsymbol{x}) + f(\boldsymbol{x})^\top \boldsymbol{J}_g(\boldsymbol{x}).$$

- $\mathbb{D} = \mathbb{R}, \mathbb{E} = \mathbb{R}^{n\times n}, \mathbb{B} = \mathbb{R}$:

$$b(x) = \operatorname{tr}\left[\boldsymbol{F}(x)\boldsymbol{G}(x)\right] \quad \Longrightarrow \quad b'(x) = \operatorname{tr}\left[\frac{\partial \boldsymbol{F}(x)}{\partial x}\boldsymbol{G}(x)\right] + \operatorname{tr}\left[\boldsymbol{F}(x)\frac{\partial \boldsymbol{G}(x)}{\partial x}\right].$$

Übung: Ableitung quadratischer Formen

Sei $b(\boldsymbol{x}) = f(\boldsymbol{x})^\top g(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}$ mit $f(\boldsymbol{x}) = \boldsymbol{x}$ und $g(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x}$.

Aufgabe:

Berechne $\nabla b(\boldsymbol{x})$.

Lösung:

- Es gilt $\boldsymbol{J}_f(\boldsymbol{x}) = \boldsymbol{I}_n$ und $\boldsymbol{J}_g(\boldsymbol{x}) = \boldsymbol{A}$.
- Dann ist nach der Produktregel

$$\begin{aligned}\nabla b(\boldsymbol{x}) &= g(\boldsymbol{x})^\top \boldsymbol{J}_f(\boldsymbol{x}) + f(\boldsymbol{x})^\top \boldsymbol{J}_g(\boldsymbol{x}) \\ &= (\boldsymbol{A}\boldsymbol{x})^\top \boldsymbol{I}_n + \boldsymbol{x}^\top \boldsymbol{A} = \boldsymbol{x}^\top \left(\boldsymbol{A} + \boldsymbol{A}^\top\right)\end{aligned}$$

- falls $\boldsymbol{A}$ symmetrisch also: $\nabla b(\boldsymbol{x}) = 2\boldsymbol{x}^\top \boldsymbol{A}$

Rechenregeln: Kettenregel

Theorem: Kettenregel

Seien $\mathbb{D}_1, \mathbb{D}_2, \mathbb{D}_3$ normierte Räume und $f\colon \mathbb{D}_1 \to \mathbb{D}_2$, $g\colon \mathbb{D}_2 \to \mathbb{D}_3$, sodass $g \circ f\colon \mathbb{D}_1 \to \mathbb{D}_3$. Ist f in $\boldsymbol{x} \in \mathbb{D}_1$ und g in $\boldsymbol{y} = f(\boldsymbol{x}) \in \mathbb{D}_2$ differenzierbar, dann ist

$$D_{\boldsymbol{x}}(g \circ f) = D_{\boldsymbol{y}} g \circ D_{\boldsymbol{x}} f.$$

Kettenregel: Beweis

- Da f differenzierbar ist, gilt

$$f(\boldsymbol{x}+\boldsymbol{h}) = f(\boldsymbol{x}) + D_{\boldsymbol{x}} f(\boldsymbol{h}) + o(\|\boldsymbol{h}\|).$$

- Daraus folgt

$$\begin{aligned}
(g \circ f)(\boldsymbol{x}+\boldsymbol{h}) &= g\left(f(\boldsymbol{x}) + D_{\boldsymbol{x}} f(\boldsymbol{h}) + o(\|\boldsymbol{h}\|)\right) \\
[g \text{ diff'bar:}] \qquad &= g\left(f(\boldsymbol{x})\right) + D_{\boldsymbol{y}} g\left(D_{\boldsymbol{x}} f(\boldsymbol{h}) + o(\|\boldsymbol{h}\|)\right) + o(\|\boldsymbol{h}\|) \\
[D_{\boldsymbol{y}} g \text{ linear:}] \qquad &= g\left(f(\boldsymbol{x})\right) + D_{\boldsymbol{y}} g\left(D_{\boldsymbol{x}} f(\boldsymbol{h})\right) + D_{\boldsymbol{y}} g\left(o(\|\boldsymbol{h}\|)\right) + o(\|\boldsymbol{h}\|) \\
[D_{\boldsymbol{y}} g \text{ beschr.:}] \qquad &= g\left(f(\boldsymbol{x})\right) + D_{\boldsymbol{y}} g\left(D_{\boldsymbol{x}} f(\boldsymbol{h})\right) + o(\|\boldsymbol{h}\|) \\
&= (g \circ f)(\boldsymbol{x}) + (D_{\boldsymbol{y}} g \circ D_{\boldsymbol{x}} f)(\boldsymbol{h}) + o(\|\boldsymbol{h}\|)
\end{aligned}$$

■

Zeile 2:
Der Restterm von g ist zunächst nur klein im weitergereichten Zuwachs $\boldsymbol{k} = D_{\boldsymbol{x}} f(\boldsymbol{h}) + o(\|\boldsymbol{h}\|)$, also $o(\|\boldsymbol{k}\|)$.
Da $D_{\boldsymbol{x}} f$ beschränkt ist ($\|D_{\boldsymbol{x}} f(\boldsymbol{h})\| \leq M_f \|\boldsymbol{h}\|$), gilt für kleine $\boldsymbol{h}$ aber $\|\boldsymbol{k}\| \leq (M_f + 1)\|\boldsymbol{h}\|$ –
damit ist $o(\|\boldsymbol{k}\|)$ auch $o(\|\boldsymbol{h}\|)$.

Rechenregeln: Kettenregel

Beispiele:

Anwendung: Kleinste Quadrate via Matrixkalkül

Problem (Kapitel 7 — Lineare Regression):
Response $\boldsymbol{y} \in \mathbb{R}^n$, Designmatrix $\boldsymbol{X} \in \mathbb{R}^{n \times p}$:

$$\ell(\boldsymbol{\beta}) = \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2 \to \min_{\boldsymbol{\beta}}$$

Schritt 1 — als Skalarprodukt schreiben:

$$\ell(\boldsymbol{\beta}) = (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta})^\top (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}) = \langle f(\boldsymbol{\beta}), f(\boldsymbol{\beta}) \rangle \quad \text{mit } f(\boldsymbol{\beta}) = \boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}$$

Schritt 2 — Jacobimatrix des inneren Terms (Linearität):

$$\boldsymbol{J}_f(\boldsymbol{\beta}) = -\boldsymbol{X}$$

Schritt 3 — Produktregel (mit $g = f$, beide Summanden gleich):

$$\begin{aligned} \nabla \ell(\boldsymbol{\beta}) &= f(\boldsymbol{\beta})^\top \boldsymbol{J}_f(\boldsymbol{\beta}) + f(\boldsymbol{\beta})^\top \boldsymbol{J}_f(\boldsymbol{\beta}) \\ &= 2\,(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta})^\top (-\boldsymbol{X}) = -2\,(\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta})^\top \boldsymbol{X} \end{aligned}$$

Anwendung: Kleinste Quadrate — Normalengleichungen

Schritt 4 — Gradient nullsetzen:
Am Minimum muss der Gradient verschwinden. (*Optimalitätsbedingung* → *Optimierung I*):

$$\nabla \ell(\widehat{\boldsymbol{\beta}}) = -2\left(\boldsymbol{y} - \boldsymbol{X}\widehat{\boldsymbol{\beta}}\right)^\top \boldsymbol{X} \overset{!}{=} \mathbf{0}^\top$$

Transponieren und umstellen:

$$\boldsymbol{X}^\top\left(\boldsymbol{y} - \boldsymbol{X}\widehat{\boldsymbol{\beta}}\right) = \mathbf{0} \quad \Longrightarrow \quad \boxed{\boldsymbol{X}^\top \boldsymbol{X}\,\widehat{\boldsymbol{\beta}} = \boldsymbol{X}^\top \boldsymbol{y}}$$

→ die *Normalengleichungen*: **die KQ-Lösung aus Kapitel 7 — jetzt hergeleitet!**

Payoff:

- Produkt- & Kettenregel im Matrixkalkül liefern die KQ-Lösung in 4 Zeilen — ohne eine einzige partielle Ableitung auszuschreiben.
- "$\nabla \ell = \mathbf{0}^\top$" als Optimalitätsbedingung wird systematisch ausgebaut: → *Optimierung I & II*.
- Gleiche Rechnung mit Strafterm $\lambda\|\boldsymbol{\beta}\|_2^2$ ergibt die Ridge-Regression (→ Übung).

Anwendung: Backprop in Neuronalen Netzen

- NN ist eine Funktion $f(\boldsymbol{x}) = (f_K \circ \cdots \circ f_2 \circ f_1)(\boldsymbol{x}) = \hat{\boldsymbol{y}}$ aufgebaut aus verketteten Funktionen:

$$\text{Input } \boldsymbol{x} \xrightarrow{f_1} \boldsymbol{z}_1 \xrightarrow{f_2} \boldsymbol{z}_2 \xrightarrow{\cdots} \cdots \xrightarrow{f_K} \text{ Output } \hat{\boldsymbol{y}}$$

- Jedes *layer* f_k
  - hat eine Gewichtsmatrix $\boldsymbol{W}_k \in \mathbb{R}^{d_k \times d_{k-1}}$ und
  - berechnet $f_k(\boldsymbol{z}_{k-1}) = \sigma_k(\boldsymbol{W}_k \boldsymbol{z}_{k-1})$,
  - wobei $\sigma_k : \mathbb{R}^{d_k} \to \mathbb{R}^{d_k}$ komponentenweise wirkt.
- Verlustfunktion $L(\hat{\boldsymbol{y}})$ misst Güte der Outputs.
- Wir brauchen $\frac{\partial L}{\partial \boldsymbol{W}_k}$ zur Optimierung.

Kettenregel mit Jacobimatrizen:

$$\frac{\partial L}{\partial \boldsymbol{W}_k} = \frac{\partial L}{\partial \hat{\boldsymbol{y}}} \cdot \boldsymbol{J}_{f_K}(\boldsymbol{z}_{K-1}) \cdot \boldsymbol{J}_{f_{K-1}}(\boldsymbol{z}_{K-2}) \cdots \boldsymbol{J}_{f_{k+1}}(\boldsymbol{z}_k) \cdot \frac{\partial f_k(\boldsymbol{z}_{k-1})}{\partial \boldsymbol{W}_k}$$

Backpropagation = effiziente Berechnung dieser Jacobimatrizen-Kette!

Beispiel:
Mit ReLU-Aktivierung $\sigma(x) = \max(0, x)$ ist $f_k(\boldsymbol{z}_{k-1}) = \max(\boldsymbol{0}, \boldsymbol{W}_k \boldsymbol{z}_{k-1})$, also:

$$\boldsymbol{J}_{f_k}(\boldsymbol{z}_{k-1}) = \operatorname{diag}\left(\mathbb{I}_{\boldsymbol{W}_k \boldsymbol{z}_{k-1} > 0}\right) \cdot \boldsymbol{W}_k$$

(nur für $(\boldsymbol{W}_k \boldsymbol{z}_{k-1})_i \neq 0$: an den Knickstellen ist $\max(0, \cdot)$ nicht differenzierbar)

Übung

Sei $h(\boldsymbol{x}) = \|\boldsymbol{x}\|_2 = \sqrt{\boldsymbol{x}^\top \boldsymbol{x}}$.

Frage:

Was ist $\nabla h(\boldsymbol{x})$ für $\boldsymbol{x} \neq \boldsymbol{0}$?

Antwort:

- Für $g(x) = \sqrt{x} = x^{1/2}$ gilt $g'(x) = \frac{1}{2x^{1/2}}$.
- Für $f(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{x}$ gilt $\nabla f(\boldsymbol{x}) = 2\boldsymbol{x}^\top$.
- Nach der Kettenregel ist

$$\nabla h(\boldsymbol{x}) = g'\big(f(\boldsymbol{x})\big) \nabla f(\boldsymbol{x}) = \frac{2\boldsymbol{x}^\top}{2f(\boldsymbol{x})^{1/2}} = \frac{\boldsymbol{x}^\top}{\|\boldsymbol{x}\|}.$$

- (In $\boldsymbol{x} = \boldsymbol{0}$ ist $\|\cdot\|_2$ *nicht* differenzierbar.)

Ableitungen höheren Grades

Ableitungen höheren Grades

Def.: k-mal (Fréchet-)differenzierbar

Seien $\mathbb{D}$ und $\mathbb{E}$ zwei normierte Räume und $f\colon \mathbb{D} \to \mathbb{E}$. Definiere $D^0_{\boldsymbol{x}} f := f(\boldsymbol{x})$.

Wir nennen f an der Stelle $\boldsymbol{x} \in \mathbb{D}$ **k-mal (Fréchet-)differenzierbar**,
falls für $j = 1, \ldots, k$ *beschränkte, multilineare Abbildungen* $D^j_{\boldsymbol{x}} f\colon \mathbb{D}^j \to \mathbb{E}$ existieren,
sodass für alle $\boldsymbol{h}_1, \ldots, \boldsymbol{h}_{j-1}$ und für $\boldsymbol{h} \to \mathbf{0}$ gilt:

$$D^j_{\boldsymbol{x}} f(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_{j-1}, \boldsymbol{h}) = D^{j-1}_{\boldsymbol{x}+\boldsymbol{h}} f(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_{j-1}) - D^{j-1}_{\boldsymbol{x}} f(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_{j-1}) + o(\|\boldsymbol{h}\|) \prod_{i=1}^{j-1} \|\boldsymbol{h}_i\|$$

- Höhere Ableitungen sind also wieder (multi-)lineare Annäherungen an die (lokale) Veränderung der nächstniedrigeren Ableitung von $f \implies$ Ableitungen berechnen wir wie gewohnt iterativ
- Höhere Ableitungen involvieren i.a. Tensoren, deren Stufe wächst mit j.

Konkret: Hesse-Matrix ($k = 2$)

Sei $f\colon \mathbb{R}^n \to \mathbb{R}$ (z.B. eine Log-Likelihood) und $f \in \mathcal{C}^2$.

- $j = 1$:
  $D_{\boldsymbol{x}} f(\boldsymbol{h}_1) = \nabla f(\boldsymbol{x})\, \boldsymbol{h}_1$ – *lineare* Abbildung, dargestellt durch den *Gradienten* (Zeilenvektor).
- $j = 2$:
  Die Definition fragt: *wie verändert sich Df selbst*, wenn wir $\boldsymbol{x}$ um $\boldsymbol{h}$ verschieben?

$$D_{\boldsymbol{x}+\boldsymbol{h}} f(\boldsymbol{h}_1) - D_{\boldsymbol{x}} f(\boldsymbol{h}_1) = [\nabla f(\boldsymbol{x} + \boldsymbol{h}) - \nabla f(\boldsymbol{x})]\boldsymbol{h}_1$$

- Lineare Approximation des Gradienten (jede Komponente von ∇f ist selbst differenzierbar):

$$\nabla f(\boldsymbol{x} + \boldsymbol{h}) - \nabla f(\boldsymbol{x}) = \boldsymbol{h}^\top \boldsymbol{H}_f(\boldsymbol{x}) + o(\|\boldsymbol{h}\|)$$

- Einsetzen ergibt die *bilineare* Abbildung

$$D^2_{\boldsymbol{x}} f(\boldsymbol{h}_1, \boldsymbol{h}) = \boldsymbol{h}^\top \boldsymbol{H}_f(\boldsymbol{x})\, \boldsymbol{h}_1$$

  dargestellt durch die Matrix $\boldsymbol{H}_f(\boldsymbol{x})$ der zweiten partiellen Ableitungen: die **Hesse-Matrix**

(Symmetrie von $\boldsymbol{H}_f$ für $f \in \mathcal{C}^2$: $\to$ Satz von Schwarz, gleich.)

Höhergradige Ableitung für Vektor-zu-Skalar

Def.: Hesse-Matrix

Die **Hesse-Matrix** $\boldsymbol{H}_f(\boldsymbol{x})$ einer zweimal differenzierbaren Funktion $f\colon S \subseteq \mathbb{R}^n \to \mathbb{R}$ (S offen) ist die *Matrix der zweiten partiellen Ableitungen*, also

$$\boldsymbol{H}_f(\boldsymbol{x}) := \left(\frac{\partial^2 f(\boldsymbol{x})}{\partial x_i \partial x_j} \right)_{i,j=1,\ldots,n} \in \mathbb{R}^{n \times n}.$$

Hier ist:

- $D_{\boldsymbol{x}} f(\boldsymbol{h}) = \nabla f(\boldsymbol{x}) \boldsymbol{h}$
- $D^2_{\boldsymbol{x}} f(\boldsymbol{h}_1, \boldsymbol{h}_2) = \boldsymbol{h}_1^\top \boldsymbol{H}_f(\boldsymbol{x}) \boldsymbol{h}_2$ mit *Hesse-Matrix* $\boldsymbol{H}_f(\boldsymbol{x}) \in \mathbb{R}^{n \times n}$.
- ($D^3_{\boldsymbol{x}} f \in \mathbb{R}^{n \times n \times n}$ wäre ein *Tensor* mit $\left(D^3_{\boldsymbol{x}} f\right)_{i,j,k} = \frac{\partial^3 f(\boldsymbol{x})}{\partial x_k \partial x_j \partial x_i}$ und $D^3_{\boldsymbol{x}} f(\boldsymbol{h}_1, \boldsymbol{h}_2, \boldsymbol{h}_3) = \sum_{i,j,k} \left(D^3_{\boldsymbol{x}} f\right)_{i,j,k} h_{1i} h_{2j} h_{3k}$)

Symmetrische Hesse-Matrizen

Theorem: Satz von Schwarz

Sei $f\colon \mathbb{R}^n \to \mathbb{R}$ zweimal stetig differenzierbar ($f \in \mathcal{C}^2$). Dann gilt:

$$\frac{\partial^2 f(\boldsymbol{x})}{\partial x_i \partial x_j} = \frac{\partial^2 f(\boldsymbol{x})}{\partial x_j \partial x_i} \quad \text{für alle } i, j.$$

$\Longrightarrow$ Hesse-Matrix für $f \in \mathcal{C}^2$ ist symmetrisch: $\boldsymbol{H}_f(\boldsymbol{x}) = \boldsymbol{H}_f(\boldsymbol{x})^\top$.

- Reihenfolge der partiellen Ableitungen spielt unter Stetigkeitsbedingungen keine Rolle
- Reduziert Anzahl der zu berechnenden Ableitungen von n^2 auf $\frac{n(n+1)}{2}$
- Wichtig für numerische Verfahren und Optimierung: nutze Symmetrie der Hesse-Matrix

Geometrische Interpretation und Konvexität

Hesse-Matrix charakterisiert die *lokale Krümmung* einer Funktion:

1. Definitheit der Hesse-Matrix:

Für $f \in \mathcal{C}^2$, $f\colon \mathbb{R}^n \to \mathbb{R}$, an einem *kritischen Punkt* $\boldsymbol{x}^*$ (d.h. $\nabla f(\boldsymbol{x}^*) = \mathbf{0}^\top$):

- $\boldsymbol{H}_f(\boldsymbol{x}^*)$ **positiv definit** ($\boldsymbol{h}^\top \boldsymbol{H}_f(\boldsymbol{x}^*)\boldsymbol{h} > 0$ für alle $\boldsymbol{h} \neq \mathbf{0}$) $\implies$ $\boldsymbol{x}^*$ ist ein **lokales Minimum**
- $\boldsymbol{H}_f(\boldsymbol{x}^*)$ **negativ definit** ($\boldsymbol{h}^\top \boldsymbol{H}_f(\boldsymbol{x}^*)\boldsymbol{h} < 0$ für alle $\boldsymbol{h} \neq \mathbf{0}$) $\implies$ $\boldsymbol{x}^*$ ist ein **lokales Maximum**
- $\boldsymbol{H}_f(\boldsymbol{x}^*)$ **indefinit** (positive und negative Eigenwerte) $\implies$ $\boldsymbol{x}^*$ ist ein **Sattelpunkt**

2. Konvexität:

$f \in \mathcal{C}^2$ ist **konvex** auf einer *offenen, konvexen* Menge $S \subseteq \mathbb{R}^n$ $\iff$
$\boldsymbol{H}_f(\boldsymbol{x}) \succeq 0$ (*positiv semidefinit*) für alle $\boldsymbol{x} \in S$.

3. Quadratische Approximation:

$$f(\boldsymbol{x} + \boldsymbol{h}) \approx f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})\boldsymbol{h} + \frac{1}{2}\boldsymbol{h}^\top \boldsymbol{H}_f(\boldsymbol{x})\boldsymbol{h}$$

- beste quadratische Approximation von f in der Umgebung von $\boldsymbol{x}$ ($\rightarrow$ *Taylorentwicklung*)
- $\boldsymbol{H}_f(\boldsymbol{x})$ beschreibt die lokale Krümmung von f in Richtung $\boldsymbol{h}$
- Eigenvektoren der Hesse-Matrix geben die Hauptkrümmungsrichtungen an, Eigenwerte die Stärke der Krümmung in diesen Richtungen

Praxisrelevanz Hesse-Matrix

- Heuristik: in typischen hochdimensionalen (nicht-konvexen) Problemen sind echte lokale Minima / Maxima **selten**, die meisten kritischen Punkte sind **Sattelpunkte**! $\Longrightarrow$ SGD & co funktionieren

  $\Longrightarrow$ *Singularity incoming* (maybe....)

- **Statistik**:
  - Wichtiger Spezialfall für $f\colon \mathbb{R}^p \to \mathbb{R}$: *(Log-)Likelihood-Funktion*
  - Hesse-Matrix der negativen Log-Likelihood ($\to$ beobachtete *Fisher-Information*) bestimmt Unsicherheit von Schätzern
- **ML**:
  Hesse-Matrix der Verlustfunktion bestimmt Konditionierung von Optimierungsproblemen
- **Optimierung**:
  Hesse-Matrix bestimmt Konvergenzverhalten von Optimierungsalgorithmen

Anwendung: Fisher-Information

Def.: Fisher-Informationsmatrix

Für ein statistisches Modell mit Log-Likelihood $\ell(\boldsymbol{\theta})$ und Parameter $\boldsymbol{\theta} \in \mathbb{R}^p$ ist die **Fisher-Informationsmatrix**, definiert als

$$\boldsymbol{I}(\boldsymbol{\theta}) := \mathbb{E}\left[-\boldsymbol{H}_\ell(\boldsymbol{\theta})\right] = -\mathbb{E}\left[\frac{\partial^2 \ell(\boldsymbol{\theta})}{\partial \theta_i \partial \theta_j}\right]_{i,j} \in \mathbb{R}^{p \times p}.$$

Intuition:

- Stark gekrümmte Log-Likelihood $\implies$ Daten sind "informativ" über optimales $\boldsymbol{\theta}$
- Flache Log-Likelihood $\implies$ wenig Information, Parameter schwer zu schätzen

Wichtige Konsequenz:

Asymptotische Effizienz des ML-Schätzers:
Für ML-Schätzer $\widehat{\boldsymbol{\theta}}_{ML}$ gilt (unter Regularitätsbedingungen) für großes n approximativ

$$\mathbb{V}\mathrm{ar}(\widehat{\boldsymbol{\theta}}_{ML}) \approx \boldsymbol{I}(\boldsymbol{\theta})^{-1},$$

der MLE erreicht damit asymptotisch die *Cramér-Rao-Schranke*, die untere Schranke für die Varianz unverzerrter Schätzer.
$\implies$ mehr Information $\rightarrow$ kleinere Varianz $\rightarrow$ präzisere Schätzung

Höhergradige Ableitung für Vektor-zu-Vektor

Wichtiger Fall:

$$f\colon \mathbb{R}^n \to \mathbb{R}^m$$

- **Beispiel:** Score-Funktion $\nabla \ell(\boldsymbol{\theta})^\top$ (in der Zeilenkonvention ist $\nabla \ell$ eine Zeile, erst die Transponierte bildet $\mathbb{R}^p \to \mathbb{R}^p$ ab), Momenten-Gleichungen
- j-te Ableitung ist multilineare Abbildung $D^j_{\boldsymbol{x}} f\colon (\mathbb{R}^n)^j \to \mathbb{R}^m$: $\to$ Stufe $j+1$-Tensor mit Dimensionen $m \times \underbrace{n \times n \times \cdots \times n}_{j-\text{mal}}$:

$$D^j_{\boldsymbol{x}} f(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_j) = \sum_{i_1,\ldots,i_j=1}^{n} \underbrace{\frac{\partial^j f(\boldsymbol{x})}{\partial x_{i_j} \cdots \partial x_{i_1}}}_{\in \mathbb{R}^m} h_{1,i_1} \cdots h_{j,i_j}$$

- Insbesondere:

$$D_{\boldsymbol{x}} f(\boldsymbol{h}) = \boldsymbol{J}_f(\boldsymbol{x})\boldsymbol{h}, \qquad D^2_{\boldsymbol{x}} f(\boldsymbol{h}_1, \boldsymbol{h}_2) = \sum_{i=1}^{n} \frac{\partial \boldsymbol{J}_f(\boldsymbol{x})}{\partial x_i} \boldsymbol{h}_1 \cdot h_{2,i}$$

(ausgeschriebene Summenformeln für $j = 1, 2, 3$: $\to$ Anhang)

Taylorapproximation

Taylorentwicklung: univariate Funktionen

Theorem: Taylorentwicklung I

Sei $f: \mathbb{R} \to \mathbb{R}$ $(k+1)$-mal *stetig differenzierbar*.

Dann gilt für $h \to 0$:

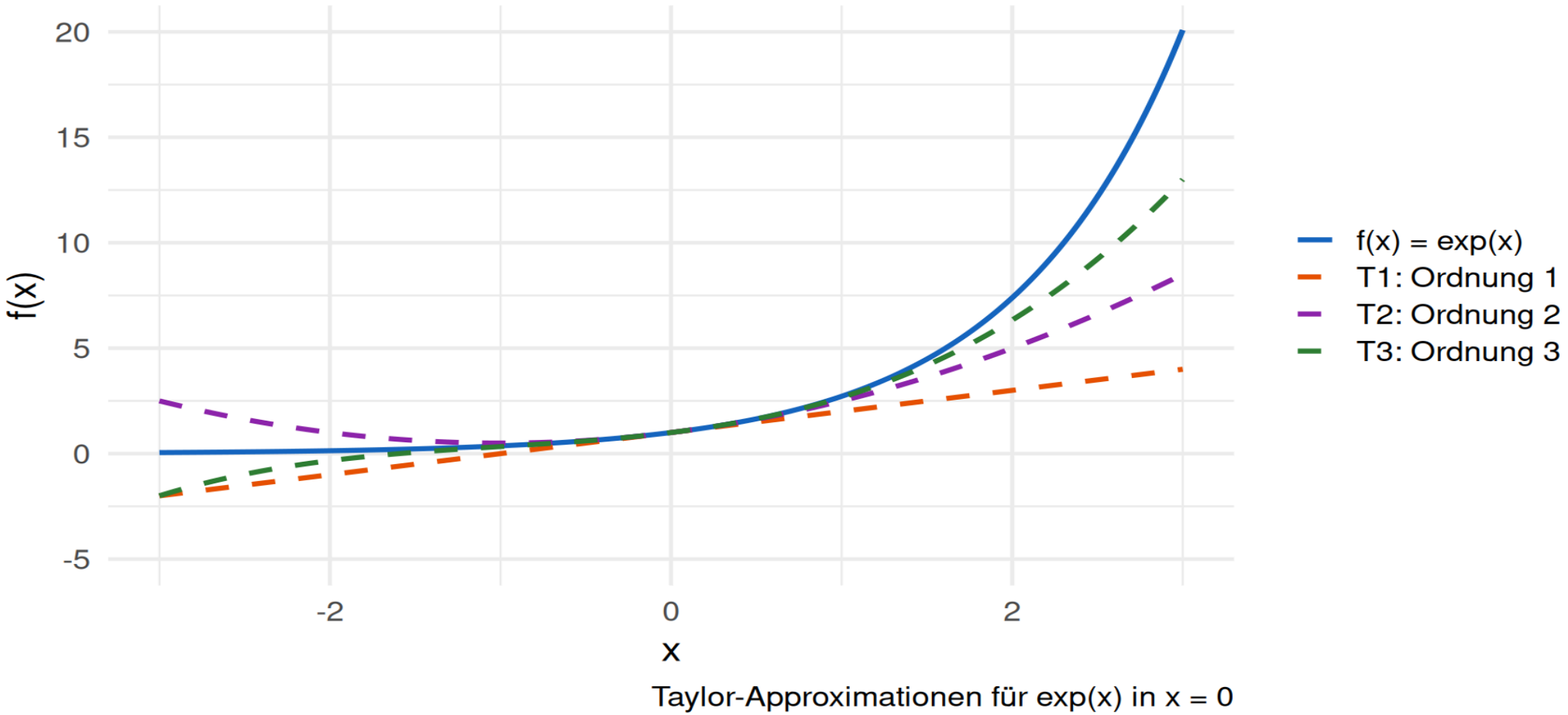
$$f(x+h) = f(x) + \sum_{j=1}^{k} \frac{1}{j!} f^{(j)}(x) h^j + o\big(|h|^k\big).$$

- *Taylorpolynom* $T_k(x+h) := \sum_{j=0}^{k} \frac{1}{j!} f^{(j)}(x) h^j$ (Entwicklungspunkt x)
- Dies ist nützlich um Funktionen lokal durch einfaches Polynom zu approximieren.
- Je höher der Grad k, desto besser die *lokale* Approximation für $h \to 0$ — *global* muss sie dadurch nicht besser werden!
- Essenzielles Tool in Konvergenzanalysen (z.B. asymptotische Normalität des Maximum-Likelihood-Schätzers).

Beweisskizze: → *Anhang*.

Taylorentwicklung: univariate Funktionen

$$f(x) \approx T_k(x) = f(0) + f'(0)x + \frac{1}{2}f''(0)x^2 + \frac{1}{6}f'''(0)x^3 + \dots$$



Taylorentwicklung: Allgemein

Theorem: Taylorentwicklung II

Seien V, W normierte Räume und $f\colon V \to W$ k-mal *stetig Fréchet-differenzierbar* ($f \in \mathcal{C}^k$). Dann gilt für $\boldsymbol{h} \to \mathbf{0}$:

$$f(\boldsymbol{x}+\boldsymbol{h}) = f(\boldsymbol{x}) + \sum_{j=1}^{k} \frac{1}{j!} D_{\boldsymbol{x}}^{j} f(\underbrace{\boldsymbol{h}, \ldots, \boldsymbol{h}}_{j\text{-mal}}) + o\big(\|\boldsymbol{h}\|^k\big).$$

- Gleiche Struktur wie univariat: Polynom in $\boldsymbol{h}$ mit multilinearen Koeffizienten $D_{\boldsymbol{x}}^{j} f$.

Beweisskizze (Reduktion auf den 1D-Fall): → *Anhang*.

Taylorapproximation für Vektor-zu-Skalar

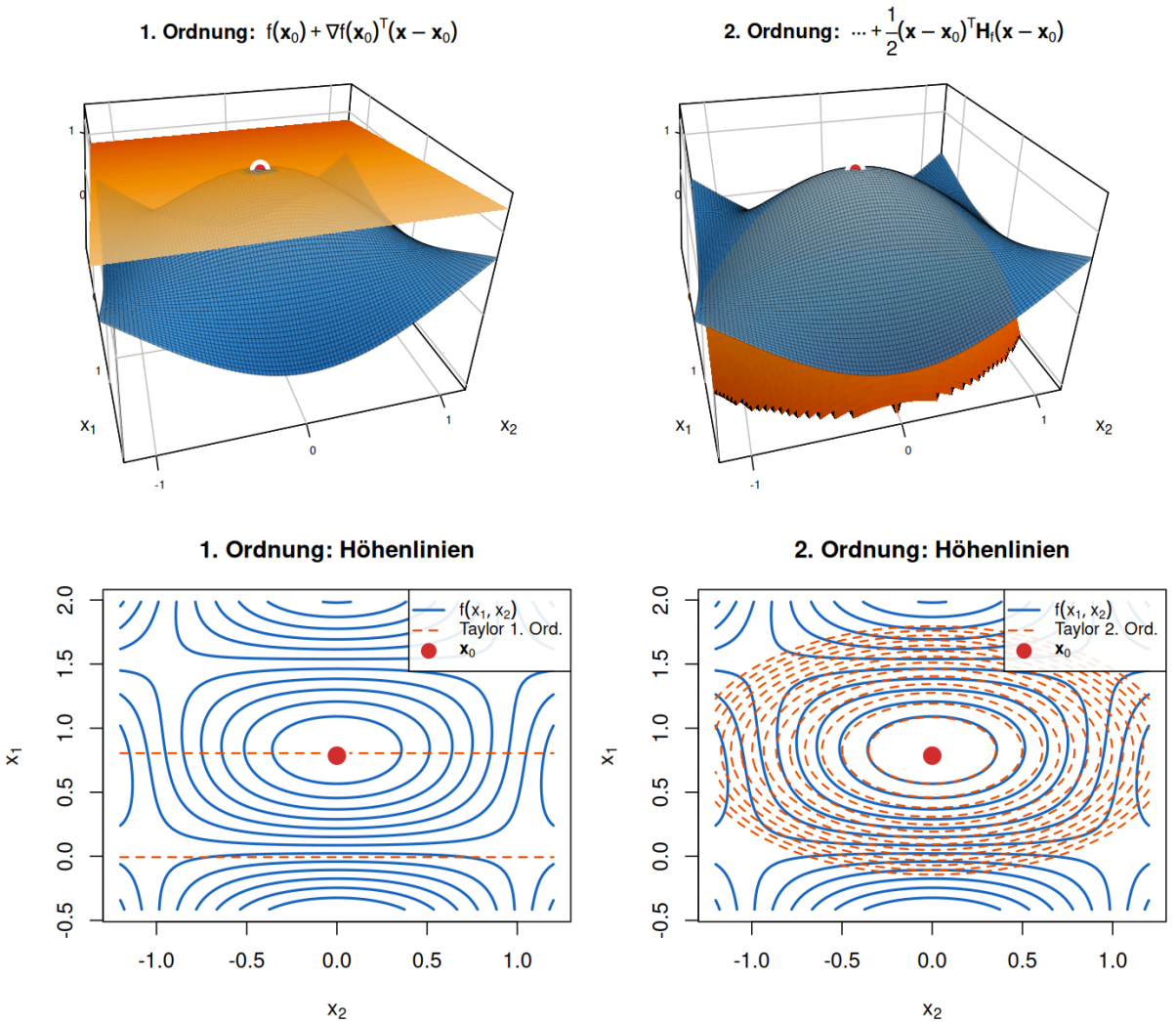
- Es gilt:

$$
\begin{aligned}
f(\boldsymbol{x}+\boldsymbol{h}) &= f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})\boldsymbol{h} + o(\|\boldsymbol{h}\|) \\
&= f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})\boldsymbol{h} + \frac{1}{2}\boldsymbol{h}^\top \boldsymbol{H}_f(\boldsymbol{x})\boldsymbol{h} + o(\|\boldsymbol{h}\|^2) \\
&= f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})\boldsymbol{h} + \frac{1}{2}\boldsymbol{h}^\top \boldsymbol{H}_f(\boldsymbol{x})\boldsymbol{h} + \frac{1}{6}\sum_{i,j,l=1}^{n} \frac{\partial^3 f(\boldsymbol{x})}{\partial x_i \partial x_j \partial x_l} h_i h_j h_l + o(\|\boldsymbol{h}\|^3) \\
&= \dots
\end{aligned}
$$

- Alternative Formulierung mit Entwicklungspunkt $\boldsymbol{x}_0$:

$$
\begin{aligned}
f(\boldsymbol{x}) &= f(\boldsymbol{x}_0) + \nabla f(\boldsymbol{x}_0)(\boldsymbol{x}-\boldsymbol{x}_0) + o(\|\boldsymbol{x}-\boldsymbol{x}_0\|) \\
&= f(\boldsymbol{x}_0) + \nabla f(\boldsymbol{x}_0)(\boldsymbol{x}-\boldsymbol{x}_0) + \frac{1}{2}(\boldsymbol{x}-\boldsymbol{x}_0)^\top \boldsymbol{H}_f(\boldsymbol{x}_0)(\boldsymbol{x}-\boldsymbol{x}_0) + o(\|\boldsymbol{x}-\boldsymbol{x}_0\|^2) \\
&= \dots
\end{aligned}
$$

Taylorapproximation für Vektor-zu-Skalar



Selbst ausprobieren: Shiny-App

Anwendung: Newton-Raphson-Verfahren

Problemstellung:
Finde $\boldsymbol{x}^* = \arg\min_{\boldsymbol{x}} f(\boldsymbol{x})$ für zweimal stetig diff.bare $f\colon \mathbb{R}^n \to \mathbb{R}$

Idee:
Approximiere $f(\boldsymbol{x})$ um aktuellen Punkt $\boldsymbol{x}^{(k)}$ durch das Taylor-Polynom 2. Ordnung (Entwicklungspunkt $\boldsymbol{x}^{(k)}$):

$$f(\boldsymbol{x}) \approx T_2(\boldsymbol{x}) = f\left(\boldsymbol{x}^{(k)}\right) + \nabla f\left(\boldsymbol{x}^{(k)}\right)\left(\boldsymbol{x} - \boldsymbol{x}^{(k)}\right) + \frac{1}{2}\left(\boldsymbol{x} - \boldsymbol{x}^{(k)}\right)^\top \boldsymbol{H}_f\left(\boldsymbol{x}^{(k)}\right)\left(\boldsymbol{x} - \boldsymbol{x}^{(k)}\right)$$

Newton-Schritt:
Minimiere $T_2(\boldsymbol{x})$ bzgl. $\boldsymbol{x}$. (für $\boldsymbol{H}_f\left(\boldsymbol{x}^{(k)}\right)$ *positiv definit* , sonst hat T_2 kein Minimum):

$$\begin{aligned}\nabla T_2(\boldsymbol{x}) = \nabla f\left(\boldsymbol{x}^{(k)}\right) + \left(\boldsymbol{x} - \boldsymbol{x}^{(k)}\right)^\top \boldsymbol{H}_f\left(\boldsymbol{x}^{(k)}\right) &\overset{!}{=} \mathbf{0}^\top \\ \implies \left(\boldsymbol{x} - \boldsymbol{x}^{(k)}\right)^\top &= -\nabla f\left(\boldsymbol{x}^{(k)}\right)\boldsymbol{H}_f\left(\boldsymbol{x}^{(k)}\right)^{-1}\end{aligned}$$

Newton-Iteration:

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \left(\nabla f\left(\boldsymbol{x}^{(k)}\right)\boldsymbol{H}_f\left(\boldsymbol{x}^{(k)}\right)^{-1}\right)^\top$$

Enorme Relevanz in ML/Statistics:

Maximum-Likelihood-Schätzung; IRLS/IWLS für GLMs; Trust-Region-Methoden ($\rightarrow$ *Optimierung II*)

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- Eigenschaften: Stetigkeit und Linearität der Ableitung
- Rechenregeln: *Produktregel* und *Kettenregel* $\longrightarrow$ Matrixkalkül liefert z.B. die *Normalengleichungen* der KQ-Schätzung in 4 Zeilen
- Höhergradige Ableitungen: multilineare Abbildungen, *Hesse-Matrix* & Krümmung
- *Taylorapproximation* und Newton-Raphson-Verfahren

Self-Check

Frage 1: Sei $b(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}$ mit *symmetrischem* $\boldsymbol{A} \in \mathbb{R}^{n\times n}$. Was ist $\nabla b(\boldsymbol{x})$?

1. $2\boldsymbol{x}^\top \boldsymbol{A}$
2. $\boldsymbol{x}^\top \boldsymbol{A}$
3. $2\boldsymbol{A}$
4. $2\boldsymbol{A}\boldsymbol{x}\boldsymbol{x}^\top$

Frage 2: Sei f $(k+1)$-mal stetig differenzierbar und T_k das Taylorpolynom vom Grad k um x. Welche Ordnung hat der Approximationsfehler $f(x+h) - T_k(x+h)$ für $h \to 0$?

1. $o\left(|h|^k\right)$
2. $o\left(|h|^{k+1}\right)$
3. $O(1)$
4. $o(|h|)$, unabhängig von k

Frage 3: Ein Newton-Schritt zur Minimierung von $f\colon \mathbb{R}^n \to \mathbb{R}$ basiert auf …

1. dem Taylorpolynom 1. Ordnung plus einer Schrittweite.
2. dem Taylorpolynom 2. Ordnung: er braucht Gradient **und** Hesse-Matrix.
3. nur der Hesse-Matrix.
4. dem Satz von Schwarz.

Nächste Vorlesung

- Konvexität

Anhang

Alternatives Beispiel (Kettenregel): Logistische Regression

Problemstellung:
Binäre Klassifikation mit $Y \in \{0, 1\}$ und Features $\boldsymbol{x} \in \mathbb{R}^p$: $P(Y = 1 \mid \boldsymbol{x}) = \sigma(\boldsymbol{\beta}^\top \boldsymbol{x})$ mit $\sigma(t) = \frac{1}{1+e^{-t}}$

Verlustfunktion für eine Beobachtung $(\boldsymbol{x}, y)$ ist:

$$\ell(\boldsymbol{\beta}) = -\left[y \log\left(\sigma(\boldsymbol{\beta}^\top \boldsymbol{x})\right) + (1-y) \log\left(1 - \sigma(\boldsymbol{\beta}^\top \boldsymbol{x})\right)\right] = -\begin{cases} \log(P(Y = 1 \mid \boldsymbol{x})) & \text{für } y = 1 \\ \log(P(Y = 0 \mid \boldsymbol{x})) & \text{für } y = 0 \end{cases}$$

Ableitung *für* $y = 1$: $\ell(\boldsymbol{\beta})|_{y=1} = -\log \sigma(\boldsymbol{\beta}^\top \boldsymbol{x}) = g(f(\boldsymbol{\beta}))$

- $f(\boldsymbol{\beta}) = \boldsymbol{\beta}^\top \boldsymbol{x}$ mit $\nabla f(\boldsymbol{\beta}) = \boldsymbol{x}^\top$
- $g(t) = -\log \sigma(t)$ mit $g'(t) = -\frac{\sigma'(t)}{\sigma(t)}$
- $\sigma'(t) = \frac{e^{-t}}{(1+e^{-t})^2} = \sigma(t)(1 - \sigma(t))$, also $g'(t) = -\frac{\sigma(t)(1-\sigma(t))}{\sigma(t)} = -(1 - \sigma(t)) = \sigma(t) - 1$

Mit Kettenregel und $\hat{y} = \sigma(\boldsymbol{\beta}^\top \boldsymbol{x})$:

$$\nabla \ell(\boldsymbol{\beta}) = g'(f(\boldsymbol{\beta})) \nabla f(\boldsymbol{\beta}) = (\sigma(\boldsymbol{\beta}^\top \boldsymbol{x}) - 1)\boldsymbol{x}^\top = (\hat{y} - 1)\boldsymbol{x}^\top$$

- analog für $y = 0$:

$$\nabla \ell(\boldsymbol{\beta})|_{y=0} = \hat{y}\boldsymbol{x}^\top$$

Taylorentwicklung I: Beweisskizze (univariat)

Vertiefung: Beweisskizze.

Sei $h > 0$ (Fall $h < 0$ analog). Definiere auf $[x, x+h]$:

$$F(t) := \sum_{j=0}^{k} \frac{f^{(j)}(t)}{j!}(x+h-t)^j, \qquad \text{also } F(x+h) = f(x+h),\ F(x) = T_k(x+h),$$

$$G(t) := (x+h-t)^{k+1}, \qquad \text{also } G(x+h) = 0,\ G(x) = h^{k+1},$$

mit $F'(t) = \frac{f^{(k+1)}(t)}{k!}(x+h-t)^k$ (Teleskopsumme!) und $G'(t) = -(k+1)(x+h-t)^k \neq 0$ auf $(x, x+h)$.

Cauchy-Mittelwertsatz: es gibt $\xi \in (x, x+h)$ mit

$$\frac{F(x+h)-F(x)}{G(x+h)-G(x)} = \frac{F'(\xi)}{G'(\xi)} \quad \Longrightarrow \quad f(x+h) - T_k(x+h) = \frac{f^{(k+1)}(\xi)}{(k+1)!}\,h^{k+1} = o\left(|h|^k\right). \ \blacksquare$$

Wieso $o(|h|^k)$? $f^{(k+1)}$ stetig $\Longrightarrow$ lokal beschränkt durch ein M, also $\left|f^{(k+1)}(\xi)\,h^{k+1}/(k+1)!\right|/|h|^k \le \frac{M}{(k+1)!}\,|h| \xrightarrow{h\to 0} 0$.

↩ zurück: „Taylorentwicklung: univariate Funktionen"

Taylorentwicklung II: Beweisskizze (allgemein)

Vertiefung: Beweisskizze.

Fixiere $\boldsymbol{x} \in V$. Für $\boldsymbol{h} = \mathbf{0}$ ist die Aussage trivial; sei also $\boldsymbol{h} \neq \mathbf{0}$.

- Setze $s := \|\boldsymbol{h}\|$ und $\boldsymbol{u} := \boldsymbol{h}/s$ (also $\|\boldsymbol{u}\| = 1$, $\boldsymbol{h} = s\boldsymbol{u}$).
- Definiere $\psi\colon \mathbb{R}_{\geq 0} \to W$ durch $\psi(w) := f(\boldsymbol{x} + w\boldsymbol{u})$.
- Wiederholte Kettenregel liefert: $\psi^{(j)}(w) = D^j_{\boldsymbol{x}+w\boldsymbol{u}} f(\underbrace{\boldsymbol{u}, \ldots, \boldsymbol{u}}_{j\text{-mal}})$.
- 1D-Taylor für ψ um $w = 0$ — in der *Peano-Form*, die nur $\psi \in \mathcal{C}^k$ braucht (Satz I setzt stärker $\mathcal{C}^{k+1}$ voraus): $\psi(s) = \sum_{j=0}^{k} \frac{s^j}{j!} \psi^{(j)}(0) + o(s^k) \quad \text{für } s \to 0.$
- Mit $\psi(s) = f(\boldsymbol{x} + \boldsymbol{h})$, $\psi(0) = f(\boldsymbol{x})$ und $\psi^{(j)}(0) = D^j_{\boldsymbol{x}} f(\boldsymbol{u}, \ldots, \boldsymbol{u})$:

$$f(\boldsymbol{x} + \boldsymbol{h}) = f(\boldsymbol{x}) + \sum_{j=1}^{k} \frac{s^j}{j!} D^j_{\boldsymbol{x}} f(\boldsymbol{u}, \ldots, \boldsymbol{u}) + o(s^k).$$

- Multilinearität von $D^j_{\boldsymbol{x}} f$: $D^j_{\boldsymbol{x}} f(\boldsymbol{u}, \ldots, \boldsymbol{u}) = D^j_{\boldsymbol{x}} f\left(\frac{\boldsymbol{h}}{s}, \ldots, \frac{\boldsymbol{h}}{s}\right) = \frac{1}{s^j} D^j_{\boldsymbol{x}} f(\boldsymbol{h}, \ldots, \boldsymbol{h})$.
- Mit $s = \|\boldsymbol{h}\|$:

$$f(\boldsymbol{x} + \boldsymbol{h}) = f(\boldsymbol{x}) + \sum_{j=1}^{k} \frac{1}{j!} D^j_{\boldsymbol{x}} f(\boldsymbol{h}, \ldots, \boldsymbol{h}) + o(\|\boldsymbol{h}\|^k). \quad \blacksquare$$

(Drei technische Feinheiten des Arguments: → Anhang.)

↩ zurück: „Taylorentwicklung: Allgemein"

Taylorentwicklung II: Beweisskizze — Feinheiten

Drei Punkte machen die Skizze zum vollständigen Beweis — hier für den im Kurs relevanten Fall $V = \mathbb{R}^n$, $W = \mathbb{R}^m$ (allgemeine normierte Räume brauchen andere Argumente):

1) Definitionsbereich: f muss auf einer Umgebung von $\boldsymbol{x}$ definiert sein, die das ganze Segment $\boldsymbol{x} + w\boldsymbol{u}$, $0 \leq w \leq s$, enthält — für hinreichend kleines $\|\boldsymbol{h}\|$ ist das erfüllt.

2) W-wertiges 1D-Taylor: Der 1D-Satz mit Peano-Restglied gilt zunächst für reellwertige Funktionen; wegen $\dim W < \infty$ wendet man ihn **koordinatenweise** auf $\psi = (\psi_1, \ldots, \psi_m)$ an und sammelt die Restglieder ein.

3) Gleichmäßigkeit in der Richtung: Das $o(s^k)$ aus dem 1D-Satz gilt zunächst *pro fester Richtung* $\boldsymbol{u}$. Damit daraus $o(\|\boldsymbol{h}\|^k)$ für **beliebige** $\boldsymbol{h} \to \boldsymbol{0}$ wird, muss der Fehlerterm gleichmäßig über alle Richtungen klein werden: Betrachte das Restglied für $j = k$ in Lagrange-Form — die Differenz $D^k_{\boldsymbol{x}+\xi\boldsymbol{u}} f - D^k_{\boldsymbol{x}} f$ wird wegen der **Stetigkeit von** $D^k f$ und der **Kompaktheit der Einheitssphäre** $\{\, \| \boldsymbol{u} \| = 1\}$ gleichmäßig klein.

↩ zurück: „Taylorentwicklung II: Beweisskizze (allgemein)"

Höhere Ableitungen für Vektor-zu-Vektor: explizite Formeln

Für $f\colon \mathbb{R}^n \to \mathbb{R}^m$ ist insbesonders:

$$D_{\boldsymbol{x}} f(\boldsymbol{h}) = \sum_{i=1}^{n} \frac{\partial f(\boldsymbol{x})}{\partial x_i} h_i = \boldsymbol{J}_f(\boldsymbol{x})\boldsymbol{h},$$

$$D^2_{\boldsymbol{x}} f(\boldsymbol{h}_1, \boldsymbol{h}_2) = \sum_{i=1}^{n}\sum_{j=1}^{n} \frac{\partial^2 f(\boldsymbol{x})}{\partial x_j \partial x_i} h_{1,i} h_{2,j} = \sum_{i=1}^{n} \frac{\partial \boldsymbol{J}_f(\boldsymbol{x})}{\partial x_i} \boldsymbol{h}_1 \cdot h_{2,i},$$

$$D^3_{\boldsymbol{x}} f(\boldsymbol{h}_1, \boldsymbol{h}_2, \boldsymbol{h}_3) = \sum_{i=1}^{n}\sum_{j=1}^{n}\sum_{k=1}^{n} \frac{\partial^3 f(\boldsymbol{x})}{\partial x_k \partial x_j \partial x_i} h_{1,i} h_{2,j} h_{3,k} = \sum_{i=1}^{n}\sum_{j=1}^{n} \frac{\partial^2 \boldsymbol{J}_f(\boldsymbol{x})}{\partial x_j \partial x_i} \boldsymbol{h}_1 \cdot h_{2,i} \cdot h_{3,j}$$

↩ zurück: „Höhergradige Ableitung für Vektor-zu-Vektor”

Konvexität

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 25.08.2026

Verwendete Vorkenntnisse

- Normen, Skalarprodukträume
- Gradient ∇f & *Hesse-Matrix* $\boldsymbol{H}_f$ (Kap. 10/11)
- **Lineare Algebra**
  - Vektorräume
  - Linearkombinationen: $\sum w_i \boldsymbol{v}_i$
  - Positiv definite Matrizen
- **Analysis**
  - Grenzwerte; Supremum & Infimum
  - Abgeschlossene Mengen
- **Wahrscheinlichkeitstheorie**
  - Erwartungswert $\mathbb{E}[X]$ als gewichteter Durchschnitt
  - Jensen-Ungleichung

Einführung

Einführung

- *Konvexität* ist ein wichtiges Konzept in der Mathematik, besonders in
  - Optimierung
  - Wahrscheinlichkeitstheorie
- Der Begriff bezieht sich meist auf die (verwandten) Konzepte
  - *konvexe Mengen*
  - *konvexe Funktionen*
- Beides ist gewissermaßen eine Verallgemeinerung von Linearitätskonzepten

Diese Folien: $\rightarrow$ wichtigste Definitionen und Eigenschaften

Warum Konvexität wichtig ist

In Optimierung / ML / Statistik:

- Konvexe Optimierungsprobleme sind *gutartig*: jedes lokale Minimum ist *global* ($\rightarrow$ *Optimierung I*)
- Viele Algorithmen basieren auf konvexer Optimierung, z.B.:
  - Maximum Likelihood (*meist* konkave Log-Likelihoods = konvexe *negative* Log-Likelihoods): GLMs etc.
  - Support Vector Machines
  - Ridge Regression

In der Wahrscheinlichkeitstheorie:

- *Jensen'sche Ungleichung* nutzt Konvexität für Erwartungswerte
- KL-Divergenz ist konvex (wichtig für *Variational Inference*)
- Varianz als Konvexitätseigenschaft: $\mathbb{E}[X^2] \geq (\mathbb{E}[X])^2$

Konvexkombinationen

Konvexkombinationen

Def.: Konvexkombination

Sei V ein Vektorraum und $\mathcal{X} = \{\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k\} \subset V$.
Dann nennen wir $\boldsymbol{x}$ eine **Konvexkombination** von Vektoren in $\mathcal{X}$,
falls es Koeffizienten $w_1, \ldots, w_k \geq 0$ gibt mit

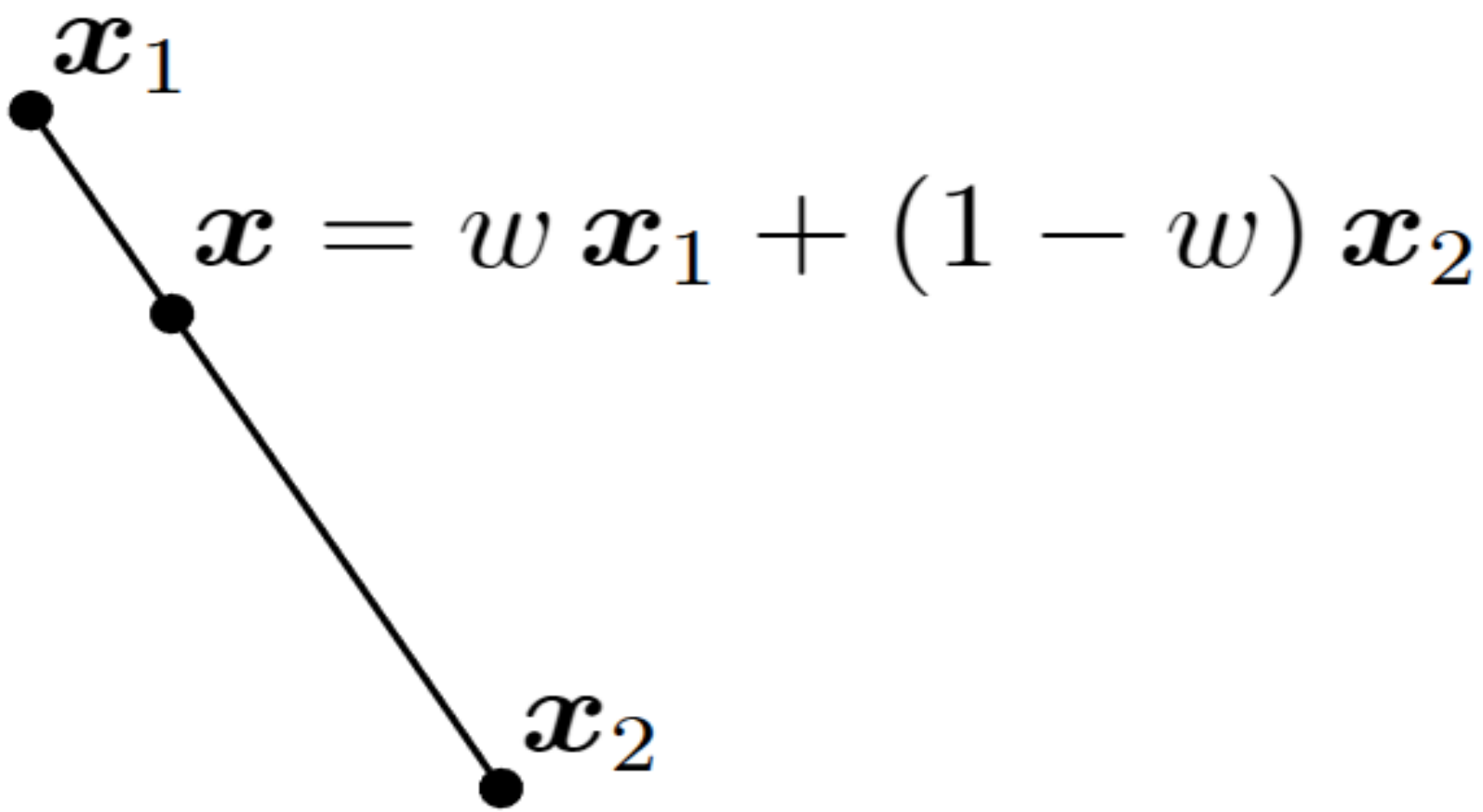
$$\boldsymbol{x} = \sum_{i=1}^{k} w_i \boldsymbol{x}_i \qquad \text{und} \qquad \sum_{i=1}^{k} w_i = 1.$$

- *Konvexkombinationen* sind spezielle Linearkombinationen.
- Sie können als gewichteter Durchschnitt interpretiert werden.
- **Bsp.:** Erwartungswert $\mathbb{E}(X)$ für diskrete Zufallsvariable X mit endlichem Träger ist Konvexkombination der Werte des Trägers $\mathcal{X} = T_X$:

$$\mathbb{E}(X) = \sum_{x_j \in T_X} \mathbb{P}(X = x_j) x_j$$

Konvexkombinationen

- Die Menge aller Konvexkombinationen zweier Vektoren $\boldsymbol{x}_1, \boldsymbol{x}_2$ ist das *Liniensegment*, das $\boldsymbol{x}_1$ und $\boldsymbol{x}_2$ verbindet.

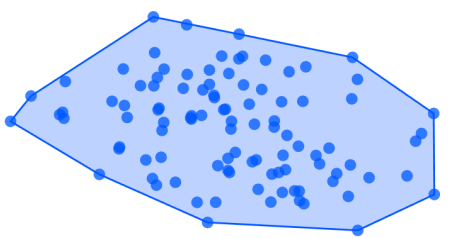


Konvexe Hülle

Def.: Konvexe Hülle

Sei $\mathcal{X} \subset V$ eine beliebige Menge von Vektoren.
Die *Menge aller Konvexkombinationen* von endlich vielen Vektoren aus $\mathcal{X}$ wird als **konvexe Hülle** $\operatorname{conv}(\mathcal{X})$ von $\mathcal{X}$ bezeichnet:

$$\operatorname{conv}(\mathcal{X}) = \left\{ \boldsymbol{x} = \sum_{i=1}^{N} w_i \boldsymbol{x}_i \colon N \in \mathbb{N}, \boldsymbol{x}_i \in \mathcal{X}, w_i \geq 0, \sum_{i=1}^{N} w_i = 1 \right\}$$



- *Extrempunkte* sind die Punkte $\boldsymbol{x}_i \in \mathcal{X}$, die sich nur mit $w_i = 1$ (und allen übrigen Gewichten 0) darstellen lassen.
- Die *konvexe Hülle* ist die kleinste konvexe Menge, die $\mathcal{X}$ enthält.

Konkretes Beispiel im $\mathbb{R}^2$: → Anhang.

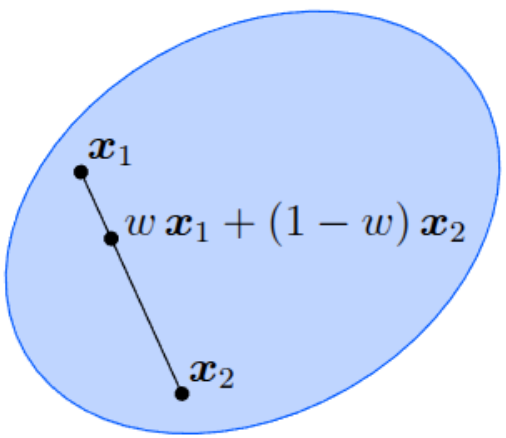
Konvexe Mengen

Konvexe Mengen: Definition

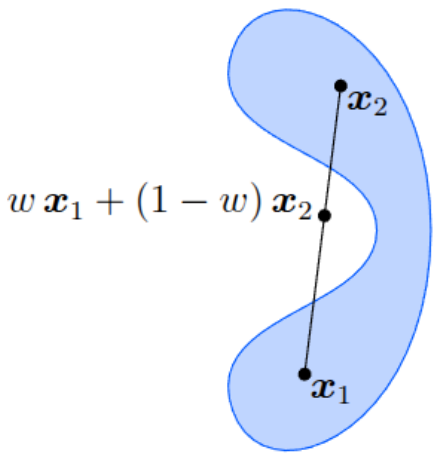
Def.: Konvexe Menge

Eine Menge $\mathcal{X} \subset V$ in Vektorraum V wird als **konvex** bezeichnet, falls für alle $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{X}$ und alle $\lambda \in [0, 1]$ gilt:

$$\lambda \boldsymbol{x} + (1 - \lambda)\boldsymbol{y} \in \mathcal{X}.$$



konvex



nicht konvex

Konvexe Mengen enthalten für alle Paare von Punkten auch das Liniensegment zwischen diesen Punkten $\implies$ sie haben keine “Einbuchtungen” oder “Löcher”.

Quiz: Konvexe Mengen

Welche der folgenden Mengen ist konvex?

1. Die leere Menge $\emptyset$.
2. Ein Vektorraum V.
3. Die Einheitssphäre $\{\boldsymbol{x} \in V \colon \|\boldsymbol{x}\| = 1\}$.
4. Der geschlossene Ball $\{\boldsymbol{x} \in V \colon \|\boldsymbol{x}\| \leq r\}$.
5. Die Menge $\{(x, y) \in \mathbb{R}^2 \colon y \leq x^2\}$.

Konvexe Mengen: Beispiele

Weitere Beispiele

- Ein offener Ball $\{\boldsymbol{x} \in V : \|\boldsymbol{x}\| < r\}$.
- Der *Simplex*

$$\Delta^N = \left\{ \boldsymbol{x} \in \mathbb{R}^N : x_i \geq 0, \sum_{i=1}^{N} x_i = 1 \right\}.$$

- Die Menge aller *positiv semi-definiten* Matrizen in $\mathbb{R}^{n \times n}$.

Simplex - Beispiel und Anwendung

Beispiel in $\mathbb{R}^3$:
Simplex Δ^3 besteht aus allen Punkten $\boldsymbol{x} = (x_1, x_2, x_3)$ mit $x_1, x_2, x_3 \geq 0$ und $x_1 + x_2 + x_3 = 1 = \|\boldsymbol{x}\|_1$

Extrempunkte:

$$\boldsymbol{e}_1 = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}, \boldsymbol{e}_2 = \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix}, \boldsymbol{e}_3 = \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

Anwendung in der Statistik:

- Δ^N definiert (die Menge aller) Wahrscheinlichkeitsverteilungen über N Ereignisse. Beispiel: $\boldsymbol{p} = (0.2, 0.5, 0.3)$ beschreibt eine kategoriale Verteilung mit drei Kategorien
- Wichtig für: Softmax-Regression/Multinomial-Modelle, Dirichlet-Verteilungen, ...

Geometrisch:

Δ^2 ist die Strecke von $(1, 0)$ nach $(0, 1)$,
Δ^3 das Dreieck zwischen $\boldsymbol{e}_1, \boldsymbol{e}_2$ und $\boldsymbol{e}_3$,
Δ^4 ein Tetraeder, ...

PSD-Matrizen bilden eine konvexe Menge

Def.: Positiv Semi-Definit

Eine *symmetrische* Matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ ist **positiv semi-definit (PSD)**
$\iff \boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x} \geq 0 \; \forall \, \boldsymbol{x} \in \mathbb{R}^n$.

Wir schreiben: $\boldsymbol{A} \succeq 0$

Seien $\boldsymbol{A}_1, \boldsymbol{A}_2 \succeq 0$ und $\lambda \in [0, 1]$.

Zu zeigen:
$\boldsymbol{A}_\lambda = \lambda \boldsymbol{A}_1 + (1 - \lambda)\boldsymbol{A}_2 \succeq 0$.

Für beliebiges $\boldsymbol{x} \in \mathbb{R}^n$:

$$\begin{aligned} \boldsymbol{x}^\top \boldsymbol{A}_\lambda \boldsymbol{x} &= \boldsymbol{x}^\top [\lambda \boldsymbol{A}_1 + (1 - \lambda)\boldsymbol{A}_2]\boldsymbol{x} \\ &= \lambda \boldsymbol{x}^\top \boldsymbol{A}_1 \boldsymbol{x} + (1 - \lambda)\boldsymbol{x}^\top \boldsymbol{A}_2 \boldsymbol{x} \\ &\geq \lambda \cdot 0 + (1 - \lambda) \cdot 0 = 0 \quad (\text{da } \boldsymbol{A}_1, \boldsymbol{A}_2 \succeq 0) \quad \blacksquare \end{aligned}$$

Anwendung:
Kovarianzmatrizen sind stets *PSD* (*SPD* nur, falls keine Komponente linear von den übrigen abhängt) und bilden eine *konvexe Menge*.

Konvexe Mengen: Eigenschaften

- Viele wichtige Operationen erhalten Konvexität.

Theorem: Konvexitätserhaltung

Seien $\mathcal{X}, \mathcal{X}_1, \mathcal{X}_2, \ldots$ konvexe Mengen. Dann sind auch die folgenden Mengen konvex:

- *Schnittmenge:*
  $\mathcal{S} = \bigcap_{i \in I} \mathcal{X}_i$ für eine beliebige Indexmenge I (alle $\mathcal{X}_i$ im selben Vektorraum)
- *Kartesisches Produkt:*
  $\mathcal{X}_1 \times \cdots \times \mathcal{X}_k$
- *Affine Abbildungen:*
  $f\colon \mathbb{R}^n \to \mathbb{R}^m$ mit $f(\boldsymbol{x}) = \boldsymbol{A}\boldsymbol{x} + \boldsymbol{b}$ und $\mathcal{X} \subseteq \mathbb{R}^n$: Bildmenge
  $f(\mathcal{X}) = \{\boldsymbol{y} = f(\boldsymbol{x})\colon \boldsymbol{x} \in \mathcal{X}\}$ ist konvex.
- *Summen:*
  $\mathcal{X}_1 \oplus \mathcal{X}_2 := \{\boldsymbol{x}_1 + \boldsymbol{x}_2\colon \boldsymbol{x}_1 \in \mathcal{X}_1, \boldsymbol{x}_2 \in \mathcal{X}_2\}$ ($\mathcal{X}_1, \mathcal{X}_2$ im selben Vektorraum)

- **Beispiel** für die Nützlichkeit der Schnitt-Eigenschaft:
  Polyeder (= Schnitte von *Halbräumen*) sind konvex – wichtig für *lineare Programmierung* ($\rightarrow$ *Optimierung unter linearen Nebenbedingungen*)!

Konvexe Hülle: Eigenschaften

- Eine *konvexe Hülle* ist selbst eine *konvexe Menge* (Konvexkombinationen von Konvexkombinationen aus $\mathcal{X}$ sind wieder Konvexkombinationen aus $\mathcal{X}$).
- Sie ist die **kleinste konvexe Menge, die ganz $\mathcal{X}$ enthält**.

Proposition

Es gilt $\operatorname{conv}(\mathcal{X}) = \bigcap_{\mathcal{X} \subseteq \mathcal{Y}, \mathcal{Y} \text{ konvex}} \mathcal{Y}$.

Beweis:

Sei $\mathcal{S} = \bigcap_{\mathcal{X} \subseteq \mathcal{Y}, \mathcal{Y} \text{ konvex}} \mathcal{Y}$.

- Da $\mathcal{X} \subseteq \mathcal{Y}$ und $\mathcal{Y}$ konvex ist, ist jede Konvexkombination von Punkten aus $\mathcal{X}$ auch in jedem $\mathcal{Y}$ (konvexe Mengen enthalten – per Induktion über die Anzahl der Punkte – alle Konvexkombinationen endlich vieler ihrer Punkte). Es folgt $\operatorname{conv}(\mathcal{X}) \subseteq \mathcal{S}$.
- Da der Schnitt auch über die Menge $\mathcal{Y} = \operatorname{conv}(\mathcal{X})$ geht, ist automatisch auch $\mathcal{S} \subseteq \operatorname{conv}(\mathcal{X})$.

Konvexe Mengen: Anwendung

- Konvexe Mengen/Hüllen sind wichtig in der Optimierung.

Beispiel:

- Sei $\mathcal{X} \subseteq \mathbb{N}^n$. Jedes $\boldsymbol{x} \in \mathcal{X}$ beschreibt eine Zusammensetzung von n verschiedenen Gegenständen: $x_i = k$ bedeutet, dass k Einheiten des i-ten Gegenstands vorhanden sind.
- Jeder Gegenstand hat einen Wert v_i und wir wollen das $\boldsymbol{x} \in \mathcal{X}$ mit maximalem Wert finden:

$$\boldsymbol{x}^\star \in \operatorname*{arg\,max}_{\boldsymbol{x} \in \mathcal{X}} \sum_{i=1}^{n} v_i x_i.$$

- Falls $\mathcal{X}$ sehr groß ist, dauert es zu lang, den Wert jeden Vektors auszurechnen.
- Man kann zeigen, dass das Maximum (auch) an einem Extrempunkt von $\operatorname{conv}(\mathcal{X})$ angenommen wird $\rightarrow$ oft viel leichter zu finden.

Konvexe Mengen: Projektionstheorem

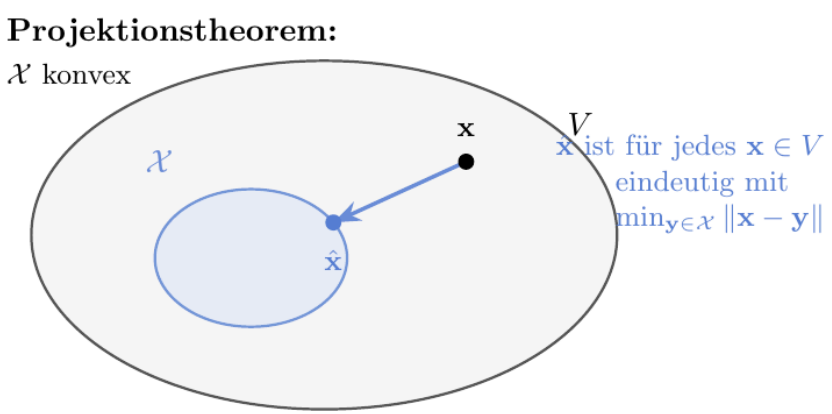
- Eine weitere wichtige Eigenschaft ist Existenz & Eindeutigkeit von *Projektionen* auf konvexe Mengen:

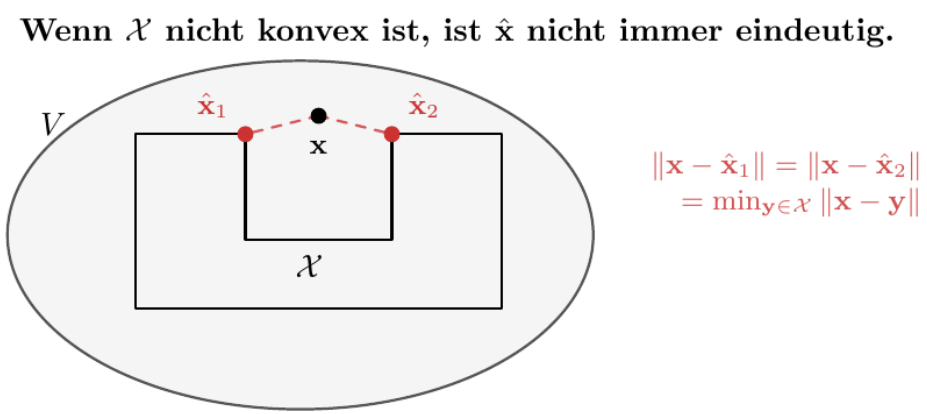
Theorem: Projektionstheorem

Sei $\mathcal{X} \neq \emptyset$ eine abgeschlossene, konvexe Teilmenge eines *vollständigen* Skalarproduktraums V (z.B. $V = \mathbb{R}^n$).
Dann existiert für jeden Punkt $\boldsymbol{x} \in V$ genau ein Punkt $\hat{\boldsymbol{x}} \in \mathcal{X}$, der den minimalen Abstand zu $\boldsymbol{x}$ hat:

$$\|\boldsymbol{x} - \hat{\boldsymbol{x}}\| = \min_{\boldsymbol{y} \in \mathcal{X}} \|\boldsymbol{x} - \boldsymbol{y}\|.$$

Dieser Punkt ist die **Projektion** von $\boldsymbol{x}$ auf $\mathcal{X}$.





Konvexe Funktionen

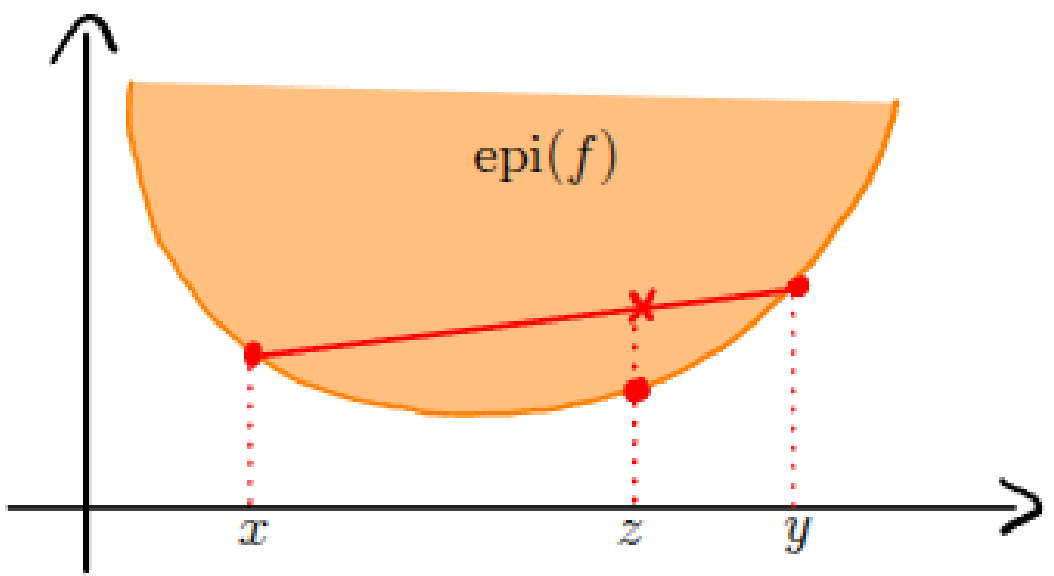
Konvexe Funktionen

Def.: Epigraph

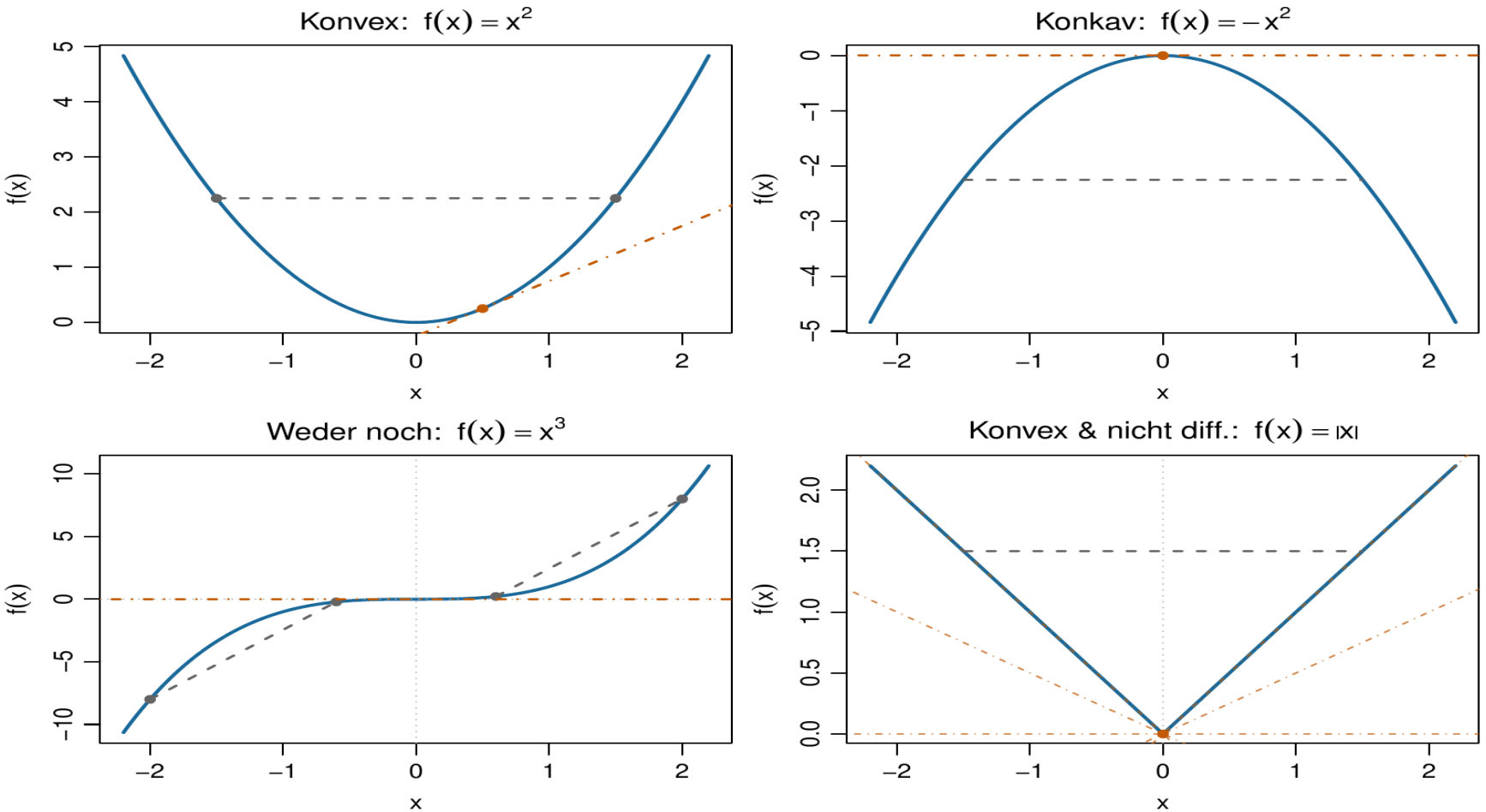
Der **Epigraph** einer Funktion $f\colon \mathcal{X} \to \mathbb{R}$ ist die Menge

$$\operatorname{epi}(f) = \{(\boldsymbol{x}, t) \in \mathcal{X} \times \mathbb{R} \colon t \geq f(\boldsymbol{x})\}$$

- Der *Epigraph* enthält alle Punkte in $\mathcal{X} \times \mathbb{R}$, die auf oder über dem Graphen von f liegen.
- *Konvexe Funktionen* $f\colon \mathcal{X} \to \mathbb{R}$ sind solche, deren *Epigraph* eine konvexe Menge ist. ($\mathcal{X}$ ist dann automatisch konvex.)



Konvex vs. Konkav



Konvexe Funktionen

- Eine gängigere Definition erfolgt direkt über Ungleichungen.

Def.: Konvexe Funktion

Sei $\mathcal{X}$ konvex und $f\colon \mathcal{X} \to \mathbb{R}$ eine Funktion.
Dann heißt f **konvex**, falls für alle $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{X}$ und alle $\lambda \in [0, 1]$ gilt

$$f(\lambda \boldsymbol{x} + (1 - \lambda)\boldsymbol{y}) \leq \lambda f(\boldsymbol{x}) + (1 - \lambda) f(\boldsymbol{y}).$$

- Intuitiv sagt die Definition (für $f : \mathbb{R}^n \to \mathbb{R}$):
  Die Strecke zwischen den Graphenpunkten $(\boldsymbol{x}, f(\boldsymbol{x}))$ und $(\boldsymbol{y}, f(\boldsymbol{y}))$ liegt *nie unter* dem Funktionsgraphen.
  Konvexe Funktionen sind also *"nach oben geöffnet"*.
- Wir nennen eine Funktion f **konkav**, wenn $-f$ *konvex* ist.

Konvexe Funktion - Verifikation

Zu zeigen:
$f(x) = |x|$ ist konvex auf $\mathbb{R}$.

Seien $x, y \in \mathbb{R}$ beliebig und $\lambda \in [0, 1]$. Zu prüfen:

$$f(\lambda x + (1-\lambda)y) \overset{?}{\leq} \lambda f(x) + (1-\lambda) f(y)$$

Beweis (Dreiecksungleichung):

$$\begin{aligned} f(\lambda x + (1-\lambda)y) &= |\lambda x + (1-\lambda)y| \\ &\leq |\lambda x| + |(1-\lambda)y| \\ &= \lambda |x| + (1-\lambda)|y| \\ &= \lambda f(x) + (1-\lambda) f(y) \quad \blacksquare \end{aligned}$$

Konvexe Funktionen: Beispiele

Die folgenden multivariaten Funktionen sind konvex:

- *Affine* Funktionen:
  $f(\boldsymbol{x}) = \boldsymbol{b}^\top \boldsymbol{x} + a$ mit $\boldsymbol{b} \in \mathbb{R}^n$, $a \in \mathbb{R}$.
  Sie sind gleichzeitig *auch* *konkav*!
- Manche *quadratischen* Funktionen:
  $f(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{Q}\boldsymbol{x} + \boldsymbol{b}^\top \boldsymbol{x} + a$ mit $\boldsymbol{Q} \in \mathbb{R}^{n \times n}$, $\boldsymbol{b} \in \mathbb{R}^n$, $a \in \mathbb{R}$, **falls der symmetrische Anteil** $\frac{1}{2}(\boldsymbol{Q} + \boldsymbol{Q}^\top)$ ***positiv semidefinit*** **ist** (für symmetrisches $\boldsymbol{Q}$ also: falls $\boldsymbol{Q} \succeq 0$).
- ***Jede*** *Norm*: $f(\boldsymbol{x}) = \|\boldsymbol{x}\|$, d.h. jede Funktion mit den Eigenschaften
  - $f(c\boldsymbol{x}) = |c| f(\boldsymbol{x})$
  - $f(\boldsymbol{x} + \boldsymbol{y}) \leq f(\boldsymbol{x}) + f(\boldsymbol{y})$
  - $f(\boldsymbol{x}) \geq 0$ und $f(\boldsymbol{x}) = 0 \implies \boldsymbol{x} = \boldsymbol{0}$

Anwendung in ML / Statistik:

- **Kleinste-Quadrate-Verlust:**
  $f(\boldsymbol{w}) = \|\boldsymbol{A}\boldsymbol{w} - \boldsymbol{y}\|_2^2 = \boldsymbol{w}^\top (\boldsymbol{A}^\top \boldsymbol{A}) \boldsymbol{w} - 2\boldsymbol{y}^\top \boldsymbol{A}\boldsymbol{w} + \|\boldsymbol{y}\|_2^2$ mit $\boldsymbol{Q} = \boldsymbol{A}^\top \boldsymbol{A} \succeq 0$ (immer PSD!) $\implies$
  konvexes *Optimierungsproblem*
- **Ridge-Regression:**
  $f(\boldsymbol{w}) = \|\boldsymbol{A}\boldsymbol{w} - \boldsymbol{y}\|_2^2 + \lambda \|\boldsymbol{w}\|_2^2$ mit $\lambda > 0$ ist **strikt konvex**,
  d.h. die Konvexitäts-Ungleichung gilt strikt für $\boldsymbol{w} \neq \boldsymbol{w}'$
  ($\rightarrow$ eindeutige Lösung; formale Definition und Konsequenzen im Kapitel *Optimierung I*)

Konvexe Funktionen: Eigenschaften

Proposition

Seien $f_1, f_2, \ldots : \mathcal{X} \to \mathbb{R}$ konvexe Funktionen. Dann sind auch die folgenden Funktionen konvex:

i. $f(\boldsymbol{x}) = f_1(\boldsymbol{x}) + f_2(\boldsymbol{x})$.

ii. $f(\boldsymbol{x}) = c f_1(\boldsymbol{x}), c \geq 0$.

iii. $f(\boldsymbol{x}) = \max_{i=1,\ldots,k} f_i(\boldsymbol{x})$.

iv. $f(\boldsymbol{x}) = \lim_{n\to\infty} f_n(\boldsymbol{x})$, vorausgesetzt der Grenzwert existiert für alle $\boldsymbol{x} \in \mathcal{X}$.

Beweis für (ii) + (iv): → *Anhang*.

Jensen-Ungleichung

Satz: Jensen-Ungleichung

Sei $\mathcal{X} \subseteq \mathbb{R}^n$ eine *konvexe Menge* und $f\colon \mathcal{X} \to \mathbb{R}$ eine *konvexe Funktion.*
Dann gilt für alle $N \in \mathbb{N}$, alle $\boldsymbol{x}_1, \ldots, \boldsymbol{x}_N \in \mathcal{X}$ und alle $w_1, \ldots, w_N \geq 0$ mit $\sum_{i=1}^{N} w_i = 1$:

$$f\left(\sum_{i=1}^{N} w_i \boldsymbol{x}_i\right) \leq \sum_{i=1}^{N} w_i f(\boldsymbol{x}_i).$$

- Die *Jensen-Ungleichung* verallgemeinert die Definition von Konvexität auf beliebig viele Punkte.
- Der Beweis erfolgt per Induktion über N.

Anwendung: Nichtnegative Varianz

Für eine Zufallsvariable X mit endlichem Träger T_X ist $\mathbb{E}[X] = \sum_{x \in T_X} P(X = x)x$ eine *Konvexkombination* der Werte in T_X (Gewichte = Wahrscheinlichkeiten).
Mit der *Jensen-Ungleichung* und da $x \mapsto x^2$ *konvex* ist gilt also:

$$\mathbb{E}[X]^2 = f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)] = \mathbb{E}[X^2]$$

Umstellen ergibt:

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \geq 0$$

Für allgemeine integrierbare X gilt die Ungleichung ebenfalls – der Beweis läuft dann über Stützgeraden statt über endliche Konvexkombinationen.

Konvexe Vektor-zu-Skalar-Funktionen

Theorem

Sei $\mathcal{X} \subseteq \mathbb{R}^n$ eine offene, *konvexe Menge* und $f\colon \mathcal{X} \to \mathbb{R}$ zweimal stetig differenzierbar.
Dann sind die folgenden Aussagen äquivalent:

i. f ist konvex.

ii. $f(\boldsymbol{y}) \geq f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})(\boldsymbol{y} - \boldsymbol{x})$ für alle $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{X}$.

iii. $\boldsymbol{H}_f(\boldsymbol{x})$ ist positiv semidefinit für alle $\boldsymbol{x} \in \mathcal{X}$.

- (i): f ist konvex genau dann wenn die Sehne zwischen zwei Punkten des Graphen nie *unter* dem Funktionsgraphen verläuft.
- (ii): f ist konvex genau dann wenn die Tangente an f nie *über* dem Funktionsgraphen verläuft.
- (iii): f ist konvex genau dann wenn die `Hesse-Matrix` positiv semidefinit ist (d.h.: nicht-negative Krümmung in alle Richtungen an jeder Stelle).

Beweis: → *Anhang*.

Konvexität & Spektrum der Hesse-Matrix

Für $f \in \mathcal{C}^2$ ist $\boldsymbol{H}_f(\boldsymbol{x})$ symmetrisch
$\implies$ $\boldsymbol{H}_f(\boldsymbol{x})$ hat reelle Eigenwerte λ_i und orthonormale Eigenvektoren $\boldsymbol{v}_i$:

- Lokale Taylor-Approximation um $\boldsymbol{x}$:
  $f(\boldsymbol{x}+\boldsymbol{h}) \approx f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})\,\boldsymbol{h} + \frac{1}{2}\boldsymbol{h}^\top \boldsymbol{H}_f(\boldsymbol{x})\boldsymbol{h} = f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})\,\boldsymbol{h} + \frac{1}{2}\sum_{i=1}^{n} \lambda_i \left(\boldsymbol{v}_i^\top \boldsymbol{h}\right)^2$
  - $\lambda_i > 0$: in $\boldsymbol{v}_i$-Richtung um $\boldsymbol{x}$ herum *konvex* (nach oben gekrümmt)
  - $\lambda_i < 0$: in $\boldsymbol{v}_i$-Richtung um $\boldsymbol{x}$ herum *konkav* (nach unten gekrümmt)
  - $|\lambda_i|$ misst Stärke der Krümmung in $\boldsymbol{v}_i$-Richtung

Wrap-up

Heute gelernt

- *Konvexkombinationen* = gewichtete Durchschnitte; *konvexe Hülle* = kleinste konvexe Obermenge
- *Konvexe Mengen* enthalten alle Verbindungsstrecken; Konvexität bleibt unter Schnitten, Produkten, affinen Abbildungen und Summen erhalten
- *Projektionen* auf nichtleere abgeschlossene konvexe Mengen (in vollständigen Skalarprodukträumen) existieren und sind eindeutig
- *Konvexe Funktionen*: *Epigraph* konvex $\iff$ Sehnen-Ungleichung; *Jensen-Ungleichung*
- $\mathcal{C}^2$-Charakterisierung: f konvex $\iff$ *Hesse-Matrix* überall PSD

Nächste Vorlesung

- Optimierung I – u.a.: warum konvexe Optimierungsprobleme “gutartig” sind

Self-Check: Konvexe Funktionen

Welche der folgenden Funktionen ist konvex?

1. $f\colon \mathbb{R} \to \mathbb{R}, f(x) = ax + b, a, b \in \mathbb{R}$.
2. $f\colon \mathbb{R} \to \mathbb{R}, f(x) = x^4$.
3. $f\colon \mathbb{R}_+ \to \mathbb{R}, f(x) = \sqrt{x}$.
4. $f\colon \mathbb{R} \to \mathbb{R}, f(x) = e^x$.
5. $f\colon \mathbb{R}_+ \to \mathbb{R}, f(x) = \ln x$.

Self-Check: Klassifikation

Klassifizieren Sie: **konvex**, **strikt konvex** (d.h., die Ungleichung gilt strikt für $\boldsymbol{x} \neq \boldsymbol{y}$, $\lambda \in (0, 1)$) oder **keins von beiden**?

1. $f\colon \mathbb{R} \to \mathbb{R}, f(x) = |x|$
2. $f\colon \mathbb{R} \to \mathbb{R}, f(x) = e^x$
3. $f\colon \mathbb{R} \to \mathbb{R}, f(x) = ax + b, a, b \in \mathbb{R}$
4. $f\colon \mathbb{R}_+ \to \mathbb{R}, f(x) = \sqrt{x}$

Anhang

Konvexe Hülle - Beispiel im $\mathbb{R}^2$

(Zusatzbeispiel)

Sei $\mathcal{X} = \{(0,0), (2,0), (1,2)\} \subset \mathbb{R}^2$ (die Ecken eines Dreiecks).

Extrempunkte:
Alle drei Punkte in $\mathcal{X}$ sind *Extrempunkte*, da sie nur durch $w_i = 1, w_j = 0 \; \forall \, j \neq i$ erreicht werden können.

Konvexe Hülle:
$\text{conv}(\mathcal{X})$ ist die Menge aller Punkte der Form

$$\boldsymbol{x} = w_1 \begin{pmatrix} 0 \\ 0 \end{pmatrix} + w_2 \begin{pmatrix} 2 \\ 0 \end{pmatrix} + w_3 \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

mit $w_1, w_2, w_3 \geq 0$ und $w_1 + w_2 + w_3 = 1$.

Konkreter Punkt: Für $w_1 = w_2 = w_3 = \frac{1}{3}$:

$$\boldsymbol{x} = \tfrac{1}{3} \begin{pmatrix} 0 \\ 0 \end{pmatrix} + \tfrac{1}{3} \begin{pmatrix} 2 \\ 0 \end{pmatrix} + \tfrac{1}{3} \begin{pmatrix} 1 \\ 2 \end{pmatrix} = \begin{pmatrix} 1 \\ \frac{2}{3} \end{pmatrix}$$

Dieser Punkt (Schwerpunkt!) liegt im Inneren des Dreiecks.

↩ zurück: „Konvexe Hülle"

Beispiel: Schnitt konvexer Mengen

(Zusatzbeispiel)

Konkrete Mengen im $\mathbb{R}^2$:

- $\mathcal{X}_1 = \{(x, y) \in \mathbb{R}^2 : x + y \leq 1\}$ (Halbraum, konvex)
- $\mathcal{X}_2 = \{(x, y) \in \mathbb{R}^2 : x \geq 0\}$ (Halbraum, konvex)
- $\mathcal{X}_3 = \{(x, y) \in \mathbb{R}^2 : y \geq 0\}$ (Halbraum, konvex)

Schnitt:

$$\mathcal{S} = \mathcal{X}_1 \cap \mathcal{X}_2 \cap \mathcal{X}_3 = \{(x, y) : x \geq 0, y \geq 0, x + y \leq 1\}$$

Dies ist ein Dreieck mit Ecken $(0, 0), (1, 0), (0, 1)$ - offensichtlich konvex!

Generell:

Polyeder (als Schnitte von Halbräumen) sind konvex – wichtig für *lineare Programmierung* (Optimierung unter linearen Nebenbedingungen)!

↩ zurück: „Konvexe Mengen: Eigenschaften”

Konvexitätserhaltung: Beweis

- *(i) Schnitt:*
  Sei $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{S}$, $\lambda \in [0,1]$ und $\boldsymbol{z} = \lambda \boldsymbol{x} + (1-\lambda)\boldsymbol{y}$.
  Dann gilt $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{S} \implies \boldsymbol{x}, \boldsymbol{y} \in \mathcal{X}_i \; \forall i \in I$.
  $\mathcal{X}_i$ konvex $\forall i \in I \implies \boldsymbol{z} \in \mathcal{X}_i \; \forall i \in I \implies \boldsymbol{z} \in \mathcal{S}$.
- *(ii) Kartesisches Produkt:*
  Seien $(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k), (\boldsymbol{y}_1, \ldots, \boldsymbol{y}_k) \in \mathcal{X}_1 \times \cdots \times \mathcal{X}_k; \lambda \in [0,1]$.
  Weil $\mathcal{X}_i$ konvex, ist auch

$$\begin{aligned} &\lambda(\boldsymbol{x}_1, \ldots, \boldsymbol{x}_k) + (1-\lambda)(\boldsymbol{y}_1, \ldots, \boldsymbol{y}_k) \\ &= (\lambda \boldsymbol{x}_1 + (1-\lambda)\boldsymbol{y}_1, \ldots, \lambda \boldsymbol{x}_k + (1-\lambda)\boldsymbol{y}_k) \in (\mathcal{X}_1 \times \cdots \times \mathcal{X}_k) \end{aligned}$$

- *(iii) Affine Abb.:*
  Sei $\boldsymbol{y}_1 = f(\boldsymbol{x}_1), \boldsymbol{y}_2 = f(\boldsymbol{x}_2) \in f(\mathcal{X})$ für $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{X}$; $\lambda \in [0,1]$. Dann gilt:

$$\begin{aligned} \lambda \boldsymbol{y}_1 + (1-\lambda)\boldsymbol{y}_2 &= \lambda(\boldsymbol{A}\boldsymbol{x}_1 + \boldsymbol{b}) + (1-\lambda)(\boldsymbol{A}\boldsymbol{x}_2 + \boldsymbol{b}) \\ &= \boldsymbol{A}(\lambda \boldsymbol{x}_1 + (1-\lambda)\boldsymbol{x}_2) + \boldsymbol{b} \\ &= f(\lambda \boldsymbol{x}_1 + (1-\lambda)\boldsymbol{x}_2) \in f(\mathcal{X}). \end{aligned}$$

- *(iv) Summe:*
  Folgt aus (ii) und (iii):
  $\mathcal{X}_1 \times \mathcal{X}_2$ ist konvex, $f(\boldsymbol{x}_1, \boldsymbol{x}_2) = \boldsymbol{x}_1 + \boldsymbol{x}_2$ ist linear, also ist auch $f(\mathcal{X}_1 \times \mathcal{X}_2) = \mathcal{X}_1 \oplus \mathcal{X}_2$ konvex.

↩ zurück: „Konvexe Mengen: Eigenschaften”

Projektionstheorem: Beweisskizze

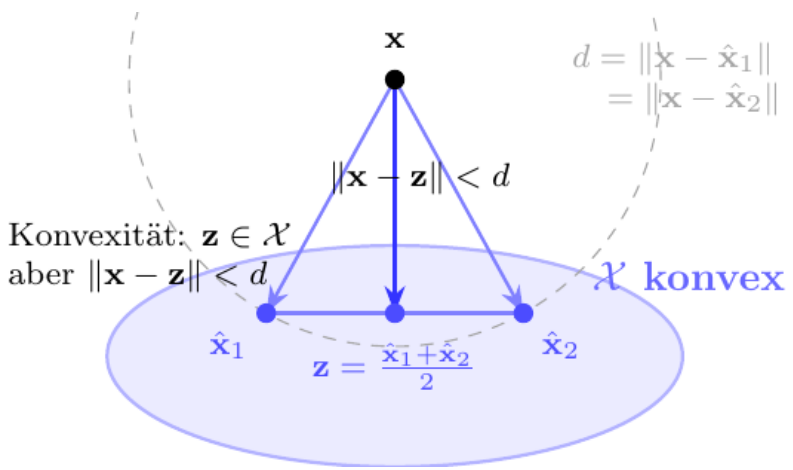
- **Existenz:**
  Eine Minimierungsfolge ist beschränkt und besitzt daher einen Grenzwert (in $\mathbb{R}^n$: Bolzano-Weierstraß; allgemein: Parallelogrammgleichung $\to$ Cauchy-Folge + Vollständigkeit von V);
  $\mathcal{X}$ abgeschlossen $\implies$ dieser Grenzwert liegt in $\mathcal{X}$

- **Eindeutigkeit:**
  Angenommen, es *gäbe* $\hat{\boldsymbol{x}}_1 \neq \hat{\boldsymbol{x}}_2$ mit demselben minimalen Abstand d zu $\boldsymbol{x}$.

Für jedes $\lambda \in [0,1]$ gilt wegen Konvexität
$\boldsymbol{z} = \lambda\hat{\boldsymbol{x}}_1 + (1-\lambda)\hat{\boldsymbol{x}}_2 \in \mathcal{X}$ und nach der Dreiecksungleichung:

$$\|\boldsymbol{x} - \boldsymbol{z}\| \leq \lambda\|\boldsymbol{x} - \hat{\boldsymbol{x}}_1\| + (1-\lambda)\|\boldsymbol{x} - \hat{\boldsymbol{x}}_2\| = d$$

Wegen der Minimalität von d gilt sogar $\|\boldsymbol{x} - \boldsymbol{z}\| = d$.
Wähle Mittelpunkt $\boldsymbol{z} = (\hat{\boldsymbol{x}}_1 + \hat{\boldsymbol{x}}_2)/2$ und setze $\boldsymbol{u} := \boldsymbol{x} - \hat{\boldsymbol{x}}_1$,
$\boldsymbol{v} := \boldsymbol{x} - \hat{\boldsymbol{x}}_2$.



Aus der Definition von $\boldsymbol{z}$ folgt $\boldsymbol{x} - \boldsymbol{z} = \frac{\boldsymbol{u}+\boldsymbol{v}}{2}$, $\quad \boldsymbol{z} - \hat{\boldsymbol{x}}_1 = \frac{\boldsymbol{u}-\boldsymbol{v}}{2}$,
und diese sind orthogonal: $\left\langle \boldsymbol{x} - \boldsymbol{z}, \boldsymbol{z} - \hat{\boldsymbol{x}}_1 \right\rangle = \frac{1}{4}\langle \boldsymbol{u}+\boldsymbol{v}, \boldsymbol{u}-\boldsymbol{v}\rangle = \frac{1}{4}\left(\|\boldsymbol{u}\|^2 - \|\boldsymbol{v}\|^2\right) = \frac{1}{4}\left(d^2 - d^2\right) = 0$

Damit also auch

$$\begin{aligned}\|\boldsymbol{x} - \hat{\boldsymbol{x}}_1\|^2 &= \|(\boldsymbol{x} - \boldsymbol{z}) + (\boldsymbol{z} - \hat{\boldsymbol{x}}_1)\|^2 \\ &= \|\boldsymbol{x} - \boldsymbol{z}\|^2 + \|\boldsymbol{z} - \hat{\boldsymbol{x}}_1\|^2 && [(\boldsymbol{x} - \boldsymbol{z}) \perp (\boldsymbol{z} - \hat{\boldsymbol{x}}_1)] \\ &> \|\boldsymbol{x} - \boldsymbol{z}\|^2,\end{aligned}$$

also: *Widersprüchlich* $d^2 > d^2 \implies$ Projektion eindeutig. ■

↩ zurück: „Konvexe Mengen: Projektionstheorem”

Konvexe Funktionen: Beweise I

(Beweis der $\mathcal{C}^2$-Charakterisierung; (i) $\iff$ (ii))

- $(i) \implies (ii)$: Setze $t = (1 - \lambda)$. Dann gilt

$$
\begin{aligned}
& f(\lambda \boldsymbol{x} + (1-\lambda)\boldsymbol{y}) \leq \lambda f(\boldsymbol{x}) + (1-\lambda) f(\boldsymbol{y}) \\
\iff \quad & f(\lambda \boldsymbol{x} + (1-\lambda)\boldsymbol{y}) - f(\boldsymbol{x}) \leq (1-\lambda)[f(\boldsymbol{y}) - f(\boldsymbol{x})] \\
\iff \quad & f(\boldsymbol{y}) - f(\boldsymbol{x}) \geq \frac{f(\boldsymbol{x} + t(\boldsymbol{y} - \boldsymbol{x})) - f(\boldsymbol{x})}{t} \quad \text{für } t > 0. \quad \blacksquare
\end{aligned}
$$

  (Für $t \to 0$ erhalten wir (ii).)

- $(ii) \implies (i)$: Sei $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{X}$, $\lambda \in [0, 1]$ und definiere $\boldsymbol{z} = \lambda \boldsymbol{x} + (1-\lambda)\boldsymbol{y}$. (ii) impliziert

$$
\begin{aligned}
\lambda f(\boldsymbol{x}) &\geq \lambda f(\boldsymbol{z}) + \lambda \nabla f(\boldsymbol{z})(\boldsymbol{x} - \boldsymbol{z}) \\
\text{und } (1-\lambda) f(\boldsymbol{y}) &\geq (1-\lambda) f(\boldsymbol{z}) + (1-\lambda) \nabla f(\boldsymbol{z})(\boldsymbol{y} - \boldsymbol{z}).
\end{aligned}
$$

  Summieren wir die beiden Ungleichungen, erhalten wir

$$
\lambda f(\boldsymbol{x}) + (1-\lambda) f(\boldsymbol{y}) \geq f(\boldsymbol{z}) + 0 = f(\lambda \boldsymbol{x} + (1-\lambda)\boldsymbol{y}). \quad \blacksquare
$$

↩ zurück: „Konvexe Vektor-zu-Skalar-Funktionen”

Konvexe Funktionen: Beweise II

(Beweis der $\mathcal{C}^2$-Charakterisierung, fortgesetzt; (ii) $\iff$ (iii))

- $(ii) \implies (iii)$: Sei $\boldsymbol{h} \in \mathbb{R}^n$ beliebig. Nach Definition der zweiten Ableitung gilt für $t \to 0$,

$$
\begin{aligned}
f(\boldsymbol{x} + t\boldsymbol{h}) - f(\boldsymbol{x}) - t\nabla f(\boldsymbol{x})\boldsymbol{h} - \frac{t^2}{2}\boldsymbol{h}^\top \boldsymbol{H}_f(\boldsymbol{x})\boldsymbol{h} &= o(t^2) \qquad \text{[Taylorpolynom]} \\
\iff \boldsymbol{h}^\top \boldsymbol{H}_f(\boldsymbol{x})\boldsymbol{h} &= 2\,\frac{f(\boldsymbol{x} + t\boldsymbol{h}) - f(\boldsymbol{x}) - t\nabla f(\boldsymbol{x})\boldsymbol{h}}{t^2} + o(1) \\
&\geq 0 + o(1) \to 0.
\end{aligned}
$$

  (Da die linke Seite nicht von t abhängt, folgt (iii).)

- $(iii) \implies (ii)$: *Taylor's Theorem*: für $f \in \mathcal{C}^2$ und jedes $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{X}$ $\exists\, t \in (0, 1)$, so dass:

$$
\begin{aligned}
f(\boldsymbol{y}) &= f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})(\boldsymbol{y} - \boldsymbol{x}) + \frac{1}{2}(\boldsymbol{y} - \boldsymbol{x})^\top \boldsymbol{H}_f(\boldsymbol{x} + t(\boldsymbol{y} - \boldsymbol{x}))(\boldsymbol{y} - \boldsymbol{x}) \\
&\geq f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})(\boldsymbol{y} - \boldsymbol{x})
\end{aligned}
$$

Konvexe Funktionen: Beweis der Eigenschaften

(Proposition Summe/Skalierung/Maximum/Grenzwert)

Seien $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{X}$, $\lambda \in [0,1]$ und $\boldsymbol{z} := \lambda \boldsymbol{x} + (1-\lambda)\boldsymbol{y}$.

(ii) Skalierung: Für $f = cf_1$ mit $c \geq 0$ gilt

$$\begin{aligned} f(\boldsymbol{z}) &= c\, f_1(\boldsymbol{z}) \\ &\leq c\,[\lambda f_1(\boldsymbol{x}) + (1-\lambda) f_1(\boldsymbol{y})] \\ &= \lambda f(\boldsymbol{x}) + (1-\lambda) f(\boldsymbol{y}). \end{aligned}$$

Die Ungleichung folgt aus der Konvexität von f_1; Multiplikation mit $c \geq 0$ erhält ihre Richtung.

(iv) Grenzwert: Wegen der Konvexität von f_n gilt für jedes n

$$f_n(\boldsymbol{z}) \leq \lambda f_n(\boldsymbol{x}) + (1-\lambda) f_n(\boldsymbol{y}).$$

Da alle beteiligten Folgen konvergieren, können wir den Grenzwert bilden:

$$\begin{aligned} f(\boldsymbol{z}) &= \lim_{n\to\infty} f_n(\boldsymbol{z}) \\ &\leq \lambda \lim_{n\to\infty} f_n(\boldsymbol{x}) + (1-\lambda) \lim_{n\to\infty} f_n(\boldsymbol{y}) \\ &= \lambda f(\boldsymbol{x}) + (1-\lambda) f(\boldsymbol{y}). \end{aligned}$$

↩ zurück: „Konvexe Funktionen: Eigenschaften”

Optimierung I: Grundlagen & Verfahren erster Ordnung

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 26.08.2026

Opener-Quiz

Opener-Quiz

Beim Reinkommen: QR-Code scannen und die 2 Einstiegsfragen beantworten.

Frage 1: Sei $[a, b] \subset \mathbb{R}$ und $f\colon [a, b] \to \mathbb{R}$. Wir wollen $f(x) = 0$ lösen. Welche der folgenden Aussagen ist wahr?

1. Falls f stetig ist, hat $f(x) = 0$ genau eine Lösung.
2. Falls $f(a) < 0$ und $f(b) > 0$, hat $f(x) = 0$ mindestens eine Lösung.
3. Falls f streng monoton ist, hat $f(x) = 0$ höchstens eine Lösung.

Frage 2: Sei $f\colon \mathbb{R}^n \to \mathbb{R}$ konvex und differenzierbar mit $\nabla f(\boldsymbol{x}^\star) = \mathbf{0}^\top$. Was folgt?

1. $\boldsymbol{x}^\star$ ist ein lokales, aber nicht unbedingt globales Minimum.
2. $\boldsymbol{x}^\star$ ist ein globales Minimum.
3. $\boldsymbol{x}^\star$ kann auch ein Sattelpunkt sein.

Verwendete Vorkenntnisse

- *Gradient* $\nabla f(\boldsymbol{x})$ für $f : \mathbb{R}^n \to \mathbb{R}$; $\nabla f(\boldsymbol{x})^\top$ ist die Richtung des steilsten *Anstiegs*
- *Hesse-Matrix* $\boldsymbol{H}_f(\boldsymbol{x})$ als Matrix zweiter Ableitungen: *Krümmung*
- *Konvexität von Mengen und Funktionen*
- **Analysis:**
  - Notwendige Bedingung für Extrema: $f'(x^\star) = 0$
  - Hinreichende Bedingung für Minima: $f'(x^\star) = 0$ **und** $f''(x^\star) > 0$
  - Zwischenwertsatz
- **Lineare Algebra:**
  - *positiv definite* Matrizen
  - *Konditionszahl* einer Matrix

Motivation & Einführung

Einführung

- Ein **nichtlineares Gleichungssystem** für $f\colon \mathbb{R}^n \to \mathbb{R}^m$ hat die Form

$$f(\boldsymbol{x}) = \mathbf{0}$$

  $\to$ Wie finden wir Lösungen $\boldsymbol{x}^\star$?
- Ein **unbeschränktes Optimierungsproblem** für $f\colon \mathbb{R}^n \to \mathbb{R}$ hat die Form

$$\boldsymbol{x}^\star = \arg\min_{\boldsymbol{x}} f(\boldsymbol{x}) \quad \text{bzw.} \quad \boldsymbol{x}^\star = \arg\max_{\boldsymbol{x}} f(\boldsymbol{x}),$$

  $\implies$ Falls f *differenzierbar* ist, muss $\nabla f(\boldsymbol{x}^\star) = \mathbf{0}^\top$ gelten.
- Ein *beschränktes Optimierungsproblem* stellt zusätzliche Bedingungen an Lösungen $\boldsymbol{x}^\star$:

$$\boldsymbol{x}^\star = \arg\min_{\boldsymbol{x} \in S} f(\boldsymbol{x})$$

  mit *Nebenbedingungen* $S \subset \mathbb{R}^n$, z.B:
  - $S = \{\boldsymbol{x} \in \mathbb{R}^n \colon g_i(\boldsymbol{x}) = 0, i = 1, \dots, m\}$ (Gleichungsnebenbedingungen) und/oder
  - $S = \{\boldsymbol{x} \in \mathbb{R}^n \colon h_j(\boldsymbol{x}) \leq 0, j = 1, \dots, p\}$ (Ungleichungsnebenbedingungen) $\to$ Kapitel *Optimierung II*
- Alle diese Probleme werden meist *iterativ* gelöst.

Warum Optimierung?

Statistik und ML bestehen großteils aus Optimierungsproblemen:

- **Maximum-Likelihood-Schätzung** für i.i.d. $y_1, \ldots, y_n$ mit Dichte $f_Y(y \mid \boldsymbol{\theta})$:

$$\widehat{\boldsymbol{\theta}}_{\text{ML}} = \arg\max_{\boldsymbol{\theta}} \left(\sum_{i=1}^{n} \log f_Y(y_i \mid \boldsymbol{\theta}) \right)$$

- **Supervised Learning** mit *Verlustfunktion* $L(y, \hat{y})$ und Vorhersagen $\hat{y}_i := p_\theta(\boldsymbol{x}_i)$ aus Features $\boldsymbol{x}_i$:

$$\widehat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \left(\sum_{i=1}^{n} L(y_i, p_\theta(\boldsymbol{x}_i)) \right)$$

  z.B. **Lineare Regression:** $\widehat{\boldsymbol{\beta}} = \arg\min_\beta \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2$ oder **Neuronale Netze**.

- **Regularisierte Regression (Ridge: $p = 2$, Lasso: $p = 1$):**

$$\widehat{\boldsymbol{\beta}} = \arg\min_{\boldsymbol{\beta}} \left(\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_p^p \right)$$

  kann als *beschränktes Optimierungsproblem* mit $S = \{\boldsymbol{\beta} : \ \|\boldsymbol{\beta}\|_p^p \leq c(\lambda)\}$ formuliert werden:

$$\arg \min_{\boldsymbol{\beta}:\ \|\boldsymbol{\beta}\|_p^p \leq c(\lambda)} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2$$

Nichtlineare Gleichungssysteme

Lösbarkeit

Theorem

Sei $f\colon [a,b] \to \mathbb{R}$ streng monoton, stetig und $f(a) \cdot f(b) \leq 0$.

Dann hat die Gleichung $f(x) = 0$ eine eindeutige Lösung.

Beweis.

- Weil f streng monoton ist, kann es höchstens eine Lösung geben.
- Falls $f(a) \cdot f(b) = 0$, ist $f(a) = 0$ oder $f(b) = 0$.
- $f(a) \cdot f(b) < 0$ bedeutet, dass $f(a)$ und $f(b)$ verschiedenes Vorzeichen haben.
- Nehmen wir o.B.d.A. an, dass $f(a) < 0 < f(b)$.
  Weil f stetig ist, gibt es nach dem Zwischenwertsatz ein $x \in (a,b)$, sodass $f(x) = 0$. ■

Anmerkung:
Diese Bedingungen sind *hinreichend*, aber nicht *notwendig* für Lösbarkeit.

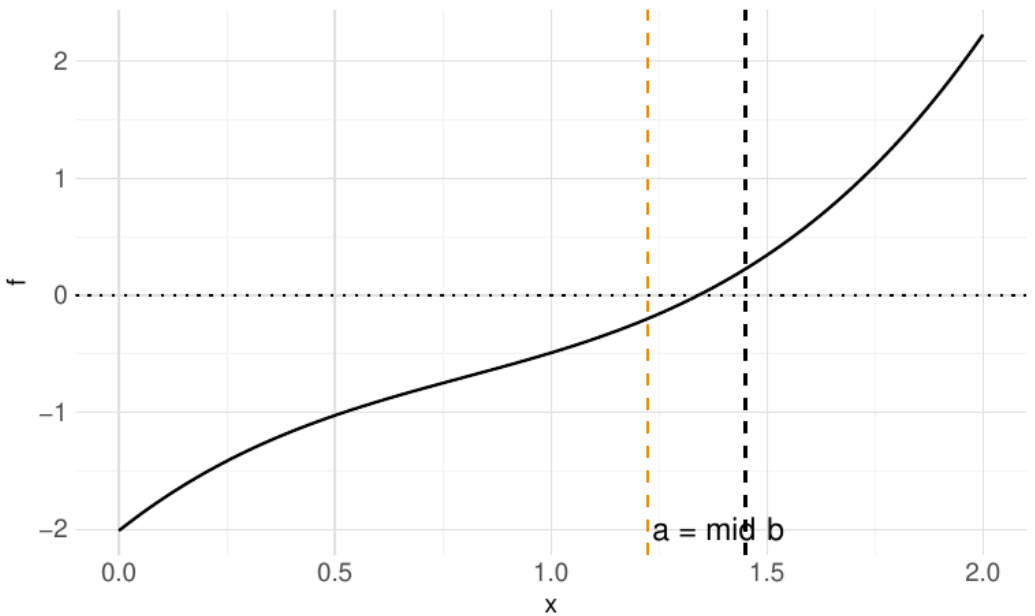
Bisektionsverfahren

Ziel:
Löse $f(x) = 0$, $f: \mathbb{R} \to \mathbb{R}$.

Bisektionsverfahren

- *Inputs:*
  - **Stetige** Funktion f
  - Zwei Punkte a, b mit $f(a) \cdot f(b) < 0$ (also: unterschiedliche Vorzeichen)
  - Ziel-Genauigkeit ϵ: $\left|x^\star - x^{(k)}\right| < \epsilon$

```
bisect <- function(f, a, b, eps) {
  while (b - a > eps) {
    mid <- (a + b) / 2
    if (f(a) * f(mid) <= 0) b <- mid else a <- mid
  }
  return((a + b) / 2)
}
```



Kommentare:

- Kann auch verwendet werden, um die **Umkehrfunktion** $f^{-1}(y)$ auszuwerten: Löse $f(x) - y = 0$.
- Der Algorithmus benötigt $\left\lceil \log_2 \left(\frac{b-a}{\epsilon} \right) \right\rceil$ Schritte.
- Eignet sich nur für **univariate** Funktionen.

Newton-Raphson-Verfahren

Ziel:
Löse $f(x) = 0$ für $f\colon \mathbb{R} \to \mathbb{R}$ differenzierbar.

Idee:
Approximiere f lokal durch eine Tangente und finde deren Nullstelle.

Taylor-Entwicklung um $x^{(k)}$:

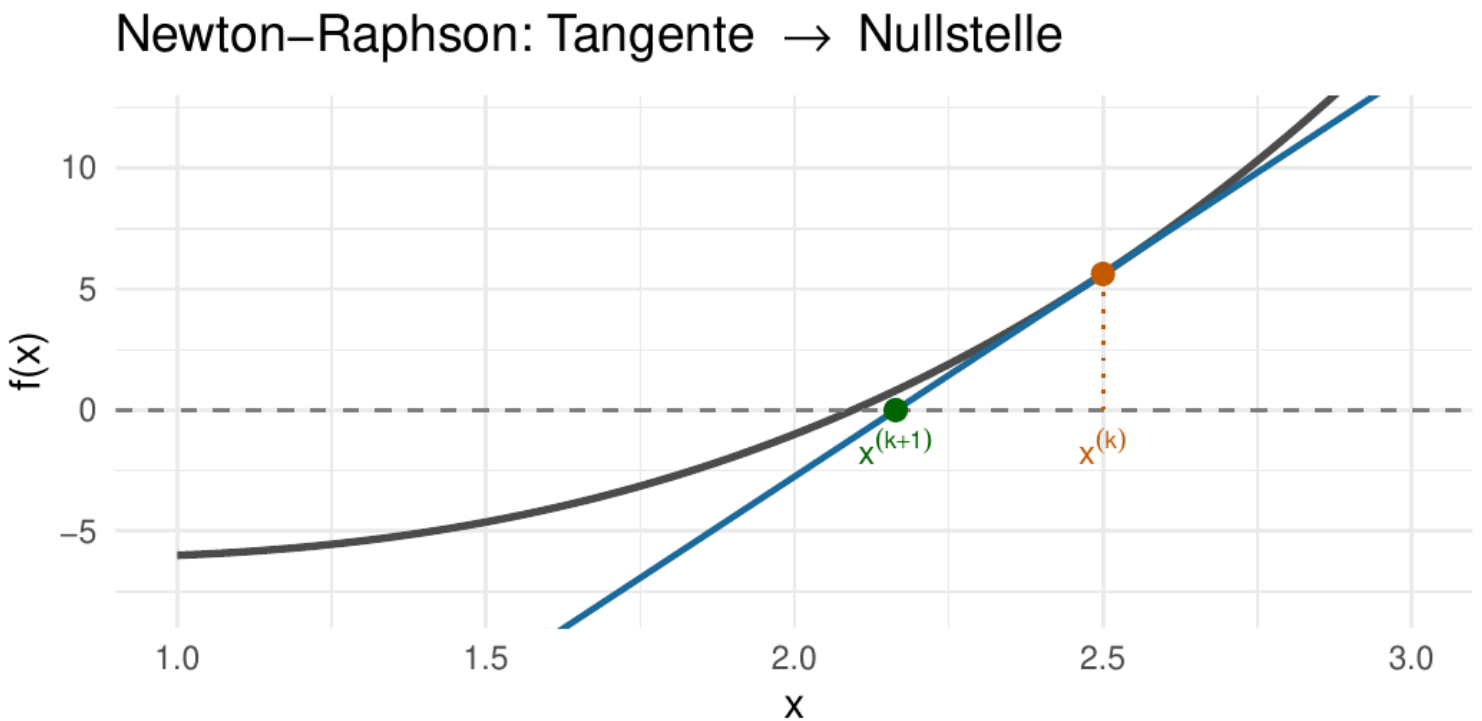
$$f(x) \approx f(x^{(k)}) + f'(x^{(k)})(x - x^{(k)}) \overset{!}{=} 0$$

Newton-Raphson-Iteration

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$

Geometrisch:
Schnittpunkt der Tangente an f im Punkt $x^{(k)}$ mit der x-Achse.

Newton-Raphson: Visualisierung



- **Konvergenz:** Quadratisch nahe einer *einfachen* Nullstelle ($f'(x^\star) \neq 0$): $e_{k+1} \approx C \cdot e_k^2$ für $e_k := \left|x^{(k)} - x^\star\right|$ mit $C = \frac{|f''(x^\star)|}{2|f'(x^\star)|}$
- **Verbindung zur Optimierung:**
  Newton für $\min f(\boldsymbol{x})$ ist Newton-Raphson für $\nabla f(\boldsymbol{x}) = \mathbf{0}^\top$!
  ($\rightarrow$ Kapitel *Optimierung II*)
- Für multivariate Probleme heute: *Gradientenverfahren*.

Das Optimierungsproblem

Das Optimierungsproblem

Ziel:
Finde das Minimum von $f(\boldsymbol{x})$, $f\colon \mathbb{R}^n \to \mathbb{R}$.

$$\boldsymbol{x}^\star = \arg\min_{\boldsymbol{x}\in\mathbb{R}^n} f(\boldsymbol{x})$$

Wichtige Beobachtungen:

- Maximierung ist äquivalent: $\max f(\boldsymbol{x}) = -\min(-f(\boldsymbol{x}))$
- Ist f differenzierbar, muss am Minimum gelten:

$$\nabla f(\boldsymbol{x}^\star) = \mathbf{0}^\top$$

  $\Longrightarrow$ Optimierung $\simeq$ Lösen von Gleichungssystemen!

- **Problem:**
  $\nabla f(\boldsymbol{x}) = \mathbf{0}^\top$ meist nicht analytisch lösbar
  $\Longrightarrow$ *iterative Verfahren* nötig

Konvexe Funktionen in der Optimierung

- Konvexe Funktionen ergeben *einfachere* Optimierungsprobleme:

Proposition

Für jede differenzierbare, konvexe Funktion $f\colon \mathbb{R}^n \to \mathbb{R}$ sind die folgenden Aussagen äquivalent:

i. $\boldsymbol{x}^\star$ ist ein globales Minimum von f.

ii. $\nabla f(\boldsymbol{x}^\star) = \mathbf{0}^\top$.

- Das bedeutet:
  - Jeder kritische Punkt ist ein lokales Minimum.
  - Jedes lokale Minimum ist ein globales Minimum.

Wenn wir ein konvexes Optimierungsproblem lösen, finden wir garantiert ein *globales* Optimum!

(Beweis → Anhang)

Strikt konvexe Funktionen

- Um die **Eindeutigkeit** von Minima zu garantieren, benötigen wir eine stärkere Form von Konvexität.

Def.: Strikte Konvexität

Sei $\mathcal{X}$ konvex. Eine Funktion $f\colon \mathcal{X} \to \mathbb{R}$ heißt **strikt konvex**, falls für alle $\boldsymbol{x}, \boldsymbol{y} \in \mathcal{X}$ mit $\boldsymbol{x} \neq \boldsymbol{y}$ und alle $\lambda \in (0, 1)$ gilt

$$f(\lambda \boldsymbol{x} + (1 - \lambda)\boldsymbol{y}) < \lambda f(\boldsymbol{x}) + (1 - \lambda) f(\boldsymbol{y}).$$

Proposition

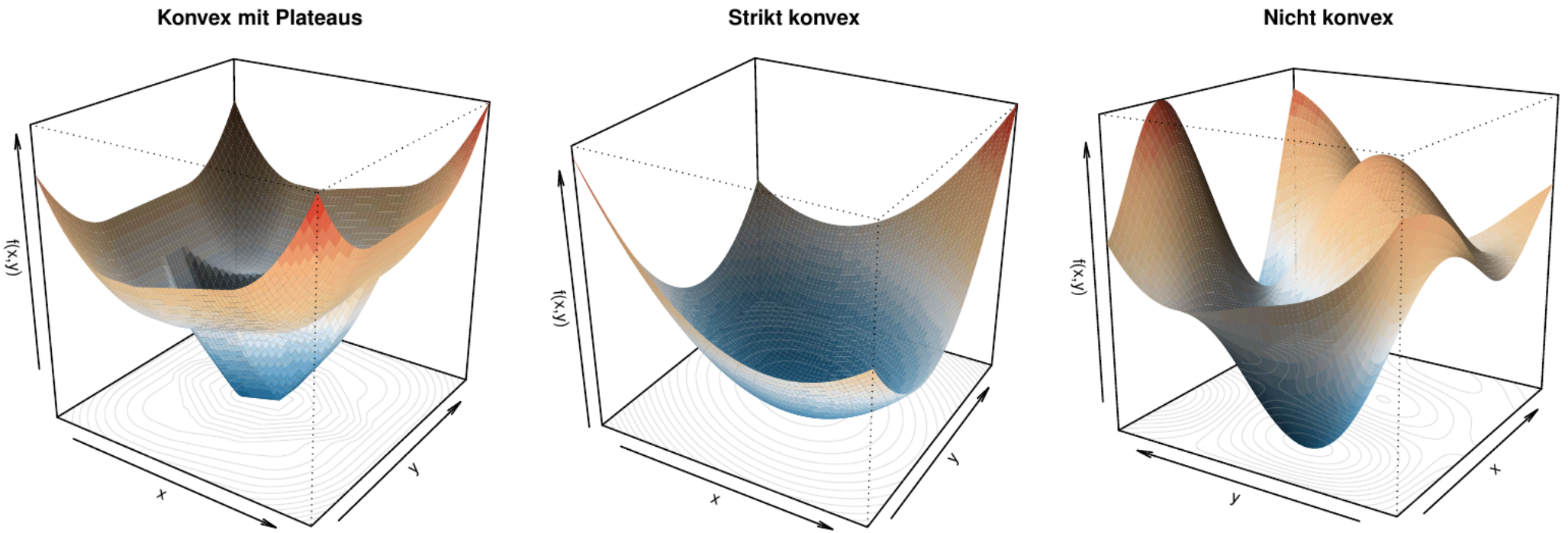
Sei $f\colon \mathbb{R}^n \to \mathbb{R}$ *strikt konvex.*

Dann hat f höchstens ein Minimum.

Wenn wir ein *strikt* konvexes Optimierungsproblem lösen, finden wir garantiert das *eindeutige globale* Optimum!

- Gilt auch mit konvexer zulässiger Menge;
  Spezialfall *Projektionstheorem*: quadrierte Distanz $\|\boldsymbol{x} - \boldsymbol{y}\|^2$ ist strikt konvex in $\boldsymbol{y}$.

Optimierungslandschaften



- Links: konvex mit Plateaus – Minimums*region*, flache Teilstücke
- Mitte: strikt konvex – “Schüssel” mit eindeutigem Minimum.
- Rechts: nicht konvex – mehrere Täler und lokale Minima ohne Garantie für Globalität.

(Non-)Konvexe Optimierung in ML & Statistik

1. **Strikt konvexe Probleme** haben eine einzige *eindeutige* Lösung, z.B.

- KQ-Probleme $\min_\beta \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2$ für $\boldsymbol{X}$ mit vollem Spaltenrang ($\boldsymbol{H} = 2\boldsymbol{X}^\top \boldsymbol{X} \succ 0$)
- genereller: ML-Schätzung für GLMs mit kanonischem Link (konkave Log-Likelihood $\implies$ neg. LL konvex)
- Graph LASSO (regularisierte Kovarianzschätzung)
- Ridge Regression $\min_\beta \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2 + \lambda\|\boldsymbol{\beta}\|_2^2$, strikt konvex für $\lambda > 0$ ($\rightarrow$ Übung) & Elastic Net
- Support Vector Machines: konvexer Hinge-Loss $L(y, \hat{y}) = \max(0, 1 - y\hat{y})$ + strikt konvexe L_2-Regularisierung (strikt konvex nur in $\boldsymbol{w}$ — der Achsenabschnitt b ist i.A. *nicht* eindeutig)

2. **Konvexe Probleme** (nicht *strikt*!) haben manchmal nur eine *Lösungsmenge*, z.B. LASSO, Quantilregression, KQ-Probleme ohne vollen Spaltenrang

3. **Nicht-konvexe Probleme** mit (lokalen Minima und) *Sattelpunkten*
   $\implies$ verschiedene Initialisierungen/Algorithmen $\rightarrow$ (evtl.) verschiedene Lösungen, zum Beispiel:

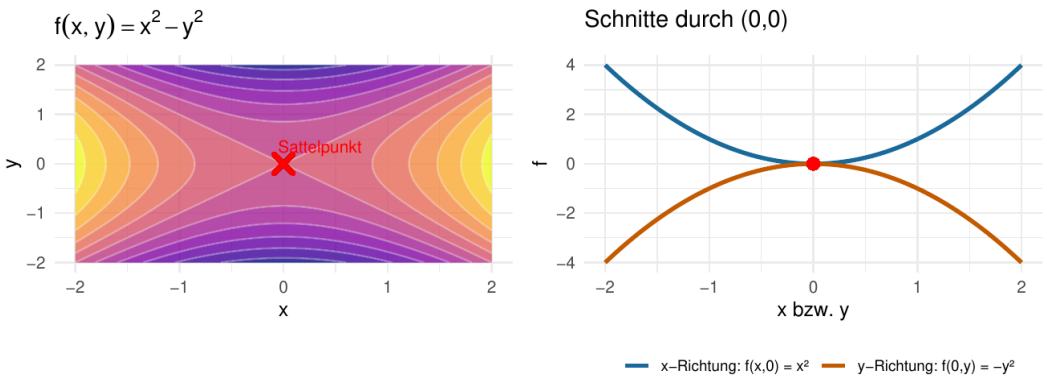
- **Neuronale Netze**
- **Clusteranalysen** (z.B. k-*means*, Mischverteilungsmodelle) - **Bäume** (CART, Random Forest, XGBoost, ...)
- *Latente Variablen* (GLMM, SEM, Faktoranalysen, ...)
- Variablenselektion (L_0, *Best Subset*, ...)
- Embeddings (tSNE, UMAP, `word2vec`, ...)
- *Matrix-Completion*
- **Hyperparameter-Tuning**: meist nichtmal stetig $\rightarrow$ Gitter-, Zufalls- oder Bayes-Optimierung

Sattelpunkte

Bei **nicht-konvexen** Funktionen: $\nabla f(\boldsymbol{x}) = \mathbf{0}^\top$ bedeutet nicht unbedingt Minimum!

Definition: Sattelpunkt

Ein *kritischer Punkt* $\boldsymbol{x}^\star$ mit $\nabla f(\boldsymbol{x}^\star) = \mathbf{0}^\top$ heißt **Sattelpunkt**, wenn $\boldsymbol{H}_f(\boldsymbol{x}^\star)$ *indefinit* ist (sowohl positive als auch negative Eigenwerte).



Beispiel $f(x, y) = x^2 - y^2$:

- $\nabla f = (2x, -2y) = \mathbf{0}^\top$ bei $(0, 0)^\top$ ✓
- $\boldsymbol{H}_f = \begin{pmatrix} 2 & 0 \\ 0 & -2 \end{pmatrix}$: Eigenwerte $2 > 0$, $-2 < 0$
  $\Longrightarrow$ **indefinit**
- $(0, 0)^\top$ ist Minimum in x-Richtung, Maximum in y-Richtung

Konsequenzen:

- In vielen hochdimensionalen, nicht-konvexen Modellen treten Sattelpunkte auf.
- In hohen Dimensionen sind sie i.A. häufiger als lokale Minima.

Optimalitätsbedingungen: Zusammenfassung

Für $f : \mathbb{R}^n \to \mathbb{R}$ zweimal stetig differenzierbar:

Notwendige Bedingung (1. Ordnung)

$\boldsymbol{x}^\star$ lokales Minimum $\implies \nabla f(\boldsymbol{x}^\star) = \mathbf{0}^\top$ ($\boldsymbol{x}^\star$ ist *stationärer Punkt*)

Hinreichende Bedingung (2. Ordnung)

$\nabla f(\boldsymbol{x}^\star) = \mathbf{0}^\top$ und $\boldsymbol{H}_f(\boldsymbol{x}^\star) \succ 0 \implies \boldsymbol{x}^\star$ ist lokales Minimum

Klassifikation *stationärer Punkte* $\left(\nabla f = \mathbf{0}^\top\right)$:

$\boldsymbol{H}_f(\boldsymbol{x}^\star)$	**Punkt ist...**
$\succ 0$ (positiv definit)	*Striktes* lokales Minimum
$\prec 0$ (negativ definit)	*Striktes* lokales Maximum
indefinit	Sattelpunkt
semidefinit, aber nicht definit	Unklar, weitere Analyse nötig

So wichtig:

Für **konvexe Funktionen** ist jedes *lokale* Minimum auch **global** (keine *Sattelpunkte*)!

Iterative Optimierungsverfahren

Taxonomie: Optimierungsverfahren

Klassifikation nach verwendeter **Ableitungsinformation**:

Ordnung	Verwendet	Beispiele
0-te Ordnung	nur $f(\boldsymbol{x})$	*Nelder-Mead*, Gittersuche
1-te Ordnung	$f, \nabla f$	*Gradient Descent*, *Momentum GD*, *SGD*
2-te Ordnung	$f, \nabla f, \boldsymbol{H}_f$	*Newton* ($\rightarrow$ Kapitel *Optimierung II*)

Trade-off:

- 0-te Ordnung: robust, keine Ableitungen nötig, aber langsam
- Höhere Ordnung $\rightarrow$ schnellere Konvergenz, aber teurer pro Schritt
- In der Praxis oft: Superlineare Verfahren (z.B. *Quasi-Newton (BFGS)*, $\rightarrow$ Kapitel *Optimierung II*) als Kompromiss zwischen 1. und 2. Ordnung

Nelder-Mead: Ableitungsfreie Optimierung

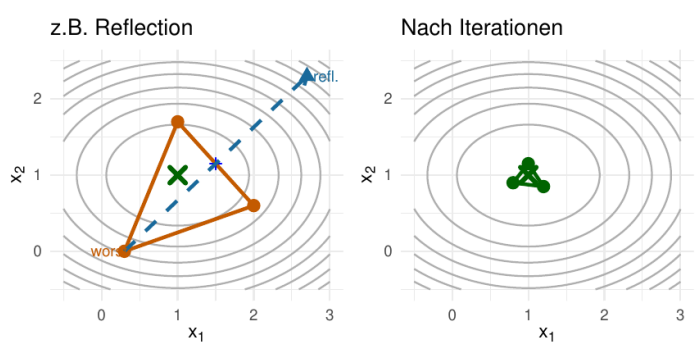
0-te Ordnung: Verwendet nur Funktionswerte $f(\boldsymbol{x})$, keine Ableitungen.

Simplex-Methode

- Arbeitet mit einem *Simplex*: $n+1$ Punkte in $\mathbb{R}^n$ (Dreieck in 2D, Tetraeder in 3D)
- Iterativ: ersetze schlechtesten Punkt durch besseren

Simplex-Operationen: (*Interaktive Demo*)

1. ***Reflection:*** Spiegle schlechtesten Punkt am Schwerpunkt der anderen
2. ***Expansion:*** Gehe weiter in vielversprechende Richtung
3. ***Contraction:*** Ziehe zusammen wenn Reflection nicht hilft
4. ***Shrink:*** Verkleinere gesamten Simplex zum besten Punkt hin



Nelder-Mead: Eigenschaften

Wann `Nelder-Mead` verwenden?

Situation	Geeignet?
Ableitungen nicht verfügbar	✓
Niedrige Dimension ($n \leq 10$)	✓
Nicht-glatte Zielfunktion	✓
Hohe Dimension ($n > 50$)	× (zu langsam)
Präzise Konvergenz nötig	× (i.A. keine Garantien)

Komplexität:
$O(n)$ Funktionsauswertungen pro Iteration, meist *viele* Iterationen nötig.

Fazit:
Robuste Methode für "schwierige" Probleme ohne Ableitungen, aber langsam. Oft als **Startpunktsuche** vor gradientenbasierten Methoden verwendet.

Gradientenabstieg

Gradientenabstieg: Die Kernidee

Der Gradient zeigt bergauf:

$\Longrightarrow$ Der negative Gradient zeigt bergab!

Gradientenabstieg (Gradient Descent / GD)

Die Iterationsvorschrift ist gegeben durch:

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \gamma \nabla f\left(\boldsymbol{x}^{(k)}\right)^\top$$

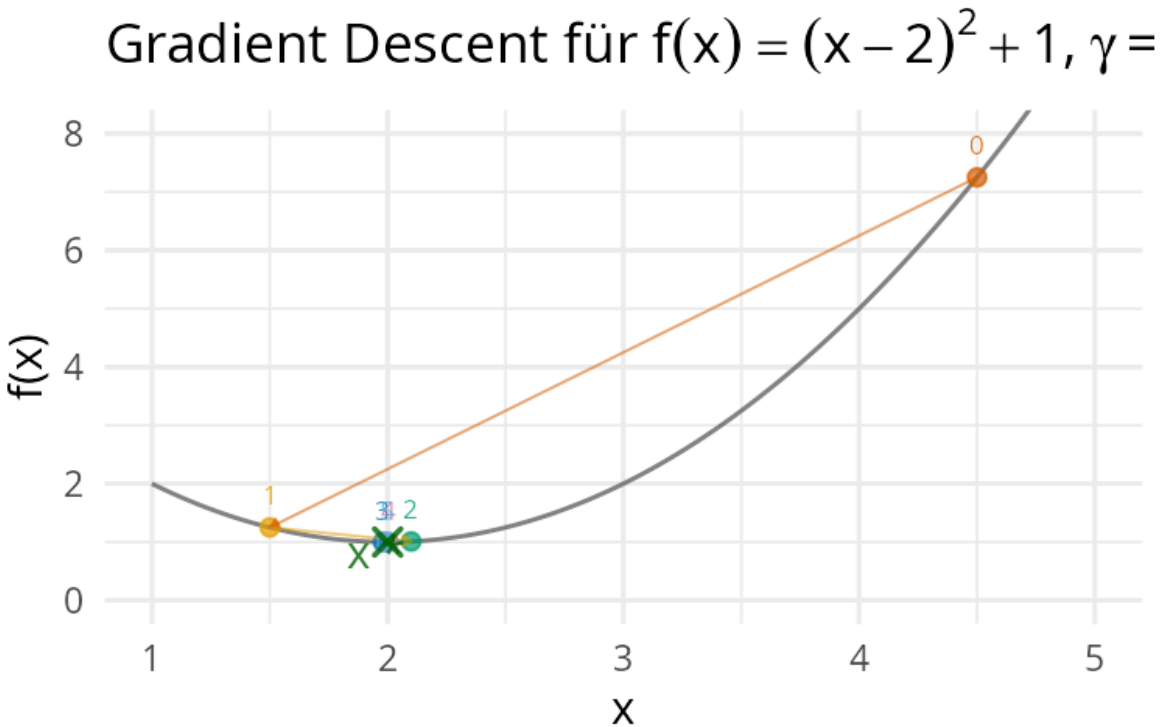
- $\gamma > 0$: *Schrittweite* (*learning rate*)
- $-\nabla f\left(\boldsymbol{x}^{(k)}\right)^\top$: Richtung des steilsten Abstiegs an Stelle $\boldsymbol{x}^{(k)}$

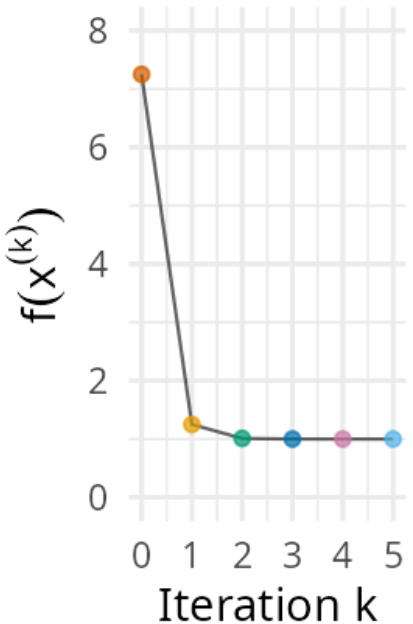
Intuition: Gehe immer in die Richtung, in der es am steilsten bergab geht.

$-\nabla f(\boldsymbol{x})^\top$ ist für *jedes* differenzierbare f eine Abstiegsrichtung, solange $\nabla f(\boldsymbol{x}) \neq \mathbf{0}^\top$ – sie zeigt aber i.A. *nicht direkt* auf das Minimum!
Falls *Konvexität* gilt, ist jeder stationäre Punkt dann bereits *global* optimal.

Bemerkung:

Gradientenabstieg: $f : \mathbb{R} \to \mathbb{R}$





k	x^(k)	f'(x^(k))
0	4.500	5.000
1	1.500	-1.000
2	2.100	0.200
3	1.980	-0.040
4	2.004	0.008
5	1.999	-0.002

Idee:

Starte bei $x^{(0)}$ und verbessere schrittweise:

$$x^{(k+1)} = x^{(k)} - \gamma f'\left(x^{(k)}\right)$$

Beispiel Gradientenabstieg: Schritt für Schritt

Problem:
Minimiere $f(x) = (x-2)^2 + 1$ mit Startwert $x^{(0)} = 4.5$, Schrittweite $\gamma = 0.6$

Ableitung:
$f'(x) = 2x - 4$

Iterationen:

$$
\begin{aligned}
x^{(0)} &= 4.5 \\
x^{(1)} &= x^{(0)} - 0.6 \cdot f'(x^{(0)}) = 4.5 - 0.6 \cdot (2 \cdot 4.5 - 4) = 4.5 - 3 = 1.5 \\
x^{(2)} &= x^{(1)} - 0.6 \cdot f'(x^{(1)}) = 1.5 - 0.6 \cdot (2 \cdot 1.5 - 4) = 1.5 + 0.6 = 2.1 \\
x^{(3)} &= 2.1 - 0.6 \cdot (2 \cdot 2.1 - 4) = 1.98 \\
&\vdots
\end{aligned}
$$

Konvergenz:
$x^{(k)} \to 2$ (analytisches Minimum: $f'(x^\star) = 0 \implies x^\star = 2$)

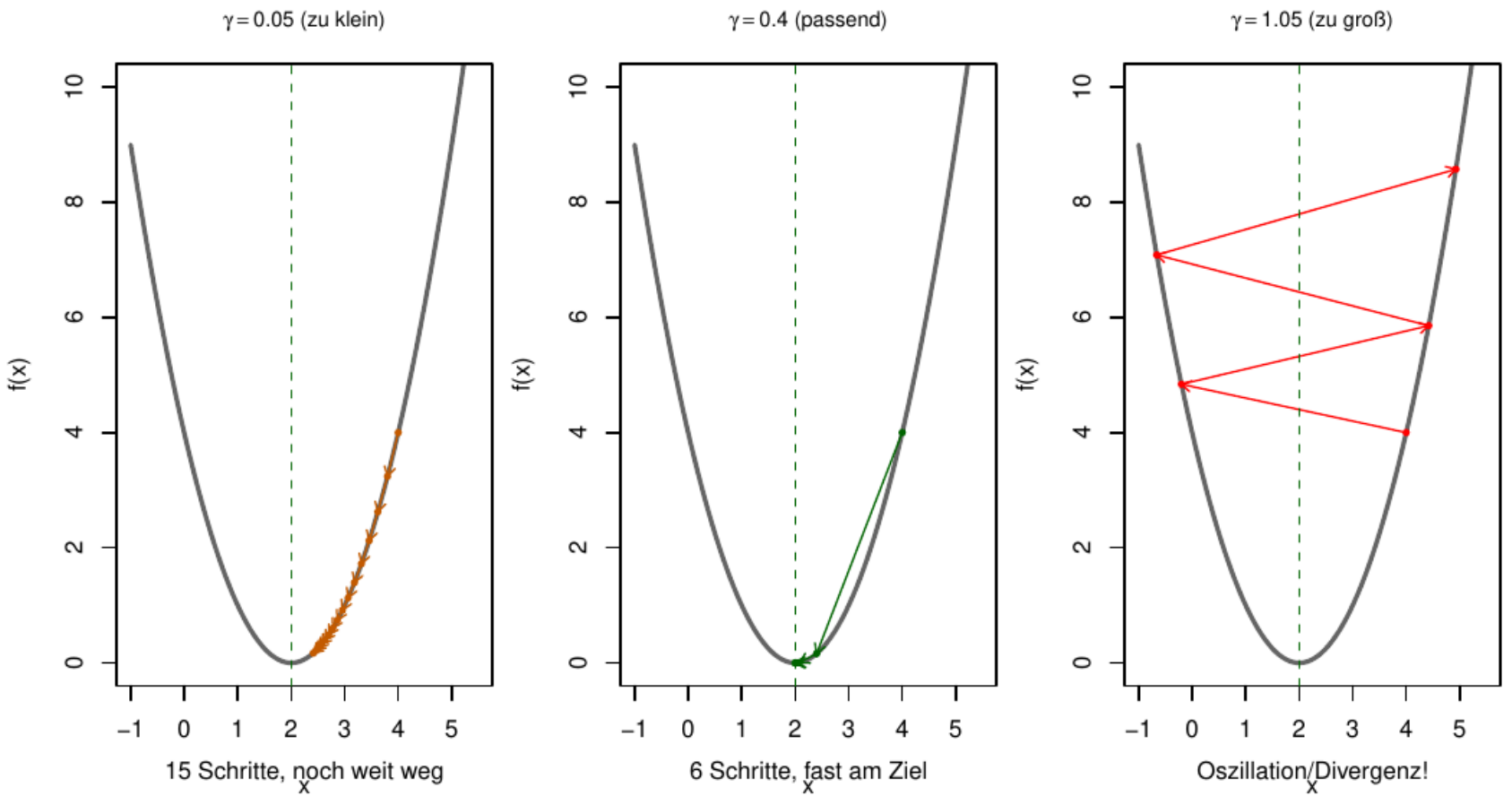
Quiz: Gradient Descent

Sei $f(\boldsymbol{x}) = x_1^2 + x_2^2$ und $\boldsymbol{x}^{(0)} = (4, 3)^\top$ mit Schrittweite $\gamma = 0.25$.

Frage: Wo liegt $\boldsymbol{x}^{(1)}$ nach einem GD-Schritt?

1. $(2, 1.5)^\top$
2. $(0, 0)^\top$
3. $(3, 2)^\top$
4. $(8, 6)^\top$

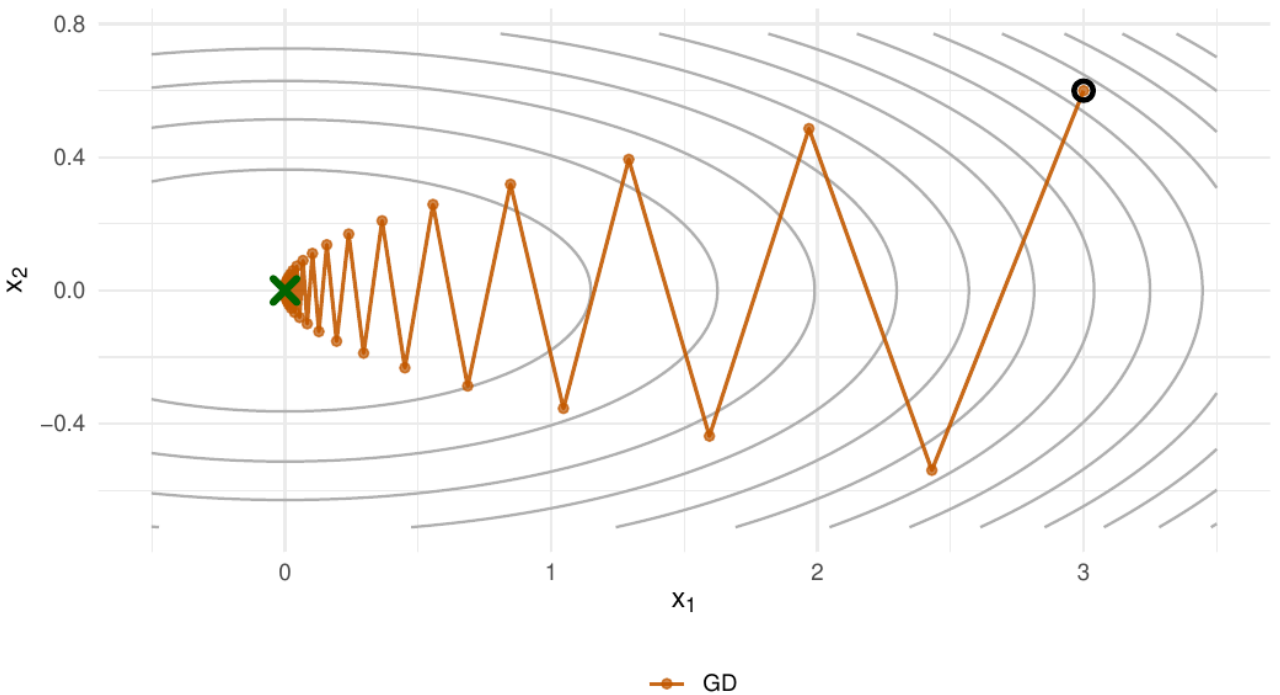
Die Rolle der Schrittweite γ



Dilemma:
γ muss

- klein genug sein für *Konvergenz*,
- aber groß genug für *Effizienz*...

Problem: Zick-Zack bei schlechter Kondition



Bei **hoher Konditionszahl** $\kappa\left(\boldsymbol{H}_f\right)$ oszilliert GD oft stark.

- $\kappa\left(\boldsymbol{H}_f\right)$ ist groß, wenn die Krümmung von f in verschiedenen Richtungen stark unterschiedlich ist (z.B. “Canyons”)
- Wann und wie schnell konvergiert GD also überhaupt? $\rightarrow$ jetzt
- **Interaktiv ausprobieren:** Shiny-App: Gradient Descent

↩ zurück: „Konvergenzrate & Kondition”

Konvergenz: Lipschitz-stetiger Gradient

Def.: Lipschitz-Stetigkeit

Eine Funktion $f : \mathbb{R}^n \to \mathbb{R}^m$ heißt **Lipschitz-stetig** mit Konstante $L > 0$, falls:

$$\|f(\boldsymbol{x}) - f(\boldsymbol{y})\| \leq L\|\boldsymbol{x} - \boldsymbol{y}\| \quad \forall \boldsymbol{x}, \boldsymbol{y}$$

Intuition:
Funktionswerte ändern sich *höchstens* L-mal so schnell wie Funktionsargumente. Je größer L, desto "gezackter" kann f sein $\implies$ kleinere Schrittweiten γ nötig.

- *Lipschitz-stetiger Gradient* ∇f:
  $\|\nabla f(\boldsymbol{x}) - \nabla f(\boldsymbol{y})\| \leq L\|\boldsymbol{x} - \boldsymbol{y}\|$
  $\to$ Steigung von f ändert sich nicht zu abrupt.
- Äquivalent (für zweimal differenzierbares f):
  $\|\boldsymbol{H}_f(\boldsymbol{x})\|_2 \leq L \; \forall \, \boldsymbol{x}$ mit $L = \sup_{\boldsymbol{x}} \max_i |\lambda_i \left(\boldsymbol{H}_f(\boldsymbol{x})\right)|$
  $\to$ Krümmung von f ist beschränkt
- L heißt auch *Lipschitz-Konstante* von f (oder ∇f)

Konvergenzrate bei Konvexität

Konvergenzrate: Gradientenabstieg

Für *konvexes* f mit *Lipschitz-stetigem* Gradient ∇f und *Lipschitz-Konstante* L, das sein Minimum in $\boldsymbol{x}^\star$ annimmt,
gilt für *Schrittweite* $\gamma \leq \frac{1}{L}$ und alle Iterationen $k \geq 1$:

$$f\left(\boldsymbol{x}^{(k)}\right) - f(\boldsymbol{x}^\star) \leq \frac{\|\boldsymbol{x}^{(0)} - \boldsymbol{x}^\star\|^2}{2\gamma k}$$

Interpretation:
Der *Funktionswertabstand* $f\left(\boldsymbol{x}^{(k)}\right) - f(\boldsymbol{x}^\star)$ sinkt mit Rate $O(1/k)$
$\implies$ es braucht $k \sim 1/\varepsilon$ Schritte, bis er $< \varepsilon$ ist (... relativ langsam!)

Achtung:
Über den Abstand $\left\|\boldsymbol{x}^{(k)} - \boldsymbol{x}^\star\right\|$ sagt diese Schranke *nichts*.

Starke Konvexität: lineare Konvergenzrate

Def.: Starke Konvexität

Eine *zweimal differenzierbare* Funktion f heißt **stark konvex** (μ-*strongly convex*) mit Parameter $\mu > 0$, falls:

$$\boldsymbol{H}_f(\boldsymbol{x}) - \mu \boldsymbol{I} \succeq 0 \quad \forall \boldsymbol{x}$$

(ableitungsfrei äquivalent: $f(\boldsymbol{x}) - \frac{\mu}{2} \|\boldsymbol{x}\|^2$ ist konvex)

- $\mu = \inf_{\boldsymbol{x}} \lambda_{\min}(\boldsymbol{H}_f(\boldsymbol{x}))$: minimale (positive!) Krümmung von f

Konvergenzrate: Stark konvexer Gradientenabstieg

Für *stark konvexes* f mit Parameter $\mu > 0$ und *Lipschitz-stetigem* Gradienten mit *Lipschitz-Konstante* L gilt für $\gamma = \frac{1}{L}$:

$$f\left(\boldsymbol{x}^{(k)}\right) - f(\boldsymbol{x}^\star) \leq \rho^k \cdot \left(f\left(\boldsymbol{x}^{(0)}\right) - f\left(\boldsymbol{x}^\star\right)\right), \quad \rho = 1 - \frac{\mu}{L} < 1$$

$\Longrightarrow$ schnellere Konvergenz bei starker Konvexität

Konvergenzrate & Kondition

Verallgemeinerte *Kondition* für Funktionen:

- $\rho = \frac{\kappa_f - 1}{\kappa_f}$ mit Konditionszahl $\kappa_f = \frac{\sup_{\boldsymbol{x}} \lambda_{\max}(\boldsymbol{H}_f(\boldsymbol{x}))}{\inf_{\boldsymbol{x}} \lambda_{\min}(\boldsymbol{H}_f(\boldsymbol{x}))} = \frac{L}{\mu}$
- Großes $\kappa_f \implies \rho \to 1 \implies$ langsame Konvergenz:
  Zick-Zack-Trajektorien durch "Canyons" mit anisotropischer Krümmung
  ($\to$ genau das vorhin gesehene Zick-Zack-Problem!)
- Für quadratische Funktionen $f(\boldsymbol{x}) = \frac{1}{2}\boldsymbol{x}^\top \boldsymbol{A}\boldsymbol{x}$ mit $\boldsymbol{A} \succ 0$ ist $\boldsymbol{H}_f = \boldsymbol{A}$,
  also $\kappa_f = \kappa(\boldsymbol{A})$
  $\to$ selbe *Konditionszahl* wie in der numerischen linearen Algebra!

Abbruchkriterien (Stopping Criteria)

Wann beenden wir die Iteration $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \gamma \nabla f(\boldsymbol{x}^{(k)})^\top$?

Typische Abbruchkriterien

1. *Gradient klein:* $\left\| \nabla f(\boldsymbol{x}^{(k)}) \right\| < \varepsilon_{\text{grad}}$
2. *Funktionswert stagniert:* $\frac{\left| f(\boldsymbol{x}^{(k+1)}) - f(\boldsymbol{x}^{(k)}) \right|}{\left| f(\boldsymbol{x}^{(k)}) \right|} < \varepsilon_f$
3. *Iterationen ausgeschöpft:* $k > k_{\max}$

- Üblicherweise kombiniert man mehrere Kriterien
- Typische Werte: $\varepsilon_{\text{grad}} \approx 10^{-6}$ bis 10^{-8}, $k_{\max} \approx 1000$ bis 10000

Achtung:

Kriterien können bei nicht stark konvexen oder schlecht konditionierten Problemen versagen — $\|\nabla f\|$ oder $\left| f^{(k+1)} - f^{(k)} \right|$ kann beliebig klein werden, obwohl $\boldsymbol{x}^{(k)}$ noch weit vom Optimum entfernt ist!

Line Search: Adaptive Schrittweite

Problem:
Eine feste Schrittweite γ ist oft suboptimal.

Idee:
Wähle $\gamma^{(k)}$ in jedem Schritt so, dass f "genug" sinkt.

Backtracking Line Search (*Armijo*)

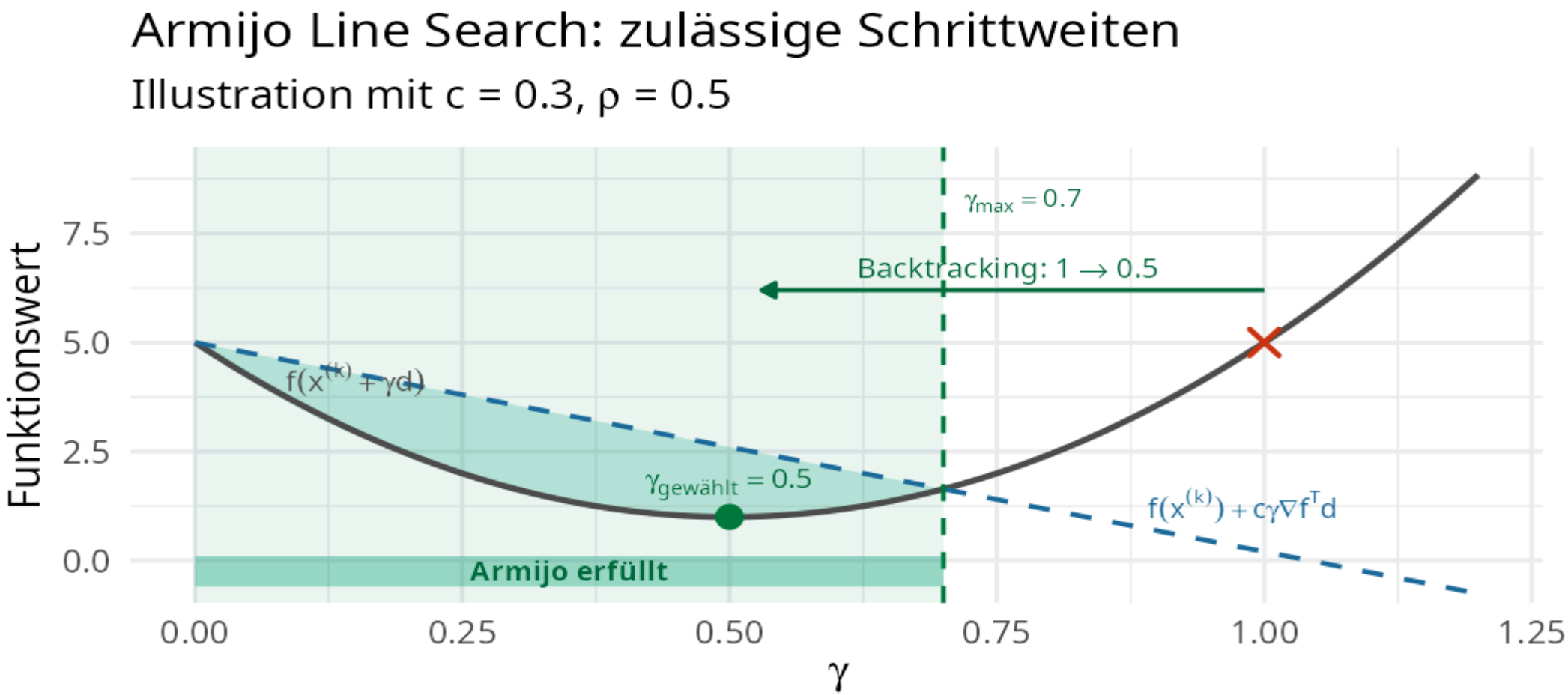
Gegeben: Richtung $\boldsymbol{d} = -\nabla f(\boldsymbol{x}^{(k)})^\top$, Parameter $c \in (0,1)$, $\rho \in (0,1)$

1. Starte mit $\gamma = 1$
2. `while` $f\left(\boldsymbol{x}^{(k)} + \gamma \boldsymbol{d}\right) > f\left(\boldsymbol{x}^{(k)}\right) + c\gamma \nabla f\left(\boldsymbol{x}^{(k)}\right)\boldsymbol{d}$:
   - $\gamma \leftarrow \rho\gamma$ (verkleinere Schrittweite)
3. `return` γ

Armijo-Bedingung:
Fordert "genügend Abstieg" — Funktionswert muss proportional zum erwarteten Abstieg sinken.

Achtung:
ρ ist hier der *Verkleinerungsfaktor* der Schrittweite — nicht die Konvergenzrate $\rho = 1 - \mu/L$ von vorhin.

Line Search: Visualisierung



Typische Parameter: $c = 10^{-4}$, $\rho = 0.5$

Quasi-Newton: *BFGS* verwendet automatisch *Line Search* ($\rightarrow$ Kapitel *Optimierung II*)

Zusammenfassung

Self-Check: Gradientenabstieg

1. Sei $f(\boldsymbol{x}) = x_1^2 + 4x_2^2$ und $\boldsymbol{x}^{(0)} = (2, 1)^\top$ mit Schrittweite $\gamma = 0.25$. Wo liegt $\boldsymbol{x}^{(1)}$?

1. $(1, -1)^\top$
2. $(1.5, 0)^\top$
3. $(1, 0)^\top$
4. $(0, 0)^\top$

2. Welche Aussagen sind wahr?

1. GD konvergiert für jede Schrittweite $\gamma > 0$ gegen ein lokales Minimum.
2. Für konvexes f mit L-Lipschitz-Gradient und $\gamma \leq 1/L$ gilt: Fehler $= O(1/k)$.
3. Für stark konvexes f konvergiert GD linear, d.h. Fehler $= O(\rho^k)$ mit $\rho < 1$.
4. Ist $\left\|\nabla f(\boldsymbol{x}^{(k)})\right\|$ klein, dann ist $\boldsymbol{x}^{(k)}$ nahe am Minimum.

Zusammenfassung

Methode	Wann verwenden?
Bisektion	univariate Nullstellen; braucht nur Stetigkeit + Vorzeichenwechsel
Newton-Raphson	Nullstellen glatter Funktionen; quadratische Konvergenz
Nelder-Mead	Optimierung ohne Ableitungen, niedrige Dimension ($n \leq 10$)
Gradient Descent	Optimierung großer differenzierbarer Probleme; einfache Implementierung

- **Konvexität** $\implies$ jedes lokale Minimum ist global;
  strikte Konvexität $\implies$ *höchstens ein* Minimum (also eindeutig, falls existent)
  - **Konvergenz von GD:**
  - Rate $O(1/k)$ (konvex) bzw. $O(\rho^k)$ (stark konvex)
  - $\rho = 1 - \frac{\mu}{L}$ bei $\gamma = \frac{1}{L}$
  - alternativ Schrittweite adaptiv mit *Line Search*

Im Kapitel Optimierung II:
Newton-Verfahren (“Newton-Raphson für $\nabla f = \mathbf{0}^\top$”), Quasi-Newton (BFGS), Momentum & SGD, Optimierung unter Nebenbedingungen (Lagrange/KKT)

Anhang

Fixpunktiteration erster Ordnung

(in der Vorlesung nur als Bemerkung: GD ist die Fixpunktiteration für $\nabla f(\boldsymbol{x})^\top = \mathbf{0}$)

Ziel: Löse $f(\boldsymbol{x}) = \mathbf{0}$, $f\colon \mathbb{R}^n \to \mathbb{R}^n$ monoton steigend (d.h. $(f(\boldsymbol{x}) - f(\boldsymbol{y}))^\top (\boldsymbol{x} - \boldsymbol{y}) \geq 0 \; \forall\, \boldsymbol{x}, \boldsymbol{y}$).

- Wir definieren die Iteration

$$\boldsymbol{x}^{(k)} = \boldsymbol{x}^{(k-1)} - \gamma f(\boldsymbol{x}^{(k-1)}).$$

- Ist f differenzierbar, erhalten wir aus einer Taylorapproximation 1. Ordnung um $\boldsymbol{x}^\star$:

$$\begin{aligned}
\left\| \boldsymbol{x}^{(k)} - \boldsymbol{x}^\star \right\| &\approx \left\| \boldsymbol{x}^{(k-1)} - \boldsymbol{x}^\star - \underbrace{\gamma f(\boldsymbol{x}^\star)}_{=\mathbf{0}} - \gamma \boldsymbol{J}_f(\boldsymbol{x}^\star) \left(\boldsymbol{x}^{(k-1)} - \boldsymbol{x}^\star \right) \right\| \\
&= \left\| \left(\boldsymbol{I}_n - \gamma \boldsymbol{J}_f(\boldsymbol{x}^\star) \right) \left(\boldsymbol{x}^{(k-1)} - \boldsymbol{x}^\star \right) \right\| \\
&\leq \left\| \boldsymbol{I}_n - \gamma \boldsymbol{J}_f(\boldsymbol{x}^\star) \right\| \left\| \boldsymbol{x}^{(k-1)} - \boldsymbol{x}^\star \right\|.
\end{aligned}$$

- Falls $\rho := \left\| \boldsymbol{I}_n - \gamma \boldsymbol{J}_f(\boldsymbol{x}^\star) \right\| < 1$, gilt *lokal* (für Startwerte nahe genug an $\boldsymbol{x}^\star$)

$$\left\| \boldsymbol{x}^{(k)} - \boldsymbol{x}^\star \right\| = O(\rho^k)$$

- Exakt mit diesem ρ und *global* gilt das nur für affines f — dort verschwindet der oben weggelassene Restterm.

↩ zurück: „Gradientenabstieg: Die Kernidee"

Fixpunktiteration erster Ordnung

Vorteile:

- Simples Verfahren auch für multivariate Funktionen
- Schnelle Konvergenz für gutartige Funktionen

Nachteile:

- Schrittweite γ muss gut gewählt sein:
  - zu klein: langsame Konvergenz
  - zu groß: gar keine Konvergenz

In der Praxis:

- Folge $\gamma_k \to 0$.
- Falls f fallend, betrachte $-f$.

Beweis: Globale Optimalität bei Konvexität

Proposition

Für jede differenzierbare, konvexe Funktion $f\colon \mathbb{R}^n \to \mathbb{R}$ sind äquivalent: (i) $\boldsymbol{x}^\star$ ist ein globales Minimum von f. (ii) $\nabla f(\boldsymbol{x}^\star) = \mathbf{0}^\top$.

Beweis.

- $(i) \implies (ii)$: Jedes globale Minimum ist ein lokales Minimum. Dort muss $\nabla f(\boldsymbol{x}^\star) = \mathbf{0}^\top$ gelten. ■
- $(ii) \implies (i)$: Sei $\boldsymbol{x}^\star$ ein Punkt mit $\nabla f(\boldsymbol{x}^\star) = \mathbf{0}^\top$. Dann gilt für alle $\boldsymbol{y} \in \mathbb{R}^n$

$$f(\boldsymbol{y}) \geq f(\boldsymbol{x}^\star) + \nabla f(\boldsymbol{x}^\star)(\boldsymbol{y} - \boldsymbol{x}^\star) = f(\boldsymbol{x}^\star).$$

  Also ist $\boldsymbol{x}^\star$ ein globales Minimum. ■

↩ zurück: „Konvexe Funktionen in der Optimierung"

Optimierung II: Verfahren 2. Ordnung, Stochastischer GD, Nebenbedingungen

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 26.08.2026

Verwendete Vorkenntnisse

- **Aus Optimierung I:**
  - *Gradient Descent* (Gradientenabstieg, GD): Iterationsvorschrift, Schrittweite, Konvergenzraten
  - *Newton-Raphson-Verfahren* für nichtlineare Gleichungen $f(x) = 0$
  - Optimalitätsbedingungen: $\nabla f(\boldsymbol{x}^\star) = \mathbf{0}^\top$, Definitheit von $\boldsymbol{H}_f(\boldsymbol{x}^\star)$
  - *Lipschitz-Konstante* und *Konditionszahl* $\kappa_f = L/\mu$
- **Aus Konvexität:**
  - *Konvexe Mengen und Funktionen*; lokale = globale Minima
- **Lineare Algebra:** *positiv definite* Matrizen, Eigenwerte
- **Analysis:** Taylor-Approximation 1. und 2. Ordnung

Opener-Quiz: Was wissen Sie noch?

Frage 1: Wie lautet die Iterationsvorschrift des *Gradientenabstiegs*?

1. $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} + \gamma \nabla f\left(\boldsymbol{x}^{(k)}\right)^\top$
2. $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \gamma \nabla f\left(\boldsymbol{x}^{(k)}\right)^\top$
3. $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \gamma f\left(\boldsymbol{x}^{(k)}\right)$
4. $\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \boldsymbol{H}_f\left(\boldsymbol{x}^{(k)}\right)^{-1} \nabla f\left(\boldsymbol{x}^{(k)}\right)^\top$

Frage 2: Sei $\nabla f(\boldsymbol{x}^\star) = \boldsymbol{0}^\top$ und $\boldsymbol{H}_f(\boldsymbol{x}^\star)$ *indefinit*. Dann ist $\boldsymbol{x}^\star$ ...

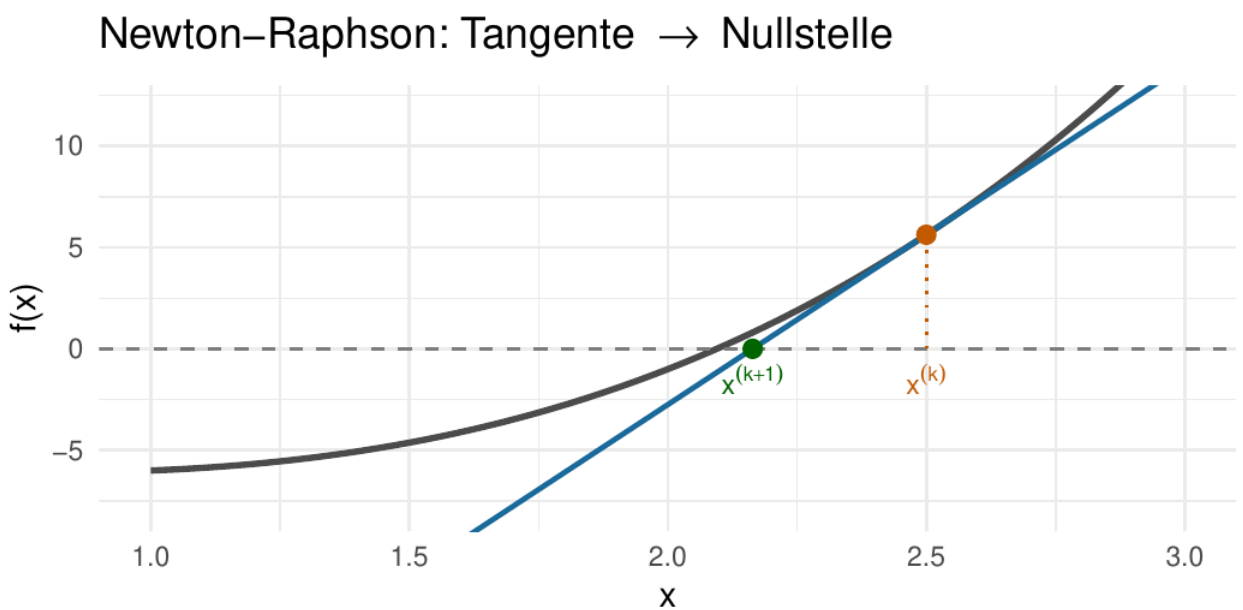
1. ein lokales Minimum
2. ein lokales Maximum
3. ein Sattelpunkt
4. ein globales Minimum

Brücke: Von Newton-Raphson zur Newton-Optimierung

Letzte Woche:

Newton-Raphson löst $f(x) = 0$ über die Nullstelle der Tangente:

$$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$$



Heute:

Am Minimum gilt $\nabla f(\boldsymbol{x}^\star) = \mathbf{0}^\top$.

$\Longrightarrow$ **Newton-Optimierung = Newton-Raphson, angewandt auf die Gleichung $\nabla f(\boldsymbol{x}) = \mathbf{0}^\top$!**

Newton-Verfahren

Newton-Verfahren: Idee & Schritt

Problem bei GD:
Berücksichtigt nur die *Steigung*, nicht die *Krümmung*
$\rightarrow$ approximiert f lokal durch Tangente (Taylor 1. Ordnung)

Newton-Idee:
Approximiere f lokal durch eine *Parabel* (Taylor 2. Ordnung):

$$f(\boldsymbol{x}+\boldsymbol{h}) \approx f(\boldsymbol{x}) + \nabla f(\boldsymbol{x})\boldsymbol{h} + \frac{1}{2}\boldsymbol{h}^\top \boldsymbol{H}_f(\boldsymbol{x})\boldsymbol{h}$$

$\rightarrow$ Minimiere diese Approximation bzgl. $\boldsymbol{h}$:

$$\nabla_{\boldsymbol{h}} [\dots] = \nabla f(\boldsymbol{x}) + \boldsymbol{h}^\top \boldsymbol{H}_f(\boldsymbol{x}) \overset{!}{=} \mathbf{0}^\top \quad \iff \quad \boldsymbol{h} = -\boldsymbol{H}_f(\boldsymbol{x})^{-1} \nabla f(\boldsymbol{x})^\top$$

Newton-Schritt

In Iteration k wird der nächste Punkt $\boldsymbol{x}^{(k+1)}$ berechnet als:

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \boldsymbol{H}_f\left(\boldsymbol{x}^{(k)}\right)^{-1} \nabla f\left(\boldsymbol{x}^{(k)}\right)^\top$$

- **Voraussetzung:** $\boldsymbol{H}_f\left(\boldsymbol{x}^{(k)}\right)$ muss invertierbar sein (Konvexität liefert nur $\boldsymbol{H}_f \succeq 0$!)
- Abbruchkriterien wie bei *Gradient Descent*
- In der Praxis löst man das LGS, statt $\boldsymbol{H}_f^{-1}$ explizit zu berechnen
  — **Invertiere niemals eine Matrix!**

Verbindung: Newton-Raphson ↔ Newton-Optimierung

Nullstellensuche	**Optimierung**
Löse $f(x) = 0$	Löse $\nabla f(\boldsymbol{x}) = \mathbf{0}^\top$
$x^{(k+1)} = x^{(k)} - \frac{f(x^{(k)})}{f'(x^{(k)})}$	$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \boldsymbol{H}_f^{-1} \nabla f(\boldsymbol{x}^{(k)})^\top$

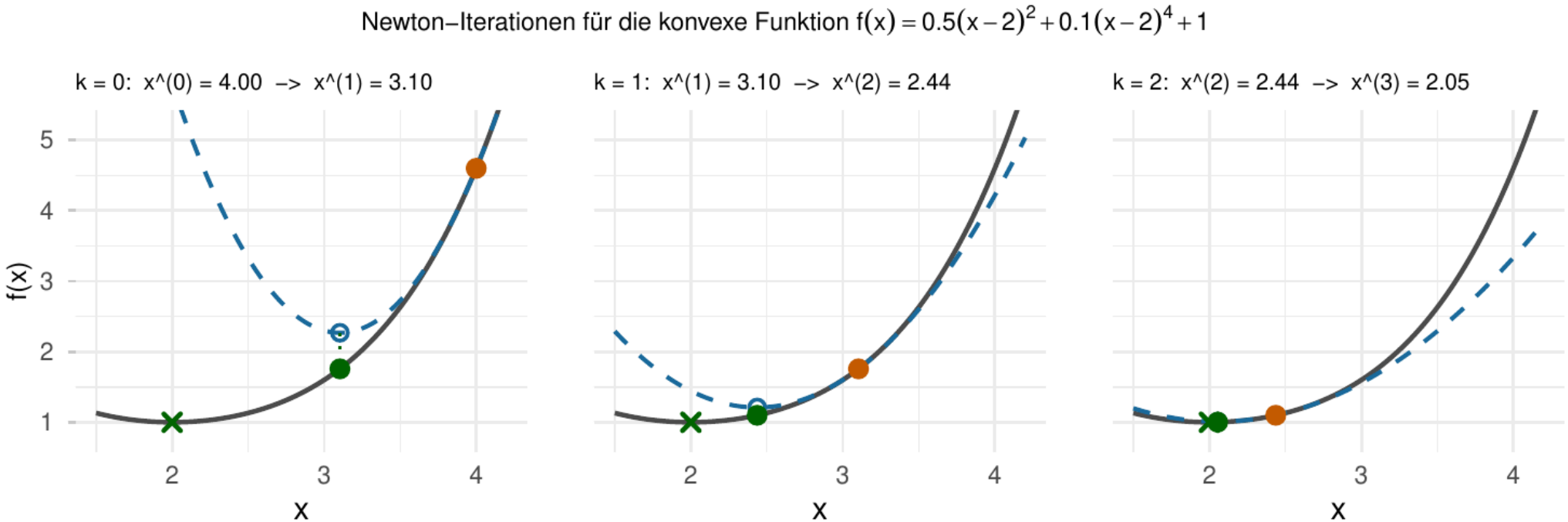
$\Longrightarrow$

Newton für Optimierung $\simeq$ Newton-Raphson für $g(\boldsymbol{x}) := \nabla f(\boldsymbol{x})^\top = \mathbf{0}$

- Jacobi-Matrix von g: $\boldsymbol{J}_g = \boldsymbol{H}_f$
- Newton-Raphson:

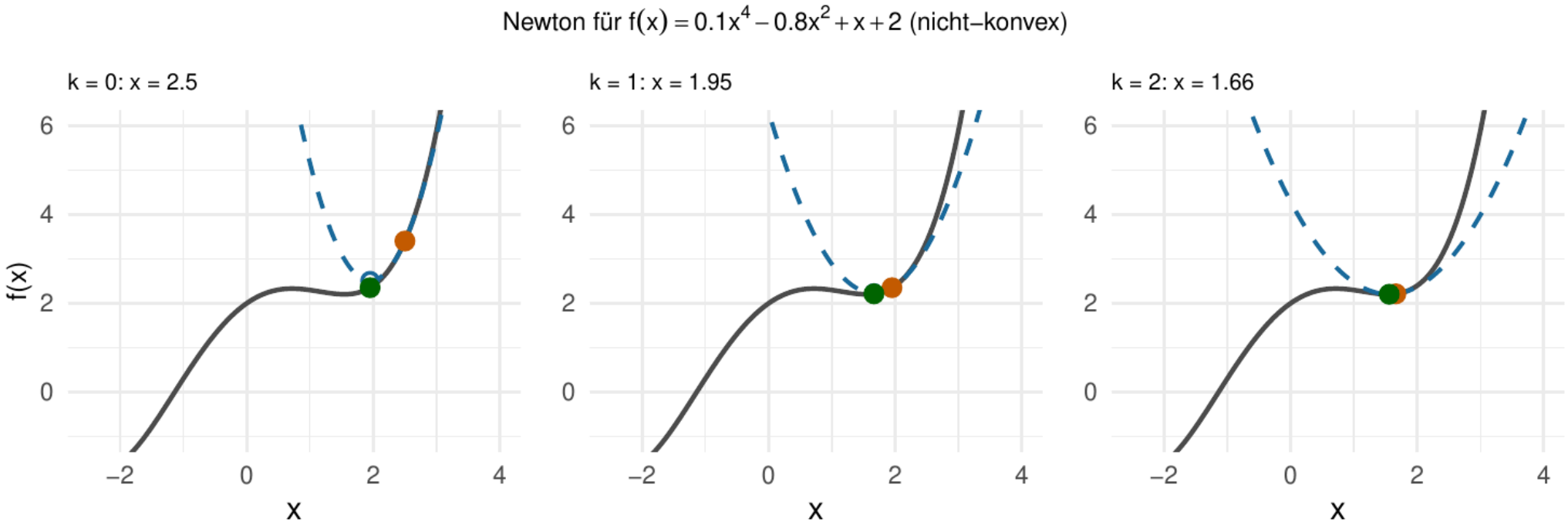
$$\begin{aligned} \boldsymbol{x}^{(k+1)} &= \boldsymbol{x}^{(k)} - \boldsymbol{J}_g^{-1}(\boldsymbol{x}^{(k)})\, g(\boldsymbol{x}^{(k)}) \\ &= \boldsymbol{x}^{(k)} - \boldsymbol{H}_f^{-1}(\boldsymbol{x}^{(k)})\, \nabla f(\boldsymbol{x}^{(k)})^\top \quad \checkmark \end{aligned}$$

Newton-Verfahren: Geometrische Intuition



orange: aktueller Punkt
blau gestrichelt: quadratische Taylor-Approximation T_2
grün: nächster Iterationspunkt.

Newton für nicht-konvexe Funktionen



Bei nicht-konvexen Funktionen kann Newton auch zu beliebigen **lokalen** oder globalen kritischen Punkten — also auch Sattelpunkten oder *Maxima* — konvergieren.

Quasi-Newton-Verfahren

Quasi-Newton-Verfahren

Problem:
Hesse-Matrix $\boldsymbol{H}_f(\boldsymbol{x})$ ist teuer zu berechnen, zu speichern ($O(n^2)$!) und zu invertieren ($O(n^3)$!).

Idee:
Approximiere $\boldsymbol{H}_f^{-1}$ iterativ aus Gradienten-Informationen!

Quasi-Newton-Update

Der nächste Punkt $\boldsymbol{x}^{(k+1)}$ wird berechnet als:

$$\boldsymbol{x}^{(k+1)} = \boldsymbol{x}^{(k)} - \gamma_k \boldsymbol{B}_k \nabla f\left(\boldsymbol{x}^{(k)}\right)^\top,$$

wobei $\boldsymbol{B}_k \approx \boldsymbol{H}_f^{-1}(\boldsymbol{x}^{(k)})$ aus vorherigen Schritten geschätzt wird und γ_k aus einer *Line Search* stammt.

- Vermeidet Invertieren von $\boldsymbol{H}_f$
- Oft *Low-Rank-Updates* von $\boldsymbol{B}_k$ (BFGS: Rang 2) oder Diagonal-Approximationen
- Schrittweite γ_k kommt automatisch aus einer *Line Search* (ohne sie kann schon der erste Schritt aus $\boldsymbol{B}_0 = \boldsymbol{I}$ den Funktionswert erhöhen)

BFGS-Verfahren

BFGS = **B**royden–**F**letcher–**G**oldfarb–**S**hanno

Aktualisiert $\boldsymbol{B}_k$ so, dass die *Sekantenbedingung* $f''(x) \approx \frac{f'(x_{k+1}) - f'(x_k)}{x_{k+1} - x_k}$ erfüllt ist:

$$\boldsymbol{B}_{k+1} \underbrace{\left(\nabla f(\boldsymbol{x}^{(k+1)}) - \nabla f(\boldsymbol{x}^{(k)})\right)^\top}_{\boldsymbol{y}_k} = \underbrace{\boldsymbol{x}^{(k+1)} - \boldsymbol{x}^{(k)}}_{\boldsymbol{s}_k}$$

- **Achtung:** wegen $\boldsymbol{B}_k \approx \boldsymbol{H}_f^{-1}$ ist die *Matrix*gleichung der **Kehrwert** des Differenzenquotienten (in 1D: $b_{k+1} = s_k / y_k$ statt y_k / s_k)
- Setze $\boldsymbol{B}_0 = \boldsymbol{I}$
- BFGS-Update mit $\rho_k = \frac{1}{\boldsymbol{y}_k^\top \boldsymbol{s}_k}$ $\left(\frac{\boldsymbol{y}_k^\top \boldsymbol{s}_k}{\boldsymbol{s}_k^\top \boldsymbol{s}_k} \approx \text{ Krümmung in Richtung } \boldsymbol{s}_k\right)$:

$$\boldsymbol{B}_{k+1} = \left(\boldsymbol{I} - \rho_k \boldsymbol{s}_k \boldsymbol{y}_k^\top\right) \boldsymbol{B}_k \left(\boldsymbol{I} - \rho_k \boldsymbol{y}_k \boldsymbol{s}_k^\top\right) + \rho_k \boldsymbol{s}_k \boldsymbol{s}_k^\top$$

- Komplexität: $O(n^2)$ pro Schritt statt $O(n^3)$ bei Newton
- Konvergenz schneller als GD, fast so gut wie Newton (superlinear)
- *L-BFGS*: *Limited-memory* Variante für großes n — speichert nur letzte m Vektoren für Speicherkomplexität $O(mn)$ statt $O(n^2)$

Methodenvergleich: Zusammenfassung

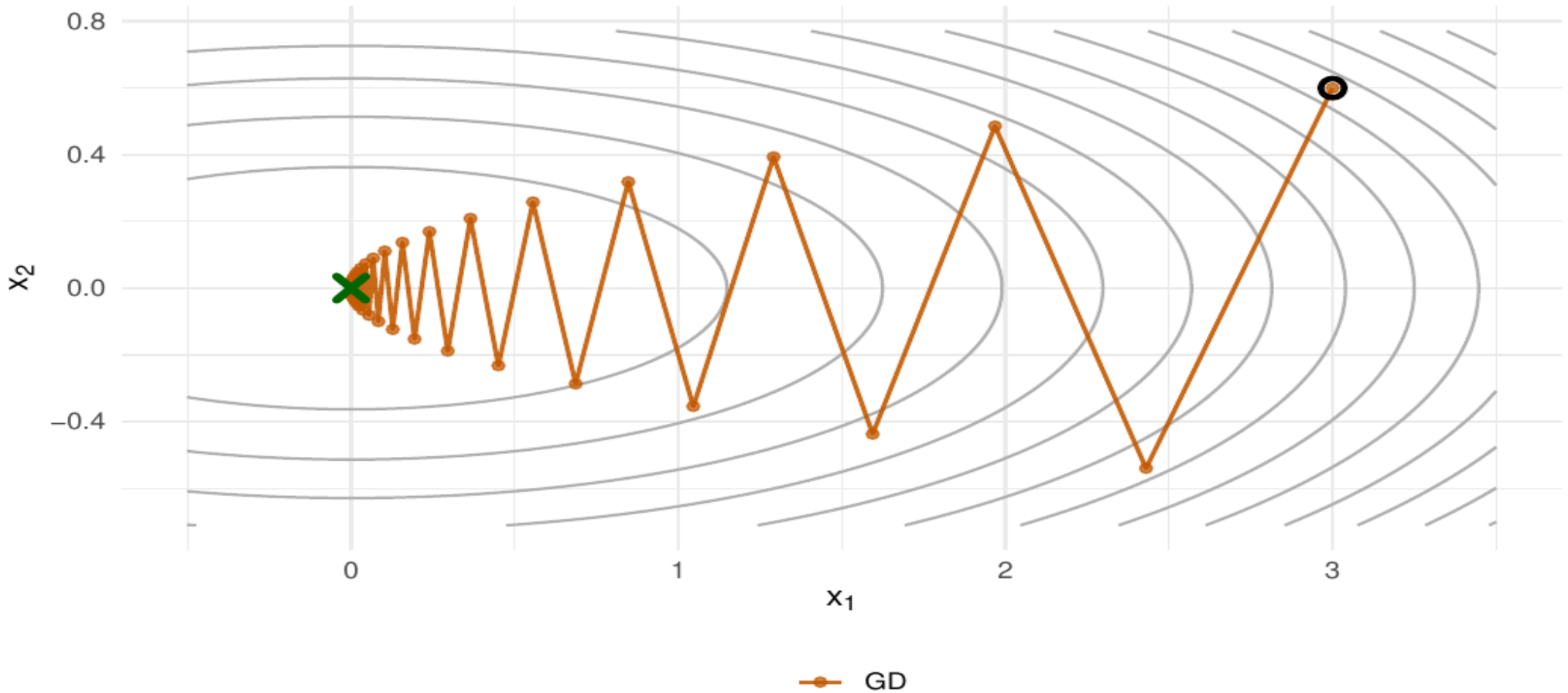
Eigenschaft	Nelder-Mead	Gradientenabstieg	Quasi-Newton	Newton
Verfahren	0. Ordnung	1. Ordnung	“1.5te” Ordnung	2. Ordnung
Benötigt	nur f	$f, \nabla f$	$f, \nabla f$	$f, \nabla f, \boldsymbol{H}_f$
Komplexität/Schritt	$O(n)$	$O(n)$	$O(n^2)$	$O(n^3)$
Konvergenz	Keine Garantie	Linear$^\times$	Superlinear$^\times$	Quadratisch$^\times$
Schrittweite	Autom.	Kritisch (γ)	*line search*	Autom.
Skalierbarkeit	Schlecht ($n \leq 10$)	Sehr gut	Gut	Schlecht

$\times$: **Konvergenzaussagen gelten lokal bzw. für glatte, stark konvexe Zielfunktionen und geeignete Schrittweiten/Line Search.**

- GD/Newton/Quasi-Newton sind *Fixpunktiterationen* für $\nabla f(\boldsymbol{x}) = \boldsymbol{0}$
- **Nelder-Mead**: Default in R's `optim()`; nützlich, wenn keine Ableitungen verfügbar
- **GD**: Grundlage moderner NN-Optimierer
- **Quasi-Newton (BFGS)**: in R: `optim(..., method = "BFGS")`, guter Kompromiss
- **Newton**: MaxLik-Inferenz (IWLS für GLMs), XGBoost, ...

Momentum & Stochastic Gradient Descent

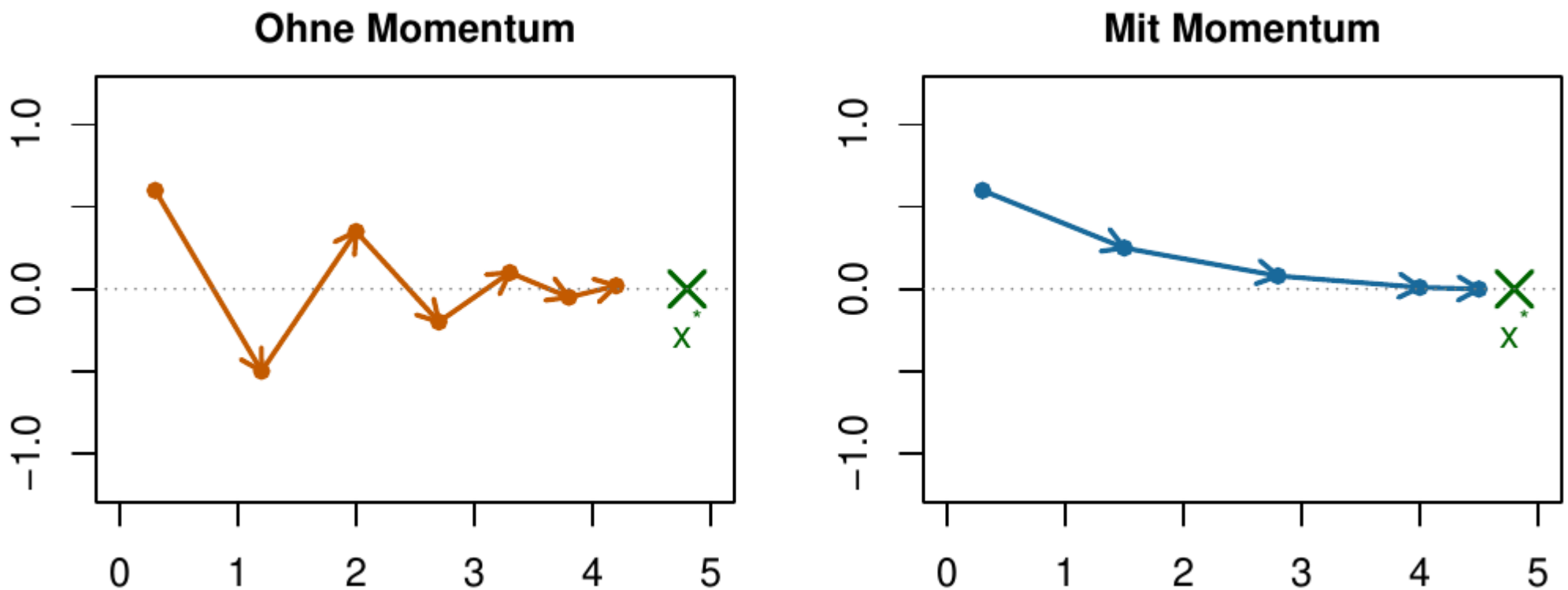
Gradientenabstieg mit Momentum: Motivation



Bei **hoher Konditionszahl** $\kappa\left(\boldsymbol{H}_f\right)$ oszilliert GD oft stark.

Idee: Momentum (“Schwung”)

Analogie:
Eine Kugel, die einen Hügel herunterrollt, sammelt Schwung (*momentum*). Ihre Bewegung wird dann nicht durch jede kleine Unebenheit beeinflusst, sondern durch die *mittlere Steigung*.



Momentum **dämpft Oszillationen** und **beschleunigt** konsistente Richtungen.

Heavy-Ball Momentum

Momentum-Update

Benutzt wird nicht mehr nur die aktuelle Steigung $\nabla f\left(\boldsymbol{x}^{(k)}\right)^\top$, sondern ein speziell "gemittelter" Gradient $\boldsymbol{v}^{(k)}$:

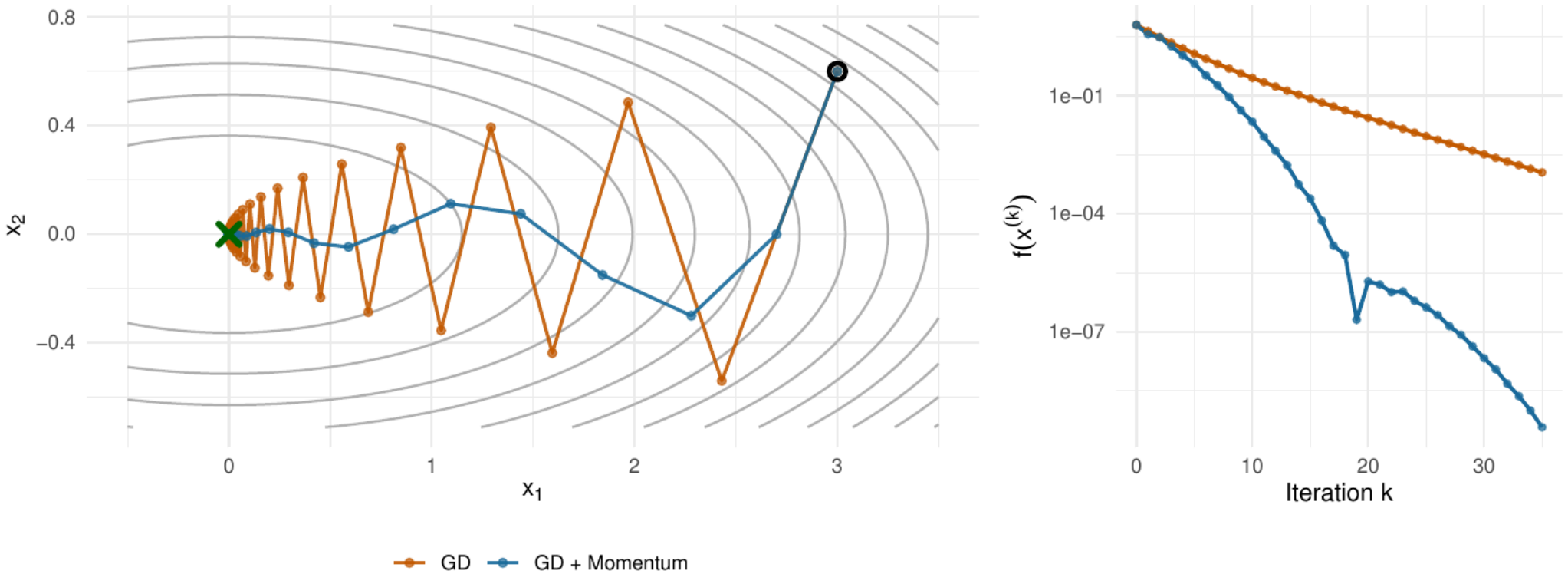
$$\begin{aligned}\boldsymbol{v}^{(k+1)} &= \alpha \boldsymbol{v}^{(k)} - \gamma \nabla f\left(\boldsymbol{x}^{(k)}\right)^\top \\ \boldsymbol{x}^{(k+1)} &= \boldsymbol{x}^{(k)} + \boldsymbol{v}^{(k+1)}\end{aligned}$$

- $\boldsymbol{v}^{(k)}$: "Geschwindigkeit" (akkumulierte Bewegungsrichtung)
- $\alpha \in [0, 1)$: *Momentum*-Parameter (typisch: 0.9); gedämpft wird mit $1 - \alpha$
- γ: Schrittweite (wie bei GD)
- (Bessere) Alternative: *Nesterov-Momentum* benutzt Gradient an *voraussichtlichem* Punkt.

Wirkung:

- Glättet Oszillationen durch Mittelung früherer Gradienten
- Beschleunigt so Iterationsfortschritt in Richtungen mit konsistentem Gradienten

Momentum: Visualisierung



s.a. Shiny-App

Stochastic Gradient Descent: Motivation

Typisches ML/Statistik-Problem:
Minimiere das *empirische Risiko*

$$R(\boldsymbol{\theta}) := \frac{1}{N} \sum_{i=1}^{N} L(y_i, p_{\boldsymbol{\theta}}(\boldsymbol{x}_i))$$

- Modell p_θ mit Features/Inputs $\boldsymbol{x}_i$, Labels/Outputs y_i und Parametern $\boldsymbol{\theta}$
- N = Anzahl Datenpunkte (kann sehr groß sein!)
- Verlustfunktion $L(y_i, p_{\boldsymbol{\theta}}(\boldsymbol{x}_i)) \simeq$ Fehler für Datenpunkt i bei Modellvorhersage $\hat{y}_i = p_{\boldsymbol{\theta}}(\boldsymbol{x}_i)$

Problem:

$$\nabla R(\boldsymbol{\theta}) = \frac{1}{N} \sum_{i=1}^{N} \nabla L(y_i, p_{\boldsymbol{\theta}}(\boldsymbol{x}_i))$$

Jeder Schritt erfordert Berechnung **aller** N Gradienten $\rightarrow$ **sehr teuer**!

Stochastic Gradient Descent (SGD)

Idee:
Gradient eines *zufälligen* Datenpunkts $\nabla L(y_i, p_{\boldsymbol{\theta}}(\boldsymbol{x}_i))$ ist ein *unverzerrter Schätzer*:

$$\mathbb{E}_{i \sim \text{Unif}\{1,\ldots,N\}} \left[\nabla L(y_i, p_{\boldsymbol{\theta}}(\boldsymbol{x}_i))\right] = \nabla R(\boldsymbol{\theta})$$

(Erwartungswert *nur* über die Ziehung des Index i — bei festem Datensatz und festem $\boldsymbol{\theta}$)

SGD-Update

Wähle zufälligen Index i_k:

$$\boldsymbol{\theta}^{(k+1)} = \boldsymbol{\theta}^{(k)} - \gamma^{(k)} \nabla L(y_{i_k}, p_{\boldsymbol{\theta}^{(k)}}(\boldsymbol{x}_{i_k}))^\top$$

- Kosten pro Schritt: $O(1)$ statt $O(N)$
- Richtung ist "im Mittel" korrekt, aber **verrauscht**

In der Praxis:
Verwende *Mini-Batches*: Gradient bestimmt aus Stichproben von $n \ll N$ Datenpunkten (oft: 32–256)

SGD: Grundlage moderner AI

> **Mini-Batch-Gradientenverfahren** sind die Grundlage fast des gesamten aktuellen NN-Trainings.

- Zwei langlebige Standardfamilien:
  - *SGD mit Momentum* (ggf. Nesterov), besonders in Computer Vision
  - *AdamW*: adaptive erste und zweite Momente + entkoppelter *Weight Decay*; Standard bei Transformern
- Adaptive, veränderliche Schrittweiten: *Learning-Rate-Schedule* oft ebenso wichtig wie der Optimierer (*Warmup* $\rightarrow$ lineare oder $\cos$-Abnahme.)
- *Gradient Clipping* begrenzt große Gradienten und stabilisiert das Training.
- Mini-Batch-Rauschen kann helfen, Sattelpunkte und Plateaus zu verlassen.

Weiterführend:

MSc-Kurs Optimization for ML behandelt Adam, Muon & Co, *learning rate schedules*, etc.

Beschränkte Optimierung (Constrained Optimization)

Optimierung mit Nebenbedingungen: Motivation

Bisher: **Unbeschränkte** Optimierung $\min_{\boldsymbol{x}} f(\boldsymbol{x})$ für $f : \mathbb{R}^n \to \mathbb{R}$.

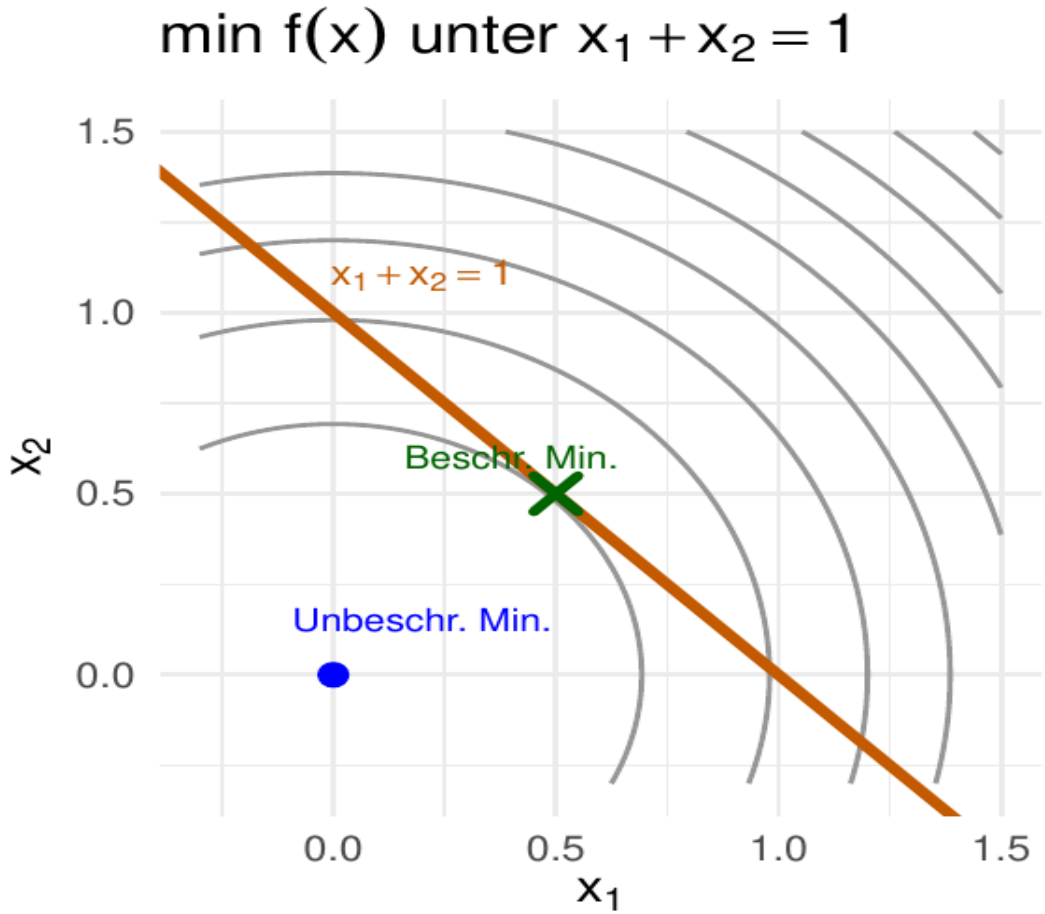
Oft haben wir aber **Einschränkungen**, z.B.:

- Parameter sind Wahrscheinlichkeiten: $p_i \geq 0$ und $\sum_i p_i = 1$
- Budget: Kosten $\leq$ Maximum
- Physik: Erhaltungsgrößen

Allgemeines Problem mit Nebenbedingungen

$$\begin{aligned} \min_{\boldsymbol{x}} \quad & f(\boldsymbol{x}) && \text{(Zielfunktion)} \\ \text{u.d.N.} \quad & g_i(\boldsymbol{x}) = 0, \; i = 1, \ldots, m && \text{(Gleichungen)} \\ & h_j(\boldsymbol{x}) \leq 0, \; j = 1, \ldots, p && \text{(Ungleichungen)} \end{aligned}$$

Geometrische Intuition



Hier also: $g(\boldsymbol{x}) = x_1 + x_2 - 1$. Das beschränkte Optimum liegt **auf** der Nebenbedingung!

Lagrange-Multiplikatoren: Idee

Intuition:

1. Wenn wir die Nebenbedingung $g(\boldsymbol{x}) = 0$ als Kurve betrachten, dürfen wir uns nur entlang dieser Kurve bewegen.
2. Im Optimum gibt es entlang der Nebenbedingung g keine Steigung mehr in f
   $\implies$ ∇f *senkrecht* zur (Tangente der) Nebenbedingung
   $\implies$ ∇f *parallel* zu ∇g

Idee: Am Optimum $\boldsymbol{x}^\star$ auf $g(\boldsymbol{x}) = 0$ muss also gelten:

$$\nabla f(\boldsymbol{x}^\star) \text{ ist parallel zu } \nabla g(\boldsymbol{x}^\star)$$

$$\implies \quad \exists\, \lambda \in \mathbb{R} \text{ mit } \nabla f(\boldsymbol{x}^\star) + \lambda \nabla g(\boldsymbol{x}^\star) = \mathbf{0}^\top$$

Lagrange-Multiplikatoren: Theorie

Lagrange-Funktion

$$\mathcal{L}(\boldsymbol{x}, \boldsymbol{\lambda}) = f(\boldsymbol{x}) + \boldsymbol{\lambda}^\top \boldsymbol{g}(\boldsymbol{x})$$

mit $\boldsymbol{g}(\boldsymbol{x}) = \begin{pmatrix} g_1(\boldsymbol{x}) \\ \vdots \\ g_m(\boldsymbol{x}) \end{pmatrix}$ und $\boldsymbol{\lambda} = \begin{pmatrix} \lambda_1 \\ \vdots \\ \lambda_m \end{pmatrix}$

Notwendige Bedingung (Lagrange)

Sind die $\nabla g_i(\boldsymbol{x}^\star)$ linear unabhängig, existiert am Optimum $\boldsymbol{x}^\star$ ein $\boldsymbol{\lambda}^\star$ mit:

$$\nabla_{\boldsymbol{x}} \mathcal{L}(\boldsymbol{x}^\star, \boldsymbol{\lambda}^\star) = \mathbf{0}^\top \quad \text{und} \quad \boldsymbol{g}(\boldsymbol{x}^\star) = \mathbf{0}$$

- Hinreichende Bedingungen sind komplexer (siehe z.B. Heath, Ch. 6.2.3)

Beispiel: Lagrange-Multiplikatoren

Problem:
Minimiere $f(x, y) = x^2 + y^2$ unter $g(x, y) = x + y - 1 = 0$.

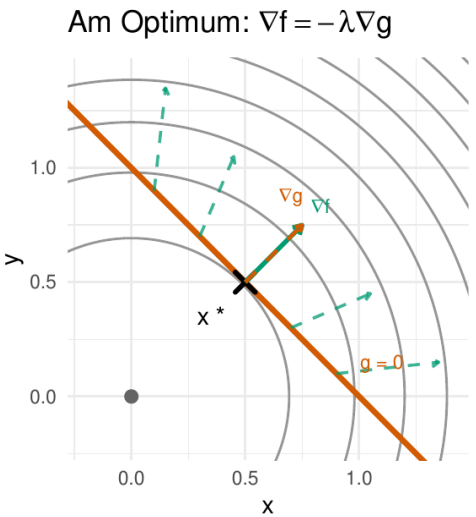
Lagrange-Funktion:

$$\mathcal{L}(x, y, \lambda) = x^2 + y^2 + \lambda(x + y - 1)$$

Notwendige Bedingungen:

$$\frac{\partial \mathcal{L}}{\partial x} = 2x + \lambda = 0 \quad \Rightarrow \quad x = -\lambda/2$$
$$\frac{\partial \mathcal{L}}{\partial y} = 2y + \lambda = 0 \quad \Rightarrow \quad y = -\lambda/2$$
$$\frac{\partial \mathcal{L}}{\partial \lambda} = g(x, y) = x + y - 1 = 0 \quad \Rightarrow \quad -\lambda = 1$$

Lösung: $x^\star = y^\star = \frac{1}{2}, \lambda^\star = -1, f(x^\star, y^\star) = \frac{1}{2}$



$\nabla f(x, y) = (2x, 2y)$; $\nabla g(x, y) = (1, 1)$

KKT-Bedingungen für Ungleichungen

Bei **Ungleichungs-Nebenbedingungen** $h_j(\boldsymbol{x}) \leq 0$ wird es komplizierter.

Karush-Kuhn-Tucker (KKT) Bedingungen

Für das Problem

$$\min f(\boldsymbol{x}) \text{ u.d.N. } g_i(\boldsymbol{x}) = 0, h_j(\boldsymbol{x}) \leq 0$$

gelten am Optimum $\boldsymbol{x}^\star$:

1. *Stationarität:* $\nabla f(\boldsymbol{x}^\star) + \sum_i \lambda_i \nabla g_i(\boldsymbol{x}^\star) + \sum_j \mu_j \nabla h_j(\boldsymbol{x}^\star) = \mathbf{0}^\top$
2. *Primale Zulässigkeit:* $g_i(\boldsymbol{x}^\star) = 0$, $h_j(\boldsymbol{x}^\star) \leq 0$
3. *Duale Zulässigkeit:* $\mu_j \geq 0$
4. *Komplementarität:* $\mu_j h_j(\boldsymbol{x}^\star) = 0$ für alle j

Komplementarität bedeutet:
Für jede Ungleichung gilt

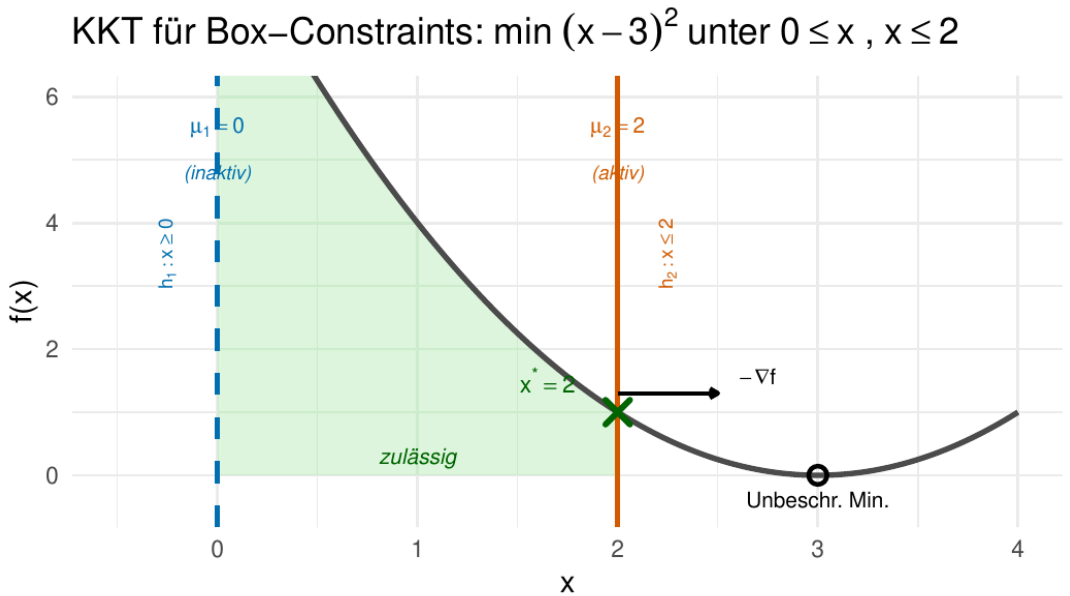
- $h_j(\boldsymbol{x}^\star) < 0$ (inaktiv) $\implies$ $\mu_j = 0$
- $\mu_j > 0$ $\implies$ $h_j(\boldsymbol{x}^\star) = 0$ (aktiv, “bindet”)

Beispiel: Ungleichungs-Nebenbedingungen

Problem: $\min_x f(x) = (x-3)^2$ unter $0 \le x \le 2$, umformuliert: $h_1(x) = -x \le 0$, $h_2(x) = x - 2 \le 0$

KKT-Bedingungen:

1. $2(x^\star - 3) - \mu_1 + \mu_2 = 0$
2. $-x^\star \le 0$, $x^\star - 2 \le 0$
3. $\mu_1, \mu_2 \ge 0$
4. $\mu_1 x^\star = 0$, $\mu_2(x^\star - 2) = 0$



Lösung:
$x^\star = 2$ (am rechten Rand), mit $\mu_1 = 0$, $\mu_2 = 2$

$\Longrightarrow$ Obere Schranke ist **aktiv** (bindet), untere ist **inaktiv**.

Am Optimum $x^\star = 2$:
neg. Gradient $-\nabla f(x^\star) = -2(x^\star - 3) = 2$ zeigt nach rechts (Richtung des unbeschränkten Minimums), aber die **aktive** Schranke h_2 blockiert weitere Bewegung.

Statistische Anwendung: Regularisierte Regression

Regularisierte Regression lässt sich als *beschränkte Optimierung* formulieren:

Ridge / Lasso (äquivalente Formulierungen)

$$\text{Penalisiert:} \quad \min_{\boldsymbol{\beta}} \left(\|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2 + \lambda \|\boldsymbol{\beta}\|_p^p \right)$$

$$\text{Beschränkt:} \quad \min_{\boldsymbol{\beta}} \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2 \quad \text{u.d.N.} \quad \|\boldsymbol{\beta}\|_p^p \leq c$$

mit $p = 2$ (Ridge) bzw. $p = 1$ (Lasso).

Lagrange-Funktion:
$\mathcal{L}(\boldsymbol{\beta}, \mu) = \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2 + \mu \left(\|\boldsymbol{\beta}\|_p^p - c \right)$

KKT-Stationarität für Ridge ($p = 2$):

$$\nabla_{\boldsymbol{\beta}} \mathcal{L}^\top = -2\boldsymbol{X}^\top (\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}) + 2\mu\boldsymbol{\beta} = \mathbf{0} \implies \widehat{\boldsymbol{\beta}}_{\text{Ridge}} = (\boldsymbol{X}^\top \boldsymbol{X} + \mu \boldsymbol{I})^{-1} \boldsymbol{X}^\top \boldsymbol{y}$$

Für jedes $\lambda > 0$ existiert ein $c > 0$, sodass beide Formulierungen dieselbe Lösung haben —
umgekehrt nur, solange die Nebenbedingung auch *bindet* (ist der KQ-Schätzer selbst zulässig, erzwingt die Komplementarität $\mu = 0$).
Die Zuordnung $\lambda \leftrightarrow c$ hängt von den Daten ab.

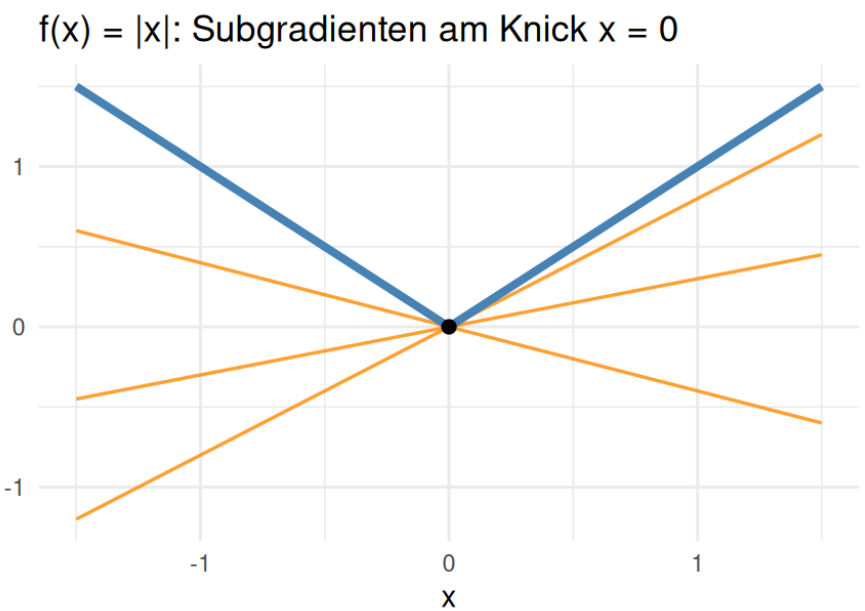
Exkurs: Subgradienten

Proposition

Sei $f\colon \mathbb{R}^n \to \mathbb{R}$ konvex.
Dann gibt es für jedes $\boldsymbol{x} \in \mathbb{R}^n$ einen Vektor $\boldsymbol{v} \in \mathbb{R}^n$, sodass

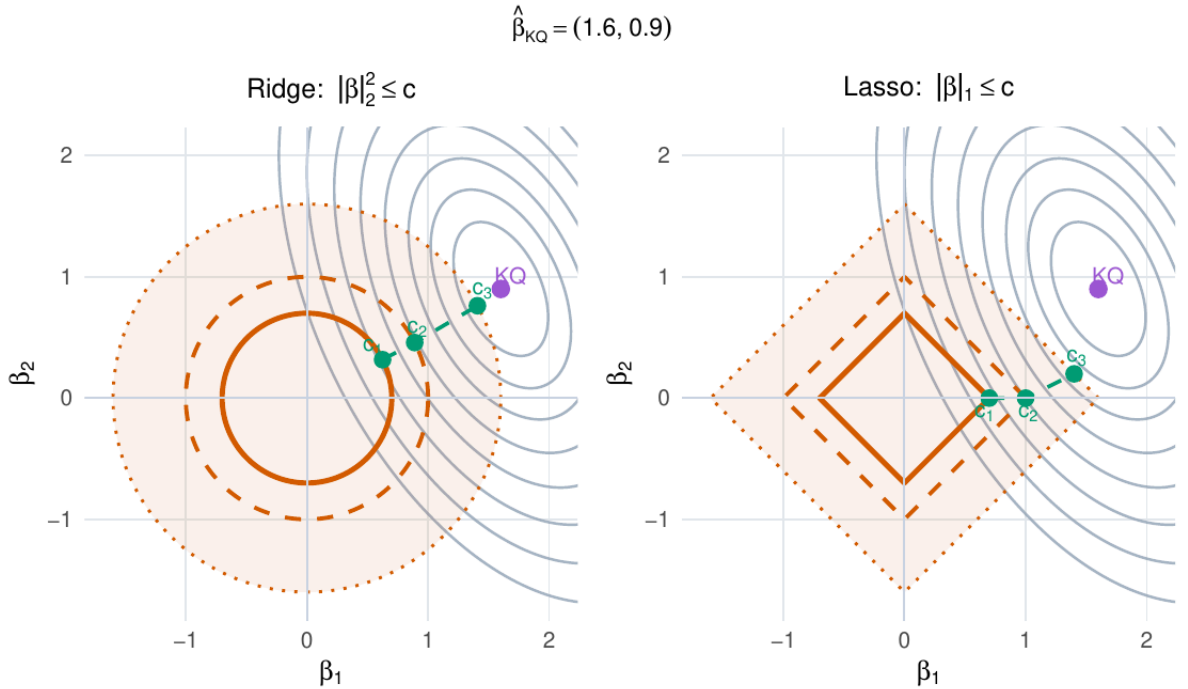
$$f(\boldsymbol{y}) \geq f(\boldsymbol{x}) + \boldsymbol{v}^\top (\boldsymbol{y} - \boldsymbol{x}) \quad \forall \boldsymbol{y} \in \mathbb{R}^n.$$

- Einfach gesagt: Wir können an jedem Punkt eine *affine* Funktion *unter* den Funktionsgraphen legen.
- Der Vektor $\boldsymbol{v}$ muss nicht eindeutig sein. *Jeder* Vektor, der diese Ungleichung erfüllt, wird **Subgradient** genannt. (Ist f in $\boldsymbol{x}$ differenzierbar, ist $\nabla f(\boldsymbol{x})^\top$ der *einzige* Subgradient.)
- Die Funktion muss dafür **nicht differenzierbar** sein $\rightarrow$ *Subgradienten* wichtig für allgemeinere Optimierungsprobleme: z.B. LASSO, Quantilregression, u.v.m.



↩ zurück: „Geometrie: Ridge vs. Lasso”

Geometrie: Ridge vs. Lasso



Grau: Höhenlinien $f(\boldsymbol{\beta}) = \|\boldsymbol{y} - \boldsymbol{X}\boldsymbol{\beta}\|_2^2$; Lila: $\arg\min f(\boldsymbol{\beta})$; Grün: Lösung unter Nebenbedingungen.

- **Ridge** ($p = 2$): Kreisförmige Nebenbedingung $\longrightarrow$ Lösungen auf dem Kreisbogen (sofern die Nebenbedingung *bindet*)
- **Lasso** ($p = 1$): Rautenförmige Nebenbedingung $\longrightarrow$ Lösungen oft an *Ecken* $\implies$ **sparse** ($\exists j : \beta_j = 0$)

Achtung:
$\|\boldsymbol{\beta}\|_1$ ist an den Ecken **nicht differenzierbar** — die KKT-Stationarität gilt fürs Lasso nur mit *Subgradienten*.
In der Praxis verwendet man Subgradienten- bzw. *Proximal-Verfahren*.

Optimierung in R

Optimierung in R: `optimize`

Univariat: `optimize()` für $f : \mathbb{R} \to \mathbb{R}$

```r
f <- function(x) (x - 3)^2
optimize(f, interval = c(0, 10))
```

```
$minimum
[1] 3

$objective
[1] 0
```

- Benutzt *Brent-Verfahren*: *Golden Section Search* (Intervall-Schrumpfung ähnlich *Bisektion*) + parabolische Interpolation
- Findet für *unimodales* f das Minimum im Intervall — sonst evtl. nur ein *lokales* Minimum

Optimierung in R: `optim`

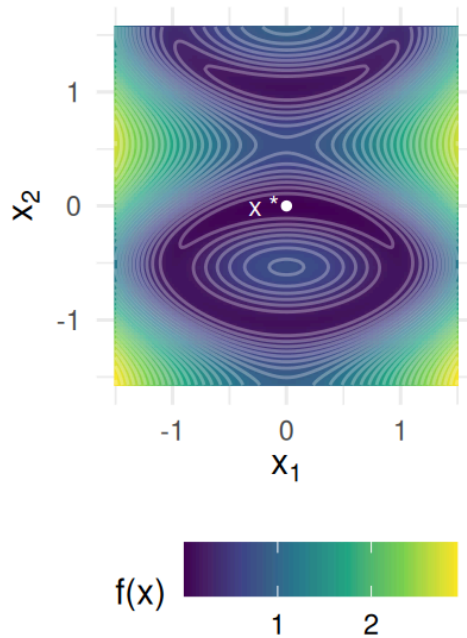
Multivariat: `optim()` für $f : \mathbb{R}^n \to \mathbb{R}$

```r
f <- function(x) log1p((x[1]^2 + sin(3*x[2]))^2) +
  .1 * x[1]^2 + .1 * x[2]^2
optim(c(-1, -0.5), f, method = "Nelder-Mead") |> #default
  str()
```

```
List of 5
 $ par        : num [1:2] 5.54e-05 7.49e-06
 $ value      : num 8.18e-10
 $ counts     : Named int [1:2] 123 NA
  ..- attr(*, "names")= chr [1:2] "function" "gradient"
 $ convergence: int 0
 $ message    : NULL
```

```r
optim(c(-1, -0.5), f, method = "BFGS") |> str()
```

```
List of 5
 $ par        : num [1:2] 6.88e-10 2.11e-10
 $ value      : num 4.52e-19
 $ counts     : Named int [1:2] 47 29
  ..- attr(*, "names")= chr [1:2] "function" "gradient"
 $ convergence: int 0
 $ message    : NULL
```



`optim`: (Falsche) Konvergenz

Startwerte weit weg vom globalen Minimum $\boldsymbol{x}^\star = (0, 0)$ bzw. zu kleines Iterationslimit:

```r
# DANGER: Konvergenz zu lokalen Minima
optim(c(-1, 1), f)[1:2] |> str()
optim(c(-.5, -1), f, method = "BFGS")[1:2] |> str()
```

```
List of 2
 $ par  : num [1:2] 8.81e-05 1.04
 $ value: num 0.108
List of 2
 $ par  : num [1:2] -1.54e-05 -1.04
 $ value: num 0.108
```

```r
# DANGER: Keine Konvergenz aber keine warnings():
optim(c(-1, -0.5), f, control = list(maxit = 50)) |> str()
```

```
List of 5
 $ par        : num [1:2] -0.584 -0.151
 $ value      : num 0.0454
 $ counts     : Named int [1:2] 51 NA
  ..- attr(*, "names")= chr [1:2] "function" "gradient"
 $ convergence: int 1
 $ message    : NULL
```

$\Longrightarrow$ `convergence = 0` heißt nur: *Abbruchkriterium erfüllt* — **keine Garantie für ein (globales) Minimum!**

`optim`: Mit Gradienten oder Constraints

```r
grad_f <- function(x) {
  log1p_dx <- 1 / (1 + (x[1]^2 + sin(3*x[2]))^2)
  c((4 *x[1]*(x[1]^2 + sin(3*x[2]))) * log1p_dx + .2 * x[1],
    (6 *cos(3*x[2])*(x[1]^2 + sin(3*x[2]))) * log1p_dx + .2 *x[2])
}
str(optim(c(-1, -0.5), f, gr = grad_f, method = "BFGS")[-5])
```

```
List of 4
 $ par        : num [1:2] 6.88e-10 2.11e-10
 $ value      : num 4.52e-19
 $ counts     : Named int [1:2] 47 29
  ..- attr(*, "names")= chr [1:2] "function" "gradient"
 $ convergence: int 0
```

```r
str(optim(c(-1, 0.5), f, gr = grad_f, method = "L-BFGS-B",
          lower = c(-1, -1), upper = c(1, 1))[-5])
```

```
List of 4
 $ par        : num [1:2] 8.71e-07 2.65e-08
 $ value      : num 8.22e-14
 $ counts     : Named int [1:2] 14 14
  ..- attr(*, "names")= chr [1:2] "function" "gradient"
 $ convergence: int 0
```

$\longrightarrow$ Analytischer Gradient schneller und genauer als numerische Approximation über finite Differenzen. $\longrightarrow$ `L-BFGS-B` erlaubt Box-Nebenbedingungen (*KKT*!).

Self-Check & Zusammenfassung

Self-Check

Frage 1: Newton-Schritt für $f(x) = x^2 + 2$ mit $x^{(0)} = 4$: Was ist $x^{(1)}$?

1. $x^{(1)} = 0$
2. $x^{(1)} = 2$
3. $x^{(1)} = -4$
4. $x^{(1)} = 3.5$

Frage 2: Welche Aussage über die KKT-Bedingungen ist **falsch**?

1. Die Multiplikatoren für Ungleichungen erfüllen $\mu_j \geq 0$.
2. Komplementarität: $\mu_j h_j(\boldsymbol{x}^\star) = 0$ für alle j.
3. Ist eine Ungleichung inaktiv ($h_j(\boldsymbol{x}^\star) < 0$), gilt $\mu_j = 0$.
4. Ist eine Ungleichung inaktiv ($h_j(\boldsymbol{x}^\star) < 0$), muss $\mu_j > 0$ gelten.

Frage 3: `optim()` liefert `convergence = 0`. Was wissen wir?

1. Das globale Minimum wurde gefunden.
2. Ein Abbruchkriterium wurde erfüllt — keine Garantie für ein (globales) Minimum.
3. Der Gradient ist exakt $\mathbf{0}$.
4. Die Funktion ist konvex.

Wrap-up: Optimierung I & II

Methode	Wann verwenden?
Bisektion / Newton-Raphson	Nullstellen $f(x) = 0$, univariat
Nelder-Mead	Keine Ableitungen, niedrige Dimension ($n \leq 10$)
Gradient Descent	Große Probleme, einfache Implementierung
Momentum	GD + schlecht konditionierte Probleme
SGD	Sehr große Datensätze & große Dimension
Quasi-Newton (BFGS)	Mittlere Probleme, guter Kompromiss
Newton	Kleine Probleme, schnelle Konvergenz nötig
Lagrange/KKT	Nebenbedingungen

Kernideen der beiden Optimierungs-Sitzungen:

- Unbeschränkte differenzierbare Optimierung $\simeq$ Lösen von $\nabla f(\boldsymbol{x}) = \mathbf{0}^\top$
  - meist *iterativ* mit - unter Nebenbedingungen: mit *Lagrange*, *KKT* prüfen.
- Mehr Ableitungsinformation $\rightarrow$ schnellere Konvergenz, teurere Schritte
- *Konvexität* garantiert globale Optima; *Konditionszahl* der *Hessematrix* steuert das Tempo
- Nebenbedingungen: *Lagrange* / *KKT* — Gradienten von Ziel und Nebenbedingung im Gleichgewicht

Nächste Vorlesung: Funktionsapproximation

Anhang

Momentum: Zusammenfassung

Wann Momentum verwenden?

- Hilfreich vor allem bei **schlecht konditionierten** Problemen; bei kleiner Konditionszahl kann $\alpha = 0.9$ sogar *langsamer* sein als reines GD
- Standard in Deep Learning ($\alpha = 0.9$ typisch)
- Kombination mit adaptiven Schrittweiten (Adagrad, RMSprop): **ADAM**, ...

Verbindung zur Konvexität:

- Bei **quadratischen** Zielfunktionen (und geeignet gewählten α, γ) beschleunigt Momentum die Konvergenz beweisbar auf die Rate $\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$; für allgemeine stark konvexe f garantiert das erst die *Nesterov*-Variante
- Reduziert effektiv die “Konditionszahl” des Problems

Funktionsapproximation: Interpolation & Splines*

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 27.08.2026

Opener-Quiz

Opener-Quiz

Beim Reinkommen: QR-Code scannen und die 2 Einstiegsfragen beantworten.

Frage 1:

Sei $\boldsymbol{v}_1, \ldots, \boldsymbol{v}_d$ eine *Basis* eines Vektorraums V. Welche Aussagen sind wahr?

1. Jedes $\boldsymbol{v} \in V$ hat eine **eindeutige** Darstellung $\boldsymbol{v} = \sum_{j=1}^{d} \alpha_j \boldsymbol{v}_j$.
2. Basisvektoren dürfen linear abhängig sein, solange sie V aufspannen.
3. Die Anzahl d der Basisvektoren ist die *Dimension* von V.

Frage 2:

Die Menge *aller* Funktionen $f\colon [a, b] \to \mathbb{R}$ bildet mit punktweiser Addition und Skalarmultiplikation einen Vektorraum. Welche Dimension hat er?

1. 2 (Intervallgrenzen a und b)
2. n (Anzahl der Datenpunkte)
3. endlich, aber sehr groß
4. Er ist unendlich-dimensional.

Verwendete Vorkenntnisse

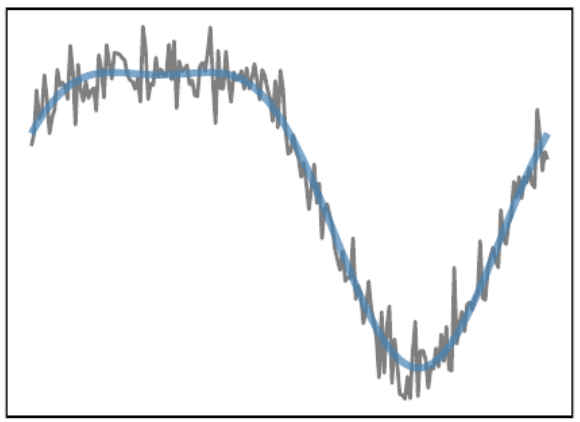
- **Lineare Algebra:**
  - *Lineare Gleichungssysteme* $\boldsymbol{Ba} = \boldsymbol{y}$ und deren Lösbarkeit
  - *Konditionszahl* einer Matrix als Maß für numerische Stabilität
  - *Basen* und *lineare Unabhängigkeit* (Basisdarstellung von Funktionen)
- **Analysis:**
  - Polynome und ihre Ableitungen (Polynomgrad, Monomialbasis)
  - Stetigkeit und Differenzierbarkeit
  - Stückweise definierte Funktionen
  - *Normen* (z.B. $\left\| \cdot \right\|_\infty$ für Konvergenzaussagen)

Einführung

Einführung

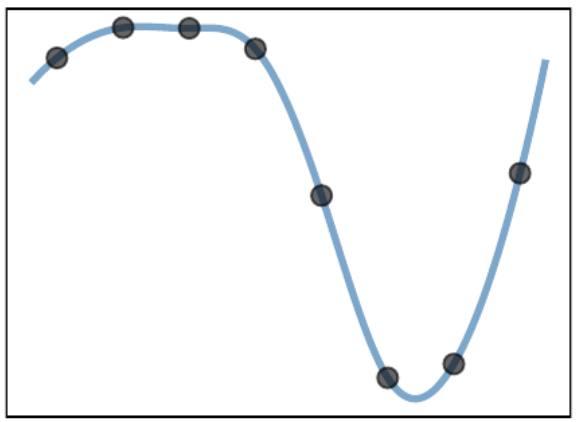
Drei Varianten: Finde $\widehat{f}$, so dass ...

Approximation



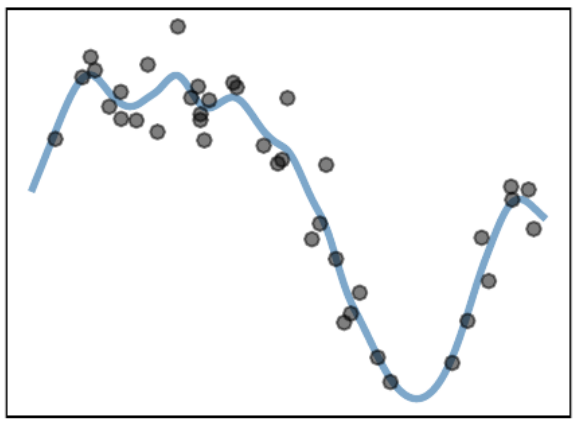
$$\|f - \widehat{f}\| \approx 0$$

Interpolation



$$f(x_i) = \widehat{f}(x_i) \quad \forall i$$

Glättung



$$y_i = \widehat{f}(x_i) + \varepsilon_i \quad \forall i$$

Bei der Interpolation sind die Daten *rauschfrei*, d.h. $y_i = f(x_i)$ – deshalb steht dort später auch $y_i = \widehat{f}(x_i)$.

Frage:
Wie sind die drei Varianten miteinander verwandt?

Kommentare

- Das *Approximationsproblem* lösen wir meist, indem wir es in ein *Interpolations*- oder *Glättungsproblem* überführen.
- Glättung ist die “schwierigste” Variante, weil die Fehler ε_i meist stochastisch sind.
- Wir kümmern uns zunächst um **Interpolation**.
- Wichtige Anwendungen:
  - Glatte Kurven zeichnen.
  - f approximativ, aber schneller auswerten.
    - **Beispiel:** $f(x) = \int_0^x g(y)dy$ oder $f(x) = g^{-1}(x)$ müssen numerisch bestimmt werden.
  - Ableitungen oder Integrale von f approximieren.

Anwendungen in ML und Data Science

- **Computergrafik:**
  Interpolation von Pixelwerten bei Reformatierung/Rotation von Bildern.
- **Zeitreihen:**
  Auffüllen fehlender Werte in Sensor- oder Finanzdaten.
- **Generative Modelle:**
  Latent Space-Interpolation in VAEs/GANs
- **NLP:**
  Positional Encodings in Transformern (“RoPE”) basieren auf Sinus-/Cosinus-Funktionen; *Long-Context*-Erweiterungen von RoPE interpolieren bzw. skalieren Positionsindizes.

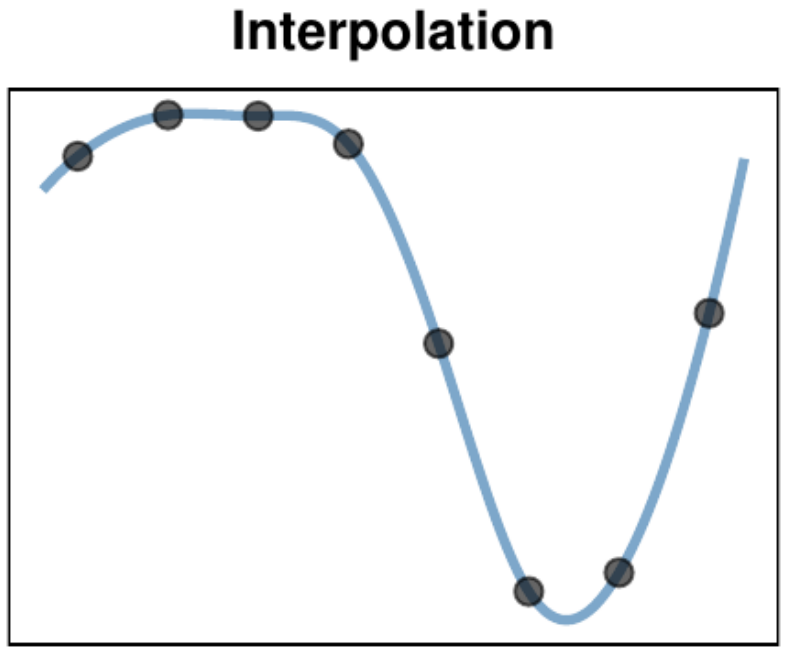
(teils Interpolation im engeren Sinn, teils verwandte Konzepte/Analogien)

Quiz: Interpolationsproblem

Problem:

Finde $\widehat{f}$, so dass

$$y_i = \widehat{f}(x_i), \quad i = 1, \ldots, n.$$



Welche der folgenden Aussagen ist wahr?

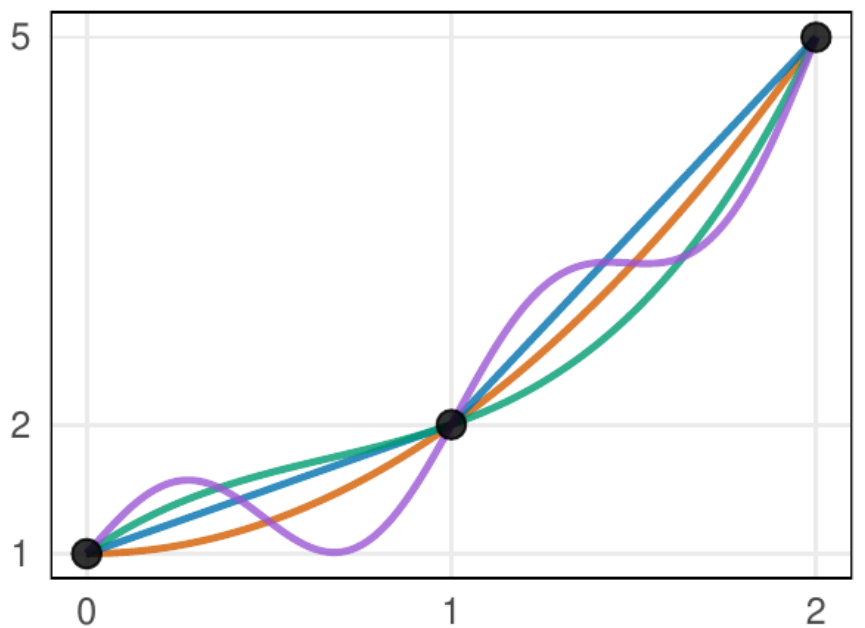
1. Es gibt unendlich viele Funktionen $\widehat{f}$, die das Problem lösen.
2. Aus $y_i = \widehat{f}(x_i), i = 1, \ldots, n$, folgt $\|f - \widehat{f}\| \approx 0$.
3. Das Problem hat eine eindeutige Lösung, wenn alle x_i verschieden sind.

Unendlich viele Lösungen: Beispiel

Betrachte $n = 3$ Punkte: $(0, 1), (1, 2), (2, 5)$.

Einige Interpolanten:

- **Parabel:** $\widehat{f}_1(x) = 1 + x^2$
- **Stückweise linear:** $\widehat{f}_2(x) = \begin{cases} x + 1 & x < 1 \\ 3x - 1 & x \geq 1 \end{cases}$
- **Kubisch:** $\widehat{f}_3(x) = x^3 - 2x^2 + 2x + 1$
- **Vogelwild:** $\widehat{f}_4(x) = 1 + x^2 + 0.5 \sin(2\pi x)$



Alle erfüllen $\widehat{f}(x_i) = y_i$, aber sehen völlig unterschiedlich aus!

Allgemein:

Jedes $\widehat{f} = \widehat{f}_1 + g$ mit $g(x_i) = 0$ für alle i ist ebenfalls ein Interpolant.

$\Longrightarrow$ Wir brauchen *Basissysteme* (Polynome, Splines), die die Lösungsmenge auf bestimmte Funktionenräume einschränken.

Funktionenräume und ihre Basen

Funktionenräume

- Menge aller Funktionen $f\colon [a,b] \to \mathbb{R}$ ist ein **Vektorraum**:
  - Addition: $(f+g)(x) = f(x) + g(x)$
  - Skalarmultiplikation: $(\alpha \cdot f)(x) = \alpha \cdot f(x)$
- Dieser Raum ist **unendlich-dimensional** $\implies$ $\exists$ unendlich viele Interpolanten für n gegebene Punkte!

Lösung:
Einschränkung auf endlichdimensionale Unterräume

Basen: von $\mathbb{R}^n$ zu Funktionenräumen

$\{\phi_1, \ldots, \phi_K\}$ ist eine **Basis** von $\mathcal{F}_K$ — *genauso* wie $\{\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n\}$ eine Basis des $\mathbb{R}^n$ ist:

	Vektorraum $\mathbb{R}^n$	**Funktionenraum $\mathcal{F}_K$**
"Vektor"	$\boldsymbol{v} \in \mathbb{R}^n$	Funktion $f\colon [a, b] \to \mathbb{R}$
Basis	$\boldsymbol{v}_1, \ldots, \boldsymbol{v}_n$	$\phi_1, \ldots, \phi_K$
eindeutige Darstellung	$\boldsymbol{v} = \sum_{i=1}^{n} \alpha_i \boldsymbol{v}_i$	$f = \sum_{k=1}^{K} a_k \phi_k$
Koordinaten	$(\alpha_1, \ldots, \alpha_n)$	Koeffizienten $(a_1, \ldots, a_K)$
Dimension	n	K

- Alle Konzepte der linearen Algebra (lineare Unabhängigkeit, *Span*, Basiswechsel, …) übertragen sich direkt auf Funktionen — eine Funktion *ist* ein Vektor (in $\mathbb{R}^\infty$…)
- Statt eine Funktion (in $\mathbb{R}^\infty$) finden zu müssen, brauchen wir jetzt nur noch ihren **Koordinatenvektor $\boldsymbol{a} = (a_1, \ldots, a_K) \in \mathbb{R}^K$**
  $\to$ ein Problem der (numerischen) linearen Algebra!

Interpolation durch Basisdarstellung

Interpolation durch Basisdarstellung

- **Problem:**
  Finde $\widehat{f}$, so dass

$$y_i = \widehat{f}(x_i) \quad \forall i.$$

- **Lösung:**
  1. Nehme an, dass $\widehat{f}(x) = \sum_{k=1}^{K} a_k \phi_k(x)$ für
     - *Basisfunktionen* $\phi_1, \ldots, \phi_K$ (bekannt)
     - *Basiskoeffizienten* $a_1, \ldots, a_K$ (unbekannt)
  2. Löse das Gleichungssystem

$$y_i = \sum_{k=1}^{K} a_k \phi_k(x_i), \qquad i = 1, \ldots, n.$$

$\rightarrow$ *Interpolationsproblem* zurückgeführt auf *Lineares Gleichungssystem*!

Quiz: Vom Interpolationsproblem zum LGS

Das System

$$y_i = \sum_{k=1}^{K} a_k \phi_k(x_i), \qquad i = 1, \ldots, n$$

lässt sich als LGS $\boldsymbol{Ba} = \boldsymbol{y}$ darstellen. Was sind $\boldsymbol{B}, \boldsymbol{a}, \boldsymbol{y}$?

Beispiel: Basisdarstellung konkret

Gegeben:

$n = 3$ Datenpunkte $(0, 1), (1, 2), (2, 5)$, Monomialbasis $\phi_k(x) = x^{k-1}$, $K = 3$

$$\implies \boldsymbol{B} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 4 \end{pmatrix}, \quad \boldsymbol{y} = \begin{pmatrix} 1 \\ 2 \\ 5 \end{pmatrix}$$

Lösung durch Einsetzen:

$$\begin{aligned} &\text{Zeile 1:} && a_1 = 1 && && \\ &\text{Zeile 2:} && a_2 + a_3 = 1 && \to a_2 && = 1 - a_3 \\ &\text{Zeile 3:} && 2a_2 + 4a_3 = 4 && \to 2 + 2a_3 && = 4 \end{aligned}$$

$$\implies a_3 = 1, a_2 = 0 \implies \widehat{f}(x) = 1 + x^2$$

Probe:

$\widehat{f}(0) = 1, \widehat{f}(1) = 2, \widehat{f}(2) = 5$ ✓

Polynominterpolation

Fundamentalsatz der Algebra

Satz: Fundamentalsatz der Algebra

Jedes Polynom

$$p(x) = a_n x^n + \ldots + a_1 x + a_0$$

mit $a_n \neq 0$ und $n \geq 1$ hat mindestens eine Nullstelle in $\mathbb{C}$.

Äquivalent:
Jedes Polynom vom Grad n hat genau n Nullstellen in $\mathbb{C}$ (mit Vielfachheit gezählt).

Folgerung:
Ein Polynom vom Grad $\leq n - 1$ mit n verschiedenen Nullstellen ist das Nullpolynom.

Beispiel:
Ein quadratisches Polynom (Grad 2) kann höchstens 2 (verschiedene) Nullstellen haben. Finden wir eines mit 3 Nullstellen, muss es $p(x) = 0$ für alle x sein.

Eindeutigkeit der Polynominterpolation

Satz: Polynominterpolation

Für n Paare $(x_i, y_i)_{i=1}^n$ mit paarweise verschiedenen x_i gibt es *genau* ein Polynom p vom Grad *höchstens* $(n-1)$ mit $p(x_i) = y_i$ für alle i.

Beweisskizze (Eindeutigkeit):

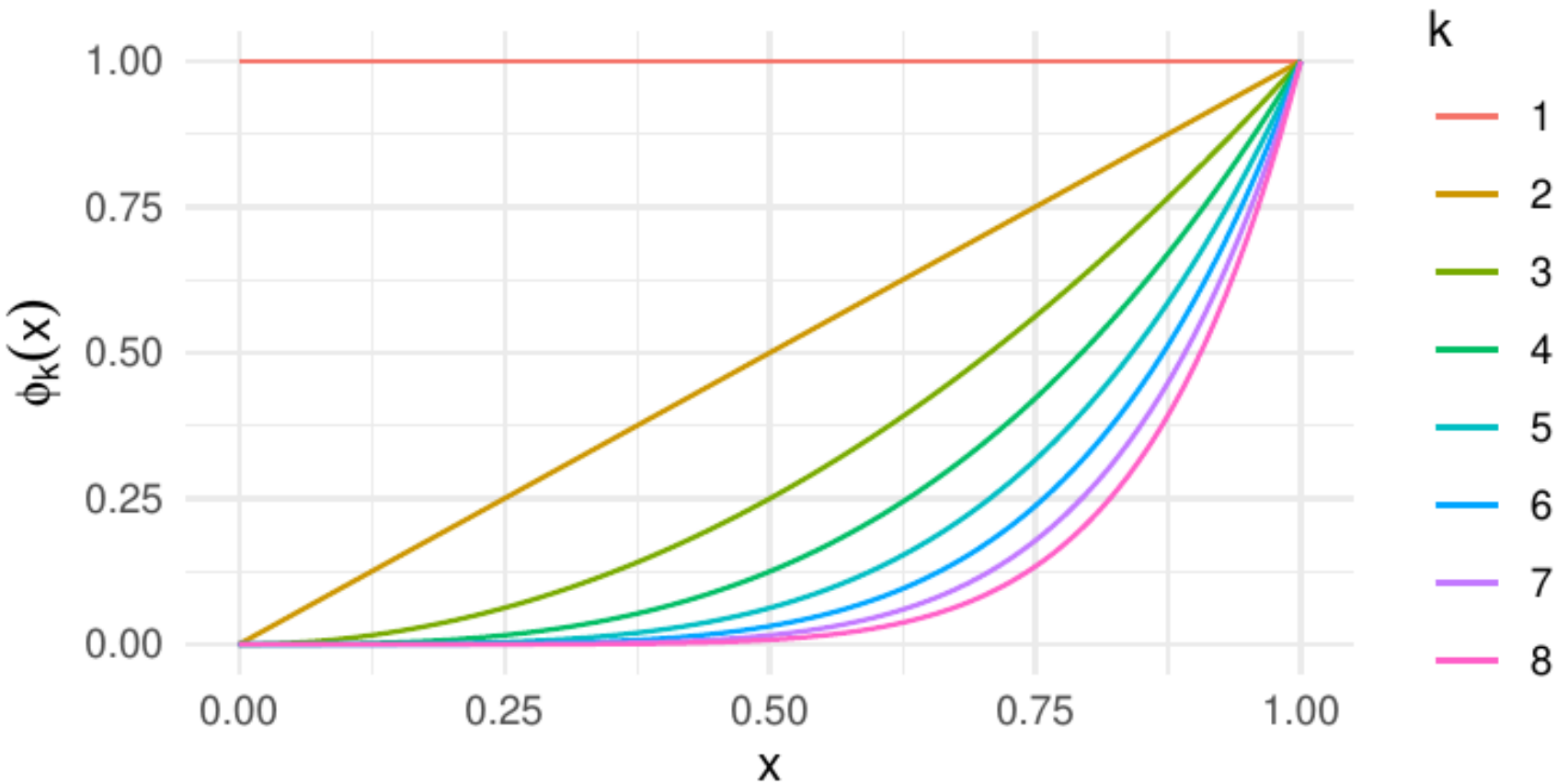
- Angenommen, p und q sind beides interpolierende Polynome vom Grad $\leq n-1$.
- Dann gilt: $(p-q)(x_i) = p(x_i) - q(x_i) = y_i - y_i = 0$ für alle $i = 1, \ldots, n$.
- Also hat $p - q$ mindestens n Nullstellen, aber Grad $\leq n-1$.
- Nach dem *Fundamentalsatz*: $p - q$ muss das Nullpolynom sein.
- Daher $p = q$. ■

Polynominterpolation

- Alle Polynome vom Grad höchstens $(n-1)$ können durch die *Monomialbasis*

$$\phi_1(x) = 1, \quad \phi_2(x) = x, \quad \ldots, \quad \phi_n(x) = x^{n-1}$$

erzeugt werden.



Achtung: Häufiges Missverständnis

Es gibt **genau ein** Polynom von Grad **höchstens** $n-1$ durch n Punkte.

NICHT: Grad **genau** $n-1$!

Beispiel: $n=3$ Punkte $(0,0),(1,1),(2,2)$ liegen auf einer Geraden.

- Interpolierendes Polynom: $p(x)=x$ (Grad 1, nicht 2!)
- Das eindeutige Polynom *höchstens* 2. Grades ist: $p(x)=0\cdot x^2+1\cdot x+0$

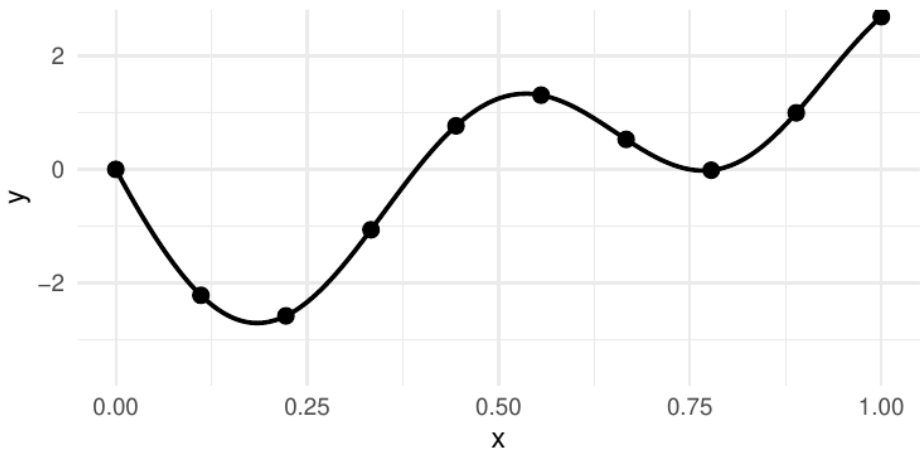
Polynominterpolation in R

```r
x <- seq(0, 1, length = 10)
y <- f(x)

# Vandermonde-Matrix: cbind(1, x, x^2, ...)
B <- outer(x, 0:9, "^")
a <- solve(B, y)

xnew <- seq(0, 1, length = 100)
fhat <- outer(xnew, 0:9, "^") %*% a

ggplot() +
  geom_point(aes(x, y)) +
  geom_line(aes(xnew, fhat))
```



Problem 1: Kondition

- *Monomialbasis* führt zu einer schlecht konditionierten Basismatrix $\boldsymbol{B}$:

$$\boldsymbol{x} = (x_1, \ldots, x_n)^\top \to \boldsymbol{B} = (1 \quad \boldsymbol{x} \quad \boldsymbol{x}^2 \quad \ldots \quad \boldsymbol{x}^{n-1})$$

  - Die Spalten sind die Vektoren $\boldsymbol{b}_k = (x_1^{k-1}, \ldots, x_n^{k-1})^\top$.
  - Für k groß sind diese Vektoren nahezu linear abhängig (vgl. Abbildung der Monomialbasis!)
    $\Longrightarrow$ Matrix nahezu singulär / schlecht *konditioniert*.

Numerisches Beispiel:
n äquidistante Punkte in $[0, 1]$, Polynom vom Grad $n - 1$:

n	**Polynomgrad**	$\kappa(\boldsymbol{B})$
5	4	$\approx 10^3$
10	9	$\approx 10^7$
15	14	$\approx 10^{12}$
20	19	$\approx 10^{16}$

(Größenordnungen für κ_2; andere Normen ergeben sehr ähnliche Werte.
Die Kondition hängt zudem von der Knotenlage ab: dieselben Punkte auf $[-1, 1]$ ergeben für $n = 20$ nur $\kappa_2 \approx 10^8$.)

Bei doppelter Präzision (≈ 16 Dezimalstellen) ist für $n \geq 20$ fast alle Genauigkeit verloren!

$\to$ verwende **orthogonalisierte** Basis (Legendre-, Chebyshev-, Hermite-Polynome) —
in `R`: **`cbind(1, poly(x, 3))`** statt `cbind(1, x, x^2, x^3)`

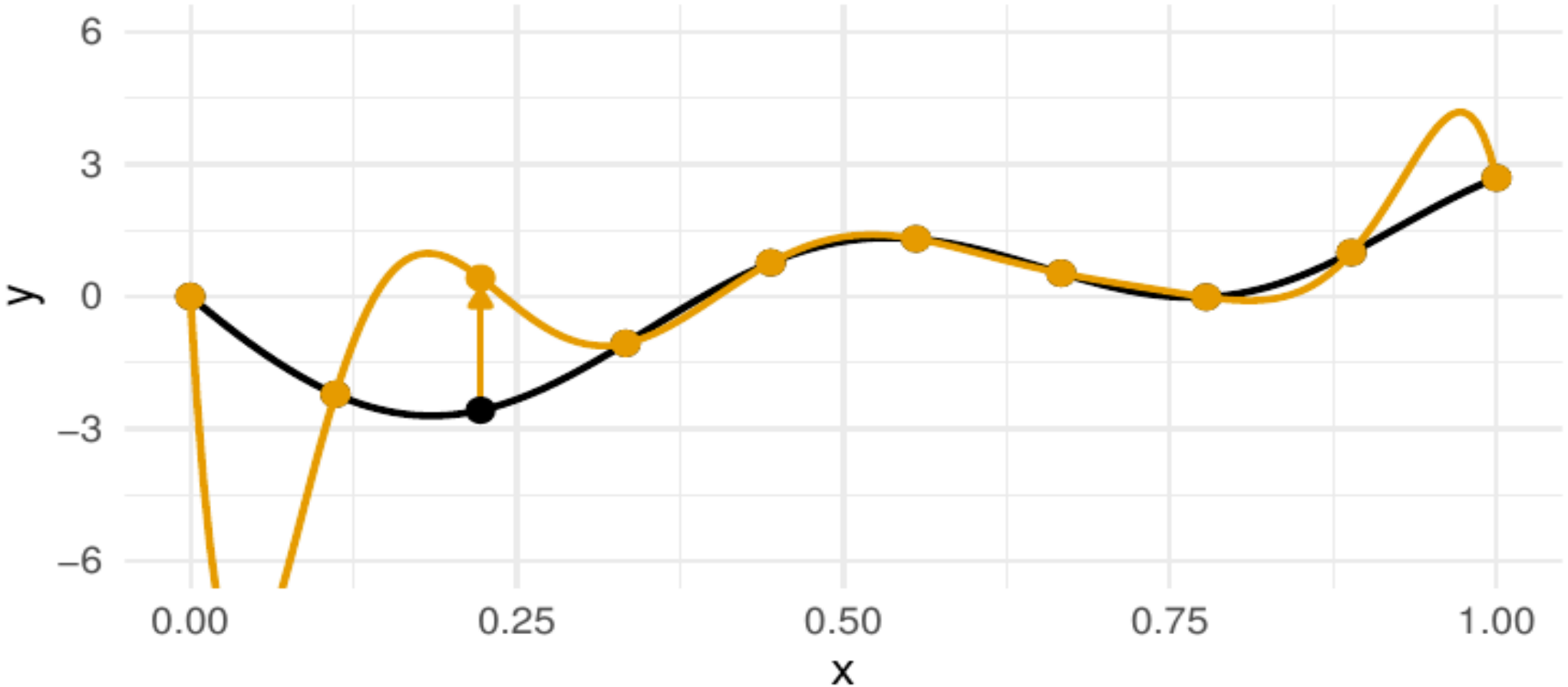
Orthogonale Polynome: Gram-Schmidt im Funktionenraum

- Auf $\mathcal{F}_K$ gibt es auch ein **Skalarprodukt**: $\langle f, g \rangle = \int f(x)\, g(x)\, dx$ (analog zu $\boldsymbol{x}^\top \boldsymbol{y}$ in $\mathbb{R}^n$)
- Damit funktioniert *Gram-Schmidt-Orthogonalisierung* ($\rightarrow$ *QR-Zerlegung*!) angewendet auf die Monome $\{1, x, x^2, \ldots\}$ über $[-1, 1]$:

$$p_2(x) = x^2 - \frac{\langle x^2, 1 \rangle}{\langle 1, 1 \rangle} \cdot 1 - \underbrace{\frac{\langle x^2, x \rangle}{\langle x, x \rangle}}_{=0} \cdot x = x^2 - \tfrac{1}{3} \propto \underbrace{\tfrac{1}{2}\left(3x^2 - 1\right)}_{\text{Legendre-Polynom } P_2}$$

- **Legendre-, Chebyshev-, Hermite-Polynome** = Gram-Schmidt bzgl. unterschiedlicher Skalarprodukte (Gewichtsfunktionen/Intervalle)
- orthogonale Basisfunktionen $\implies \boldsymbol{B}$ deutlich besser konditioniert.
- `poly(x, k)` in `R` orthogonalisiert bzgl. des *diskreten* (vektoriellen) Skalarprodukts $\sum_i f(x_i) g(x_i)$.

Problem 2: Stabilität und Lokalität



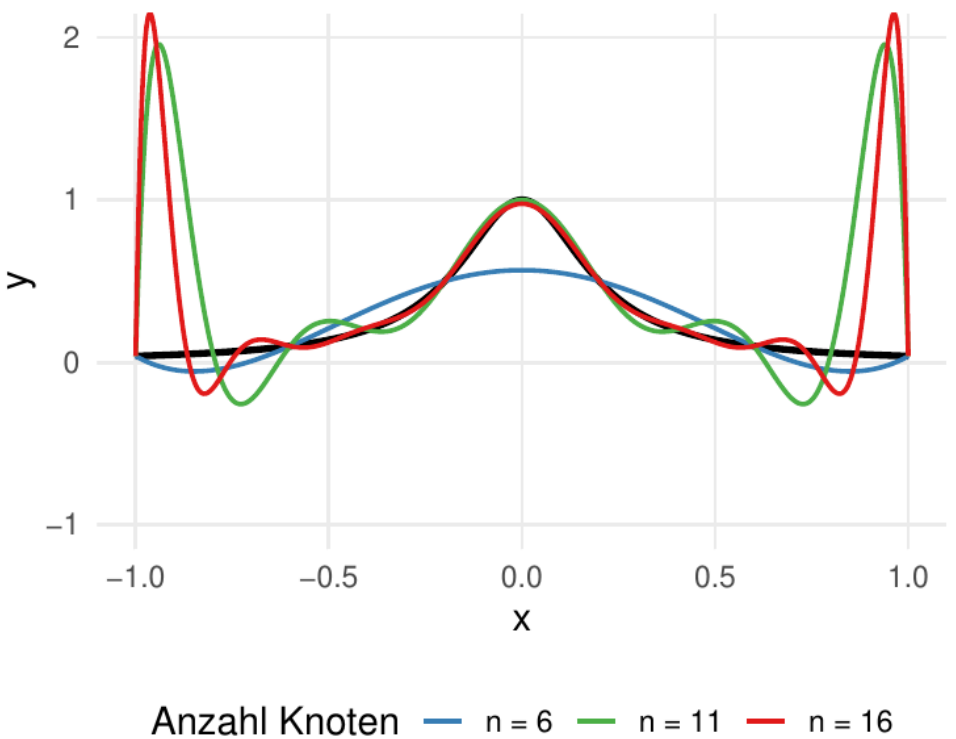
- Ändern wir nur ein y_i, verändert sich (potentiell) die gesamte Funktion. $\Longrightarrow$ Die Methode ist instabil.
- Dies ist ein grundsätzliches Problem von Polynombasen.

Das Runge-Phänomen

Runge-Phänomen (1901):

Interpoliere $f(x) = \frac{1}{1+25x^2}$ auf $[-1, 1]$ mit n äquidistanten Knoten durch p_{n-1} (Grad $n-1$).

- f ist glatt ($\in \mathcal{C}^\infty$!)
- Aber: $\|f - p_{n-1}\|_\infty \to \infty$ für $n \to \infty$!
- Das Wachstum ist **nicht monoton** (von $n = 5$ auf $n = 10$ fällt der Fehler zunächst); die Fehlermaxima wandern an den Rand.



Lösung:

Stückweise Polynome statt eines globalen Polynoms $\to$ **Splines**

Splines

Splines

Def.: Polynom-Spline

Sei $[a, b] \subset \mathbb{R}$ ein Intervall.

Ein **Polynom-Spline q-ten Grades** ist eine Funktion $s\colon [a, b] \to \mathbb{R}$,

- definiert auf einem *Gitter* von *Knoten*

$$a = \xi_0 < \xi_1 < \ldots < \xi_m = b,$$

- so dass

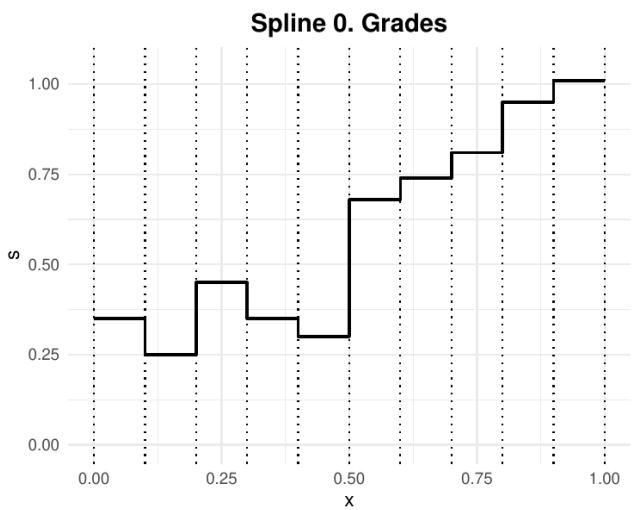
$$s(x) = p_k(x) \quad \text{für } x \in [\xi_{k-1}, \xi_k) \quad (\text{bzw. } x \in [\xi_{m-1}, \xi_m] \text{ für } k = m)$$

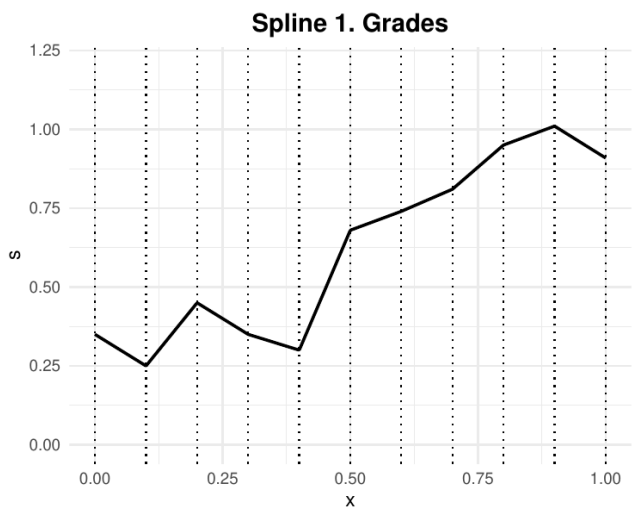
  mit *Polynomen p_k vom Grad höchstens q*, $k = 1, \ldots, m$,
- mit $(q-1)$ stetigen Ableitungen: $s \in \mathcal{C}^{q-1}$.

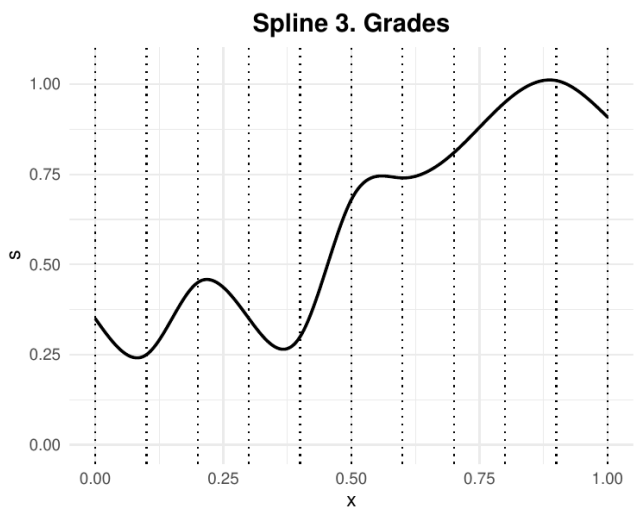
Splines

Weniger formal:

> Ein Spline ist eine Funktion, die Polynome an den Punkten $\xi_1, \ldots, \xi_{m-1}$ zusammensetzt.







Warum $m + q$ Parameter?

Annahme: einfache innere Knoten, maximale Glattheit C^{q-1}

Parameterzählung:

- Ohne Stetigkeitsbedingungen: $m(q+1)$ Parameter (m Intervalle, je $q+1$ Koeffizienten)
- An jedem der $(m-1)$ inneren Knoten: q Bedingungen (Stetigkeit von $s, s', \ldots, s^{(q-1)}$)
- $\Longrightarrow \; m(q+1) - (m-1) \cdot q = m + q$

Interpolation **an $n = m + 1$ Knoten** mit kubischem Spline ($q = 3$):

$m + 3$ Freiheitsgrade aber nur $n = m + 1$ Interpolationsbedingungen
$\Longrightarrow$ Es fehlen **2 Bedingungen**!

Gängige *Randbedingungen*:

Typ	**Bedingung**
Natürlich	$s''(a) = s''(b) = 0$ (lineare Enden)
Eingespannt	$s'(a), s'(b)$ vorgegeben
Periodisch	$s'(a) = s'(b),\ s''(a) = s''(b)$ (für $y_0 = y_m$)

Natürliche Splines sind der Standard, wenn keine Randinformation vorliegt.

Kubischer Spline: Konstruktion Schritt für Schritt

Gegeben:
4 Punkte $(0,0), (1,1), (2,0), (3,-1) \implies m = 3$ Intervalle

Ansatz:
Je Intervall ein kubisches Polynom $p_k(x)$ mit 4 Koeffizienten.

Bedingungen:

Typ	Anzahl	Beispiel
Interpolation	4	$p_1(\xi_0) = y_0,\ p_k(\xi_k) = y_k$
Stetigkeit	2	$p_1(1) = p_2(1)$
s' stetig	2	$p_1'(1) = p_2'(1)$
s'' stetig	2	$p_1''(1) = p_2''(1)$
Randbedingung (natürlich)	2	$s''(0) = s''(3) = 0$

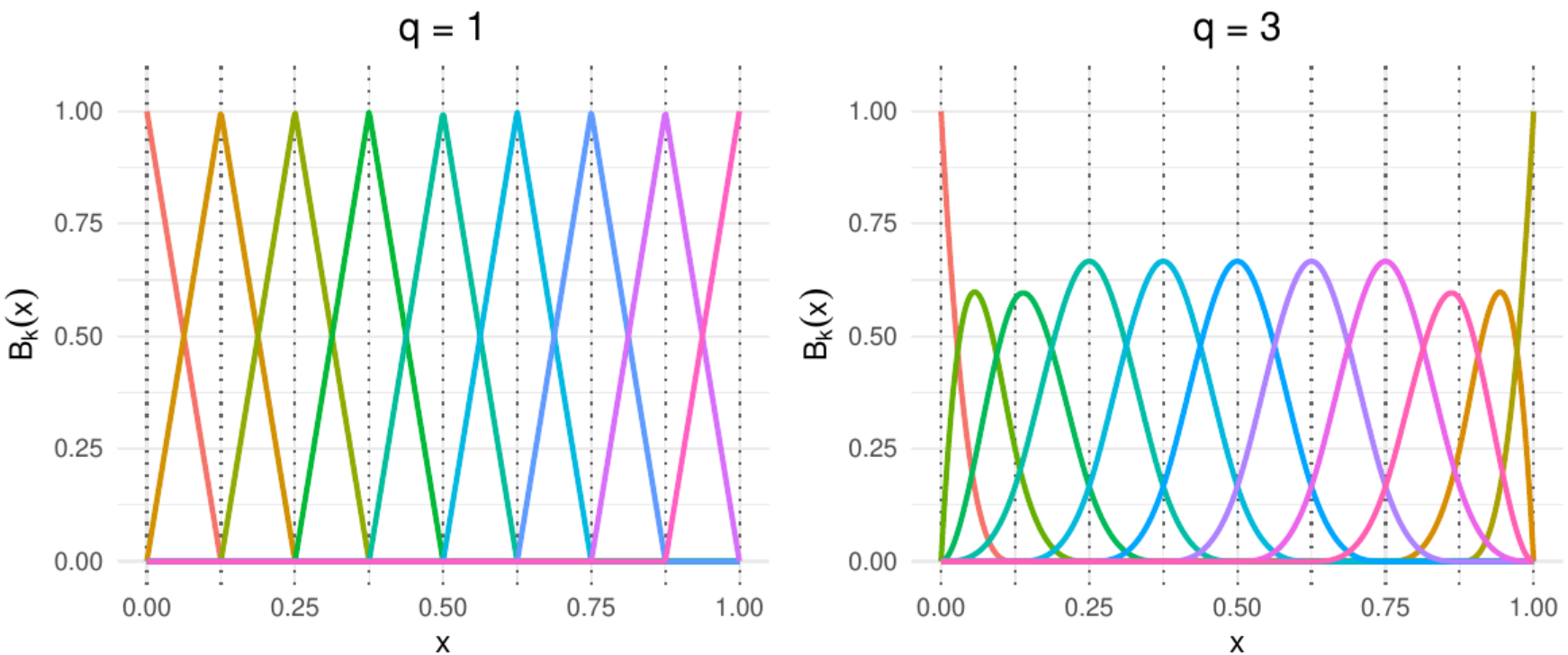
Summe: $4 + 2 + 2 + 2 + 2 = 12$ Bedingungen für $3 \times 4 = 12$ Unbekannte

$\implies$ LGS ist eindeutig lösbar!

B-Splines

- Jeder Spline von Grad q kann auch durch eine Linearkombination von $m + q$ **B-Spline**-Basisfunktionen B_k zum Grad q repräsentiert werden:

$$s(x) = \sum_{k=1}^{m+q} a_k B_k(x)$$



B-Splines: Rekursive Definition

- B-Splines können auf verschiedene Arten definiert werden.
- Rekursive Definition (*Cox-de-Boor-Rekursion*; nur zur Referenz/Transparenz):
  - Erweitere die Knotenfolge zu $\tau_1, \ldots, \tau_{m+2q+1}$:

$$\tau_1 = \cdots = \tau_{q+1} = \xi_0, \quad \tau_{q+1+i} = \xi_i \ \ (i = 1, \ldots, m-1), \quad \tau_{m+q+1} = \cdots = \tau_{m+2q+1} = \xi$$

  - $q = 0$:

$$B_k^{(0)}(x) = \mathbb{1}(\tau_k \leq x < \tau_{k+1})$$

  - $q > 0$:

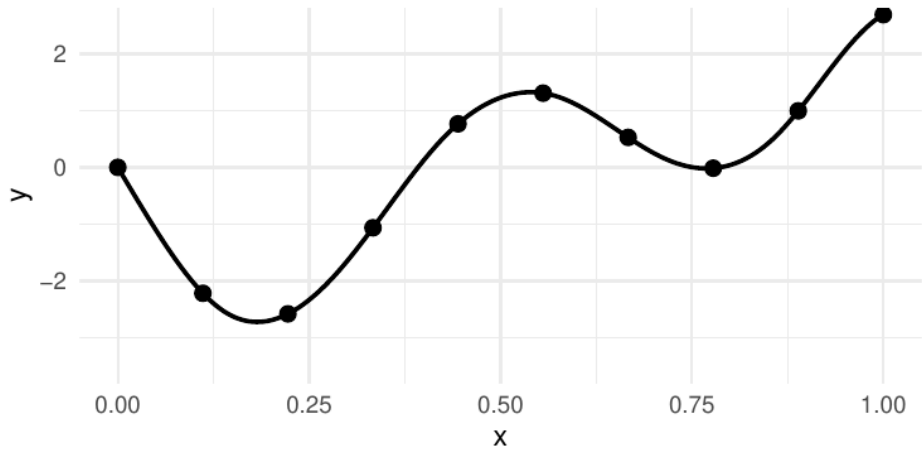
$$B_k^{(q)}(x) = \frac{x - \tau_k}{\tau_{k+q} - \tau_k} B_k^{(q-1)}(x) + \frac{\tau_{k+q+1} - x}{\tau_{k+q+1} - \tau_{k+1}} B_{k+1}^{(q-1)}(x)$$

- in R: `splines::bs()`

B-Splines in R

```r
y <- f(x)
B <- splines::bs(x, df = 10, degree = 3,
                 intercept = TRUE)
a <- solve(B, y)

xnew <- seq(0, 1, length = 100)
Bnew <- splines::bs(xnew,
                    knots = attr(B, "knots"),
                    intercept = TRUE)
fhat <- Bnew %*% a
```



B-Splines: Numerische Stabilität & Effizienz

Vergleich der Konditionszahlen für $n = 20$ Punkte in $[0, 1]$:

Basis	Polynomgrad	Konditionszahl
Monomialbasis	19	$\kappa\left(\boldsymbol{B}_{\text{mono}}\right) \approx 10^{16}$
B-Splines (kubisch, $K = 20$ Basisfunktionen)	3	$\kappa\left(\boldsymbol{B}_{\text{spline}}\right) \approx 10^{2}$

B-Splines sind **rund 14 Größenordnungen** besser konditioniert!

(Größenordnungen, hier κ_2 mit den Standardknoten von `splines::bs()`: $1{,}2 \cdot 10^{16}$ vs. 35; die genauen Werte hängen von Norm und Knotenwahl ab.)

Außerdem: Dünn besetzte *Bandstruktur* (nur $q + 1$ Einträge pro Zeile) der B-Spline-Matrix ermöglicht

- LGS-Lösung in $O(nq^2)$ statt $O(n^3)$
- Für $n = 1000$, $q = 3$: Beschleunigungsfaktor $\approx 100\,000$

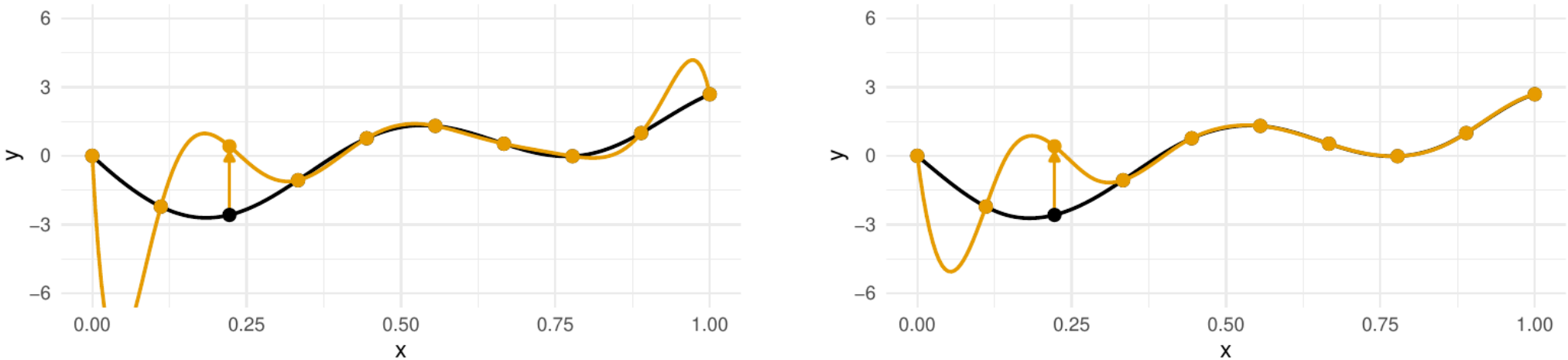
$\Longrightarrow$ B-Splines sind der Standard in der Praxis, trotz komplizierter Definition.

↩ zurück: „(B-)Splines: Eigenschaften"

(B-)Splines: Eigenschaften

- Splines sind das gängigste Basissystem zur Funktionsapproximation, meist **kubisch** ($q = 3$):

1. Wenn $\max_k |\xi_k - \xi_{k-1}|$ hinreichend klein ist, kann jede stetige Funktion f beliebig genau approximiert werden.
2. Die Ableitung eines Grad-q-Splines ist ein Grad-$(q-1)$-Spline, das Integral ein Grad-$(q+1)$-Spline.
3. Ableitungen/Integrale eines Splines approximieren auch die Ableitungen/Integrale der Zielfunktion — für die Ableitungen allerdings nur, wenn f selbst entsprechend oft differenzierbar ist, und mit je einer Ordnung langsamerer Konvergenz.
4. B-Splines sind numerisch stabil (vgl. B-Splines: Numerische Stabilität & Effizienz);
   haben *lokale Träger*: $B_k(x) = 0$ für $x \notin [\tau_k, \tau_{k+q+1}] \implies$ Approximation ist **lokal**, also stabiler;
   $\boldsymbol{B}$ hat *Band-Struktur* $\implies$ effizient lösbar.



Polynominterpolation (links) vs. Spline-Interpolation (rechts): Änderung *eines* Datenpunkts.

Self-Check & Zusammenfassung

Wrap-up

Heute gelernt

- *Approximation, Interpolation, Glättung*
- *Funktionenräume*: Funktionen als Vektoren, *Basisfunktionen* wie Basisvektoren
- Lösungsansatz durch *Basisdarstellung*: Interpolation $\rightarrow$ LGS $\boldsymbol{Ba} = \boldsymbol{y}$
- Polynombasen und ihre Probleme (Kondition, *Runge-Phänomen*); Ausweg: orthogonale Polynome = Gram-Schmidt im Funktionenraum
- *Splines, B-Splines* und ihre Vorteile (Lokalität, Stabilität, Effizienz)

Nächste Vorlesung: Funktionsapproximation II — Glättungsproblem, Bias-Varianz-Trade-off

Self-Check

Frage 1: Zur Charakterisierung eines Splines q-ten Grades auf $m+1$ Knoten benötigen wir...

1. $q+1$ Parameter.
2. $m(q+1)$ Parameter.
3. $m+q$ Parameter.

Frage 2: Ein kubischer Spline auf dem Knotengitter $\xi_0 < \xi_1 < \cdots < \xi_5$ (6 Knoten). Wie viele Basiskoeffizienten hat er?

1. 6
2. 8
3. 9
4. 24

Frage 3: Warum verwendet man in der Praxis B-Splines statt der Monomialbasis? *(mehrere Antworten möglich)*

1. Die B-Spline-Basismatrix ist deutlich besser konditioniert.
2. Nur B-Splines können jede stetige Funktion beliebig genau approximieren.
3. Lokale Träger $\rightarrow$ Bandstruktur $\rightarrow$ effiziente Lösung des LGS.
4. Mit der Monomialbasis hat das Interpolationsproblem keine eindeutige Lösung.

Ausblick: Weiterführende Themen

Weiterführende Themen & Vorlesungen:

- **Mehrdimensionale Interpolation:** Tensor-Produkt-Splines, radiale Basisfunktionen
- **Glättung mit Regularisierung:** P-Splines, Generalisierte Additive Modelle ($\rightarrow$ *Statistik V: Konzepte Statistischer Modellierung*)
- **Moderne ML-Anwendungen:** Spline-basierte Aktivierungsfunktionen (*KANs*), *Normalizing Flows*

Funktionsapproximation: Krümmung, Approximation, Glättung

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 27.08.2026

Verwendete Vorkenntnisse

- **Lineare Algebra:**
  - *Kleinste Quadrate* und *Pseudoinverse*
- **Analysis:**
  - $\mathcal{C}^k$-Funktionen (insb. $\mathcal{C}^2$ und $\mathcal{C}^4$): k-mal stetig differenzierbar
  - Integration ($\int |f''(x)|^2 dx$)
- **Funktionsapproximation I:**
  - *Basen von Funktionenräumen*: $\widehat{f}(x) = \sum_k a_k \phi_k(x)$ mit *Basisfunktionen* ϕ_k und *Basiskoeffizienten* a_k
  - *(B-)Splines*
  - *Interpolation* vs. *Glättung*
- **Statistik:**
  - Designmatrix eines Linearen Modells
  - Bias-Varianz-Zerlegung und MSE
  - Kreuzvalidierung, Informationskriterien (AIC, BIC)

Opener-Quiz: Was wissen Sie noch?

Frage 1:
Was bedeutet $f \in \mathcal{C}^4[a, b]$?

1. f ist auf $[a, b]$ viermal differenzierbar.
2. f ist auf $[a, b]$ viermal *stetig* differenzierbar.
3. f ist stetig und durch 4 beschränkt.
4. f ist ein Polynom vom Grad höchstens 4.

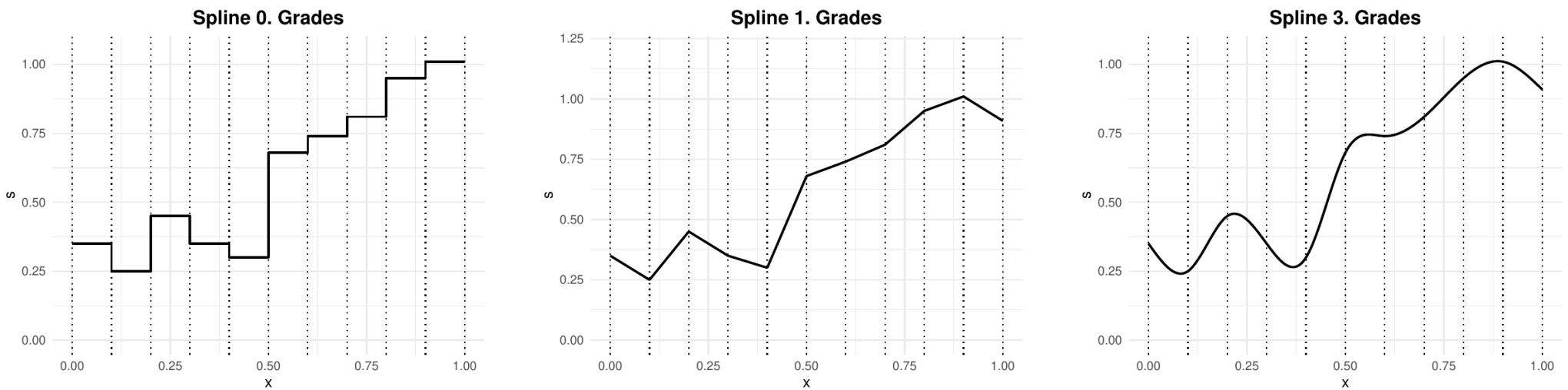
Frage 2:
Sei $\boldsymbol{B} \in \mathbb{R}^{n \times K}$ mit vollem Spaltenrang ($K < n$). Die KQ-Lösung von $\min_{\boldsymbol{a}} \|\boldsymbol{y} - \boldsymbol{B}\boldsymbol{a}\|_2^2$ ist ...

1. $\widehat{\boldsymbol{a}} = \boldsymbol{B}^{-1}\boldsymbol{y}$
2. $\widehat{\boldsymbol{a}} = \boldsymbol{B}^{\top}\boldsymbol{y}$
3. $\widehat{\boldsymbol{a}} = \boldsymbol{B}^{+}\boldsymbol{y} = \left(\boldsymbol{B}^{\top}\boldsymbol{B}\right)^{-1}\boldsymbol{B}^{\top}\boldsymbol{y}$
4. $\widehat{\boldsymbol{a}} = \left(\boldsymbol{B}\boldsymbol{B}^{\top}\right)^{-1}\boldsymbol{B}\boldsymbol{y}$

Nützliche Theorie zu Splines

Splines

Erinnerung: Ein *Spline* vom Grad q ist eine Funktion, die Polynome an den Punkten $\xi_1, \ldots, \xi_{m-1}$ **glatt** zusammensetzt: dort stimmen alle Ableitungen bis zur Ordnung $q-1$ überein, d.h. $s \in \mathcal{C}^{q-1}$.



- Splines von Grad ℓ zu festen Knoten bilden einen *Funktionenraum*
- *B-Splines* sind eine besonders gutartige **Basis** dieses Raums.

Minimale Krümmung

Kubische Splines erzeugen besonders einfache “glatte” Kurven.
Das ist kein Zufall:

Man kann sie herleiten, indem man Interpolanden mit *minimaler Krümmung* sucht (diese Eigenschaft hat auf englisch auch den schönen Namen *minimal* ***wiggliness***).

Theorem: Kubische Splines haben minimale Krümmung

Seien $(x_1, y_1), \dots, (x_n, y_n) \in [a, b] \times \mathbb{R}$ mit $a = x_1 < x_2 < \cdots < x_n = b$ und s ein *natürlicher kubischer Spline*, der diese Punkte interpoliert (d.h. mit $s''(a) = s''(b) = 0$).

Sei $g \in \mathcal{C}^2[a, b]$ eine weitere Funktion mit $g(x_i) = y_i \quad \forall i.$

Dann gilt:

$$\int_a^b \left|s''(x)\right|^2 dx \leq \int_a^b \left|g''(x)\right|^2 dx.$$

“Krümmung” meint hier $\int_a^b |f''(x)|^2 dx$, nicht das Integral der geometrischen Krümmung $\kappa = |f''| / \left(1 + (f')^2\right)^{3/2}$ – beide fallen nur für flache Kurven mit $|f'| \ll 1$ zusammen. Der Name ist trotzdem eingebürgert.

Minimale Krümmung: Beweis

Sei s der natürliche kubische Spline-Interpolant zu (x_i, y_i).
Für einen anderen Interpolanten $g \in \mathcal{C}^2$ setze $g = s + h$ mit $h(x_i) = 0 \; \forall i$.
Sei $J(g) := \int_a^b (g''(x))^2 dx$. Dann gilt

$$J(g) = J(s) + 2\int_a^b s''(x)h''(x)dx + \int_a^b (h''(x))^2 dx.$$

Auf jedem Intervall $[x_{i-1}, x_i]$, $i = 2, \ldots, n$, gilt durch partielle Integration

$$\int s''h''dx = \left[s''h' - s^{(3)}h\right]_{x_{i-1}}^{x_i} + \int s^{(4)}hdx.$$

- für kubische Splines ist $s^{(4)} = 0$
- $\sum_{i=2}^n \left[s''h' - s^{(3)}h\right]_{x_{i-1}}^{x_i} = \left[s''h'\right]_a^b = 0$, weil
  - $\left[s^{(3)}h\right]_{x_{i-1}}^{x_i} = 0$ wegen $h(x_i) = 0$,
  - sich die inneren $s''h'$-Randterme paarweise aufheben (s'' und h' sind an den inneren Knoten stetig,
  - und $s''(a) = s''(b) = 0$ (natürlicher Spline).

$$\implies J(g) = J(s) + \int_a^b \left(h''(x)\right)^2 dx \;\geq\; J(s) \implies \text{Gleichheit nur für } h \equiv 0. \qquad \blacksquare$$

Letzter Schritt:
Gleichheit heißt $\int_a^b (h'')^2 dx = 0$, *also* $h'' \equiv 0$ (h'' *ist stetig*), *also* h *affin;*
da h *an den* $n \geq 2$ *Stellen* x_i *verschwindet, folgt* $h \equiv 0 \implies$ *Der Spline* s *ist der* ***einzige*** *Minimierer.*

Minimale Krümmung: Beispiel

Gegeben:
Punkte $(0,0), (1,1), (2,0)$

Natürlicher kubischer Spline:

$$s(x) = \begin{cases} \frac{3}{2}x - \frac{1}{2}x^3 & x < 1 \\ \frac{3}{2}(2-x) - \frac{1}{2}(2-x)^3 & x \geq 1 \end{cases}$$

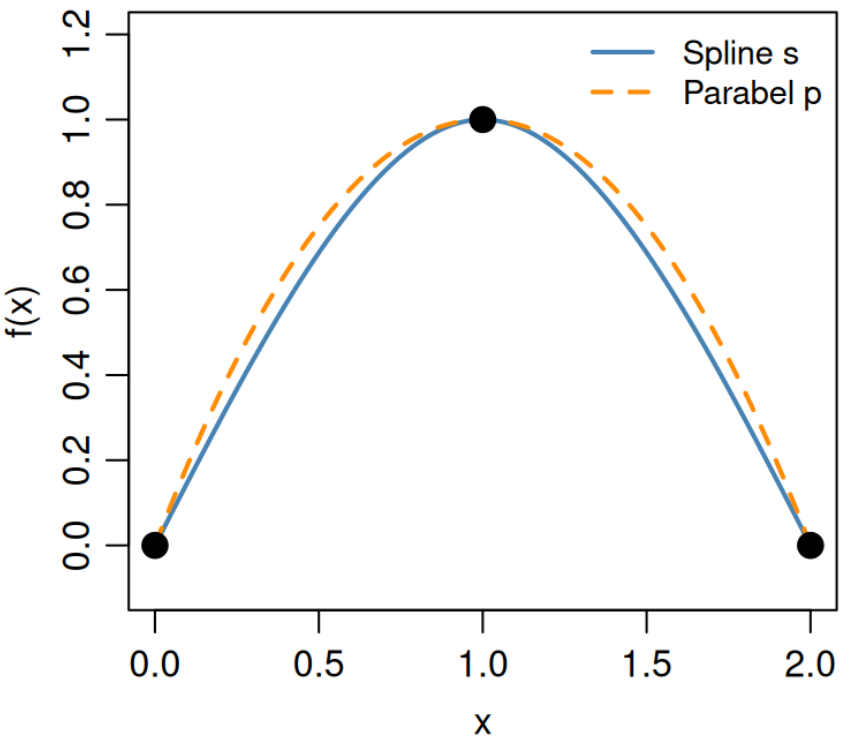
$s''(x) = -3x$ bzw. $s''(x) = -3(2-x)$
$\Longrightarrow s''(0) = s''(2) = 0$ ✓

Vergleich mit Parabel:

$$p(x) = -x^2 + 2x$$

Interpolant	$\int_0^2 \lvert f''(x)\rvert^2 dx$
Spline s	6
Parabel p	8

$\Longrightarrow$ Spline hat kleinere Krümmung!



Approximationseigenschaft

- Bereits erwähnt: Wenn die *Gitterweite* (der größte Abstand zweier benachbarter Knoten) hinreichend klein ist, kann jede stetige Funktion f beliebig genau approximiert werden. Genauer:

Theorem: Approximationsfehler kubischer Splines

Sei $a = x_0 < x_1 < \cdots < x_n = b$ eine Partition mit **Gitterweite** $h := \max_i |x_i - x_{i-1}|$. Für jedes $f \in \mathcal{C}^4[a,b]$ sei s der eindeutig bestimmte *eingespannte kubische Spline-Interpolant* mit

$$f(x_i) = s(x_i)\ \forall i, \qquad s'(a) = f'(a), \qquad s'(b) = f'(b).$$

Dann gilt

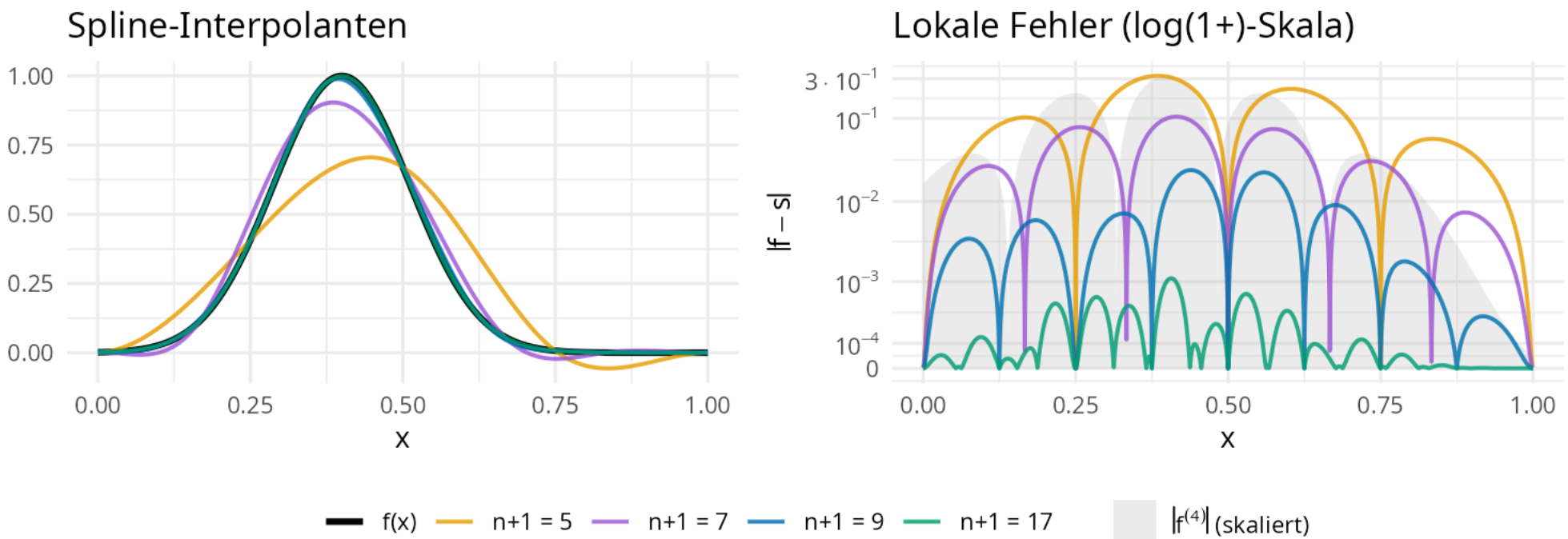
$$\max_{x\in[a,b]} |f(x) - s(x)| \leq \frac{5}{384} h^4 \max_{x\in[a,b]} \left|f^{(4)}(x)\right|.$$

Erkenntnisse:

Approximation: Lokaler Fehler

Gegeben:

$f(x) = \exp\left(-40\left(x - \frac{2}{5}\right)^2\right)$ auf $[0, 1]$ mit $\max \left|f^{(4)}(x)\right| = 19200$ (am schmalen Peak $x = \frac{2}{5}$, der kein Knoten ist)



Beobachtung:

Mit nur fünf Knoten ist die Abweichung deutlich sichtbar. Sie konzentriert sich um den schmalen Peak, wo $\left|f^{(4)}\right|$ groß ist; an den Knoten selbst ist der Interpolationsfehler natürlich null.

Approximation: h^4-Rate

Knoten $n+1$	**Gitterweite** h	**Fehlerschranke** $\frac{5}{384} h^4 \max_x \lvert f^{(4)}(x) \rvert$	**Tatsächlicher Fehler** $\max_x \lvert f(x) - s(x) \rvert$
5	0.25	≈ 0.977	≈ 0.324
7	0.1667	≈ 0.193	≈ 0.1048
9	0.125	≈ 0.061	≈ 0.0239
17	0.0625	≈ 0.0038	≈ 0.00111

Beobachtung:

- Halbierung von h verkleinert die *Schranke* exakt um $2^4 = 16$.
- Der tatsächliche Fehler muss bei groben Gittern nicht exakt diesem Faktor folgen (hier: ≈ 14 und ≈ 22): $O(h^4)$ beschreibt die *asymptotische* Rate.

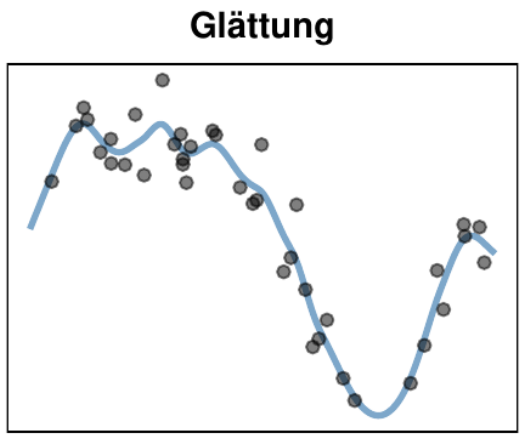
*Berechnet für den **eingespannten** Spline-Interpolanten aus dem Theorem, also mit $s'(0) = f'(0)$ und $s'(1) = f'(1)$.*

Approximation: Zusammenfassung

- (B-)Splines sind extrem mächtige Werkzeuge in der *Funktionsapproximation*.
- Sie haben hervorragende theoretische und praktische Eigenschaften:
  - Effiziente Berechnung (B-Splines)
  - Stabilität/Lokalität (B-Splines)
  - Minimale Krümmung des natürlichen kubischen Spline-Interpolanten
  - Approximationsfehler $O(h^4)$ für kubische Splines

Glättungs-/Regressionsprobleme

Das Problem



Modell:

$$y_i = f(x_i) + \epsilon_i, \qquad \epsilon_i \text{ i.i.d.} \quad \text{mit } \mathbb{E}[\epsilon_i | x_i] = 0.$$

Problem:

Finde eine Funktion $\widehat{f}$ mit kleinem Fehler

$$\sum_{i=1}^{n} \left(y_i - \widehat{f}(x_i) \right)^2.$$

*Über **alle** Funktionen ist dieses Minimum 0 und wird von jedem Interpolanten angenommen (davon gibt es unendlich viele; siehe Kapitel* Funktionsapproximation I*) $\rightarrow$ entartetes Problem.*
Wir brauchen eine Einschränkung des Suchraums!

Die Lösung

1. Nehme an, dass $\widehat{f} \in \mathcal{F}_K$, d.h. $\widehat{f}(x) = \sum_{k=1}^{K} a_k \phi_k(x)$ für

- *Basisfunktionen* $\phi_1, \ldots, \phi_K$ (bekannt; z.B. *B-Spline-Basis*)
- *Basiskoeffizienten* $a_1, \ldots, a_K$ (unbekannt)

2. Löse das KQ-Problem

$$\min_{\widehat{f} \in \mathcal{F}_K} \sum_{i=1}^{n} \left(y_i - \widehat{f}(x_i) \right)^2 = \min_{\boldsymbol{a}} \left\| \begin{pmatrix} y_1 \\ \vdots \\ y_n \end{pmatrix} - \begin{pmatrix} \phi_1(x_1) & \cdots & \phi_K(x_1) \\ \vdots & \ddots & \vdots \\ \phi_1(x_n) & \cdots & \phi_K(x_n) \end{pmatrix} \begin{pmatrix} a_1 \\ \vdots \\ a_K \end{pmatrix} \right\|_2^2 .$$

Unterschied zu Interpolation:

- *Interpolation*: $K = n$
- *Glättung*: $K < n$

Warum Glättung statt Interpolation?

Interpolation ($K = n$):

- Passt **alle** Datenpunkte exakt an:

$$\$\widehat{f}(x_i) = y_i \; \forall i,$$

  sofern $\boldsymbol{B}$ invertierbar ist (mit ungünstig platzierten Knoten kann $\boldsymbol{B}$ auch für $K = n$ singulär sein)
- **Problem:**
  $\widehat{f}$ modelliert auch das Rauschen ε_i!

Glättung ($K < n$):

- Akzeptiert, dass $\widehat{f}(x_i) \neq y_i$
- Erzielt eine **glattere, generalisierbarere** Funktion
- *Alternative:*
  - *Smoothing Splines* mit $K = n$ + Glattheits-Penalty $\lambda \int |f''|^2$
    (auch: *Penalized Splines* mit $K < n$ + Glattheits-Penalty $\rightarrow$ *Additive Modelle*)

Wann welche Methode?

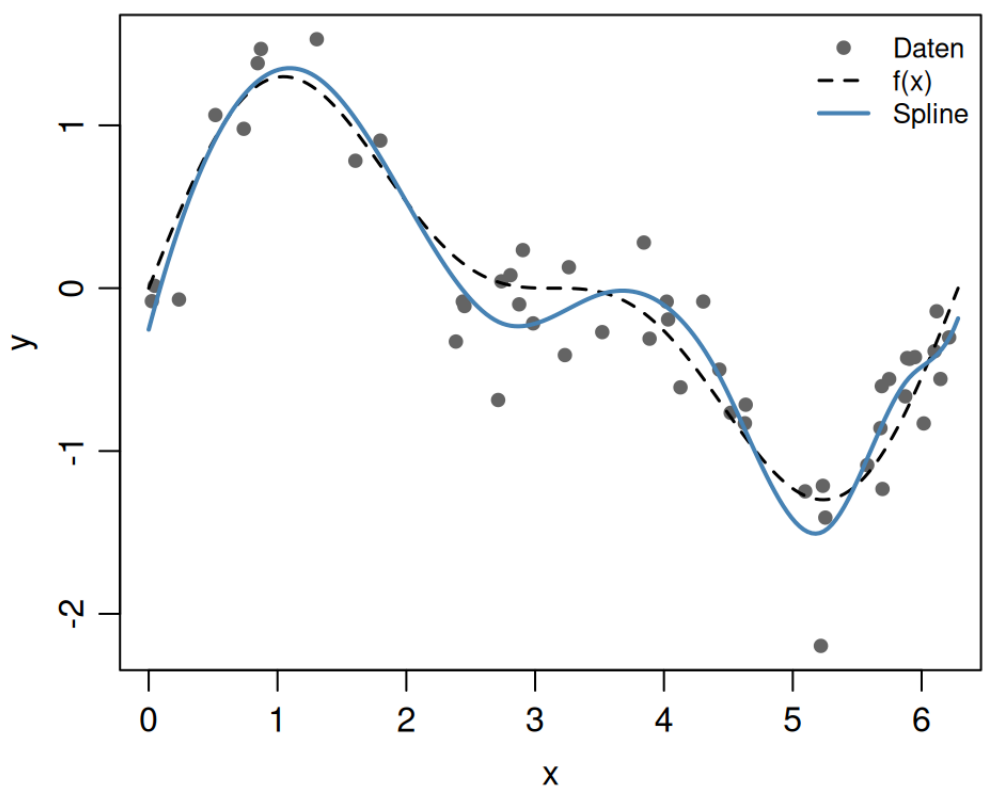
Situation	Methode
Präzise, rauschfreie Daten	Interpolation
Verrauschte Messungen	Glättung
Vorhersage für neue Daten	Glättung

Beispiel

```r
set.seed(42)
n <- 50
x <- sort(runif(n, 0, 2 * pi))
f_true <- function(x) sin(x) + 0.5 * sin(2 * x)
y <- f_true(x) + rnorm(n, sd = 0.3)

# Designmatrix: B-Spline-Basis, K = 10
B <- splines::bs(x, df = 10, intercept = TRUE,
  Boundary.knots = c(0, 2 * pi))
a_hat <- qr.solve(B, y) # KQ via QR
#! äquivalent:
#! Q * R * a = y --> R * a = Q^T * y --> a = R^- * Q^T * y
#! qrB <- qr(B)
#! a_hat <- backsolve(qr.R(qrB), t(qr.Q(qrB)) %*% y)

# Auswertung auf neuen Punkten
x_new <- seq(0, 2 * pi, length = 200)
B_new <- splines::bs(x_new,
  knots = attr(B, "knots"),
  Boundary.knots = attr(B, "Boundary.knots"),
  intercept = TRUE)
f_hat <- B_new %*% a_hat
```



Bias-Varianz Trade-off: Bias

Annahme: gleichmäßige Knoten auf $[a, b]$, $f \in \mathcal{C}^4$, kubische Splines

- Für Spline mit K Basisfunktionen:

$$h = \max_i |\xi_i - \xi_{i-1}| = O(1/K).$$

- Wäre $\epsilon_i = 0$, hätten wir

$$\max_{x \in [a,b]} \left| f(x) - \widehat{f}(x) \right| = O(h^4) = O(K^{-4}).$$

- Der Approximationsfehler wird im statistischen Kontext auch **Bias** genannt.
- Für **kleinen Bias** müssen wir also K **möglichst groß** wählen.

Bias-Varianz Trade-off: Varianz

Annahme: $\mathbb{V}\mathrm{ar}[\epsilon_i] = \sigma^2 \; \forall \, i (\text{homoskedastisch})$*

- Da $\boldsymbol{\epsilon} = (\epsilon_1, \ldots, \epsilon_n)^\top$ zufällig ist, ist auch

$$\widehat{\boldsymbol{a}} = \boldsymbol{B}^+ \boldsymbol{y} = \boldsymbol{B}^+ \boldsymbol{f} + \boldsymbol{B}^+ \boldsymbol{\epsilon}$$

  zufällig.
  $\boldsymbol{B}^+$ ist *Pseudoinverse* der Designmatrix $\boldsymbol{B} = [\phi_k(x_i)] \in \mathbb{R}^{n \times K}$ des KQ-Problems für $\boldsymbol{f} = (f(x_1), \ldots, f(x_n))^\top$.
- Man kann zeigen, dass

$$\mathbb{V}\mathrm{ar}\left[\widehat{f}(x)\right] = \mathbb{V}\mathrm{ar}\left[\sum_{k=1}^{K} \widehat{a}_k \phi_k(x)\right] = O(K/n).$$

- Für **kleine Varianz** wollen wir also K **möglichst klein** wählen.

Exakt ist das im Mittel über die Designpunkte: $\frac{1}{n}\sum_{i=1}^{n} \mathbb{V}\mathrm{ar}\left[\widehat{f}(x_i)\right] = \sigma^2 K/n$ (Spur der Hatmatrix $= K$, falls $\boldsymbol{B}$ vollen Spaltenrang hat); punktweise braucht die Rate Zusatzannahmen an Knoten und Design.

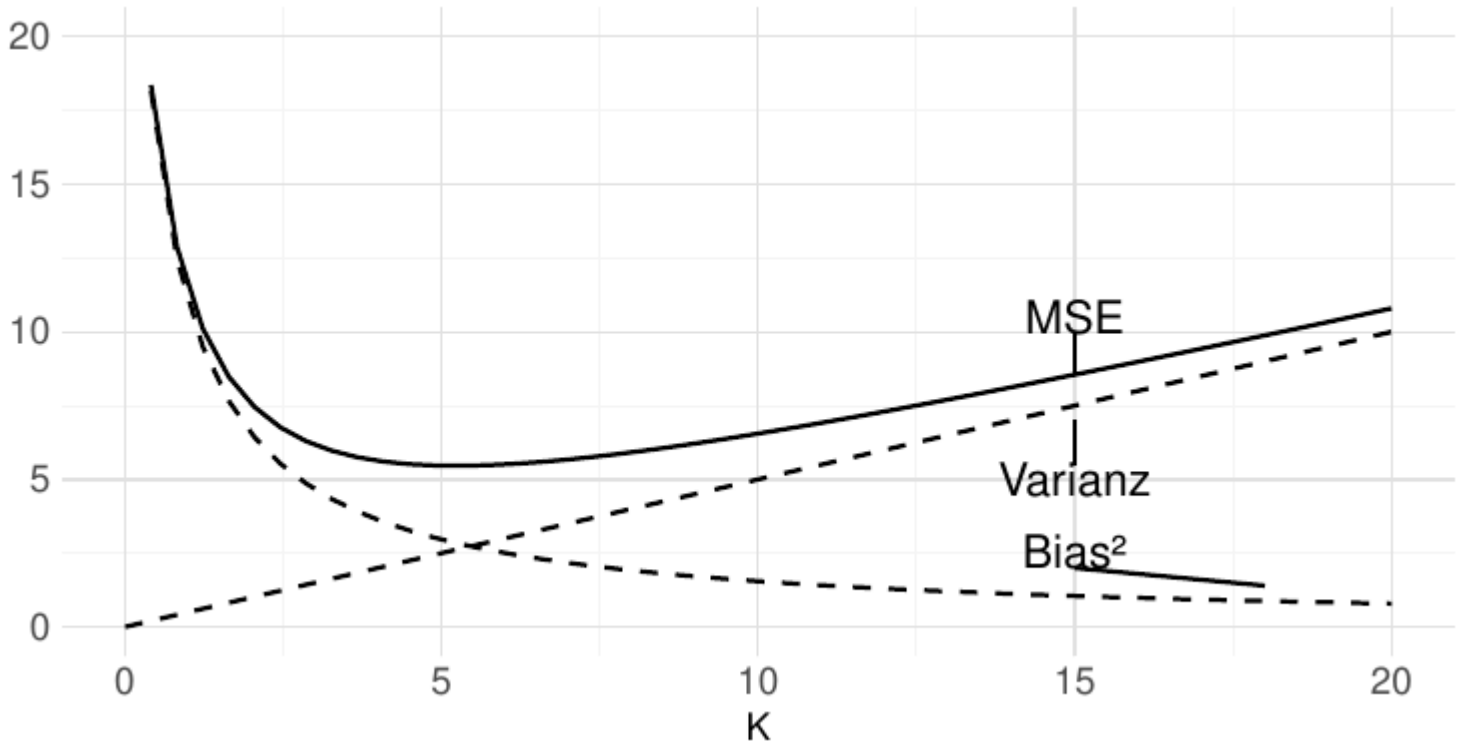
Vorhersage: Wenn K wächst — steigt Bias oder Varianz?

Think–Pair–Share (2 Minuten, mit Sitznachbar:in):

- Wenn K wächst: Was passiert mit dem **Bias**? Was mit der **Varianz**?
- Skizzieren Sie **Bias², Varianz und MSE** $= \text{Bias}^2 + \text{Varianz}$ als Funktionen von K (auf Papier!).
- Wo liegt das optimale K — eher klein, eher groß, oder dazwischen? Woran erkennt man es in Ihrer Skizze?

$\rightarrow$ Auflösung im folgenden Abschnitt zum Bias-Varianz-Trade-off.

Bias-Varianz Trade-off



- **Auflösung:**
  Bias² *fällt* in K, Varianz *steigt* in K — der MSE setzt sich aus beiden zusammen:

$$\mathbb{E}\left[\left(\widehat{f}(x) - f(x)\right)^2\right] = \left(\mathbb{E}\left[\widehat{f}(x)\right] - f(x)\right)^2 + \mathbb{V}\mathrm{ar}\left[\widehat{f}(x)\right] = O(K^{-8} + K/n).$$

- Das optimale K liegt *dazwischen*: am Minimum der MSE-Kurve, wo sich fallender Bias und steigende Varianz gerade ausbalancieren.

Die (theoretisch) optimale Wahl von K

$$\mathbb{E}\left[\left(\widehat{f}(x) - f(x)\right)^2\right] = O(K^{-8} + K/n)$$

- Die (theoretisch!) optimale Wahl $K \sim n^{1/9}$ balanciert beide Teile aus und es gilt dann:

$$\mathbb{E}\left[\left(\widehat{f}(x) - f(x)\right)^2\right] = O(n^{-8/9}).$$

- *Rein asymptotische Ordnung* — taugliches K in der Praxis deutlich größer, da Konstanten bei moderatem n dominieren.

Bias-Varianz: Beispiel

Setup:
$f(x) = \sin(3x), n = 100, \sigma = 0.3$

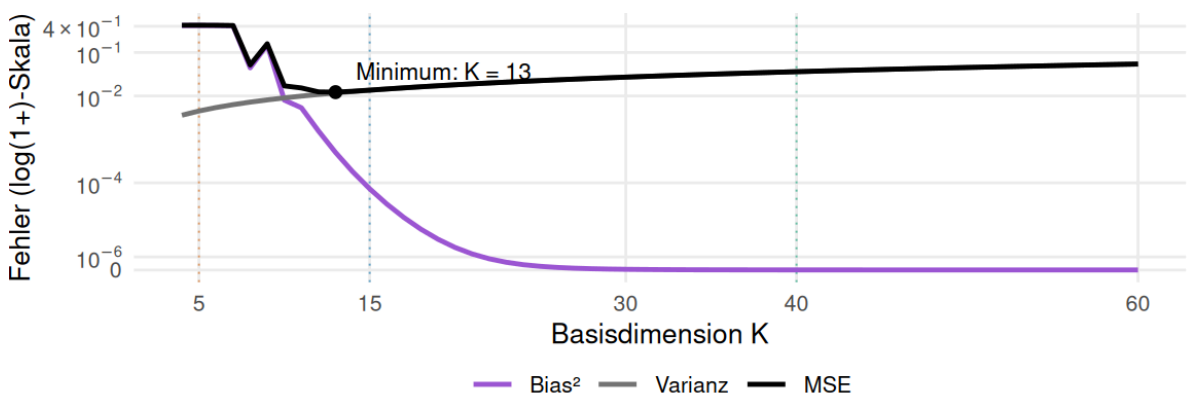
Modell	K	Bias²	Var	MSE
Unteranpassung	5	**0.416**	0.005	0.421
Nahezu optimal	15	0.000	0.014	0.014
Überanpassung	40	0.000	**0.036**	0.036

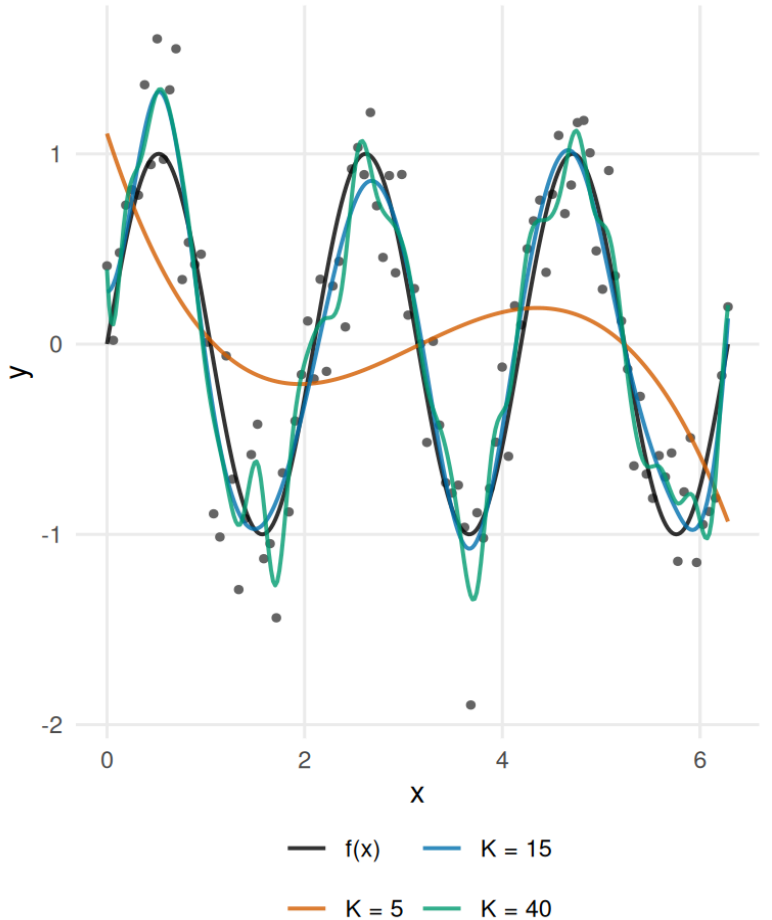
(Bias² und Varianz exakt berechnet für den KQ-Fit, gemittelt über äquidistante Designpunkte; $\mathrm{Var} = \sigma^2 K/n$)

Interpretation:

- $K = 5$: Spline zu starr $\Longrightarrow$ hoher Bias
- $K = 40$: Spline passt Rauschen an $\Longrightarrow$ hohe Varianz
- $K = 15$: Balanciert Bias und Varianz

Über $K \in 4, \ldots, 60$





Wie wählt man K in der Praxis?

Problem:
Wir kennen f nicht, also wissen wir nicht, ob K optimal ist!

***Mögliche* Ansätze:**

1. *Kreuzvalidierung*: Wähle K, das den Validierungsfehler minimiert
2. *Informationskriterien*: AIC und BIC balancieren Anpassung und Komplexität
3. *GCV*: $\mathrm{GCV}(K) = \frac{\mathrm{RSS}_K/n}{(1-K/n)^2}$

Praktisch verwendeter Ansatz:

- wähle K "groß genug"
- steuere Glattheit der Funktionsschätzung über **Penalisierung**.
  $\rightarrow$ *P-Splines*, *Additive Modelle*
  In R z.B. `mgcv::gam()`

Multivariate Funktionsapproximation

Multivariate Funktionsapproximation

- Zur Approximation von multivariaten $f\colon \mathbb{R}^p \to \mathbb{R}$ kann man eine *Tensor-Produkt-Basis* bilden (vgl. Tensorproduktbasen im Kapitel *Tensoren*):

$$\widehat{f}(\boldsymbol{x}) = \sum_{j_1=1}^{K} \cdots \sum_{j_p=1}^{K} a_{j_1,\ldots,j_p} \left(\phi_{j_1}(x_1) \times \cdots \times \phi_{j_p}(x_p)\right)$$

  mit Koeffizienten

$$\boldsymbol{A} = (a_{j_1,\ldots,j_p})_{j_i=1,\ldots,K, i=1,\ldots,p} \in \mathbb{R}^{K\times\cdots\times K}.$$

- $\boldsymbol{A}$ kann wieder durch Lösung eines gewöhnlichen KQ-Problems gefunden werden.
- Aber: *Fluch der Dimensionalität*
  - $\widehat{f}$ auswerten kostet $O(K^p)$ (mit B-Splines wegen der lokalen Träger nur $(q+1)^p$ Summanden — ebenfalls exponentiell in p).
  - $\mathrm{MSE}(\widehat{f}) = O(K^{-8} + K^p/n)$; mit "optimalem" $K \sim n^{1/(8+p)}$: $\mathrm{MSE} = O(n^{-8/(8+p)})$

  $\Longrightarrow$ sehr langsam für großes p.

Fluch der Dimensionalität

Szenario: $f\colon \mathbb{R}^p \to \mathbb{R}$ mit $K = 10$ Basisfunktionen pro Dimension

Speicher:

Dim. p	Koeffizienten K^p	Speicher
1	10	80 Bytes
2	100	800 Bytes
3	$1\,000$	8 kB
5	$100\,000$	800 kB
10	10^{10}	80 GB

Dezimale Präfixe: 1 kB $= 10^3$ Bytes, 1 GB $= 10^9$ Bytes.

Konvergenz (optimales $K \sim n^{1/(8+p)}$):

Dim. p	Konvergenzrate	n für MSE ≤ 0.01
1	$n^{-8/9}$	≈ 180
2	$n^{-0.8}$	≈ 320
5	$n^{-8/13}$	$\approx 1\,800$
10	$n^{-8/18}$	$\approx 32\,000$

illustrative Werte — entscheidende Konstanten sind problemabhängig

$\Longrightarrow$ Speicherbedarf und Datenbedarf wachsen **exponentiell** mit p!

GAMs: Lösung für multivariate Probleme

Problem:
Tensor-Produkt mit K^p Parametern für $p > 3$ unpraktikabel

Lösungsansatz:
Benutze *additive* Struktur statt vollem Tensor-*Produkt*:

Statt

$$\widehat{f}(\boldsymbol{x}) = \sum_{j_1=1}^{K} \cdots \sum_{j_p=1}^{K} a_{j_1,\ldots,j_p} \prod_{i=1}^{p} \phi_{j_i}(x_i) \quad (K^p \text{ Koeffizienten})$$

verwende **Generalisiertes Additives Modell (GAM)**:

$$\widehat{f}(\boldsymbol{x}) = \beta_0 + \sum_{i=1}^{p} f_i(x_i) \quad (pK + 1 \text{ Koeffizienten})$$

Genauer:
$pK + 1$ *Koeffizienten, wegen Identifizierbarkeit (*f_i *sind nur bis auf additive Konstanten bestimmt) nur* $p(K-1)+1$ *frei wählbar — Größenordnung* $O(pK)$.

Aber:
rein additiv — *Interaktionseffekte* (z.B. $f(x_1, x_2) = x_1 \cdot x_2$) werden nicht erfasst; dafür: zusätzliche niedrigdimensionale Tensorprodukt-Terme.

Vorteile:

- *Skalierbar:* $O(pK)$ statt $O(K^p)$ Parameter
- *Interpretierbar:* Effekt jeder Variable einzeln visualisierbar
- **In R:** `mgcv::gam()` mit automatischer Glättungsparameterwahl

Self-Check & Zusammenfassung

Ausblick

Mehr zu

- theoretischer Analyse von statistischen Methoden
- Eigenschaften von Parameterschätzung
- Funktionsapproximation

in Thomas Naglers Vorlesungen:

- *Statistical Learning Theory* (MSc)
- *Mathematical Statistics* (BSc/MSc)

Mehr zu GAMs & co in *Statistik V: Konzepte Statistischer Modellierung* (BSc)

Self-Check

Frage 1: Unter allen Interpolanten $g \in \mathcal{C}^2[a,b]$ der Punkte (x_i, y_i) …

1. minimiert der natürliche kubische Spline $\int_a^b |g''(x)|^2 dx$.
2. ist der natürliche kubische Spline der *einzige* Interpolant, der $\sum_i (g(x_i) - y_i)^2$ minimiert.
3. hat jeder kubische Spline kleinere Krümmung als jedes Polynom.
4. minimiert die Parabel die Krümmung.

Frage 2: Wir verdoppeln die Anzahl der Basisfunktionen K (bei festem n). Dann gilt (asymptotisch):

1. Bias sinkt, Varianz steigt.
2. Bias steigt, Varianz sinkt.
3. Bias und Varianz sinken beide.
4. Bias und Varianz steigen beide.

Frage 3: Was bedeutet die "optimale Wahl" $K \sim n^{1/9}$?

1. In der Praxis ist $K = \lceil n^{1/9} \rceil$ immer die beste Wahl.
2. Rein asymptotische Ordnung — praktisch wählt man K deutlich größer, da die Konstanten dominieren.
3. K muss linear mit n wachsen.
4. Der MSE fällt dann mit Rate $n^{-1/9}$.

Wrap-up

Heute gelernt

- Eigenschaften kubischer Splines (*Minimale Krümmung* des natürlichen Spline-Interpolanten, Approximationsfehler $O(h^4)$)
- *Bias-Varianz Trade-off* bei der Glättung: $\mathrm{MSE} = O(K^{-8} + K/n)$
- Praktische Modellwahl mit CV, AIC, GCV — oder K groß + Penalisierung
- *Fluch der Dimensionalität* und GAMs als Lösung

Finito!